MDN JavaScript Documentation

Volume 1 of 9 — JavaScript Guide

Unofficial print compilation. The MDN JavaScript landing page and the complete JavaScript Guide (fundamental, intermediate and advanced guides). This volume contains 38 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

JavaScript 7
Beginner's tutorials 7
JavaScript guides 8
Reference 9

JavaScript Guide 10
Introduction 10
Grammar and types 10
Control flow and error handling 10
Loops and iteration 10
Functions 11
Expressions and operators 11
Numbers and strings 11
Representing dates & times 11
Regular expressions 11
Indexed collections 12
Keyed collections 12
Working with objects 12
Using classes 12
Promises 12
Typed arrays 12
Iterators and generators 12
Resource management 13
Internationalization 13
JavaScript modules 13
Advanced topics 13

Introduction 14
What you should already know 14
Where to find JavaScript information 14
What is JavaScript? 14
JavaScript and Java 15
JavaScript and the ECMAScript specification 15
Getting started with JavaScript 16
What's next 18

Grammar and types 19
Basics 19
Comments 19
Declarations 20
Data structures and types 24
Literals 25
More information 33

Control flow and error handling 34
Block statement 34
Conditional statements 35
Exception handling statements 38

Loops and iteration 43
for statement 43
do...while statement 44
while statement 45
labeled statement 46
break statement 46
continue statement 47
for...in statement 48
for...of statement 49

Functions 51
Defining functions 51
Calling functions 53
Function scopes and closures 56
Using the arguments object 60

Function parameters 61
Arrow functions 62

Expressions and operators 64
Assignment operators 65
Comparison operators 69
Arithmetic operators 70
Bitwise operators 71
Logical operators 72
BigInt operators 74
String operators 74
Conditional (ternary) operator 75
Comma operator 75
Unary operators 75
Relational operators 77
Basic expressions 78

Numbers and strings 81
Numbers 81
Number object 82
Math object 83
BigInts 84
Strings 85
String object 86
Template literals 87

Representing dates & times 88
Date object 88

Regular expressions 91
Creating a regular expression 91
Writing a regular expression pattern 91
Using regular expressions in JavaScript 93
Examples 96
Tools 97
Assertions 99
Try it 99
Types 99
Examples 100
See also 103
Character classes 104
Try it 104
Types 104
Examples 106
See also 107
Groups and backreferences 109
Try it 109
Types 109
Examples 110
See also 111
Quantifiers 113
Try it 113
Types 113
Examples 114
See also 115
Regular expression syntax cheat sheet 116
Character classes 116
Assertions 118
Groups and backreferences 119
Quantifiers 120

Indexed collections 122
Creating an array 122
Referring to array elements 123
Populating an array 124
Array transformations 131

Sparse arrays 132
Multi-dimensional arrays 133
Using arrays to store other properties 133
Working with array-like objects 133

Keyed collections 135
Maps 135
Sets 136
Key and value equality of Map and Set 138

Working with objects 139
Creating new objects 139
Objects and properties 142
Inheritance 145
Defining methods 146
Defining getters and setters 147
Comparing objects 148
See also 149

Using classes 150
Overview of classes 150
Declaring a class 151
Constructor 152
Instance methods 154
Private fields 155
Accessor fields 159
Public fields 160
Static properties 160
Extends and inheritance 161
Why classes? 163

Using promises 166
Chaining 166
Error handling 170
Composition 173
Cancellation 174
Creating a Promise around an old callback API 174
Timing 175
See also 176

JavaScript typed arrays 177
Buffers 177
Views 178
Web APIs using typed arrays 180
Examples 181
See also 183

Iterators and generators 184
Iterators 184
Generator functions 185
Iterables 186
Advanced generators 187

JavaScript resource management 189
Problem 189
The using and await using declarations 191
The DisposableStack and AsyncDisposableStack objects 193
Error handling 195
Examples 197
Pitfalls 198
Conclusion 198

Internationalization 200
Overview 200
Locale information 201
Figuring out the locale 203
Understanding the return value 204
Formatting data 205

Collation 212
Plural rules 213
Segmentation 214
Display names 215

JavaScript modules 219
Module philosophy 219
Working with examples 223
Named exports 224
Named imports 224
Default exports and imports 225
Avoiding naming conflicts 226
Aggregating modules 229
Importing a module for its side effects 230
Import declarations are hoisted 230
Importing JSON modules 231
Dynamic module loading 231
Top-level await 232
Module metadata 234
Language versus host responsibilities 234
See also 235
Using modules on the web 236
Server configuration 236
Applying modules to your HTML 236
Modules in workers 237
Module specifiers on the web 237
Importing modules using import maps 238
Loading non-JavaScript resources 244
Module caching 245
The import.meta object 246
See also 246
Understanding the module graph 247
What is the module graph? 247
From module source to execution 247
Errors during the module pipeline 253
Cyclic imports 256
Top-level await and asynchronous evaluation 260
Dynamic imports 264
Module caching 266
Controlling import phases 266
See also 270
Modules across platforms 271
Other module formats 271
Using modules in Node.js 273
Transpilers 276
Bundlers 277
How TypeScript understands modules 281
"Isomorphic" modules 283
See also 284

JavaScript language overview 286
Data types 286
Variables 289
Operators 290
Grammar 291
Control structures 291
Objects 294
Arrays 295
Functions 297
Classes 300
Asynchronous programming 301
Modules 302
Language and runtime 303
Further exploration 303

JavaScript data types and data structures 304
Dynamic and weak typing 304
Primitive values 304
Objects 308
Type coercion 310
See also 311

Equality comparisons and sameness 312
Strict equality using === 312
Loose equality using == 313
Same-value equality using Object.is() 314
Same-value-zero equality 315
Comparing equality methods 315
See also 318

Enumerability and ownership of properties 319
Querying object properties 319
Traversing object properties 319
Obtaining properties by enumerability/ownership 320
See also 321

Closures 322
Lexical scoping 322
Closure 323
Practical closures 324
Emulating private methods with closures 325
Closure scope chain 327
Creating closures in loops: A common mistake 329
Performance considerations 332

Inheritance and the prototype chain 334
Inheritance with the prototype chain 334
Constructors 336
Inspecting prototypes: a deeper dive 340
Different ways of creating and mutating prototype chains 343
Performance 346
Conclusion 347

Meta programming 348
Proxies 348
Handlers and traps 348
Revocable Proxy 350
Reflection 350

Memory management 352
Memory life cycle 352
Garbage collection 353
Configuring an engine's memory model 355
Data structures aiding memory management 355

Agent communication and coordination 359
Communication between two agents 359
Sharing memory with SharedArrayBuffer 364
Accessing shared memory with Atomics 367
Waiting and notifying 373
Useful shared memory patterns 375
Number-guessing game using shared memory 381
See also 383

JavaScript

Source: developer.mozilla.org/en-US/docs/Web/JavaScript · MDN page last modified 2026-10-28

JavaScript (**JS**) is a lightweight interpreted (or just-in-time compiled) programming language with first-class functions. While it is most well-known as the scripting language for Web pages, many non-browser environments also use it, such as Node.js, Apache CouchDB and Adobe Acrobat. JavaScript is a prototype-based, garbage-collected, dynamic language, supporting multiple paradigms such as imperative, functional, and object-oriented.

JavaScript's dynamic capabilities include runtime object construction, variable parameter lists, function variables, dynamic script creation (via `eval`), object introspection (via `for...in` and `Object` utilities), and source-code recovery (JavaScript functions store their source text and can be retrieved through `toString()`).

This section is dedicated to the JavaScript language itself, and not the parts that are specific to Web pages or other host environments. For information about APIs that are specific to Web pages, please see Web APIs and DOM.

The standards for JavaScript are the ECMAScript Language Specification (ECMA-262) and the ECMAScript Internationalization API specification (ECMA-402). As soon as one browser implements a feature, we try to document it. This means that cases where some proposals for new ECMAScript features have already been implemented in browsers, documentation and examples in MDN articles may use some of those new features. Most of the time, this happens between the stages 3 and 4, and is usually before the spec is officially published.

Do not confuse JavaScript with the Java programming language — **JavaScript is *not* "Interpreted Java"**. Both "Java" and "JavaScript" are trademarks or registered trademarks of Oracle in the U.S. and other countries. However, the two programming languages have very different syntax, semantics, and use.

JavaScript documentation of core language features (pure ECMAScript, for the most part) includes the following:

- The JavaScript guide
- The JavaScript reference

For more information about JavaScript specifications and related technologies, see JavaScript technologies overview.

Beginner's tutorials

Our learn web development core modules contain modern, up-to-date tutorials covering JavaScript fundamentals.

Your first website: Adding interactivity

This article provides a brief tour of what JavaScript is and how to use it, aimed at people who are completely new to web development.

Dynamic scripting with JavaScript

This module focuses on the essentials of the core JavaScript language, plus some key surrounding topics — learning these topics will give you a solid basis to work from.

JavaScript frameworks and libraries

JavaScript frameworks are an essential part of modern front-end web development, providing developers with tried and tested tools for building scalable, interactive web applications. Many modern companies use frameworks as a standard part of their tooling, so many front-end development jobs now require framework experience. This set of articles provides a comfortable starting point to help you begin learning frameworks.

JavaScript guides

Fundamental language guides

JavaScript Guide

A much more detailed guide to the JavaScript language, aimed at those with previous programming experience either in JavaScript or another language.

Intermediate

Advanced JavaScript objects

The object-oriented nature of JavaScript is important to understand if you want to go further with your knowledge of the language and write more efficient code, therefore we've provided this module to help you.

Asynchronous JavaScript

In this module, we take a look at asynchronous JavaScript, why it is important, and how it can be used to effectively handle potential blocking operations, such as fetching resources from a server.

Client-side web APIs

Explores what APIs are, and how to use some of the most common APIs you'll come across often in your development work.

JavaScript language overview

An overview of the basic syntax and semantics of JavaScript for those coming from other programming languages to get up to speed.

JavaScript data structures

Overview of available data structures in JavaScript.

Equality comparisons and sameness

JavaScript provides three different value comparison operations: strict equality using ===, loose equality using ==, and the `Object.is()` method.

Enumerability and ownership of properties

How different methods that visit a group of object properties one-by-one handle the enumerability and ownership of properties.

Closures

A closure is the combination of a function and the lexical environment within which that function was declared.

Advanced

Inheritance and the prototype chain

Explanation of the widely misunderstood and underestimated prototype-based inheritance.

Memory Management

Memory life cycle and garbage collection in JavaScript.

Agent communication and coordination

Sharing memory and coordinating access with `SharedArrayBuffer`, `Atomics`, and `postMessage()`.

Reference

Browse the complete JavaScript reference documentation.

Standard objects

Get to know standard built-in objects: `Array`, `Boolean`, `Error`, `Function`, `JSON`, `Math`, `Number`, `Object`, `RegExp`, `String`, `Map`, `Set`, `WeakMap`, `WeakSet`, and others.

Expressions and operators

Learn more about the behavior of JavaScript's operators `instanceof`, `typeof`, `new`, `this`, the operator precedence, and more.

Statements and declarations

Learn how `do-while`, `for-in`, `for-of`, `try-catch`, `let`, `var`, `const`, `if-else`, `switch`, and more JavaScript statements and keywords work.

Functions

Learn how to work with JavaScript's functions to develop your applications.

Classes

JavaScript classes are the most appropriate way to do object-oriented programming.

JavaScript

JavaScript Guide

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide · MDN page last modified 2026-11-05

The JavaScript Guide shows you how to use JavaScript and gives an overview of the language. If you need exhaustive information about a language feature, have a look at the JavaScript reference.

This Guide is divided into the following chapters.

Introduction

Overview: Introduction

- About this guide
- About JavaScript
- JavaScript and Java
- ECMAScript
- Tools
- What's next

Grammar and types

Overview: Grammar and types

- Basic syntax & comments
- Declarations
- Variable scope
- Variable hoisting
- Data structures and types
- Literals

Control flow and error handling

Overview: Control flow and error handling

- `if...else`
- `switch`
- `try/catch/throw`
- Error objects

Loops and iteration

Overview: Loops and iteration

- `for`
- `while`
- `do...while`
- `continue`
- `break`
- `for...in`

- `for...of`

Functions

Overview: Functions

- Defining functions
- Calling functions
- Function scopes and closures
- Arguments & parameters
- Arrow functions

Expressions and operators

Overview: Expressions and operators

- Assignment & Comparisons
- Arithmetic operators
- Bitwise & logical operators
- Conditional (ternary) operator

Numbers and strings

Overview: Numbers and strings

- Numbers
- `Number` object
- `Math` object
- Strings
- `String` object
- Template literals

Representing dates & times

Overview: Representing dates & times

- `Date` object

Regular expressions

Overview: Regular expressions

- Creating a regular expression
- Writing a regular expression pattern
  - Assertions
  - Character classes
  - Groups and backreferences
  - Quantifiers

Indexed collections

Overview: Indexed collections

Keyed collections

Overview: Keyed collections

- `Map`
- `WeakMap`
- `Set`
- `WeakSet`

Working with objects

Overview: Working with objects

- Objects and properties
- Creating objects
- Defining methods
- Getter and setter

Using classes

Overview: Using classes

- Declaring a class
- Various class features
- Extends and inheritance
- Why classes?

Promises

Overview: Promises

- Guarantees
- Chaining
- Error handling
- Composition
- Timing

Typed arrays

Overview: Typed arrays

Iterators and generators

Overview: Iterators and generators

- Iterators
- Iterables

- Generators

Resource management

Overview: JavaScript resource management

- The `using` and `await using` declarations
- The `DisposableStack` and `AsyncDisposableStack` objects
- Error handling

Internationalization

Overview: Internationalization

- Date and time formatting
- Number formatting
- Collation

JavaScript modules

Overview: JavaScript modules

- Exporting
- Importing
- Default exports and imports
- Avoiding naming conflicts
- Importing JSON modules
- Dynamic module loading
- Top-level await
- Module metadata
- Using modules on the web
- Understanding the module graph
- Modules across platforms

Advanced topics

After you have learned all fundamental features of JavaScript, you can explore some more niche features, or dive deeper into the language's mechanisms and concepts.

- Language overview
- Data structures
- Enumerability and ownership of properties
- Inheritance and the prototype chain
- Equality comparisons and sameness
- Closures
- Meta programming
- Memory management
- Agent communication and coordination

JavaScript › Guide

Introduction

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Introduction · MDN page last modified 2026-07-08

This chapter introduces JavaScript and discusses some of its fundamental concepts.

What you should already know

This guide assumes you have the following basic background:

- A general understanding of the Internet and the World Wide Web (WWW).
- Good working knowledge of HyperText Markup Language (HTML).
- Some programming experience. If you are new to programming, try one of the tutorials linked on the main page about JavaScript.

Where to find JavaScript information

The JavaScript documentation on MDN includes the following:

- Dynamic scripting with JavaScript provides structured JavaScript guides for beginners and introduces basic concepts of programming and the Internet.
- JavaScript Guide (this guide) provides an overview about the JavaScript language and its objects.
- JavaScript Reference provides detailed reference material for JavaScript.

What is JavaScript?

JavaScript is a cross-platform, object-oriented scripting language used to make webpages interactive (e.g., having complex animations, clickable buttons, popup menus, etc.). There are also more advanced server side versions of JavaScript such as Node.js, which allow you to add more functionality to a website than downloading files (such as realtime collaboration between multiple computers). Inside a host environment (for example, a web browser), JavaScript can be connected to the objects of its environment to provide programmatic control over them.

JavaScript contains a standard library of objects, such as `Array`, `Map`, and `Math`, and a core set of language elements such as operators, control structures, and statements. Core JavaScript can be extended for a variety of purposes by supplementing it with additional objects; for example:

- *Client-side JavaScript* extends the core language by supplying objects to control a browser and its *Document Object Model* (DOM). For example, client-side extensions allow an application to place elements on an HTML form and respond to user events such as mouse clicks, form input, and page navigation.
- *Server-side JavaScript* extends the core language by supplying objects relevant to running JavaScript on a server. For example, server-side extensions allow an application to communicate with a database, provide continuity of information from one invocation to another of the application, or perform file manipulations on a server.

This means that in the browser, JavaScript can change the way the webpage (DOM) looks. And, likewise, Node.js JavaScript on the server can respond to custom requests sent by code executed in the browser.

JavaScript and Java

JavaScript and Java are similar in some ways but fundamentally different in some others. The JavaScript language resembles Java but does not have Java's static typing and strong type checking. JavaScript follows most Java expression syntax, naming conventions and basic control-flow constructs which was the reason why it was renamed from LiveScript to JavaScript.

In contrast to Java's compile-time system of classes built by declarations, JavaScript supports a runtime system based on a small number of data types representing numeric, Boolean, and string values. JavaScript has a prototype-based object model instead of the more common class-based object model. The prototype-based model provides dynamic inheritance; that is, what is inherited can vary for individual objects. JavaScript also supports functions without any special declarative requirements. Functions can be properties of objects, executing as loosely typed methods.

JavaScript is a very free-form language compared to Java. You do not have to declare all variables, classes, and methods. You do not have to be concerned with whether methods are public, private, or protected, and you do not have to implement interfaces. Variables, parameters, and function return types are not explicitly typed.

Java is a class-based programming language designed for fast execution and type safety. Type safety means, for instance, that you can't cast a Java integer into an object reference or access private memory by corrupting the Java bytecode. Java's class-based model means that programs consist exclusively of classes and their methods. Java's class inheritance and strong typing generally require tightly coupled object hierarchies. These requirements make Java programming more complex than JavaScript programming.

In contrast, JavaScript descends in spirit from a line of smaller, dynamically typed languages such as HyperTalk and dBase. These scripting languages offer programming tools to a much wider audience because of their easier syntax, specialized built-in functionality, and minimal requirements for object creation.

JavaScript	Java
Object-oriented. No distinction between types of objects. Inheritance is through the prototype mechanism, and properties and methods can be added to any object dynamically.	Class-based. Objects are divided into classes and instances with all inheritance through the class hierarchy. Classes and instances cannot have properties or methods added dynamically.
Variable data types are not declared (dynamic typing, loosely typed).	Variable data types must be declared (static typing, strongly typed).
Cannot automatically write to hard disk.	Can automatically write to hard disk.

JavaScript and the ECMAScript specification

JavaScript is standardized at Ecma International — the European association for standardizing information and communication systems (ECMA was formerly an acronym for the European Computer Manufacturers Association) to deliver a standardized, international programming language based on JavaScript. This standardized version of JavaScript, called ECMAScript, behaves the same way in all applications that support the standard. Companies can use the open standard language to develop their implementation of JavaScript. The ECMAScript standard is documented in the ECMA-262 specification.

The ECMA-262 standard is also approved by the ISO (International Organization for Standardization) as ISO-16262. You can also find the specification on the Ecma International website. The ECMAScript

specification does not describe the Document Object Model (DOM), which is standardized by the World Wide Web Consortium (W3C) and/or WHATWG (Web Hypertext Application Technology Working Group). The DOM defines the way in which HTML document objects are exposed to your script. To get a better idea about the different technologies that are used when programming with JavaScript, consult the article JavaScript technologies overview.

JavaScript documentation versus the ECMAScript specification

The ECMAScript specification is a set of requirements for implementing ECMAScript. It is useful if you want to implement standards-compliant language features in your ECMAScript implementation or engine (such as SpiderMonkey in Firefox, or V8 in Chrome).

The ECMAScript document is *not* intended to help script programmers. Use the JavaScript documentation for information when writing scripts.

The ECMAScript specification uses terminology and syntax that may be unfamiliar to a JavaScript programmer. Although the description of the language may differ in ECMAScript, the language itself remains the same. JavaScript supports all functionality outlined in the ECMAScript specification.

The JavaScript documentation describes aspects of the language that are appropriate for a JavaScript programmer.

Getting started with JavaScript

To get started with JavaScript, all you need is a modern web browser. Recent versions of Firefox, Chrome, Microsoft Edge, and Safari all support the features discussed in this guide.

A very useful tool for exploring JavaScript is the JavaScript Console (sometimes called the Web Console, or just the console): this is a tool which enables you to enter JavaScript and run it in the current page.

The screenshots here show the Firefox Web Console, but all modern browsers ship with a console that works in a similar way.

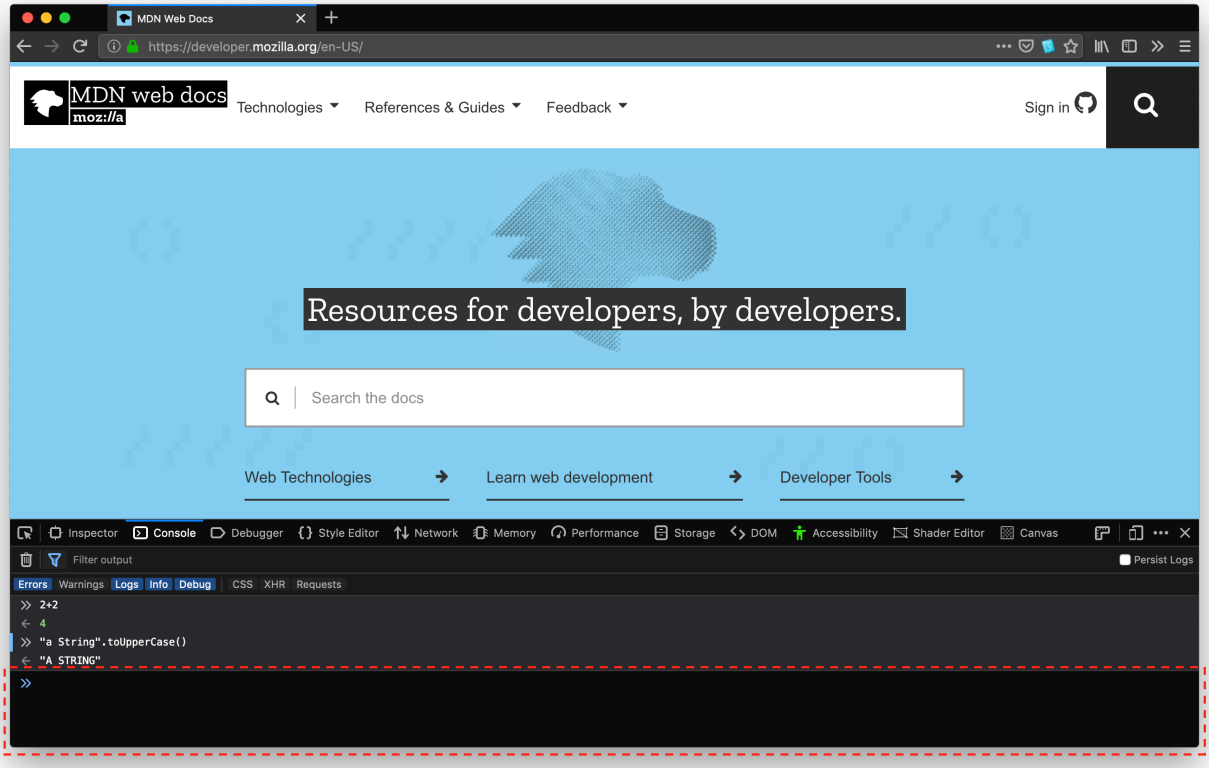
Opening the console

The exact instructions for opening the console vary from one browser to another:

- Opening the console in Firefox
- Opening the console in Chrome
- Opening the console in Microsoft Edge

Entering and running JavaScript

The console appears at the bottom of the browser window. Along the bottom of the console is an input line that you can use to enter JavaScript, and the output appears in the panel above:



The console works the exact same way as `eval`: the last expression entered is returned. For the sake of simplicity, it can be imagined that every time something is entered into the console, it is actually surrounded by `console.log` around `eval`, like so:

```js
console.log(eval("3 + 5"));
```

Multi-line input in the console

By default, if you press `Enter` (or `Return`, depending on your keyboard) after entering a line of code, then the string you typed is executed. To enter multi-line input:

- If the string you typed was incomplete (for example, you typed `function foo() {`) then the console will treat `Enter` as a line break, and let you type another line.
- If you hold down `Shift` while pressing `Enter`, then the console will treat this as a line break, and let you type another line.
- In Firefox only, you can activate multi-line input mode, in which you can enter multiple lines in a mini-editor, then run the whole thing when you are ready.

To get started with writing JavaScript, open the console, copy the following code, and paste it in at the prompt:

```js
(function () {
  "use strict";
  /* Start of your code */
  function greetMe(yourName) {
    alert(`Hello ${yourName}`);
  }

  greetMe("World");
  /* End of your code */
})();
```

Press Enter to watch it unfold in your browser!

What's next

In the following pages, this guide introduces you to the JavaScript syntax and language features, so that you will be able to write more complex applications.

But for now, remember to always include the `(function(){"use strict";` before your code, and add `})();` to the end of your code. The strict mode and IIFE articles explain what those do, but for now they can be thought of as doing the following:

1. Prevent semantics in JavaScript that trip up beginners.
2. Prevent code snippets executed in the console from interacting with one another (e.g., having something created in one console execution being used for a different console execution).

JavaScript › Guide

Grammar and types

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Grammar_and_types · MDN page last modified 2026-09-07

This chapter discusses JavaScript's basic grammar, variable declarations, data types and literals.

Basics

JavaScript borrows most of its syntax from Java, C, and C++, but it has also been influenced by Awk, Perl, and Python.

JavaScript is **case-sensitive** and uses the **Unicode** character set. For example, the word Früh (which means "early" in German) could be used as a variable name.

```js
const Früh = "foobar";
```

But, the variable `früh` is not the same as `Früh` because JavaScript is case sensitive.

In JavaScript, instructions are called statements and are separated by semicolons (;).

A semicolon is not necessary after a statement if it is written on its own line. But if more than one statement on a line is desired, then they *must* be separated by semicolons.

> **Note:** ECMAScript also has rules for automatic insertion of semicolons (ASI) to end statements. (For more information, see the detailed reference about JavaScript's lexical grammar.)

It is considered best practice, however, to always write a semicolon after a statement, even when it is not strictly needed. This practice reduces the chances of bugs getting into the code.

The source text of JavaScript script gets scanned from left to right, and is converted into a sequence of input elements which are *tokens*, *control characters*, *line terminators*, *comments*, or whitespace. (Spaces, tabs, and newline characters are considered whitespace.)

Comments

The syntax of **comments** is the same as in C++ and in many other languages:

```js
// a one line comment

/* this is a longer,
 * multi-line comment
 */
```

You can't nest block comments. This often happens when you accidentally include a `*/` sequence in your comment, which will terminate the comment.

js · ✗ bad example

```
/* You can't, however, /* nest comments */ SyntaxError */
```

In this case, you need to break up the `*/` pattern. For example, by inserting a backslash:

```js
/* You can /* nest comments *\/ by escaping slashes */
```

Comments behave like whitespace, and are discarded during script execution.

> **Note:** You might also see a third type of comment syntax at the start of some JavaScript files, which looks something like this: `#!/usr/bin/env node`.
>
> This is called **hashbang comment** syntax, and is a special comment used to specify the path to a particular JavaScript engine that should execute the script. See Hashbang comments for more details.

Declarations

JavaScript has five kinds of variable declarations.

`var`
: Declares a variable, optionally initializing it to a value.

`let`
: Declares a block-scoped variable, optionally initializing it to a value.

`const`
: Declares a block-scoped variable that cannot be re-assigned, which must be initialized at declaration.

`using`
: Declares a variable like `const` that is *synchronously disposed*.

`await using`
: Declares a variable like `const` that is *asynchronously disposed*.

We will only talk about the first three: `var`, `let`, and `const`, in this article. The `using` and `await using` declarations will be introduced in Resource management.

Variables

You use variables as symbolic names for values in your application. The names of variables, called identifiers, conform to certain rules.

A JavaScript identifier usually starts with a letter, underscore (_), or dollar sign ($). Subsequent characters can also be digits (`0` – `9`). Because JavaScript is case sensitive, letters include the characters `A` through `Z` (uppercase) as well as `a` through `z` (lowercase).

You can use most Unicode letters such as `å` and `ü` in identifiers. (For more details, see the lexical grammar reference.) You can also use Unicode escape sequences to represent characters in identifiers.

Some examples of legal names are `Number_hits`, `temp99`, `$credit`, and `_name`.

Declaring variables

You can declare a variable in two ways:

- With the keyword `var`. For example, `var x = 42`. This syntax can be used to declare both **local** and **global** variables, depending on the *execution context*.
- With the keyword `const` or `let`. For example, `let y = 13`. This syntax can be used to declare a block-scope local variable. (See Variable scope below.)

You can declare variables to unpack values using the destructuring syntax. For example, `const { bar } = foo`. This will create a variable named `bar` and assign to it the value corresponding to the key of the same name from our object `foo`.

Variables should always be declared before they are used. JavaScript used to allow assigning to undeclared variables, which creates an **undeclared global** variable. This is an error in strict mode and should be avoided altogether.

Declaration and initialization

In a statement like `let x = 42`, the `let x` part is called a *declaration*, and the `= 42` part is called an *initializer*. The declaration allows the variable to be accessed later in code without throwing a `ReferenceError`, while the initializer assigns a value to the variable. In `var` and `let` declarations, the initializer is optional. If a variable is declared without an initializer, it is assigned the value `undefined`.

js

```
let x;
console.log(x); // logs "undefined"
```

In essence, `let x = 42` is equivalent to `let x; x = 42`.

`const` declarations always need an initializer, because they forbid any kind of assignment after declaration, and implicitly initializing it with `undefined` is likely a programmer mistake.

js · ✗ bad example

```
const x; // SyntaxError: Missing initializer in const declaration
```

Variable scope

A variable may belong to one of the following scopes:

- Global scope: The default scope for all code running in script mode.
- Module scope: The scope for code running in module mode.
- Function scope: The scope created with a function.

In addition, variables declared with `let` or `const` can belong to an additional scope:

- Block scope: The scope created with a pair of curly braces (a block).

When you declare a variable outside of any function, it is called a *global* variable, because it is available to any other code in the current document. When you declare a variable within a function, it is called a *local* variable, because it is available only within that function.

`let` and `const` declarations can also be scoped to the block statement that they are declared in.

js

```
if (Math.random() > 0.5) {
  const y = 5;
}
console.log(y); // ReferenceError: y is not defined
```

However, variables created with `var` are not block-scoped, but only local to the *function (or global scope)* that the block resides within.

For example, the following code will log `5`, because the scope of `x` is the global context (or the function context if the code is part of a function). The scope of `x` is not limited to the immediate `if` statement block.

```js
if (true) {
  var x = 5;
}
console.log(x); // x is 5
```

Variable hoisting

`var`-declared variables are hoisted, meaning you can refer to the variable anywhere in its scope, even if its declaration isn't reached yet. You can see `var` declarations as being "lifted" to the top of its function or global scope. However, if you access a variable before it's declared, the value is always `undefined`, because only its *declaration* and *default initialization (with `undefined`)* is hoisted, but not its *value assignment*.

```js
console.log(x === undefined); // true
var x = 3;

(function () {
  console.log(x); // undefined
  var x = "local value";
})();
```

The above examples will be interpreted the same as:

```js
var x;
console.log(x === undefined); // true
x = 3;

(function () {
  var x;
  console.log(x); // undefined
  x = "local value";
})();
```

Because of hoisting, all `var` statements in a function should be placed as near to the top of the function as possible. This best practice increases the clarity of the code.

Whether `let` and `const` are hoisted is a matter of definition debate. Referencing the variable in the block before the variable declaration always results in a `ReferenceError`, because the variable is in a "temporal dead zone" from the start of the block until the declaration is processed.

```js
console.log(x); // ReferenceError
const x = 3;

console.log(y); // ReferenceError
let y = 3;
```

Unlike `var` declarations, which only hoist the declaration but not its value, function declarations are hoisted entirely — you can safely call the function anywhere in its scope. See the hoisting glossary entry for more discussion.

Global variables

Global variables are in fact properties of the *global object*.

In web pages, the global object is `window`, so you can read and set global variables using the `window.variable` syntax. In all environments, the `globalThis` variable (which itself is a global variable) may be used to read and set global variables. This is to provide a consistent interface among various JavaScript runtimes.

Consequently, you can access global variables declared in one window or frame from another window or frame by specifying the `window` or `frame` name. For example, if a variable called `phoneNumber` is declared in a document, you can refer to this variable from an `iframe` as `parent.phoneNumber`.

Constants

You can create a read-only, named constant with the `const` keyword. The syntax of a constant identifier is the same as any variable identifier: it must start with a letter, underscore, or dollar sign (`$`), and can contain alphabetic, numeric, or underscore characters.

js

```
const PI = 3.14;
```

A constant cannot change value through assignment or be re-declared while the script is running. It must be initialized to a value. The scope rules for constants are the same as those for `let` block-scope variables.

You cannot declare a constant with the same name as a function or variable in the same scope. For example:

js · ✗ bad example

```
// THIS WILL CAUSE AN ERROR
function f() {}
const f = 5;

// THIS WILL CAUSE AN ERROR TOO
function f() {
  const g = 5;
  var g;
}
```

However, `const` only prevents *re-assignments*, but doesn't prevent *mutations*. The properties of objects assigned to constants are not protected, so the following statement is executed without problems.

js

```
const MY_OBJECT = { key: "value" };
MY_OBJECT.key = "otherValue";
```

Also, the contents of an array are not protected, so the following statement is executed without problems.

js

```
const MY_ARRAY = ["HTML", "CSS"];
MY_ARRAY.push("JAVASCRIPT");
console.log(MY_ARRAY); // ['HTML', 'CSS', 'JAVASCRIPT'];
```

Data structures and types

Data types

The latest ECMAScript standard defines eight data types:

- Seven data types that are primitives:
  1. Boolean. `true` and `false`.
  2. null. A special keyword denoting a null value. (Because JavaScript is case-sensitive, `null` is not the same as `Null`, `NULL`, or any other variant.)
  3. undefined. A top-level property whose value is not defined.
  4. Number. An integer or floating point number. For example: `42` or `3.14159`.
  5. BigInt. An integer with arbitrary precision. For example: `9007199254740992n`.
  6. String. A sequence of characters that represent a text value. For example: `"Howdy"`.
  7. Symbol. A data type whose instances are unique and immutable.
- and Object

Although these data types are relatively few, they enable you to perform useful operations with your applications. Functions are the other fundamental elements of the language. While functions are technically a kind of object, you can think of objects as named containers for values, and functions as procedures that your script can perform.

Data type conversion

JavaScript is a *dynamically typed* language. This means you don't have to specify the data type of a variable when you declare it. It also means that data types are automatically converted as-needed during script execution.

So, for example, you could define a variable as follows:

```js
let answer = 42;
```

And later, you could assign the same variable a string value, for example:

```js
answer = "Thanks for all the fish!";
```

Because JavaScript is dynamically typed, this assignment does not cause an error message.

Numbers and the '+' operator

In expressions involving numeric and string values with the + operator, JavaScript converts numeric values to strings. For example, consider the following statements:

```js
x = "The answer is " + 42; // "The answer is 42"
y = 42 + " is the answer"; // "42 is the answer"
z = "37" + 7; // "377"
```

With all other operators, JavaScript does *not* convert numeric values to strings. For example:

```js
"37" - 7; // 30
"37" * 7; // 259
```

Converting strings to numbers

In the case that a value representing a number is in memory as a string, there are methods for conversion.

- `parseInt()`
- `parseFloat()`
- `Number()`

`parseInt` only returns whole numbers, so its use is diminished for decimals.

> **Note:** Additionally, a best practice for `parseInt` is to always include the *radix* parameter. The radix parameter is used to specify which numerical system is to be used.

```js
parseInt("101", 2); // 5
```

An alternative method of retrieving a number from a string is with the + (unary plus) operator. This implicitly performs number conversion, which is the same process as the `Number()` function.

```js
"1.1" + "1.1"; // '1.11.1'
(+"1.1") + (+"1.1"); // 2.2
// Note: the parentheses are added for clarity, not required.
```

Literals

Literals represent values in JavaScript. These are fixed values—not variables—that you *literally* provide in your script. This section describes the following types of literals:

- Array literals
- Boolean literals
- Numeric literals
- Object literals
- RegExp literals
- String literals

Array literals

An array literal is a list of zero or more expressions, each of which represents an array element, enclosed in square brackets (`[]`). When you create an array using an array literal, it is initialized with the specified values as its elements, and its `length` is set to the number of arguments specified.

The following example creates the `coffees` array with three elements and a `length` of three:

```js
const coffees = ["French Roast", "Colombian", "Kona"];
```

An array literal creates a new array object every time the literal is evaluated. For example, an array defined with a literal in the global scope is created once when the script loads. However, if the array

literal is inside a function, a new array is instantiated every time that function is called.

> **Note:** Array literals create `Array` objects. See `Array` and Indexed collections for details on `Array` objects.

Extra commas in array literals

If you put two commas in a row in an array literal, the array leaves an empty slot for the unspecified element. The following example creates the `fish` array:

```js
const fish = ["Lion", , "Angel"];
```

When you log this array, you will see:

```js
console.log(fish);
// [ 'Lion', <1 empty item>, 'Angel' ]
```

Note that the second item is "empty", which is not exactly the same as the actual `undefined` value. When using array-traversing methods like `Array.prototype.map`, empty slots are skipped. However, index-accessing `fish[1]` still returns `undefined`.

If you include a trailing comma at the end of the list of elements, the comma is ignored.

In the following example, the `length` of the array is three. There is no `myList[3]` and `myList[1]` is empty. All other commas in the list indicate a new element.

```js
const myList = ["home", , "school"];
```

In the following example, the `length` of the array is four, and `myList[0]` and `myList[2]` are missing.

```js
const myList = [, "home", , "school"];
```

In the following example, the `length` of the array is four, and `myList[1]` and `myList[3]` are missing. **Only the last comma is ignored.**

```js
const myList = ["home", , "school", ,];
```

> **Note:** Trailing commas help keep git diffs clean when you have a multi-line array, because appending an item to the end only adds one line, but does not modify the previous line.
>
> ```diff
> const myList = [
>   "home",
>   "school",
> + "hospital",
> ];
> ```

Understanding the behavior of extra commas is important to understanding JavaScript as a language.

However, when writing your own code, you should explicitly declare the missing elements as `undefined`, or at least insert a comment to highlight its absence. Doing this increases your code's clarity and maintainability.

```js
const myList = ["home", /* empty */, "school", /* empty */, ];
```

Boolean literals

The Boolean type has two literal values: `true` and `false`.

> **Note:** Do not confuse the primitive Boolean values `true` and `false` with the true and false values of the `Boolean` object.
>
> The Boolean object is a wrapper around the primitive Boolean data type. See `Boolean` for more information.

Numeric literals

JavaScript numeric literals include integer literals in different bases as well as floating-point literals in base-10.

Note that the language specification requires numeric literals to be unsigned. Nevertheless, code fragments like `-123.4` are fine, being interpreted as a unary `-` operator applied to the numeric literal `123.4`.

Integer literals

Integer and `BigInt` literals can be written in decimal (base 10), hexadecimal (base 16), octal (base 8) and binary (base 2).

- A *decimal* integer literal is a sequence of digits without a leading `0` (zero).
- A leading `0` (zero) on an integer literal, or a leading `0o` (or `0O`) indicates it is in *octal*. Octal integer literals can include only the digits `0` – `7`.
- A leading `0x` (or `0X`) indicates a *hexadecimal* integer literal. Hexadecimal integers can include digits (`0` – `9`) and the letters `a` – `f` and `A` – `F`. (The case of a character does not change its value. Therefore: `0xa` = `0xA` = `10` and `0xf` = `0xF` = `15`.)
- A leading `0b` (or `0B`) indicates a *binary* integer literal. Binary integer literals can only include the digits `0` and `1`.
- A trailing `n` suffix on an integer literal indicates a `BigInt` literal. The `BigInt` literal can use any of the above bases. Note that leading-zero octal syntax like `0123n` is not allowed, but `0o123n` is fine.

Some examples of integer literals are:

```
0, 117, 123456789123456789n             (decimal, base 10)
015, 0001, 0o777777777777n              (octal, base 8)
0x1123, 0x00111, 0x123456789ABCDEFn     (hexadecimal, "hex" or base 16)
0b11, 0b0011, 0b1110100101010101010101n (binary, base 2)
```

For more information, see Numeric literals in the Lexical grammar reference.

Floating-point literals

A floating-point literal can have the following parts:

- An unsigned decimal integer,
- A decimal point (`.`),
- A fraction (another decimal number),
- An exponent.

The exponent part is an `e` or `E` followed by an integer, which can be signed (preceded by `+` or `-`). A floating-point literal must have at least one digit, and either a decimal point or `e` (or `E`).

More succinctly, the syntax is:

```
[digits].[digits][(E|e)[(+|-)]digits]
```

For example:

```js
3.1415926
.123456789
3.1E+12
.1e-23
```

Object literals

An object literal is a list of zero or more pairs of property names and associated values of an object, enclosed in curly braces ({}).

> **Warning:** Do not use an object literal at the beginning of a statement! This will lead to an error (or not behave as you expect), because the `{` will be interpreted as the beginning of a block.

The following is an example of an object literal. The first element of the `car` object defines a property, `myCar`, and assigns to it a new string, `"Saturn"`; the second element, the `getCar` property, is immediately assigned the result of invoking the function `(carTypes("Honda"))`; the third element, the `special` property, uses an existing variable (`sales`).

```js
const sales = "Toyota";

function carTypes(name) {
  return name === "Honda" ? name : `Sorry, we don't sell ${name}.`;
}

const car = { myCar: "Saturn", getCar: carTypes("Honda"), special: sales };

console.log(car.myCar); // Saturn
console.log(car.getCar); // Honda
console.log(car.special); // Toyota
```

Additionally, you can use a numeric or string literal for the name of a property or nest an object inside another. The following example uses these options.

```js
const car = { manyCars: { a: "Saab", b: "Jeep" }, 7: "Mazda" };

console.log(car.manyCars.b); // Jeep
console.log(car[7]); // Mazda
```

Object property names can be any string, including the empty string. If the property name would not be a valid JavaScript identifier or number, it must be enclosed in quotes.

Property names that are not valid identifiers cannot be accessed as a dot (`.`) property.

js · ✗ bad example

```
const unusualPropertyNames = {
  "": "An empty string",
  "!": "Bang!",
};
console.log(unusualPropertyNames.""); // SyntaxError: Unexpected string
console.log(unusualPropertyNames.!); // SyntaxError: Unexpected token !
```

Instead, they must be accessed with the bracket notation (`[]`).

js · ✓ good example

```
console.log(unusualPropertyNames[""]); // An empty string
console.log(unusualPropertyNames["!"]); // Bang!
```

Enhanced Object literals

Object literals support a range of shorthand syntaxes that include setting the prototype at construction, shorthand for `foo: foo` assignments, defining methods, making `super` calls, and computing property names with expressions.

Together, these also bring object literals and class declarations closer together, and allow object-based design to benefit from some of the same conveniences.

js

```
const theProtoObj = {};
const handler = {};
const obj = {
  // __proto__
  __proto__: theProtoObj,
  // Shorthand for 'handler: handler'
  handler,
  // Methods
  toString() {
    // Super calls
    return `d ${super.toString()}`;
  },
  // Computed (dynamic) property names
  ["prop_" + (() => 42)()]: 42,
};
```

RegExp literals

A regex literal (which is defined in detail later) is a pattern enclosed between slashes. The following is an example of a regex literal.

js

```
const re = /ab+c/;
```

String literals

A string literal is zero or more characters enclosed in double (") or single (') quotation marks. A string must be delimited by quotation marks of the same type (that is, either both single quotation marks, or both double quotation marks).

The following are examples of string literals:

```js
'foo'
"bar"
'1234'
'one line \n another line'
"Joyo's cat"
```

You should use string literals unless you specifically need to use a `String` object. See `String` for details on `String` objects.

You can call any of the `String` object's methods on a string literal value. JavaScript automatically converts the string literal to a temporary String object, calls the method, then discards the temporary String object. You can also use the `length` property with a string literal:

```js
// Will print the number of symbols in the string including whitespace.
console.log("Joyo's cat".length); // In this case, 10.
```

Template literals are also available. Template literals are enclosed by the back-tick (`` ` ``) (grave accent) character instead of double or single quotes.

Template literals provide syntactic sugar for constructing strings. (This is similar to string interpolation features in Perl, Python, and more.)

```js
// Basic literal string creation
`In JavaScript '\n' is a line-feed.`;

// Multiline strings
`In JavaScript, template strings can run
 over multiple lines, but double and single
 quoted strings cannot.`;

// String interpolation
const name = "Lev",
  time = "today";
`Hello ${name}, how are you ${time}?`;
```

Tagged templates are a compact syntax for specifying a template literal along with a call to a "tag" function for parsing it. A tagged template is just a more succinct and semantic way to invoke a function that processes a string and a set of relevant values. The name of the template tag function precedes the template literal — as in the following example, where the template tag function is named `print`. The `print` function will interpolate the arguments and serialize any objects or arrays that may come up, avoiding the pesky `[object Object]`.

```js
const formatArg = (arg) => {
  if (Array.isArray(arg)) {
    // Print a bulleted list
    return arg.map((part) => `- ${part}`).join("\n");
  }
  if (arg.toString === Object.prototype.toString) {
    // This object will be serialized to "[object Object]".
    // Let's print something nicer.
    return JSON.stringify(arg);
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  return arg;
};

const print = (segments, ...args) => {
  // For any well-formed template literal, there will always be N args and
  // (N+1) string segments.
  let message = segments[0];
  segments.slice(1).forEach((segment, index) => {
    message += formatArg(args[index]) + segment;
  });
  console.log(message);
};

const todos = [
  "Learn JavaScript",
  "Learn Web APIs",
  "Set up my website",
  "Profit!",
];

const progress = { javascript: 20, html: 50, css: 10 };

print`I need to do:
${todos}
My current progress is: ${progress}
`;

// I need to do:
// - Learn JavaScript
// - Learn Web APIs
// - Set up my website
// - Profit!
// My current progress is: {"javascript":20,"html":50,"css":10}
```

Since tagged template literals are just sugar of function calls, you can re-write the above as an equivalent function call:

js

```
print(["I need to do:\n", "\nMy current progress is: ", "\n"], todos, progress);
```

This may be reminiscent of the `console.log`-style interpolation:

js

```
console.log("I need to do:\n%o\nMy current progress is: %o\n", todos, progress);
```

You can see how the tagged template reads more naturally than a traditional "formatter" function, where the variables and the template itself have to be declared separately.

Using special characters in strings

In addition to ordinary characters, you can also include special characters in strings, as shown in the following example.

js

```
"one line \n another line";
```

The following table lists the special characters that you can use in JavaScript strings.

Character	Meaning
`\0`	Null Byte
`\b`	Backspace
`\f`	Form Feed
`\n`	New Line
`\r`	Carriage Return
`\t`	Tab
`\v`	Vertical tab
`\'`	Apostrophe or single quote
`\"`	Double quote
`\\`	Backslash character
`\XXX`	The character with the Latin-1 encoding specified by up to three octal digits `XXX` between `0` and `377`. For example, `\251` is the octal sequence for the copyright symbol.
`\xXX`	The character with the Latin-1 encoding specified by the two hexadecimal digits `XX` between `00` and `FF`. For example, `\xA9` is the hexadecimal sequence for the copyright symbol.
`\uXXXX`	The Unicode character specified by the four hexadecimal digits `XXXX`. For example, `\u00A9` is the Unicode sequence for the copyright symbol. See Unicode escape sequences.
`\u{XXXXX}`	Unicode code point escapes. For example, `\u{2F804}` is the same as the Unicode escapes `\uD87E\uDC04`.

Escaping characters

For characters not listed in the table, a preceding backslash is ignored, but this usage is deprecated and should be avoided.

You can insert a quotation mark inside a string by preceding it with a backslash. This is known as *escaping* the quotation mark. For example:

```js
const quote = "He read \"The Cremation of Sam McGee\" by R.W. Service.";
console.log(quote);
```

The result of this would be:

```
He read "The Cremation of Sam McGee" by R.W. Service.
```

To include a literal backslash inside a string, you must escape the backslash character. For example, to assign the file path `c:\temp` to a string, use the following:

```js
const home = "c:\\temp";
```

You can also escape line breaks by preceding them with backslash. The backslash and line break are both removed from the value of the string.

```js
const str =
  "this string \
is broken \
across multiple \
lines.";
console.log(str); // this string is broken across multiple lines.
```

More information

This chapter focuses on basic syntax for declarations and types. To learn more about JavaScript's language constructs, see also the following chapters in this guide:

- Control flow and error handling guide
- Loops and iteration
- Functions
- Expressions and operators guide

In the next chapter, we will have a look at control flow constructs and error handling.

JavaScript › Guide

Control flow and error handling

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Control_flow_and_error_handling · MDN page last modified 2025-09-19

JavaScript supports a compact set of statements, specifically control flow statements, that you can use to incorporate a great deal of interactivity in your application. This chapter provides an overview of these statements.

The JavaScript reference contains exhaustive details about the statements in this chapter. The semicolon (`;`) character is used to separate statements in JavaScript code.

Any JavaScript expression is also a statement. See Expressions and operators for complete information about expressions.

Block statement

The most basic statement is a *block statement*, which is used to group statements. The block is delimited by a pair of curly braces:

```js
{
  statement1;
  statement2;
  // …
  statementN;
}
```

Example

Block statements are commonly used with control flow statements (`if`, `for`, `while`).

```js
while (x < 10) {
  x++;
}
```

Here, `{ x++; }` is the block statement.

> **Note:** `var`-declared variables are not block-scoped, but are scoped to the containing function or script, and the effects of setting them persist beyond the block itself. For example:
>
> ```js
> var x = 1;
> {
>   var x = 2;
> }
> console.log(x); // 2
> ```
>
> This outputs `2` because the `var x` statement within the block is in the same scope as the `var x` statement before the block. (In C or Java, the equivalent code would have output `1`.)
>
> This scoping effect can be mitigated by using `let` or `const`.

Conditional statements

A conditional statement is a set of commands that executes if a specified condition is true. JavaScript supports two conditional statements: `if...else` and `switch`.

if...else statement

Use the `if` statement to execute a statement if a logical condition is `true`. Use the optional `else` clause to execute a statement if the condition is `false`.

An `if` statement looks like this:

```js
if (condition) {
  statement1;
} else {
  statement2;
}
```

Here, the `condition` can be any expression that evaluates to `true` or `false`. (See Boolean for an explanation of what evaluates to `true` and `false`.)

If `condition` evaluates to `true`, `statement1` is executed. Otherwise, `statement2` is executed. `statement1` and `statement2` can be any statement, including further nested `if` statements.

You can also compound the statements using `else if` to have multiple conditions tested in sequence, as follows:

```js
if (condition1) {
  statement1;
} else if (condition2) {
  statement2;
} else if (conditionN) {
  statementN;
} else {
  statementLast;
}
```

In the case of multiple conditions, only the first logical condition which evaluates to `true` will be executed. To execute multiple statements, group them within a block statement (`{ /* … */ }`).

Best practice

In general, it's good practice to always use block statements—*especially* when nesting `if` statements:

```js
if (condition) {
  // Statements for when condition is true
  // …
} else {
  // Statements for when condition is false
  // …
}
```

In general it's good practice to not have an `if...else` with an assignment like `x = y` as a condition:

js · ✗ bad example

```
if (x = y) {
  // statements here
}
```

However, in the rare case you find yourself wanting to do something like that, the `while` documentation has a Using an assignment as a condition section with guidance on a general best-practice syntax you should know about and follow.

Falsy values

The following values evaluate to `false` (also known as Falsy values):

- `false`
- `undefined`
- `null`
- `0`
- `NaN`
- the empty string (`""`)

All other values—including all objects—evaluate to `true` when passed to a conditional statement.

> **Note:** Do not confuse the primitive boolean values `true` and `false` with the true and false values of the `Boolean` object!
>
> For example:
>
> js
>
> ```
> const b = new Boolean(false);
> if (b) {
>   // this condition evaluates to true
> }
> if (b == true) {
>   // this condition evaluates to false
> }
> ```

Example

In the following example, the function `checkData` returns `true` if the number of characters in a `Text` object is three. Otherwise, it displays an alert and returns `false`.

js

```
function checkData() {
  if (document.form1.threeChar.value.length === 3) {
    return true;
  }
  alert(
    `Enter exactly three characters. ${document.form1.threeChar.value} is not valid.`,
  );
  return false;
}
```

switch statement

A `switch` statement allows a program to evaluate an expression and attempt to match the expression's value to a `case` label. If a match is found, the program executes the associated statement.

A `switch` statement looks like this:

```js
switch (expression) {
  case label1:
    statements1;
    break;
  case label2:
    statements2;
    break;
  // …
  default:
    statementsDefault;
}
```

JavaScript evaluates the above switch statement as follows:

- The program first looks for a `case` clause with a label matching the value of expression and then transfers control to that clause, executing the associated statements.
- If no matching label is found, the program looks for the optional `default` clause:
  - If a `default` clause is found, the program transfers control to that clause, executing the associated statements.
  - If no `default` clause is found, the program resumes execution at the statement following the end of `switch`.
  - (By convention, the `default` clause is written as the last clause, but it does not need to be so.)

break statements

The optional `break` statement associated with each `case` clause ensures that the program breaks out of `switch` once the matched statement is executed, and then continues execution at the statement following `switch`. If `break` is omitted, the program continues execution inside the `switch` statement (and will execute statements under the next `case`, and so on).

Example

In the following example, if `fruitType` evaluates to `"Bananas"`, the program matches the value with `case "Bananas"` and executes the associated statement. When `break` is encountered, the program exits the `switch` and continues execution from the statement following `switch`. If `break` were omitted, the statement for `case "Cherries"` would also be executed.

```js
switch (fruitType) {
  case "Oranges":
    console.log("Oranges are $0.59 a pound.");
    break;
  case "Apples":
    console.log("Apples are $0.32 a pound.");
    break;
  case "Bananas":
    console.log("Bananas are $0.48 a pound.");
    break;
  case "Cherries":
    console.log("Cherries are $3.00 a pound.");
    break;
  case "Mangoes":
    console.log("Mangoes are $0.56 a pound.");
    break;
```

(code continues on the next page)

(code continued from the previous page)

```
  case "Papayas":
    console.log("Papayas are $2.79 a pound.");
    break;
  default:
    console.log(`Sorry, we are out of ${fruitType}.`);
}
console.log("Is there anything else you'd like?");
```

Exception handling statements

You can throw exceptions using the `throw` statement and handle them using the `try...catch` statements.

- `throw` statement
- `try...catch` statement

Exception types

Just about any object can be thrown in JavaScript. Nevertheless, not all thrown objects are created equal. While it is common to throw numbers or strings as errors, it is frequently more effective to use one of the exception types specifically created for this purpose:

- ECMAScript exceptions
- `DOMException`

throw statement

Use the `throw` statement to throw an exception. A `throw` statement specifies the value to be thrown:

js

```
throw expression;
```

You may throw any expression, not just expressions of a specific type. The following code throws several exceptions of varying types:

js

```
throw "Error2"; // String type
throw 42; // Number type
throw true; // Boolean type
throw {
  toString() {
    return "I'm an object!";
  },
};
```

try...catch statement

The `try...catch` statement marks a block of statements to try, and specifies one or more responses should an exception be thrown. If an exception is thrown, the `try...catch` statement catches it.

The `try...catch` statement consists of a `try` block, which contains one or more statements, and a `catch` block, containing statements that specify what to do if an exception is thrown in the `try` block.

In other words, you want the `try` block to succeed—but if it does not, you want control to pass to the `catch` block. If any statement within the `try` block (or in a function called from within the `try` block) throws an exception, control *immediately* shifts to the `catch` block. If no exception is thrown in the `try`

block, the `catch` block is skipped. The `finally` block executes after the `try` and `catch` blocks execute but before the statements following the `try...catch` statement.

The following example uses a `try...catch` statement. The example calls a function that retrieves a month name from an array based on the value passed to the function. If the value does not correspond to a month number (`1` – `12`), an exception is thrown with the value `'Invalid month code'` and the statements in the `catch` block set the `monthName` variable to `'unknown'`.

```js
function getMonthName(mo) {
  mo--; // Adjust month number for array index (so that 0 = Jan, 11 = Dec)
  // prettier-ignore
  const months = [
    "Jan", "Feb", "Mar", "Apr", "May", "Jun",
    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
  ];
  if (!months[mo]) {
    throw new Error("Invalid month code"); // throw keyword is used here
  }
  return months[mo];
}

try {
  // statements to try
  monthName = getMonthName(myMonth); // function could throw exception
} catch (e) {
  monthName = "unknown";
  logMyErrors(e); // pass exception object to error handler (i.e. your own function)
}
```

The catch block

You can use a `catch` block to handle all exceptions that may be generated in the `try` block.

```js
catch (exception) {
  statements
}
```

The `catch` block specifies an identifier (`exception` in the preceding syntax) that holds the value specified by the `throw` statement. You can use this identifier to get information about the exception that was thrown.

JavaScript creates this identifier when the `catch` block is entered. The identifier lasts only for the duration of the `catch` block. Once the `catch` block finishes executing, the identifier no longer exists.

For example, the following code throws an exception. When the exception occurs, control transfers to the `catch` block.

```js
try {
  throw "myException"; // generates an exception
} catch (err) {
  // statements to handle any exceptions
  logMyErrors(err); // pass exception object to error handler
}
```

> **Note:** When logging errors to the console inside a `catch` block, using `console.error()` rather than `console.log()` is advised for debugging. It formats the message as an error, and adds it to the list of error messages generated by the page.

The finally block

The `finally` block contains statements to be executed *after* the `try` and `catch` blocks execute. Additionally, the `finally` block executes *before* the code that follows the `try...catch...finally` statement.

It is also important to note that the `finally` block will execute *whether or not* an exception is thrown. If an exception is thrown, however, the statements in the `finally` block execute even if no `catch` block handles the exception that was thrown.

You can use the `finally` block to make your script fail gracefully when an exception occurs. For example, you may need to release a resource that your script has tied up.

The following example opens a file and then executes statements that use the file. (Server-side JavaScript allows you to access files.) If an exception is thrown while the file is open, the `finally` block closes the file before the script fails. Using `finally` here *ensures* that the file is never left open, even if an error occurs.

```js
openMyFile();
try {
  writeMyFile(theData); // This may throw an error
} catch (e) {
  handleError(e); // If an error occurred, handle it
} finally {
  closeMyFile(); // Always close the resource
}
```

If the `finally` block returns a value, this value becomes the return value of the entire `try...catch...finally` production, regardless of any `return` statements in the `try` and `catch` blocks:

```js
function f() {
  try {
    console.log(0);
    throw "bogus";
  } catch (e) {
    console.log(1);
    // This return statement is suspended
    // until finally block has completed
    return true;
    console.log(2); // not reachable
  } finally {
    console.log(3);
    return false; // overwrites the previous "return"
    // `f` exits here
    console.log(4); // not reachable
  }
  console.log(5); // not reachable
}
console.log(f()); // 0, 1, 3, false
```

Overwriting of return values by the `finally` block also applies to exceptions thrown or re-thrown inside of the `catch` block:

```js
function f() {
  try {
    throw "bogus";
  } catch (e) {
    console.log('caught inner "bogus"');
    // This throw statement is suspended until
    // finally block has completed
    throw e;
  } finally {
    return false; // overwrites the previous "throw"
    // `f` exits here
  }
}

try {
  console.log(f());
} catch (e) {
  // this is never reached!
  // while f() executes, the `finally` block returns false,
  // which overwrites the `throw` inside the above `catch`
  console.log('caught outer "bogus"');
}

// Logs:
// caught inner "bogus"
// false
```

Nesting try...catch statements

You can nest one or more `try...catch` statements.

If an inner `try` block does *not* have a corresponding `catch` block:

1. it *must* contain a `finally` block, and
2. the enclosing `try...catch` statement's `catch` block is checked for a match.

For more information, see nested try-blocks on the `try...catch` reference page.

Utilizing Error objects

Depending on the type of error, you may be able to use the `name` and `message` properties to get a more refined message.

The `name` property provides the general class of `Error` (such as `DOMException` or `Error`), while `message` generally provides a more succinct message than one would get by converting the error object to a string.

If you are throwing your own exceptions, in order to take advantage of these properties (such as if your `catch` block doesn't discriminate between your own exceptions and system ones), you can use the `Error` constructor.

For example:

```js
function doSomethingErrorProne() {
  if (ourCodeMakesAMistake()) {
    throw new Error("The message");
  }
  doSomethingToGetAJavaScriptError();
}

try {
  doSomethingErrorProne();
} catch (e) {
  // Now, we actually use `console.error()`
  console.error(e.name); // 'Error'
  console.error(e.message); // 'The message', or a JavaScript error message
}
```

JavaScript › Guide

Loops and iteration

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration · MDN page last modified 2026-05-21

Loops offer a quick and easy way to do something repeatedly. This chapter of the JavaScript Guide introduces the different iteration statements available to JavaScript.

You can think of a loop as a computerized version of the game where you tell someone to take *X* steps in one direction, then *Y* steps in another. For example, the idea "Go five steps to the east" could be expressed this way as a loop:

```js
for (let step = 0; step < 5; step++) {
  // Runs 5 times, with values of step 0 through 4.
  console.log("Walking east one step");
}
```

There are many different kinds of loops, but they all essentially do the same thing: they repeat an action some number of times. (Note that it's possible that number could be zero!)

The various loop mechanisms offer different ways to determine the start and end points of the loop. There are various situations that are more easily served by one type of loop over the others.

The statements for loops provided in JavaScript are:

- for statement
- do...while statement
- while statement
- labeled statement
- break statement
- continue statement
- for...in statement
- for...of statement

for statement

A `for` loop repeats until a specified condition evaluates to false. The JavaScript `for` loop is similar to the Java and C `for` loop.

A `for` statement looks as follows:

```js
for (initialization; condition; afterthought)
  statement
```

When a `for` loop executes, the following occurs:

1. The initializing expression `initialization`, if any, is executed. This expression usually initializes one or more loop counters, but the syntax allows an expression of any degree of complexity. This expression can also declare variables.
2. The `condition` expression is evaluated. If the value of `condition` is true, the loop statements execute. Otherwise, the `for` loop terminates. (If the `condition` expression is omitted entirely, the condition is assumed to be true.)

3. The `statement` executes. To execute multiple statements, use a block statement ({ }) to group those statements.
4. If present, the update expression `afterthought` is executed.
5. Control returns to Step 2.

Example

In the example below, the function contains a `for` statement that counts the number of selected options in a scrolling list (a `<select>` element that allows multiple selections).

HTML

```html
<form name="selectForm">
  <label for="musicTypes"
    >Choose some music types, then click the button below:</label
  >
  <select id="musicTypes" name="musicTypes" multiple>
    <option selected>R&amp;B</option>
    <option>Jazz</option>
    <option>Blues</option>
    <option>New Age</option>
    <option>Classical</option>
    <option>Opera</option>
  </select>
  <button id="btn" type="button">How many are selected?</button>
</form>
```

JavaScript

Here, the `for` statement declares the variable `i` and initializes it to `0`. It checks that `i` is less than the number of options in the `<select>` element, performs the succeeding `if` statement, and increments `i` by 1 after each pass through the loop.

```js
function countSelected(selectObject) {
  let numberSelected = 0;
  for (let i = 0; i < selectObject.options.length; i++) {
    if (selectObject.options[i].selected) {
      numberSelected++;
    }
  }
  return numberSelected;
}

const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
  const musicTypes = document.selectForm.musicTypes;
  console.log(`You have selected ${countSelected(musicTypes)} option(s).`);
});
```

do...while statement

The `do...while` statement repeats until a specified condition evaluates to false.

A `do...while` statement looks as follows:

```js
do
  statement
while (condition);
```

`statement` is always executed once before the condition is checked. (To execute multiple statements, use a block statement (`{ }`) to group those statements.)

If `condition` is `true`, the statement executes again. At the end of every execution, the condition is checked. When the condition is `false`, execution stops, and control passes to the statement following `do...while`.

Example

In the following example, the `do` loop iterates at least once and reiterates until `i` is no longer less than `5`.

```js
let i = 0;
do {
  i += 1;
  console.log(i);
} while (i < 5);
```

while statement

A `while` statement executes its statements as long as a specified condition evaluates to `true`. A `while` statement looks as follows:

```js
while (condition)
  statement
```

If the `condition` becomes `false`, `statement` within the loop stops executing and control passes to the statement following the loop.

The condition test occurs *before* `statement` in the loop is executed. If the condition returns `true`, `statement` is executed and the `condition` is tested again. If the condition returns `false`, execution stops, and control is passed to the statement following `while`.

To execute multiple statements, use a block statement (`{ }`) to group those statements.

Example 1

The following `while` loop iterates as long as `n` is less than `3`:

```js
let n = 0;
let x = 0;
while (n < 3) {
  n++;
  x += n;
}
```

With each iteration, the loop increments `n` and adds that value to `x`. Therefore, `x` and `n` take on the following values:

- After the first pass: `n` = `1` and `x` = `1`

- After the second pass: `n = 2` and `x = 3`
- After the third pass: `n = 3` and `x = 6`

After completing the third pass, the condition `n < 3` is no longer `true`, so the loop terminates.

Example 2

Avoid infinite loops. Make sure the condition in a loop eventually becomes `false`—otherwise, the loop will never terminate! The statements in the following `while` loop execute forever because the condition never becomes `false`:

js · ✗ bad example

```
// Infinite loops are bad!
while (true) {
  console.log("Hello, world!");
}
```

labeled statement

A `label` provides a statement with an identifier that lets you refer to it elsewhere in your program. For example, you can use a label to identify a loop, and then use the `break` or `continue` statements to indicate whether a program should interrupt the loop or continue its execution.

The syntax of the labeled statement looks like the following:

js

```
label:
  statement
```

The value of `label` may be any JavaScript identifier that is not a reserved word. The `statement` that you identify with a label may be any statement. For examples of using labeled statements, see the examples of `break` and `continue` below.

break statement

Use the `break` statement to terminate a loop, `switch`, or in conjunction with a labeled statement.

- When you use `break` without a label, it terminates the innermost enclosing `while`, `do-while`, `for`, or `switch` immediately and transfers control to the following statement.
- When you use `break` with a label, it terminates the specified labeled statement.

The syntax of the `break` statement looks like this:

js

```
break;
break label;
```

1. The first form of the syntax terminates the innermost enclosing loop or `switch`.
2. The second form of the syntax terminates the specified enclosing labeled statement.

Example 1

The following example iterates through the elements in an array until it finds the index of an element whose value is `theValue`:

```js
for (let i = 0; i < a.length; i++) {
  if (a[i] === theValue) {
    break;
  }
}
```

Example 2: Breaking to a label

```js
let x = 0;
let z = 0;
labelCancelLoops: while (true) {
  console.log("Outer loops:", x);
  x += 1;
  z = 1;
  while (true) {
    console.log("Inner loops:", z);
    z += 1;
    if (z === 10 && x === 10) {
      break labelCancelLoops;
    } else if (z === 10) {
      break;
    }
  }
}
```

continue statement

The `continue` statement can be used to restart a `while`, `do-while`, `for`, or `label` statement.

- When you use `continue` without a label, it terminates the current iteration of the innermost enclosing `while`, `do-while`, or `for` statement and continues execution of the loop with the next iteration. In contrast to the `break` statement, `continue` does not terminate the execution of the loop entirely. In a `while` loop, it jumps back to the condition. In a `for` loop, it jumps to the `increment-expression`.
- When you use `continue` with a label, it applies to the looping statement identified with that label.

The syntax of the `continue` statement looks like the following:

```js
continue;
continue label;
```

Example 1

The following example shows a `while` loop with a `continue` statement that executes when the value of `i` is `3`. Thus, `n` takes on the values `1`, `3`, `7`, and `12`.

```js
let i = 0;
let n = 0;
while (i < 5) {
  i++;
  if (i === 3) {
    continue;
  }
  n += i;
  console.log(n);
}
// Logs:
// 1 3 7 12
```

If you comment out the `continue;`, the loop would run till the end and you would see `1,3,6,10,15`.

Example 2

A statement labeled `checkIandJ` contains a statement labeled `checkJ`. If `continue` is encountered, the program terminates the current iteration of `checkJ` and begins the next iteration. Each time `continue` is encountered, `checkJ` reiterates until its condition returns `false`. When `false` is returned, the remainder of the `checkIandJ` statement is completed, and `checkIandJ` reiterates until its condition returns `false`. When `false` is returned, the program continues at the statement following `checkIandJ`.

If `continue` had a label of `checkIandJ`, the program would continue at the top of the `checkIandJ` statement.

```js
let i = 0;
let j = 10;
checkIandJ: while (i < 4) {
  console.log(i);
  i += 1;
  checkJ: while (j > 4) {
    console.log(j);
    j -= 1;
    if (j % 2 === 0) {
      continue;
    }
    console.log(j, "is odd.");
  }
  console.log("i =", i);
  console.log("j =", j);
}
```

for...in statement

The `for...in` statement iterates a specified variable over all the enumerable properties of an object. For each distinct property, JavaScript executes the specified statements. A `for...in` statement looks as follows:

```js
for (variable in object)
  statement
```

Example

The following function takes as its argument an object and the object's name. It then iterates over all the object's properties and returns a string that lists the property names and their values.

```js
function dumpProps(obj, objName) {
  let result = "";
  for (const i in obj) {
    result += `${objName}.${i} = ${obj[i]}<br>`;
  }
  result += "<hr>";
  return result;
}
```

For an object `car` with properties `make` and `model`, `result` would be:

```
car.make = Ford
car.model = Mustang
```

Arrays

Although it may be tempting to use this as a way to iterate over `Array` elements, the `for...in` statement will return the name of your user-defined properties in addition to the numeric indexes.

Therefore, it is better to use a traditional `for` loop with a numeric index when iterating over arrays, because the `for...in` statement iterates over user-defined properties in addition to the array elements, if you modify the Array object (such as adding custom properties or methods).

for...of statement

The `for...of` statement creates a loop Iterating over iterable objects (including `Array`, `Map`, `Set`, `arguments` object and so on), invoking a custom iteration hook with statements to be executed for the value of each distinct property.

```js
for (variable of iterable)
  statement
```

The following example shows the difference between a `for...of` loop and a `for...in` loop. While `for...in` iterates over property names, `for...of` iterates over property values:

```js
const arr = [3, 5, 7];
arr.foo = "hello";

for (const i in arr) {
  console.log(i);
}
// "0" "1" "2" "foo"

for (const i of arr) {
  console.log(i);
}
// Logs: 3 5 7
```

The `for...of` and `for...in` statements can also be used with destructuring. For example, you can simultaneously loop over the keys and values of an object using `Object.entries()`.

```js
const obj = { foo: 1, bar: 2 };

for (const [key, val] of Object.entries(obj)) {
  console.log(key, val);
}
// "foo" 1
// "bar" 2
```

JavaScript › Guide

Functions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions · MDN page last modified 2025-07-08

Functions are one of the fundamental building blocks in JavaScript. A function in JavaScript is similar to a procedure—a set of statements that performs a task or calculates a value, but for a procedure to qualify as a function, it should take some input and return an output where there is some obvious relationship between the input and the output. To use a function, you must define it somewhere in the scope from which you wish to call it.

See also the exhaustive reference chapter about JavaScript functions to get to know the details.

Defining functions

Function declarations

A **function definition** (also called a **function declaration**, or **function statement**) consists of the `function` keyword, followed by:

- The name of the function.
- A list of parameters to the function, enclosed in parentheses and separated by commas.
- The JavaScript statements that define the function, enclosed in curly braces, `{ /* … */ }`.

For example, the following code defines a function named `square`:

```js
function square(number) {
  return number * number;
}
```

The function `square` takes one parameter, called `number`. The function consists of one statement that says to return the parameter of the function (that is, `number`) multiplied by itself. The `return` statement specifies the value returned by the function, which is `number * number`.

Parameters are essentially passed to functions **by value** — so if the code within the body of a function assigns a completely new value to a parameter that was passed to the function, **the change is not reflected globally or in the code which called that function**.

When you pass an object as a parameter, if the function changes the object's properties, that change is visible outside the function, as shown in the following example:

```js
function myFunc(theObject) {
  theObject.make = "Toyota";
}

const myCar = {
  make: "Honda",
  model: "Accord",
  year: 1998,
};

console.log(myCar.make); // "Honda"
myFunc(myCar);
console.log(myCar.make); // "Toyota"
```

When you pass an array as a parameter, if the function changes any of the array's values, that change is visible outside the function, as shown in the following example:

```js
function myFunc(theArr) {
  theArr[0] = 30;
}

const arr = [45];

console.log(arr[0]); // 45
myFunc(arr);
console.log(arr[0]); // 30
```

Function declarations and expressions can be nested, which forms a *scope chain*. For example:

```js
function addSquares(a, b) {
  function square(x) {
    return x * x;
  }
  return square(a) + square(b);
}
```

See function scopes and closures for more information.

Function expressions

While the function declaration above is syntactically a statement, functions can also be created by a function expression.

Such a function can be **anonymous**; it does not have to have a name. For example, the function `square` could have been defined as:

```js
const square = function (number) {
  return number * number;
};

console.log(square(4)); // 16
```

However, a name *can* be provided with a function expression. Providing a name allows the function to refer to itself, and also makes it easier to identify the function in a debugger's stack traces:

```js
const factorial = function fac(n) {
  return n < 2 ? 1 : n * fac(n - 1);
};

console.log(factorial(3)); // 6
```

Function expressions are convenient when passing a function as an argument to another function. The following example defines a `map` function that should receive a function as first argument and an array as second argument. Then, it is called with a function defined by a function expression:

```js
function map(f, a) {
  const result = new Array(a.length);
  for (let i = 0; i < a.length; i++) {
    result[i] = f(a[i]);
  }
  return result;
}

const numbers = [0, 1, 2, 5, 10];
const cubedNumbers = map(function (x) {
  return x * x * x;
}, numbers);
console.log(cubedNumbers); // [0, 1, 8, 125, 1000]
```

In JavaScript, a function can be defined based on a condition. For example, the following function definition defines `myFunc` only if `num` equals `0`:

```js
let myFunc;
if (num === 0) {
  myFunc = function (theObject) {
    theObject.make = "Toyota";
  };
}
```

In addition to defining functions as described here, you can also use the `Function` constructor to create functions from a string at runtime, much like `eval()`.

A **method** is a function that is a property of an object. Read more about objects and methods in Working with objects.

Calling functions

Defining a function does not *execute* it. Defining it names the function and specifies what to do when the function is called.

Calling the function actually performs the specified actions with the indicated parameters. For example, if you define the function `square`, you could call it as follows:

```js
square(5);
```

The preceding statement calls the function with an argument of `5`. The function executes its statements and returns the value `25`.

Functions must be *in scope* when they are called, but the function declaration can be hoisted (appear below the call in the code). The scope of a function declaration is the function in which it is declared (or the entire program, if it is declared at the top level).

The arguments of a function are not limited to strings and numbers. You can pass whole objects to a function. The `showProps()` function (defined in Working with objects) is an example of a function that takes an object as an argument.

A function can call itself. For example, here is a function that computes factorials recursively:

```js
function factorial(n) {
  if (n === 0 || n === 1) {
    return 1;
  }
  return n * factorial(n - 1);
}
```

You could then compute the factorials of `1` through `5` as follows:

```js
console.log(factorial(1)); // 1
console.log(factorial(2)); // 2
console.log(factorial(3)); // 6
console.log(factorial(4)); // 24
console.log(factorial(5)); // 120
```

There are other ways to call functions. There are often cases where a function needs to be called dynamically, or the number of arguments to a function vary, or in which the context of the function call needs to be set to a specific object determined at runtime.

It turns out that *functions are themselves objects* — and in turn, these objects have methods. (See the `Function` object.) The `call()` and `apply()` methods can be used to achieve this goal.

Function hoisting

Consider the example below:

```js
console.log(square(5)); // 25

function square(n) {
  return n * n;
}
```

This code runs without any error, despite the `square()` function being called before it's declared. This is because the JavaScript interpreter hoists the entire function declaration to the top of the current scope, so the code above is equivalent to:

```js
// All function declarations are effectively at the top of the scope
function square(n) {
  return n * n;
}

console.log(square(5)); // 25
```

Function hoisting only works with function *declarations* — not with function *expressions*. The following code will not work:

js · ✗ bad example

```
console.log(square(5)); // ReferenceError: Cannot access 'square' before initialization
const square = function (n) {
  return n * n;
};
```

Recursion

A function can refer to and call itself. It can be referred to either by the function expression or declaration's name, or via any in-scope variable that refers to the function object. For example, consider the following function definition:

```js
const foo = function bar() {
  // statements go here
};
```

Within the function body, you can refer to the function itself either as `bar` or `foo`, and call itself using `bar()` or `foo()`.

A function that calls itself is called a *recursive function*. In some ways, recursion is analogous to a loop. Both execute the same code multiple times, and both require a condition (to avoid an infinite loop, or rather, infinite recursion in this case).

For example, consider the following loop:

```js
let x = 0;
// "x < 10" is the loop condition
while (x < 10) {
  // do stuff
  x++;
}
```

It can be converted into a recursive function declaration, followed by a call to that function:

```js
function loop(x) {
  // "x >= 10" is the exit condition (equivalent to "!(x < 10)")
  if (x >= 10) {
    return;
  }
  // do stuff
  loop(x + 1); // the recursive call
}
loop(0);
```

However, some algorithms cannot be simple iterative loops. For example, getting all the nodes of a tree structure (such as the DOM) is easier via recursion:

```js
function walkTree(node) {
  if (node === null) {
    return;
  }
  // do something with node
  for (const child of node.childNodes) {
    walkTree(child);
  }
}
```

Compared to the function `loop`, each recursive call itself makes many recursive calls here.

It is possible to convert any recursive algorithm to a non-recursive one, but the logic is often much more complex, and doing so requires the use of a stack.

In fact, recursion itself uses a stack: the function stack. The stack-like behavior can be seen in the following example:

```js
function foo(i) {
  if (i < 0) {
    return;
  }
  console.log(`begin: ${i}`);
  foo(i - 1);
  console.log(`end: ${i}`);
}
foo(3);

// Logs:
// begin: 3
// begin: 2
// begin: 1
// begin: 0
// end: 0
// end: 1
// end: 2
// end: 3
```

Immediately Invoked Function Expressions (IIFE)

An Immediately Invoked Function Expression (IIFE) is a code pattern that directly calls a function defined as an expression. It looks like this:

```js
(function () {
  // Do something
})();

const value = (function () {
  // Do something
  return someValue;
})();
```

Instead of saving the function in a variable, the function is immediately invoked. This is almost equivalent to just writing the function body, but there are a few unique benefits:

- It creates an extra scope of variables, which helps to confine variables to the place where they are useful.
- It is now an *expression* instead of a sequence of *statements*. This allows you to write complex computation logic when initializing variables.

For more information, see the IIFE glossary entry.

Function scopes and closures

Functions form a scope for variables—this means variables defined inside a function cannot be accessed from anywhere outside the function. The function scope inherits from all the upper scopes. For example, a function defined in the global scope can access all variables defined in the global scope.

A function defined inside another function can also access all variables defined in its parent function, and any other variables to which the parent function has access. On the other hand, the parent function (and any other parent scope) does *not* have access to the variables and functions defined inside the inner function. This provides a sort of encapsulation for the variables in the inner function.

js

```
// The following variables are defined in the global scope
const num1 = 20;
const num2 = 3;
const name = "Chamakh";

// This function is defined in the global scope
function multiply() {
  return num1 * num2;
}

console.log(multiply()); // 60

// A nested function example
function getScore() {
  const num1 = 2;
  const num2 = 3;

  function add() {
    return `${name} scored ${num1 + num2}`;
  }

  return add();
}

console.log(getScore()); // "Chamakh scored 5"
```

Closures

We also refer to the function body as a *closure*. A closure is any piece of source code (most commonly, a function) that refers to some variables, and the closure "remembers" these variables even when the scope in which these variables were declared has exited.

Closures are usually illustrated with nested functions to show that they remember variables beyond the lifetime of its parent scope; but in fact, nested functions are unnecessary. Technically speaking, all functions in JavaScript form closures—some just don't capture anything, and closures don't even have to be functions. The key ingredients for a *useful* closure are the following:

- A parent scope that defines some variables or functions. It should have a clear lifetime, which means it should finish execution at some point. Any scope that's not the global scope satisfies this requirement; this includes blocks, functions, modules, and more.
- An inner scope defined within the parent scope, which refers to some variables or functions defined in the parent scope.
- The inner scope manages to survive beyond the lifetime of the parent scope. For example, it is saved to a variable that's defined outside the parent scope, or it's returned from the parent scope (if the parent scope is a function).
- Then, when you call the function outside of the parent scope, you can still access the variables or functions that were defined in the parent scope, even though the parent scope has finished execution.

The following is a typical example of a closure:

```js
// The outer function defines a variable called "name"
const pet = function (name) {
  const getName = function () {
    // The inner function has access to the "name" variable of the outer function
    return name;
  };
  return getName; // Return the inner function, thereby exposing it to outer scopes
};
const myPet = pet("Vivie");

console.log(myPet()); // "Vivie"
```

It can be much more complex than the code above. An object containing methods for manipulating the inner variables of the outer function can be returned.

```js
const createPet = function (name) {
  let sex;

  const pet = {
    // setName(newName) is equivalent to setName: function (newName)
    // in this context
    setName(newName) {
      name = newName;
    },

    getName() {
      return name;
    },

    getSex() {
      return sex;
    },

    setSex(newSex) {
      if (
        typeof newSex === "string" &&
        (newSex.toLowerCase() === "male" || newSex.toLowerCase() === "female")
      ) {
        sex = newSex;
      }
    },
  };

  return pet;
};

const pet = createPet("Vivie");
console.log(pet.getName()); // Vivie

pet.setName("Oliver");
pet.setSex("male");
console.log(pet.getSex()); // male
console.log(pet.getName()); // Oliver
```

In the code above, the `name` variable of the outer function is accessible to the inner functions, and there is no other way to access the inner variables except through the inner functions. The inner variables of the inner functions act as safe stores for the outer arguments and variables. They hold "persistent" and "encapsulated" data for the inner functions to work with. The functions do not even have to be assigned to a variable, or have a name.

```js
const getCode = (function () {
  const apiCode = "0]Eal(eh&2"; // A code we do not want outsiders to be able to modify…

  return function () {
    return apiCode;
  };
})();

console.log(getCode()); // "0]Eal(eh&2"
```

In the code above, we use the IIFE pattern. Within this IIFE scope, two values exist: a variable `apiCode` and an unnamed function that gets returned and gets assigned to the variable `getCode`. `apiCode` is in the scope of the returned unnamed function but not in the scope of any other part of the program, so there is no way for reading the value of `apiCode` apart from via the `getCode` function.

Multiply-nested functions

Functions can be multiply-nested. For example:

- A function (`A`) contains a function (`B`), which itself contains a function (`C`).
- Both functions `B` and `C` form closures here. So, `B` can access `A`, and `C` can access `B`.
- In addition, since `C` can access `B` which can access `A`, `C` can also access `A`.

Thus, the closures can contain multiple scopes; they recursively contain the scope of the functions containing it. This is called *scope chaining*. Consider the following example:

```js
function A(x) {
  function B(y) {
    function C(z) {
      console.log(x + y + z);
    }
    C(3);
  }
  B(2);
}
A(1); // Logs 6 (which is 1 + 2 + 3)
```

In this example, `C` accesses `B`'s `y` and `A`'s `x`. This can be done because:

1. `B` forms a closure including `A` (i.e., `B` can access `A`'s arguments and variables).
2. `C` forms a closure including `B`.
3. Because `C`'s closure includes `B` and `B`'s closure includes `A`, then `C`'s closure also includes `A`. This means `C` can access *both* `B` *and* `A`'s arguments and variables. In other words, `C` *chains* the scopes of `B` and `A`, *in that order*.

The reverse, however, is not true. `A` cannot access `C`, because `A` cannot access any argument or variable of `B`, which `C` is a variable of. Thus, `C` remains private to only `B`.

Name conflicts

When two arguments or variables in the scopes of a closure have the same name, there is a *name conflict*. More nested scopes take precedence. So, the innermost scope takes the highest precedence, while the outermost scope takes the lowest. This is the scope chain. The first on the chain is the innermost scope, and the last is the outermost scope. Consider the following:

```js
function outside() {
  const x = 5;
  function inside(x) {
    return x * 2;
  }
  return inside;
}

console.log(outside()(10)); // 20 (instead of 10)
```

The name conflict happens at the statement `return x * 2` and is between `inside`'s parameter `x` and `outside`'s variable `x`. The scope chain here is `inside` => `outside` => global object. Therefore, `inside`'s `x` takes precedences over `outside`'s `x`, and `20` (`inside`'s `x`) is returned instead of `10` (`outside`'s `x`).

Using the arguments object

The arguments of a function are maintained in an array-like object. Within a function, you can address the arguments passed to it as follows:

```js
arguments[i];
```

where `i` is the ordinal number of the argument, starting at `0`. So, the first argument passed to a function would be `arguments[0]`. The total number of arguments is indicated by `arguments.length`.

Using the `arguments` object, you can call a function with more arguments than it is formally declared to accept. This is often useful if you don't know in advance how many arguments will be passed to the function. You can use `arguments.length` to determine the number of arguments actually passed to the function, and then access each argument using the `arguments` object.

For example, consider a function that concatenates several strings. The only formal argument for the function is a string that specifies the characters that separate the items to concatenate. The function is defined as follows:

```js
function myConcat(separator) {
  let result = ""; // initialize list
  // iterate through arguments
  for (let i = 1; i < arguments.length; i++) {
    result += arguments[i] + separator;
  }
  return result;
}
```

You can pass any number of arguments to this function, and it concatenates each argument into a string "list":

```js
console.log(myConcat(", ", "red", "orange", "blue"));
// "red, orange, blue, "

console.log(myConcat("; ", "elephant", "giraffe", "lion", "cheetah"));
// "elephant; giraffe; lion; cheetah; "

console.log(myConcat(". ", "sage", "basil", "oregano", "pepper", "parsley"));
// "sage. basil. oregano. pepper. parsley. "
```

> **Note:** The `arguments` variable is "array-like", but not an array. It is array-like in that it has a numbered index and a `length` property. However, it does *not* possess all of the array-manipulation methods.

See the `Function` object in the JavaScript reference for more information.

Function parameters

There are two special kinds of parameter syntax: *default parameters* and *rest parameters*.

Default parameters

In JavaScript, parameters of functions default to `undefined`. However, in some situations it might be useful to set a different default value. This is exactly what default parameters do.

In the past, the general strategy for setting defaults was to test parameter values in the body of the function and assign a value if they are `undefined`.

In the following example, if no value is provided for `b`, its value would be `undefined` when evaluating `a*b`, and a call to `multiply` would normally have returned `NaN`. However, this is prevented by the second line in this example:

```js
function multiply(a, b) {
  b = typeof b !== "undefined" ? b : 1;
  return a * b;
}

console.log(multiply(5)); // 5
```

With *default parameters*, a manual check in the function body is no longer necessary. You can put `1` as the default value for `b` in the function head:

```js
function multiply(a, b = 1) {
  return a * b;
}

console.log(multiply(5)); // 5
```

For more details, see default parameters in the reference.

Rest parameters

The rest parameter syntax allows us to represent an indefinite number of arguments as an array.

In the following example, the function `multiply` uses *rest parameters* to collect arguments from the second one to the end. The function then multiplies these by the first argument.

```js
function multiply(multiplier, ...theArgs) {
  return theArgs.map((x) => multiplier * x);
}

const arr = multiply(2, 1, 2, 3);
console.log(arr); // [2, 4, 6]
```

Arrow functions

An arrow function expression (also called a *fat arrow* to distinguish from a hypothetical `->` syntax in future JavaScript) has a shorter syntax compared to function expressions and does not have its own `this`, `arguments`, `super`, or `new.target`. Arrow functions are always anonymous.

Two factors influenced the introduction of arrow functions: *shorter functions* and *non-binding* of `this`.

Shorter functions

In some functional patterns, shorter functions are welcome. Compare:

```js
const a = ["Hydrogen", "Helium", "Lithium", "Beryllium"];

const a2 = a.map(function (s) {
  return s.length;
});

console.log(a2); // [8, 6, 7, 9]

const a3 = a.map((s) => s.length);

console.log(a3); // [8, 6, 7, 9]
```

No separate this

Until arrow functions, every new function defined its own `this` value (a new object in the case of a constructor, undefined in strict mode function calls, the base object if the function is called as an "object method", etc.). This proved to be less than ideal with an object-oriented style of programming.

```js
function Person() {
  // The Person() constructor defines `this` as itself.
  this.age = 0;

  setInterval(function growUp() {
    // In nonstrict mode, the growUp() function defines `this`
    // as the global object, which is different from the `this`
    // defined by the Person() constructor.
    this.age++;
  }, 1000);
}

const p = new Person();
```

In ECMAScript 3/5, this issue was fixed by assigning the value in `this` to a variable that could be closed over.

```js
function Person() {
  // Some choose `that` instead of `self`.
  // Choose one and be consistent.
  const self = this;
  self.age = 0;

  setInterval(function growUp() {
    // The callback refers to the `self` variable of which
    // the value is the expected object.
    self.age++;
  }, 1000);
}
```

Alternatively, a bound function could be created so that the proper `this` value would be passed to the `growUp()` function.

An arrow function does not have its own `this`; the `this` value of the enclosing execution context is used. Thus, in the following code, the `this` within the function that is passed to `setInterval` has the same value as `this` in the enclosing function:

```js
function Person() {
  this.age = 0;

  setInterval(() => {
    this.age++; // `this` properly refers to the person object
  }, 1000);
}

const p = new Person();
```

JavaScript › Guide

Expressions and operators

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Expressions_and_operators · MDN page last modified 2025-07-08

This chapter describes JavaScript's expressions and operators, including assignment, comparison, arithmetic, bitwise, logical, string, ternary and more.

At a high level, an *expression* is a valid unit of code that resolves to a value. There are two types of expressions: those that have side effects (such as assigning values) and those that purely *evaluate*.

The expression `x = 7` is an example of the first type. This expression uses the = *operator* to assign the value seven to the variable `x`. The expression itself evaluates to `7`.

The expression `3 + 4` is an example of the second type. This expression uses the + operator to add `3` and `4` together and produces a value, `7`. However, if it's not eventually part of a bigger construct (for example, a variable declaration like `const z = 3 + 4`), its result will be immediately discarded — this is usually a programmer mistake because the evaluation doesn't produce any effects.

As the examples above also illustrate, all complex expressions are joined by *operators*, such as = and +. In this section, we will introduce the following operators:

- Assignment operators
- Comparison operators
- Arithmetic operators
- Bitwise operators
- Logical operators
- BigInt operators
- String operators
- Conditional (ternary) operator
- Comma operator
- Unary operators
- Relational operators

These operators join operands either formed by higher-precedence operators or one of the basic expressions. A complete and detailed list of operators and expressions is also available in the reference.

The *precedence* of operators determines the order they are applied when evaluating an expression. For example:

```js
const x = 1 + 2 * 3;
const y = 2 * 3 + 1;
```

Despite * and + coming in different orders, both expressions would result in `7` because * has precedence over +, so the *-joined expression will always be evaluated first. You can override operator precedence by using parentheses (which creates a grouped expression — the basic expression). To see a complete table of operator precedence as well as various caveats, see the Operator Precedence Reference page.

JavaScript has both *binary* and *unary* operators, and one special ternary operator, the conditional operator. A binary operator requires two operands, one before the operator and one after the operator:

```
operand1 operator operand2
```

For example, `3 + 4` or `x * y`. This form is called an *infix* binary operator, because the operator is placed between two operands. All binary operators in JavaScript are infix.

A unary operator requires a single operand, either before or after the operator:

```
operator operand
operand operator
```

For example, `x++` or `++x`. The `operator operand` form is called a *prefix* unary operator, and the `operand operator` form is called a *postfix* unary operator. `++` and `--` are the only postfix operators in JavaScript — all other operators, like `!`, `typeof`, etc. are prefix.

Assignment operators

An assignment operator assigns a value to its left operand based on the value of its right operand. The simple assignment operator is equal (`=`), which assigns the value of its right operand to its left operand. That is, `x = f()` is an assignment expression that assigns the value of `f()` to `x`.

There are also compound assignment operators that are shorthand for the operations listed in the following table:

Name	Shorthand operator	Meaning
Assignment	`x = f()`	`x = f()`
Addition assignment	`x += f()`	`x = x + f()`
Subtraction assignment	`x -= f()`	`x = x - f()`
Multiplication assignment	`x *= f()`	`x = x * f()`
Division assignment	`x /= f()`	`x = x / f()`
Remainder assignment	`x %= f()`	`x = x % f()`
Exponentiation assignment	`x **= f()`	`x = x ** f()`
Left shift assignment	`x <<= f()`	`x = x << f()`
Right shift assignment	`x >>= f()`	`x = x >> f()`
Unsigned right shift assignment	`x >>>= f()`	`x = x >>> f()`
Bitwise AND assignment	`x &= f()`	`x = x & f()`
Bitwise XOR assignment	`x ^= f()`	`x = x ^ f()`
Bitwise OR assignment	`x |= f()`	`x = x | f()`
Logical AND assignment	`x &&= f()`	`x && (x = f())`
Logical OR assignment	`x ||= f()`	`x || (x = f())`
Nullish coalescing assignment	`x ??= f()`	`x ?? (x = f())`

Assigning to properties

If an expression evaluates to an object, then the left-hand side of an assignment expression may make assignments to properties of that expression. For example:

```js
const obj = {};

obj.x = 3;
console.log(obj.x); // Prints 3.
console.log(obj); // Prints { x: 3 }.

const key = "y";
obj[key] = 5;
console.log(obj[key]); // Prints 5.
console.log(obj); // Prints { x: 3, y: 5 }.
```

For more information about objects, read Working with Objects.

If an expression does not evaluate to an object, then assignments to properties of that expression do not assign:

```js
const val = 0;
val.x = 3;

console.log(val.x); // Prints undefined.
console.log(val); // Prints 0.
```

In strict mode, the code above throws, because one cannot assign properties to primitives.

It is an error to assign values to unmodifiable properties or to properties of an expression without properties (`null` or `undefined`).

Destructuring

For more complex assignments, the destructuring syntax is a JavaScript expression that makes it possible to extract data from arrays or objects using a syntax that mirrors the construction of array and object literals.

Without destructuring, it takes multiple statements to extract values from arrays and objects:

```js
const foo = ["one", "two", "three"];

const one = foo[0];
const two = foo[1];
const three = foo[2];
```

With destructuring, you can extract multiple values into distinct variables using a single statement:

```js
const [one, two, three] = foo;
```

Evaluation and nesting

In general, assignments are used within a variable declaration (i.e., with `const`, `let`, or `var`) or as standalone statements.

```js
// Declares a variable x and initializes it to the result of f().
// The result of the x = f() assignment expression is discarded.
let x = f();

x = g(); // Reassigns the variable x to the result of g().
```

However, like other expressions, assignment expressions like `x = f()` evaluate into a result value. Although this result value is usually not used, it can then be used by another expression.

Chaining assignments or nesting assignments in other expressions can result in surprising behavior. For this reason, some JavaScript style guides discourage chaining or nesting assignments. Nevertheless, assignment chaining and nesting may occur sometimes, so it is important to be able to understand how they work.

By chaining or nesting an assignment expression, its result can itself be assigned to another variable. It can be logged, it can be put inside an array literal or function call, and so on.

```js
let x;
const y = (x = f()); // Or equivalently: const y = x = f();
console.log(y); // Logs the return value of the assignment x = f().

console.log(x = f()); // Logs the return value directly.

// An assignment expression can be nested in any place
// where expressions are generally allowed,
// such as array literals' elements or as function calls' arguments.
console.log([0, x = f(), 0]);
console.log(f(0, x = f(), 0));
```

The evaluation result matches the expression to the right of the = sign in the "Meaning" column of the table above. That means that `x = f()` evaluates into whatever `f()`'s result is, `x += f()` evaluates into the resulting sum `x + f()`, `x **= f()` evaluates into the resulting power `x ** f()`, and so on.

In the case of logical assignments, `x &&= f()`, `x ||= f()`, and `x ??= f()`, the return value is that of the logical operation without the assignment, so `x && f()`, `x || f()`, and `x ?? f()`, respectively.

When chaining these expressions without parentheses or other grouping operators like array literals, the assignment expressions are **grouped right to left** (they are right-associative), but they are **evaluated left to right**.

Note that, for all assignment operators other than = itself, the resulting values are always based on the operands' values *before* the operation.

For example, assume that the following functions `f` and `g` and the variables `x` and `y` have been declared:

```js
function f() {
  console.log("F!");
  return 2;
}
function g() {
  console.log("G!");
  return 3;
}
let x, y;
```

Consider these three examples:

```js
y = x = f();
y = [f(), x = g()];
x[f()] = g();
```

Evaluation example 1

`y = x = f()` is equivalent to `y = (x = f())`, because the assignment operator `=` is right-associative. However, it evaluates from left to right:

1. The assignment expression `y = x = f()` starts to evaluate.
   1. The `y` on this assignment's left-hand side evaluates into a reference to the variable named `y`.
   2. The assignment expression `x = f()` starts to evaluate.
      1. The `x` on this assignment's left-hand side evaluates into a reference to the variable named `x`.
      2. The function call `f()` prints "F!" to the console and then evaluates to the number `2`.
      3. That `2` result from `f()` is assigned to `x`.
   3. The assignment expression `x = f()` has now finished evaluating; its result is the new value of `x`, which is `2`.
   4. That `2` result in turn is also assigned to `y`.
2. The assignment expression `y = x = f()` has now finished evaluating; its result is the new value of `y` – which happens to be `2`. `x` and `y` are assigned to `2`, and the console has printed "F!".

Evaluation example 2

`y = [ f(), x = g() ]` also evaluates from left to right:

1. The assignment expression `y = [ f(), x = g() ]` starts to evaluate.
   1. The `y` on this assignment's left-hand evaluates into a reference to the variable named `y`.
   2. The inner array literal `[ f(), x = g() ]` starts to evaluate.
      1. The function call `f()` prints "F!" to the console and then evaluates to the number `2`.
      2. The assignment expression `x = g()` starts to evaluate.
         1. The `x` on this assignment's left-hand side evaluates into a reference to the variable named `x`.
         2. The function call `g()` prints "G!" to the console and then evaluates to the number `3`.
         3. That `3` result from `g()` is assigned to `x`.
      3. The assignment expression `x = g()` has now finished evaluating; its result is the new value of `x`, which is `3`. That `3` result becomes the next element in the inner array literal (after the `2` from the `f()`).
   3. The inner array literal `[ f(), x = g() ]` has now finished evaluating; its result is an array with two values: `[ 2, 3 ]`.
   4. That `[ 2, 3 ]` array is now assigned to `y`.
2. The assignment expression `y = [ f(), x = g() ]` has now finished evaluating; its result is the new value of `y` – which happens to be `[ 2, 3 ]`. `x` is now assigned to `3`, `y` is now assigned to `[ 2, 3 ]`, and the console has printed "F!" then "G!".

Evaluation example 3

`x[f()] = g()` also evaluates from left to right. (This example assumes that `x` is already assigned to some object. For more information about objects, read Working with Objects.)

1. The assignment expression `x[f()] = g()` starts to evaluate.
   1. The `x[f()]` property access on this assignment's left-hand starts to evaluate.
      1. The `x` in this property access evaluates into a reference to the variable named `x`.
      2. Then the function call `f()` prints "F!" to the console and then evaluates to the number `2`.
   2. The `x[f()]` property access on this assignment has now finished evaluating; its result is a variable property reference: `x[2]`.
   3. Then the function call `g()` prints "G!" to the console and then evaluates to the number `3`.
   4. That `3` is now assigned to `x[2]`. (This step will succeed only if `x` is assigned to an object.)
2. The assignment expression `x[f()] = g()` has now finished evaluating; its result is the new value of `x[2]` – which happens to be `3`. `x[2]` is now assigned to `3`, and the console has printed "F!" then "G!".

Avoid assignment chains

Chaining assignments or nesting assignments in other expressions can result in surprising behavior. For this reason, chaining assignments in the same statement is discouraged.

In particular, putting a variable chain in a `const`, `let`, or `var` statement often does *not* work. Only the outermost/leftmost variable would get declared; other variables within the assignment chain are *not* declared by the `const/let/var` statement. For example:

```
const z = y = x = f();
```

This statement seemingly declares the variables `x`, `y`, and `z`. However, it only actually declares the variable `z`. `y` and `x` are either invalid references to nonexistent variables (in strict mode) or, worse, would implicitly create global variables for `x` and `y` in sloppy mode.

Comparison operators

A comparison operator compares its operands and returns a logical value based on whether the comparison is true. The operands can be numerical, string, logical, or object values. Strings are compared based on standard lexicographical ordering, using Unicode values. In most cases, if the two operands are not of the same type, JavaScript attempts to convert them to an appropriate type for the comparison. This behavior generally results in comparing the operands numerically. The sole exceptions to type conversion within comparisons involve the `===` and `!==` operators, which perform strict equality and inequality comparisons. These operators do not attempt to convert the operands to compatible types before checking equality. The following table describes the comparison operators in terms of this sample code:

```
const var1 = 3;
const var2 = 4;
```

Comparison operators

Operator	Description	Examples returning true

Operator	Description	Examples returning true

Equal (==)	Returns `true` if the operands are equal.	`3 == var1`
		`"3" == var1`
		`3 == '3'`

Not equal (!=)	Returns `true` if the operands are not equal.	`var1 != 4` `var2 != "3"`
Strict equal (===)	Returns `true` if the operands are equal and of the same type. See also `Object.is` and sameness in JS.	`3 === var1`
Strict not equal (!==)	Returns `true` if the operands are of the same type but not equal, or are of different type.	`var1 !== "3"` `3 !== '3'`
Greater than (>)	Returns `true` if the left operand is greater than the right operand.	`var2 > var1` `"12" > 2`
Greater than or equal (>=)	Returns `true` if the left operand is greater than or equal to the right operand.	`var2 >= var1` `var1 >= 3`
Less than (<)	Returns `true` if the left operand is less than the right operand.	`var1 < var2` `"2" < 12`
Less than or equal (<=)	Returns `true` if the left operand is less than or equal to the right operand.	`var1 <= var2` `var2 <= 5`

> **Note:** => is not a comparison operator but rather is the notation for Arrow functions.

Arithmetic operators

An arithmetic operator takes numerical values (either literals or variables) as their operands and returns a single numerical value. The standard arithmetic operators are addition (+), subtraction (-), multiplication (*), and division (/). These operators work as they do in most other programming languages when used with floating point numbers (in particular, note that division by zero produces `Infinity`). For example:

```js
1 / 2; // 0.5
1 / 2 === 1.0 / 2.0; // this is true
```

In addition to the standard arithmetic operations (+, -, *, /), JavaScript provides the arithmetic operators listed in the following table:

Arithmetic operators

Operator	Description	Example
Remainder (%)	Binary operator. Returns the integer remainder of dividing the two operands.	12 % 5 returns 2.
Increment (++)	Unary operator. Adds one to its operand. If used as a prefix operator (++x), returns the value of its operand after adding one; if used as a postfix operator (x++), returns the value of its operand before adding one.	If x is 3, then ++x sets x to 4 and returns 4, whereas x++ returns 3 and, only then, sets x to 4.
Decrement (--)	Unary operator. Subtracts one from its operand. The return value is analogous to that for the increment operator.	If x is 3, then --x sets x to 2 and returns 2, whereas x-- returns 3 and, only then, sets x to 2.

Operator	Description	Example
Unary negation (-)	Unary operator. Returns the negation of its operand.	If x is 3, then -x returns -3.
Unary plus (+)	Unary operator. Attempts to convert the operand to a number, if it is not already.	+"3" returns 3. +true returns 1.
Exponentiation operator (**)	Calculates the base to the exponent power, that is, base^exponent	2 ** 3 returns 8. 10 ** -1 returns 0.1.

Bitwise operators

A bitwise operator treats their operands as a set of 32 bits (zeros and ones), rather than as decimal, hexadecimal, or octal numbers. For example, the decimal number nine has a binary representation of 1001. Bitwise operators perform their operations on such binary representations, but they return standard JavaScript numerical values.

The following table summarizes JavaScript's bitwise operators.

Operator	Usage	Description
Bitwise AND	a & b	Returns a one in each bit position for which the corresponding bits of both operands are ones.
Bitwise OR	a | b	Returns a zero in each bit position for which the corresponding bits of both operands are zeros.
Bitwise XOR	a ^ b	Returns a zero in each bit position for which the corresponding bits are the same. [Returns a one in each bit position for which the corresponding bits are different.]
Bitwise NOT	~ a	Inverts the bits of its operand.
Left shift	a << b	Shifts a in binary representation b bits to the left, shifting in zeros from the right.
Sign-propagating right shift	a >> b	Shifts a in binary representation b bits to the right, discarding bits shifted off.
Zero-fill right shift	a >>> b	Shifts a in binary representation b bits to the right, discarding bits shifted off, and shifting in zeros from the left.

Bitwise logical operators

Conceptually, the bitwise logical operators work as follows:

- The operands are converted to thirty-two-bit integers and expressed by a series of bits (zeros and ones). Numbers with more than 32 bits get their most significant bits discarded. For example, the following integer with more than 32 bits will be converted to a 32-bit integer:

```
Before: 1110 0110 1111 1010 0000 0000 0000 0110 0000 0000 0001
After:                  1010 0000 0000 0000 0110 0000 0000 0001
```

- Each bit in the first operand is paired with the corresponding bit in the second operand: first bit to first bit, second bit to second bit, and so on.
- The operator is applied to each pair of bits, and the result is constructed bitwise.

For example, the binary representation of nine is 1001, and the binary representation of fifteen is 1111. So, when the bitwise operators are applied to these values, the results are as follows:

Expression	Result	Binary Description
`15 & 9`	`9`	`1111 & 1001 = 1001`
`15 | 9`	`15`	`1111 | 1001 = 1111`
`15 ^ 9`	`6`	`1111 ^ 1001 = 0110`
`~15`	`-16`	`~ 0000 0000 … 0000 1111 = 1111 1111 … 1111 0000`
`~9`	`-10`	`~ 0000 0000 … 0000 1001 = 1111 1111 … 1111 0110`

Note that all 32 bits are inverted using the Bitwise NOT operator, and that values with the most significant (left-most) bit set to 1 represent negative numbers (two's-complement representation). `~x` evaluates to the same value that `-x - 1` evaluates to.

Bitwise shift operators

The bitwise shift operators take two operands: the first is a quantity to be shifted, and the second specifies the number of bit positions by which the first operand is to be shifted. The direction of the shift operation is controlled by the operator used.

Shift operators convert their operands to thirty-two-bit integers and return a result of either type `Number` or `BigInt`: specifically, if the type of the left operand is `BigInt`, they return `BigInt`; otherwise, they return `Number`.

The shift operators are listed in the following table.

Bitwise shift operators

Operator	Description	Example
Left shift (`<<`)	This operator shifts the first operand the specified number of bits to the left. Excess bits shifted off to the left are discarded. Zero bits are shifted in from the right.	`9<<2` yields 36, because 1001 shifted 2 bits to the left becomes 100100, which is 36.
Sign-propagating right shift (`>>`)	This operator shifts the first operand the specified number of bits to the right. Excess bits shifted off to the right are discarded. Copies of the leftmost bit are shifted in from the left.	`9>>2` yields 2, because 1001 shifted 2 bits to the right becomes 10, which is 2. Likewise, `-9>>2` yields -3, because the sign is preserved.
Zero-fill right shift (`>>>`)	This operator shifts the first operand the specified number of bits to the right. Excess bits shifted off to the right are discarded. Zero bits are shifted in from the left.	`19>>>2` yields 4, because 10011 shifted 2 bits to the right becomes 100, which is 4. For non-negative numbers, zero-fill right shift and sign-propagating right shift yield the same result.

Logical operators

Logical operators are typically used with Boolean (logical) values; when they are, they return a Boolean value. However, the `&&`, `||`, and `??` operators actually return the value of one of the specified operands, so if these operators are used with non-Boolean values, they may return a non-Boolean value. As such, they are more adequately called "value selection operators". The logical operators are described in the following table.

Logical operators

Operator	Usage	Description
Logical AND (`&&`)	`expr1 && expr2`	Returns `expr1` if it can be converted to `false`; otherwise, returns `expr2`. Thus, when used with Boolean values, `&&` returns `true` if both operands are true; otherwise, returns `false`.

Operator	Usage	Description
Logical OR (`||`)	`expr1 || expr2`	Returns `expr1` if it can be converted to `true`; otherwise, returns `expr2`. Thus, when used with Boolean values, `||` returns `true` if either operand is true; if both are false, returns `false`.
Nullish coalescing operator (`??`)	`expr1 ?? expr2`	Returns `expr1` if it is neither `null` nor `undefined`; otherwise, returns `expr2`.
Logical NOT (`!`)	`!expr`	Returns `false` if its single operand can be converted to `true`; otherwise, returns `true`.

Examples of expressions that can be converted to `false` are those that evaluate to `null`, `0`, `0n`, `NaN`, the empty string (`""`), or `undefined`.

The following code shows examples of the `&&` (logical AND) operator.

```js
const a1 = true && true; // t && t returns true
const a2 = true && false; // t && f returns false
const a3 = false && true; // f && t returns false
const a4 = false && 3 === 4; // f && f returns false
const a5 = "Cat" && "Dog"; // t && t returns Dog
const a6 = false && "Cat"; // f && t returns false
const a7 = "Cat" && false; // t && f returns false
```

The following code shows examples of the `||` (logical OR) operator.

```js
const o1 = true || true; // t || t returns true
const o2 = false || true; // f || t returns true
const o3 = true || false; // t || f returns true
const o4 = false || 3 === 4; // f || f returns false
const o5 = "Cat" || "Dog"; // t || t returns Cat
const o6 = false || "Cat"; // f || t returns Cat
const o7 = "Cat" || false; // t || f returns Cat
```

The following code shows examples of the `??` (nullish coalescing) operator.

```js
const n1 = null ?? 1; // 1
const n2 = undefined ?? 2; // 2
const n3 = false ?? 3; // false
const n4 = 0 ?? 4; // 0
```

Note how `??` works like `||`, but it only returns the second expression when the first one is "nullish", i.e., `null` or `undefined`. `??` is a better alternative than `||` for setting defaults for values that might be `null` or `undefined`, in particular when values like `''` or `0` are valid values and the default should not apply.

The following code shows examples of the `!` (logical NOT) operator.

```js
const n1 = !true; // !t returns false
const n2 = !false; // !f returns true
const n3 = !"Cat"; // !t returns false
```

Short-circuit evaluation

As logical expressions are evaluated left to right, they are tested for possible "short-circuit" evaluation using the following rules:

- `falsy && anything` is short-circuit evaluated to the falsy value.
- `truthy || anything` is short-circuit evaluated to the truthy value.
- `nonNullish ?? anything` is short-circuit evaluated to the non-nullish value.

The rules of logic guarantee that these evaluations are always correct. Note that the *anything* part of the above expressions is not evaluated, so any side effects of doing so do not take effect.

BigInt operators

Most operators that can be used between numbers can be used between `BigInt` values as well.

js

```
// BigInt addition
const a = 1n + 2n; // 3n
// Division with BigInts round towards zero
const b = 1n / 2n; // 0n
// Bitwise operations with BigInts do not truncate either side
const c = 40000000000000000n >> 2n; // 10000000000000000n
```

One exception is unsigned right shift (`>>>`), which is not defined for BigInt values. This is because a BigInt does not have a fixed width, so technically it does not have a "highest bit".

js

```
const d = 8n >>> 2n; // TypeError: BigInts have no unsigned right shift, use >> instead
```

BigInts and numbers are not mutually replaceable — you cannot mix them in calculations.

js · ✗ bad example

```
const a = 1n + 2; // TypeError: Cannot mix BigInt and other types
```

This is because BigInt is neither a subset nor a superset of numbers. BigInts have higher precision than numbers when representing large integers, but cannot represent decimals, so implicit conversion on either side might lose precision. Use explicit conversion to signal whether you wish the operation to be a number operation or a BigInt one.

js · ✓ good example

```
const a = Number(1n) + 2; // 3
const b = 1n + BigInt(2); // 3n
```

You can compare BigInts with numbers.

js

```
const a = 1n > 2; // false
const b = 3 > 2n; // true
```

String operators

In addition to the comparison operators, which can be used on string values, the concatenation operator (+) concatenates two string values together, returning another string that is the union of the two operand strings.

For example,

js

```
console.log("my " + "string"); // console logs the string "my string".
```

The shorthand assignment operator += can also be used to concatenate strings.

For example,

```js
let myString = "alpha";
myString += "bet"; // evaluates to "alphabet" and assigns this value to myString.
```

Conditional (ternary) operator

The conditional operator is the only JavaScript operator that takes three operands. The operator can have one of two values based on a condition. The syntax is:

```js
condition ? val1 : val2
```

If `condition` is true, the operator has the value of `val1`. Otherwise it has the value of `val2`. You can use the conditional operator anywhere you would use a standard operator.

For example,

```js
const status = age >= 18 ? "adult" : "minor";
```

This statement assigns the value "adult" to the variable `status` if age is eighteen or more. Otherwise, it assigns the value "minor" to `status`.

Comma operator

The comma operator (`,`) evaluates both of its operands and returns the value of the last operand. This operator is primarily used inside a `for` loop, to allow multiple variables to be updated each time through the loop. It is regarded bad style to use it elsewhere, when it is not necessary. Often two separate statements can and should be used instead.

For example, if `a` is a 2-dimensional array with 10 elements on a side, the following code uses the comma operator to update two variables at once. The code prints the values of the diagonal elements in the array:

```js
const x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9];
const a = [x, x, x, x, x];

for (let i = 0, j = 9; i <= j; i++, j--) {
  //                               ^
  console.log(`a[${i}][${j}]= ${a[i][j]}`);
}
```

Unary operators

A unary operation is an operation with only one operand.

delete

The `delete` operator deletes an object's property. The syntax is:

```js
delete object.property;
delete object[propertyKey];
delete objectName[index];
```

where `object` is the name of an object, `property` is an existing property, and `propertyKey` is a string or symbol referring to an existing property.

If the `delete` operator succeeds, it removes the property from the object. Trying to access it afterwards will yield `undefined`. The `delete` operator returns `true` if the operation is possible; it returns `false` if the operation is not possible.

```js
delete Math.PI; // returns false (cannot delete non-configurable properties)

const myObj = { h: 4 };
delete myObj.h; // returns true (can delete user-defined properties)
```

Deleting array elements

Since arrays are just objects, it's technically possible to `delete` elements from them. This is, however, regarded as a bad practice — try to avoid it. When you delete an array property, the array length is not affected and other elements are not re-indexed. To achieve that behavior, it is much better to just overwrite the element with the value `undefined`. To actually manipulate the array, use the various array methods such as `splice`.

typeof

The `typeof operator` returns a string indicating the type of the unevaluated operand. `operand` is the string, variable, keyword, or object for which the type is to be returned. The parentheses are optional.

Suppose you define the following variables:

```js
const myFun = () => 5 + 2;
const shape = "round";
const size = 1;
const foo = ["Apple", "Mango", "Orange"];
const today = new Date();
```

The `typeof` operator returns the following results for these variables:

```js
typeof myFun; // returns "function"
typeof shape; // returns "string"
typeof size; // returns "number"
typeof foo; // returns "object"
typeof today; // returns "object"
typeof doesntExist; // returns "undefined"
```

For the keywords `true` and `null`, the `typeof` operator returns the following results:

```js
typeof true; // returns "boolean"
typeof null; // returns "object"
```

For a number or string, the `typeof` operator returns the following results:

```js
typeof 62; // returns "number"
typeof "Hello world"; // returns "string"
```

For property values, the `typeof` operator returns the type of value the property contains:

```js
typeof document.lastModified; // returns "string"
typeof window.length; // returns "number"
typeof Math.LN2; // returns "number"
```

For methods and functions, the `typeof` operator returns results as follows:

```js
typeof blur; // returns "function"
typeof parseInt; // returns "function"
typeof shape.split; // returns "function"
```

For predefined objects, the `typeof` operator returns results as follows:

```js
typeof Date; // returns "function"
typeof Function; // returns "function"
typeof Math; // returns "object"
typeof Option; // returns "function"
typeof String; // returns "function"
```

void

The `void` operator specifies an expression to be evaluated without returning a value. `expression` is a JavaScript expression to evaluate. The parentheses surrounding the expression are optional, but it is good style to use them to avoid precedence issues.

Relational operators

A relational operator compares its operands and returns a Boolean value based on whether the comparison is true.

in

The `in` operator returns `true` if the specified property is in the specified object. The syntax is:

```js
propNameOrNumber in objectName
```

where `propNameOrNumber` is a string, numeric, or symbol expression representing a property name or array index, and `objectName` is the name of an object.

The following examples show some uses of the `in` operator.

```js
// Arrays
const trees = ["redwood", "bay", "cedar", "oak", "maple"];
0 in trees; // returns true
3 in trees; // returns true
6 in trees; // returns false
"bay" in trees; // returns false
// (you must specify the index number, not the value at that index)
"length" in trees; // returns true (length is an Array property)

// built-in objects
"PI" in Math; // returns true
const myString = new String("coral");
"length" in myString; // returns true

// Custom objects
const myCar = { make: "Honda", model: "Accord", year: 1998 };
"make" in myCar; // returns true
"model" in myCar; // returns true
```

instanceof

The `instanceof` operator returns `true` if the specified object is of the specified object type. The syntax is:

```js
object instanceof objectType
```

where `object` is the object to test against `objectType`, and `objectType` is a constructor representing a type, such as `Map` or `Array`.

Use `instanceof` when you need to confirm the type of an object at runtime. For example, when catching exceptions, you can branch to different exception-handling code depending on the type of exception thrown.

For example, the following code uses `instanceof` to determine whether `obj` is a `Map` object. Because `obj` is a `Map` object, the statements inside the `if` block execute.

```js
const obj = new Map();
if (obj instanceof Map) {
  // statements to execute
}
```

Basic expressions

All operators eventually operate on one or more basic expressions. These basic expressions include identifiers and literals, but there are a few other kinds as well. They are briefly introduced below, and their semantics are described in detail in their respective reference sections.

this

The `this` keyword is usually used within a function. In general, when the function is attached to an object as a method, `this` refers to the object that the method is called on. It functions like a hidden parameter that is passed to the function. `this` is an expression that evaluates to the object, so you can use all the object operations we introduced.

```js
this["propertyName"];
this.propertyName;
doSomething(this);
```

For example, suppose a function is defined as follows:

```js
function getFullName() {
  return `${this.firstName} ${this.lastName}`;
}
```

We can now attach this function to an object, and it will use the properties of that object when called:

```js
const person1 = {
  firstName: "Chris",
  lastName: "Martin",
};

const person2 = {
  firstName: "Chester",
  lastName: "Bennington",
};

// Attach the same function
person1.getFullName = getFullName;
person2.getFullName = getFullName;

console.log(person1.getFullName()); // "Chris Martin"
console.log(person2.getFullName()); // "Chester Bennington"
```

Grouping operator

The grouping operator `( )` controls the precedence of evaluation in expressions. For example, you can override multiplication and division first, then addition and subtraction to evaluate addition first.

```js
const a = 1;
const b = 2;
const c = 3;

// default precedence
a + b * c; // 7
// evaluated by default like this
a + (b * c); // 7

// now overriding precedence
// addition before multiplication
(a + b) * c; // 9

// which is equivalent to
a * c + b * c; // 9
```

Property accessor

The property accessor syntax gets property values on objects, using either dot notation or bracket notation.

```js
object.property;
object["property"];
```

The working with objects guide goes into more details about object properties.

Optional chaining

The optional chaining syntax (`?.`) performs the chained operation on an object if it is defined and non-`null`, and otherwise short-circuits the operation and returns `undefined`. This allows you to operate on a value that may be `null` or `undefined` without causing a `TypeError`.

```js
maybeObject?.property;
maybeObject?.[property];
maybeFunction?.();
```

new

You can use the `new` operator to create an instance of a user-defined object type or of one of the built-in object types. Use `new` as follows:

```js
const objectName = new ObjectType(param1, param2, /* …, */ paramN);
```

super

The `super` keyword is used to call functions on an object's parent. It is useful with classes to call the parent constructor, for example.

```js
super(args); // calls the parent constructor.
super.functionOnParent(args);
```

JavaScript › Guide

Numbers and strings

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Numbers_and_strings · MDN page last modified 2025-08-26

This chapter introduces the two most fundamental data types in JavaScript: numbers and strings. We will introduce their underlying representations, and functions used to work with and perform calculations on them.

Numbers

In JavaScript, numbers are implemented in double-precision 64-bit binary format IEEE 754 (i.e., a number between ±2^−1022 and ±2^+1023, or about ±10^−308 to ±10^+308, with a numeric precision of 53 bits). Integer values up to ±2^53 − 1 can be represented exactly.

In addition to being able to represent floating-point numbers, the number type has three symbolic values: `Infinity`, `-Infinity`, and `NaN` (not-a-number).

See also JavaScript data types and structures for context with other primitive types in JavaScript.

You can use four types of number literals: decimal, binary, octal, and hexadecimal.

Decimal numbers

js

```
1234567890
42
```

Decimal literals can start with a zero (`0`) followed by another decimal digit, but if all digits after the leading `0` are smaller than 8, the number is interpreted as an octal number. This is considered a legacy syntax, and number literals prefixed with `0`, whether interpreted as octal or decimal, cause a syntax error in strict mode — so, use the `0o` prefix instead.

js · ✗ bad example

```
0888 // 888 parsed as decimal
0777 // parsed as octal, 511 in decimal
```

Binary numbers

Binary number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "B" (`0b` or `0B`). If the digits after the `0b` are not 0 or 1, the following `SyntaxError` is thrown: "Missing binary digits after 0b".

js

```
0b10000000000000000000000000000000 // 2147483648
0b01111111100000000000000000000000 // 2139095040
0B00000000011111111111111111111111 // 8388607
```

Octal numbers

The standard syntax for octal numbers is to prefix them with `0o`. For example:

js

```
0O755 // 493
0o644 // 420
```

There's also a legacy syntax for octal numbers — by prefixing the octal number with a zero: `0644 === 420` and `"\045" === "%"`. If the digits after the `0` are outside the range 0 through 7, the number will be interpreted as a decimal number.

```js
const n = 0755; // 493
const m = 0644; // 420
```

Strict mode forbids this octal syntax.

Hexadecimal numbers

Hexadecimal number syntax uses a leading zero followed by a lowercase or uppercase Latin letter "X" (`0x` or `0X`). If the digits after 0x are outside the range (0123456789ABCDEF), the following `SyntaxError` is thrown: "Identifier starts immediately after numeric literal".

```js
0xFFFFFFFFFFFFF // 4503599627370495
0xabcdef123456  // 188900967593046
0XA             // 10
```

Exponentiation

```js
0e-5   // 0
0e+5   // 0
5e1    // 50
175e-2 // 1.75
1e3    // 1000
1e-3   // 0.001
1E3    // 1000
```

Numeric separators

For all literal syntaxes shown above, an underscore (_) can be inserted between digits to improve readability.

```js
1_000_000_000_000
1_050.95
0b1010_0001_1000_0101
0o2_2_5_6
0xA0_B0_C0
1_000_000_000_000_000_000_000n
```

See the lexical grammar reference for more details about number literals.

Number object

The built-in `Number` object has properties for numerical constants, such as maximum value, not-a-number, and infinity. You cannot change the values of these properties and you use them as follows:

```js
const biggestNum = Number.MAX_VALUE;
const smallestNum = Number.MIN_VALUE;
const infiniteNum = Number.POSITIVE_INFINITY;
const negInfiniteNum = Number.NEGATIVE_INFINITY;
const notANum = Number.NaN;
```

You always refer to a property of the predefined `Number` object as shown above, and not as a property of a `Number` object you create yourself.

The following table summarizes the `Number` object's properties.

Property	Description
`Number.MAX_VALUE`	The largest positive representable number (`1.7976931348623157e+308`)
`Number.MIN_VALUE`	The smallest positive representable number (`5e-324`)
`Number.NaN`	Special "not a number" value
`Number.NEGATIVE_INFINITY`	Special negative infinite value; returned on overflow
`Number.POSITIVE_INFINITY`	Special positive infinite value; returned on overflow
`Number.EPSILON`	Difference between `1` and the smallest value greater than `1` that can be represented as a `Number` (`2.220446049250313e-16`)
`Number.MIN_SAFE_INTEGER`	Minimum safe integer in JavaScript (–2^53 + 1, or `–9007199254740991`)
`Number.MAX_SAFE_INTEGER`	Maximum safe integer in JavaScript (+2^53 – 1, or `+9007199254740991`)

Method	Description
`Number.parseFloat()`	Parses a string argument and returns a floating point number. Same as the global `parseFloat()` function.
`Number.parseInt()`	Parses a string argument and returns an integer of the specified radix or base. Same as the global `parseInt()` function.
`Number.isFinite()`	Determines whether the passed value is a finite number.
`Number.isInteger()`	Determines whether the passed value is an integer.
`Number.isNaN()`	Determines whether the passed value is `NaN`. More robust version of the original global `isNaN()`.
`Number.isSafeInteger()`	Determines whether the provided value is a number that is a *safe integer*.

The `Number` prototype provides methods for retrieving information from `Number` objects in various formats. The following table summarizes the methods of `Number.prototype`.

Method	Description
`toExponential()`	Returns a string representing the number in exponential notation.
`toFixed()`	Returns a string representing the number in fixed-point notation.
`toPrecision()`	Returns a string representing the number to a specified precision in fixed-point notation.

Math object

The built-in `Math` object has properties and methods for mathematical constants and functions. For example, the `Math` object's `PI` property has the value of pi (3.141…), which you would use in an application as

```js
Math.PI;
```

Similarly, standard mathematical functions are methods of `Math`. These include trigonometric, logarithmic, exponential, and other functions. For example, if you want to use the trigonometric function sine, you would write

```js
Math.sin(1.56);
```

Note that all trigonometric methods of `Math` take arguments in radians.

The following table summarizes the `Math` object's methods.

Methods of `Math`

Method	Description
`abs()`	Absolute value
`sin()`, `cos()`, `tan()`	Standard trigonometric functions; with the argument in radians.
`asin()`, `acos()`, `atan()`, `atan2()`	Inverse trigonometric functions; return values in radians.
`sinh()`, `cosh()`, `tanh()`	Hyperbolic functions; argument in hyperbolic angle.
`asinh()`, `acosh()`, `atanh()`	Inverse hyperbolic functions; return values in hyperbolic angle.
`pow()`, `exp()`, `expm1()`, `log()`, `log10()`, `log1p()`, `log2()`	Exponential and logarithmic functions.
`floor()`, `ceil()`	Returns the largest/smallest integer less/greater than or equal to an argument.
`min()`, `max()`	Returns the minimum or maximum (respectively) value of a comma separated list of numbers as arguments.
`random()`	Returns a random number between 0 and 1.
`round()`, `fround()`, `trunc()`,	Rounding and truncation functions.
`sqrt()`, `cbrt()`, `hypot()`	Square root, cube root, Square root of the sum of square arguments.
`sign()`	The sign of a number, indicating whether the number is positive, negative or zero.
`clz32()`, `imul()`	Number of leading zero bits in the 32-bit binary representation. The result of the C-like 32-bit multiplication of the two arguments.

Unlike many other objects, you never create a `Math` object of your own. You always use the built-in `Math` object.

BigInts

One shortcoming of number values is they only have 64 bits. In practice, due to using IEEE 754 encoding, they cannot represent any integer larger than `Number.MAX_SAFE_INTEGER` (which is 2^{53} - 1) accurately. To solve the need of encoding binary data and to interoperate with other languages that offer wide integers like `i64` (64-bit integers) and `i128` (128-bit integers), JavaScript also offers another data type to represent *arbitrarily large integers*: `BigInt`.

A BigInt can be defined as an integer literal suffixed by `n`:

```js
const b1 = 123n;
// Can be arbitrarily large.
const b2 = -1234567890987654321n;
```

BigInts can also be constructed from number values or string values using the `BigInt` constructor.

```js
const b1 = BigInt(123);
// Using a string prevents loss of precision, since long number
// literals don't represent what they seem like.
const b2 = BigInt("-1234567890987654321");
```

Conceptually, a BigInt is just an arbitrarily long sequence of bits which encodes an integer. You can safely do any arithmetic operations without losing precision or over-/underflowing.

```js
const integer = 12 ** 34; // 4.9222352429520264e+36; only has limited precision
const bigint = 12n ** 34n; // 4922235242952026704037113243122008064n
```

Compared to numbers, BigInt values yield higher precision when representing large *integers*; however, they cannot represent *floating-point numbers*. For example, division would round to zero:

```js
const bigintDiv = 5n / 2n; // 2n, because there's no 2.5 in BigInt
```

`Math` functions cannot be used on BigInt values; they only work with numbers.

Choosing between BigInt and number depends on your use-case and your input's range. The precision of numbers should be able to accommodate most day-to-day tasks already, and BigInts are most suitable for handling binary data.

Read more about what you can do with BigInt values in the Expressions and Operators section, or the BigInt reference.

Strings

JavaScript's String type is used to represent textual data. It is a set of "elements" of 16-bit unsigned integer values (UTF-16 code units). Each element in the String occupies a position in the String. The first element is at index 0, the next at index 1, and so on. The length of a String is the number of elements in it. You can create strings using string literals or string objects.

String literals

You can declare strings in source code using either single or double quotes:

```js
'foo'
"bar"
```

Within a string literal, most characters can be entered literally. The only exceptions are the backslash (`\`, which starts an escape sequence), the quote character being used to enclose the string, which terminates the string, and the newline character, which is a syntax error if not preceded by a backslash.

More advanced strings can be created using escape sequences:

Hexadecimal escape sequences

The number after \x is interpreted as a hexadecimal number.

```js
"\xA9" // "©"
```

Unicode escape sequences

The Unicode escape sequences require at least four hexadecimal digits following \u.

```js
"\u00A9" // "©"
```

Unicode code point escapes

With Unicode code point escapes, any character can be escaped using hexadecimal numbers so that it is possible to use Unicode code points up to `0x10FFFF`. With the four-digit Unicode escapes it is often necessary to write the surrogate halves separately to achieve the same result.

See also `String.fromCodePoint()` or `String.prototype.codePointAt()`.

```js
"\u{2F804}"

// the same with simple Unicode escapes
"\uD87E\uDC04"
```

String object

You can call methods directly on a string value:

```js
console.log("hello".toUpperCase()); // HELLO
```

The following methods are available on `String` values:

- Query: get the character or character code at a particular string index. Methods include `at()`, `charAt()`, `charCodeAt()`, and `codePointAt()`.
- Search: get information about a substring that conforms to a pattern, or test if a particular substring exists. Methods include `indexOf()`, `lastIndexOf()`, `startsWith()`, `endsWith()`, `includes()`, `match()`, `matchAll()`, and `search()`
- Composition: combine strings into a longer string. Methods include `padStart()`, `padEnd()`, `concat()`, and `repeat()`.
- Decomposition: break a string into smaller strings. Methods include `split()`, `slice()`, `substring()`, `substr()`, `trim()`, `trimStart()`, and `trimEnd()`.
- Transformation: return a new string based on the current string's content. Methods include `toLowerCase()`, `toUpperCase()`, `toLocaleLowerCase()`, `toLocaleUpperCase()`, `normalize()`, and `toWellFormed()`.

When working with strings, there are two other objects that provide important functionality for string manipulation: `RegExp` and `Intl`. They are introduced in regular expressions and internationalization respectively.

Template literals

Template literals are string literals allowing embedded expressions. You can use multi-line strings and string interpolation features with them.

Template literals are enclosed by backtick (grave accent) characters (`) instead of double or single quotes. Template literals can contain placeholders. These are indicated by the dollar sign and curly braces (`${expression}`).

Multi-lines

Any new line characters inserted in the source are part of the template literal. Using normal strings, you would have to use the following syntax in order to get multi-line strings:

```js
console.log(
  "string text line 1\n\
string text line 2",
);
// "string text line 1
// string text line 2"
```

To get the same effect with multi-line strings, you can now write:

```js
console.log(`string text line 1
string text line 2`);
// "string text line 1
// string text line 2"
```

Embedded expressions

In order to embed expressions within normal strings, you would use the following syntax:

```js
const five = 5;
const ten = 10;
console.log(
  "Fifteen is " + (five + ten) + " and not " + (2 * five + ten) + ".",
);
// "Fifteen is 15 and not 20."
```

Now, with template literals, you are able to make use of the syntactic sugar making substitutions like this more readable:

```js
const five = 5;
const ten = 10;
console.log(`Fifteen is ${five + ten} and not ${2 * five + ten}.`);
// "Fifteen is 15 and not 20."
```

For more information, read about Template literals in the JavaScript reference.

JavaScript › Guide

Representing dates & times

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Representing_dates_times · MDN page last modified 2025-07-08

> **Note:** The `Date` object is now considered legacy and should be avoided in new code. We will update this page with modern alternatives soon.

Date object

JavaScript does not have a date data type. However, you can use the `Date` object and its methods to work with dates and times in your applications. The `Date` object has a large number of methods for setting, getting, and manipulating dates. It does not have any properties.

JavaScript handles dates similarly to Java. The two languages have many of the same date methods, and both languages store dates as the number of milliseconds since midnight at the beginning of January 1, 1970, UTC, with a Unix Timestamp being the number of seconds since the same instant. The instant at the midnight at the beginning of January 1, 1970, UTC is called the epoch.

The `Date` object range is -100,000,000 days to 100,000,000 days relative to the epoch.

To create a `Date` object:

```js
const dateObjectName = new Date([parameters]);
```

where `dateObjectName` is the name of the `Date` object being created; it can be a new object or a property of an existing object.

Calling `Date` without the `new` keyword returns a string representing the current date and time.

The `parameters` in the preceding syntax can be any of the following:

- Nothing: creates today's date and time. For example, `today = new Date();`.
- A string representing a date, in many different forms. The exact forms supported differ among engines, but the following form is always supported: `YYYY-MM-DDTHH:mm:ss.sssZ`. For example, `xmas95 = new Date("1995-12-25")`. If you omit hours, minutes, or seconds, the value will be set to zero.
- A set of integer values for year, month, and day. For example, `xmas95 = new Date(1995, 11, 25)`.
- A set of integer values for year, month, day, hour, minute, and seconds. For example, `xmas95 = new Date(1995, 11, 25, 9, 30, 0);`.

Methods of the Date object

The `Date` object methods for handling dates and times fall into these broad categories:

- "set" methods, for setting date and time values in `Date` objects.
- "get" methods, for getting date and time values from `Date` objects.
- "to" methods, for returning string values from `Date` objects.
- parse and UTC methods, for parsing `Date` strings.

With the "get" and "set" methods you can get and set seconds, minutes, hours, day of the month, day of the week, months, and years separately. There is a `getDay` method that returns the day of the week, but

no corresponding `setDay` method, because the day of the week is set automatically. These methods use integers to represent these values as follows:

- Seconds and minutes: 0 to 59
- Hours: 0 to 23
- Day: 0 (Sunday) to 6 (Saturday)
- Date: 1 to 31 (day of the month)
- Months: 0 (January) to 11 (December)
- Year: years since 1900

For example, suppose you define the following date:

```js
const xmas95 = new Date("1995-12-25");
```

Then `xmas95.getMonth()` returns 11, and `xmas95.getFullYear()` returns 1995.

The `getTime` and `setTime` methods are useful for comparing dates. The `getTime` method returns the number of milliseconds since the epoch for a `Date` object.

For example, the following code displays the number of days left in the current year:

```js
const today = new Date();
const endYear = new Date(1995, 11, 31, 23, 59, 59, 999); // Set day and month
endYear.setFullYear(today.getFullYear()); // Set year to this year
const msPerDay = 24 * 60 * 60 * 1000; // Number of milliseconds per day
let daysLeft = (endYear.getTime() - today.getTime()) / msPerDay;
daysLeft = Math.round(daysLeft); // Returns days left in the year
```

This example creates a `Date` object named `today` that contains today's date. It then creates a `Date` object named `endYear` and sets the year to the current year. Then, using the number of milliseconds per day, it computes the number of days between `today` and `endYear`, using `getTime` and rounding to a whole number of days.

The `parse` method is useful for assigning values from date strings to existing `Date` objects. For example, the following code uses `parse` and `setTime` to assign a date value to the `ipoDate` object:

```js
const ipoDate = new Date();
ipoDate.setTime(Date.parse("Aug 9, 1995"));
```

Example

In the following example, the function `JSClock()` returns the time in the format of a digital clock.

```js
function JSClock() {
  const time = new Date();
  const hour = time.getHours();
  const minute = time.getMinutes();
  const second = time.getSeconds();
  let temp = String(hour % 12);
  if (temp === "0") {
    temp = "12";
  }
  temp += (minute < 10 ? ":0" : ":") + minute;
  temp += (second < 10 ? ":0" : ":") + second;
  temp += hour >= 12 ? " P.M." : " A.M.";
  return temp;
}
```

The `JSClock` function first creates a new `Date` object called `time`; since no arguments are given, time is created with the current date and time. Then calls to the `getHours`, `getMinutes`, and `getSeconds` methods assign the value of the current hour, minute, and second to `hour`, `minute`, and `second`.

The following statements build a string value based on the time. The first statement creates a variable `temp`. Its value is `hour % 12`, which is `hour` in the 12-hour system. Then, if the hour is `0`, it gets re-assigned to `12`, so that midnights and noons are displayed as `12:00` instead of `0:00`.

The next statement appends a `minute` value to `temp`. If the value of `minute` is less than 10, the conditional expression adds a string with a preceding zero; otherwise it adds a string with a demarcating colon. Then a statement appends a seconds value to `temp` in the same way.

Finally, a conditional expression appends "P.M." to `temp` if `hour` is 12 or greater; otherwise, it appends "A.M." to `temp`.

JavaScript › Guide

Regular expressions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions · MDN page last modified 2026-03-17

Regular expressions are patterns used to match character combinations in strings. In JavaScript, regular expressions are also objects. These patterns are used with the `exec()` and `test()` methods of `RegExp`, and with the `match()`, `matchAll()`, `replace()`, `replaceAll()`, `search()`, and `split()` methods of `String`. This chapter describes JavaScript regular expressions. It provides a brief overview of each syntax element. For a detailed explanation of each one's semantics, read the regular expressions reference.

Creating a regular expression

You construct a regular expression in one of two ways:

- Using a regular expression literal, which consists of a pattern enclosed between slashes, as follows:

  ```js
  const re = /ab+c/;
  ```

  Regular expression literals provide compilation of the regular expression when the script is loaded. If the regular expression remains constant, using this can improve performance.

- Or calling the constructor function of the `RegExp` object, as follows:

  ```js
  const re = new RegExp("ab+c");
  ```

  Using the constructor function provides runtime compilation of the regular expression. Use the constructor function when you know the regular expression pattern will be changing, or you don't know the pattern and are getting it from another source, such as user input.

Writing a regular expression pattern

A regular expression pattern is composed of simple characters, such as `/abc/`, or a combination of simple and special characters, such as `/ab*c/` or `/Chapter (\d+)\.\d*/`. The last example includes parentheses, which are used as a memory device. The match made with this part of the pattern is remembered for later use, as described in Using groups.

Using simple patterns

Simple patterns are constructed of characters for which you want to find a direct match. For example, the pattern `/abc/` matches character combinations in strings only when the exact sequence `"abc"` occurs (all characters together and in that order). Such a match would succeed in the strings `"Hi, do you know your abc's?"` and `"The latest airplane designs evolved from slabcraft."`. In both cases the match is with the substring `"abc"`. There is no match in the string `"Grab crab"` because while it contains the substring `"ab c"`, it does not contain the exact substring `"abc"`.

Using special characters

When the search for a match requires something more than a direct match, such as finding one or more b's, or finding white space, you can include special characters in the pattern. For example, to match *a single "a" followed by zero or more "b"s followed by "c"*, you'd use the pattern `/ab*c/`: the `*` after `"b"` means "0 or more occurrences of the preceding item." In the string `"cbbabbbbcdebc"`, this pattern will match the substring `"abbbbc"`.

The following pages provide lists of the different special characters that fit into each category, along with descriptions and examples.

Assertions guide
: Assertions include boundaries, which indicate the beginnings and endings of lines and words, and other patterns indicating in some way that a match is possible (including look-ahead, look-behind, and conditional expressions).

Character classes guide
: Distinguish different types of characters. For example, distinguishing between letters and digits.

Groups and backreferences guide
: Groups group multiple patterns as a whole, and capturing groups provide extra submatch information when using a regular expression pattern to match against a string. Backreferences refer to a previously captured group in the same regular expression.

Quantifiers guide
: Indicate numbers of characters or expressions to match.

If you want to look at all the special characters that can be used in regular expressions in a single table, see the following:

Special characters in regular expressions.

Characters / constructs	Corresponding article
`[xyz]`, `[^xyz]`, `.`, `\d`, `\D`, `\w`, `\W`, `\s`, `\S`, `\t`, `\r`, `\n`, `\v`, `\f`, `[\b]`, `\0`, `\c`*X*, `\x`*HH*, `\u`*HHHH*, `\u{`*H…H*`}`, *x*`|`*y*	Character classes
`^`, `$`, `\b`, `\B`, `x(?=y)`, `x(?!y)`, `(?<=y)x`, `(?<!y)x`	Assertions
`(`*x*`)`, `(?<Name>x)`, `(?:`*x*`)`, `\`*n*, `\k<Name>`	Groups and backreferences
x`*`, *x*`+`, *x*`?`, *x*`{`*n*`}`, *x*`{`*n*`,}`, *x*`{`*n*`,`*m*`}`	Quantifiers

> **Note:** A larger cheat sheet is also available (only aggregating parts of those individual articles).

Escaping

If you need to use any of the special characters literally (actually searching for a `"*"`, for instance), you should escape it by putting a backslash in front of it. For instance, to search for `"a"` followed by `"*"` followed by `"b"`, you'd use `/a\*b/` — the backslash "escapes" the `"*"`, making it literal instead of special.

> **Note:** In many cases, when trying match a special character, you can wrap it in a character class as an alternative to escaping, for example `/a[*]b/`.

Similarly, if you're writing a regular expression literal and need to match a slash ("/"), you need to escape that (otherwise, it terminates the pattern). For instance, to search for the string "/example/" followed by one or more alphabetic characters, you'd use `/\/example\/[a-z]+/i`—the backslashes before each slash make them literal.

To match a literal backslash, you need to escape the backslash. For instance, to match the string "C:\" where "C" can be any letter, you'd use `/[A-Z]:\\/` — the first backslash escapes the one after it, so the expression searches for a single literal backslash.

If using the `RegExp` constructor with a string literal, remember that the backslash is an escape in string literals, so to use it in the regular expression, you need to escape it at the string literal level. `/a\*b/` and `new RegExp("a\\*b")` create the same expression, which searches for "a" followed by a literal "*" followed by "b".

The `RegExp.escape()` function returns a new string where all special characters in regex syntax are escaped. This allows you to do `new RegExp(RegExp.escape("a*b"))` to create a regular expression that matches only the string `"a*b"`.

Using parentheses

Parentheses around any part of the regular expression pattern causes that part of the matched substring to be remembered. Once remembered, the substring can be recalled for other use. See Groups and backreferences for more details.

Using regular expressions in JavaScript

Regular expressions are used with the `RegExp` methods `test()` and `exec()` and with the `String` methods `match()`, `matchAll()`, `replace()`, `replaceAll()`, `search()`, and `split()`.

Method	Description
`exec()`	Executes a search for a match in a string. It returns an array of information or `null` on a mismatch.
`test()`	Tests for a match in a string. It returns `true` or `false`.
`match()`	Returns an array containing all of the matches, including capturing groups, or `null` if no match is found.
`matchAll()`	Returns an iterator containing all of the matches, including capturing groups.
`search()`	Tests for a match in a string. It returns the index of the match, or `-1` if the search fails.
`replace()`	Executes a search for a match in a string, and replaces the matched substring with a replacement substring.
`replaceAll()`	Executes a search for all matches in a string, and replaces the matched substrings with a replacement substring.
`split()`	Uses a regular expression or a fixed string to break a string into an array of substrings.

When you want to know whether a pattern is found in a string, use the `test()` or `search()` methods; for more information (but slower execution) use the `exec()` or `match()` methods. If you use `exec()` or `match()` and if the match succeeds, these methods return an array and update properties of the associated regular expression object and also of the predefined regular expression object, `RegExp`. If the match fails, the `exec()` method returns `null` (which coerces to `false`).

In the following example, the script uses the `exec()` method to find a match in a string.

```js
const myRe = /d(b+)d/g;
const myArray = myRe.exec("cdbbdbsbz");
```

If you do not need to access the properties of the regular expression, an alternative way of creating `myArray` is with this script:

```js
const myArray = /d(b+)d/g.exec("cdbbdbsbz");
// similar to 'cdbbdbsbz'.match(/d(b+)d/g); however,
// 'cdbbdbsbz'.match(/d(b+)d/g) outputs [ "dbbd" ]
// while /d(b+)d/g.exec('cdbbdbsbz') outputs [ 'dbbd', 'bb', index: 1, input: 'cdbbdbsbz' ]
```

(See Using the global search flag with `exec()` for further info about the different behaviors.)

If you want to construct the regular expression from a string, yet another alternative is this script:

```js
const myRe = new RegExp("d(b+)d", "g");
const myArray = myRe.exec("cdbbdbsbz");
```

With these scripts, the match succeeds and returns the array and updates the properties shown in the following table.

Results of regular expression execution.

Object	Property or index	Description	In this example
`myArray`		The matched string and all remembered substrings.	`['dbbd', 'bb', index: 1, input: 'cdbbdbsbz']`
	`index`	The 0-based index of the match in the input string.	`1`
	`input`	The original string.	`'cdbbdbsbz'`
	`[0]`	The last matched characters.	`'dbbd'`
`myRe`	`lastIndex`	The index at which to start the next match. (This property is set only if the regular expression uses the g option, described in Advanced Searching With Flags.)	`5`
	`source`	The text of the pattern. Updated at the time that the regular expression is created, not executed.	`'d(b+)d'`

As shown in the second form of this example, you can use a regular expression created with an object initializer without assigning it to a variable. If you do, however, every occurrence is a new regular expression. For this reason, if you use this form without assigning it to a variable, you cannot subsequently access the properties of that regular expression. For example, assume you have this script:

```js
const myRe = /d(b+)d/g;
const myArray = myRe.exec("cdbbdbsbz");
console.log(`The value of lastIndex is ${myRe.lastIndex}`);

// "The value of lastIndex is 5"
```

However, if you have this script:

```js
const myArray = /d(b+)d/g.exec("cdbbdbsbz");
console.log(`The value of lastIndex is ${/d(b+)d/g.lastIndex}`);

// "The value of lastIndex is 0"
```

The occurrences of `/d(b+)d/g` in the two statements are different regular expression objects and hence have different values for their `lastIndex` property. If you need to access the properties of a regular

expression created with an object initializer, you should first assign it to a variable.

Advanced searching with flags

Regular expressions have optional flags that allow for functionality like global searching and case-insensitive searching. These flags can be used separately or together in any order, and are included as part of the regular expression.

Flag	Description	Corresponding property
`d`	Generate indices for substring matches.	`hasIndices`
`g`	Global search.	`global`
`i`	Case-insensitive search.	`ignoreCase`
`m`	Makes `^` and `$` match the start and end of each line instead of those of the entire string.	`multiline`
`s`	Allows `.` to match newline characters.	`dotAll`
`u`	"Unicode"; treat a pattern as a sequence of Unicode code points.	`unicode`
`v`	An upgrade to the `u` mode with more Unicode features.	`unicodeSets`
`y`	Perform a "sticky" search that matches starting at the current position in the target string.	`sticky`

To include a flag with the regular expression, use this syntax:

```js
const re = /pattern/flags;
```

or

```js
const re = new RegExp("pattern", "flags");
```

Note that the flags are an integral part of a regular expression. They cannot be added or removed later.

For example, `re = /\w+\s/g` creates a regular expression that looks for one or more characters followed by a space, and it looks for this combination throughout the string.

```js
const re = /\w+\s/g;
const str = "fee fi fo fum";
const myArray = str.match(re);
console.log(myArray);

// ["fee ", "fi ", "fo "]
```

You could replace the line:

```js
const re = /\w+\s/g;
```

with:

```js
const re = new RegExp("\\w+\\s", "g");
```

and get the same result.

The `m` flag is used to specify that a multiline input string should be treated as multiple lines. If the `m` flag is used, `^` and `$` match at the start or end of any line within the input string instead of the start or end of the entire string.

The `i`, `m`, and `s` flags can be enabled or disabled for specific parts of a regex using the modifier syntax.

Using the global search flag with exec()

`RegExp.prototype.exec()` method with the `g` flag returns each match and its position iteratively.

```js
const str = "fee fi fo fum";
const re = /\w+\s/g;

console.log(re.exec(str)); // ["fee ", index: 0, input: "fee fi fo fum"]
console.log(re.exec(str)); // ["fi ", index: 4, input: "fee fi fo fum"]
console.log(re.exec(str)); // ["fo ", index: 7, input: "fee fi fo fum"]
console.log(re.exec(str)); // null
```

In contrast, `String.prototype.match()` method returns all matches at once, but without their position.

```js
console.log(str.match(re)); // ["fee ", "fi ", "fo "]
```

Using unicode regular expressions

The `u` flag is used to create "unicode" regular expressions; that is, regular expressions which support matching against unicode text. An important feature that's enabled in unicode mode is Unicode property escapes. For example, the following regular expression might be used to match against an arbitrary unicode "word":

```js
/\p{L}*/u;
```

Unicode regular expressions have different execution behavior as well. `RegExp.prototype.unicode` contains more explanation about this.

Examples

> **Note:** Several examples are also available in:
>
> - The reference pages for `exec()`, `test()`, `match()`, `matchAll()`, `search()`, `replace()`, `split()`
> - The guide articles: character classes, assertions, groups and backreferences, quantifiers

Using special characters to verify input

In the following example, the user is expected to enter a phone number. When the user presses the "Check" button, the script checks the validity of the number. If the number is valid (matches the character sequence specified by the regular expression), the script shows a message thanking the user and confirming the number. If the number is invalid, the script informs the user that the phone number is not valid.

The regular expression looks for:

1. the beginning of the line of data: ^

2. followed by three numeric characters `\d{3}` OR | a left parenthesis `\(`, followed by three digits `\d{3}`, followed by a close parenthesis `\)`, in a non-capturing group `(?:)`
3. followed by one dash, forward slash, or decimal point in a capturing group `()`
4. followed by three digits `\d{3}`
5. followed by the match remembered in the (first) captured group `\1`
6. followed by four digits `\d{4}`
7. followed by the end of the line of data: `$`

HTML

html

```
<p>
  Enter your phone number (with area code) and then click "Check".
  <br />
  The expected format is like ###-###-####.
</p>
<form id="form">
  <input id="phone" />
  <button type="submit">Check</button>
</form>
<p id="output"></p>
```

JavaScript

js

```
const form = document.querySelector("#form");
const input = document.querySelector("#phone");
const output = document.querySelector("#output");

const re = /^(?:\d{3}|\(\d{3}\))([-/.])\d{3}\1\d{4}$/;

function testInfo(phoneInput) {
  const ok = re.exec(phoneInput.value);

  output.textContent = ok
    ? `Thanks, your phone number is ${ok[0]}`
    : `${phoneInput.value} isn't a phone number with area code!`;
}

form.addEventListener("submit", (event) => {
  event.preventDefault();
  testInfo(input);
});
```

Result

▷ Live sample output ("Using special characters to verify input sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions#using_special_characters_to_verify_input .

Tools

RegExr

An online tool to learn, build, & test Regular Expressions.

Regex tester

An online regex builder/debugger

Regex interactive tutorial

An online interactive tutorials, Cheat sheet, & Playground.

Regex visualizer

An online visual regex tester.

JavaScript › Guide › Regular expressions

Assertions

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions/Assertions · MDN page last modified 2026-10-08

Assertions include boundaries, which indicate the beginnings and endings of lines and words, and other patterns indicating in some way that a match is possible (including look-ahead, look-behind, and conditional expressions).

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp Assertions". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const text = "A quick fox";

const regexpLastWord = /\w+$/;
console.log(text.match(regexpLastWord));
// Expected output: Array ["fox"]

const regexpWords = /\b\w+\b/g;
console.log(text.match(regexpWords));
// Expected output: Array ["A", "quick", "fox"]

const regexpFoxQuality = /\w+(?= fox)/;
console.log(text.match(regexpFoxQuality));
// Expected output: Array ["quick"]
```

Types

Boundary-type assertions

Characters	Meaning
^	**Input boundary beginning assertion:** Matches the beginning of input. If the `multiline` (m) flag is enabled, also matches immediately after a line break character. For example, `/^A/` does not match the "A" in "an A", but does match the first "A" in "An A". **Note:** This character has a different meaning when it appears at the start of a character class.
$	**Input boundary end assertion:** Matches the end of input. If the `multiline` (m) flag is enabled, also matches immediately before a line break character. For example, `/t$/` does not match the "t" in "eater", but does match it in "eat".
\A	**Buffer boundary start assertion:** Matches the start of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
\z	**Buffer boundary end assertion:** Matches the end of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
\Z	**Buffer boundary end assertion with optional newline:** Matches the end of the entire string, but allows an optional trailing newline character sequence (either a line terminator or a `\r\n` sequence). Only valid in Unicode-aware mode.

Characters	Meaning
`\b`	**Word boundary assertion:** Matches a word boundary. This is the position where a word character is not followed or preceded by another word-character, such as between a letter and a space. Note that a matched word boundary is not included in the match. In other words, the length of a matched word boundary is zero. Examples: • `/\bm/` matches the "m" in "moon". • `/oo\b/` does not match the "oo" in "moon", because "oo" is followed by "n" which is a word character. • `/oon\b/` matches the "oon" in "moon", because "oon" is the end of the string, thus not followed by a word character. • `/\w\b\w/` will never match anything, because a word character can never be followed by both a non-word and a word character. To match a backspace character (`[\b]`), see Character Classes.
`\B`	**Non-word-boundary assertion:** Matches a non-word boundary. This is a position where the previous and next character are of the same type: Either both must be words, or both must be non-words, for example between two letters or between two spaces. The beginning and end of a string are considered non-words. Same as the matched word boundary, the matched non-word boundary is also not included in the match. For example, `/\Bon/` matches "on" in "at noon", and `/ye\B/` matches "ye" in "possibly yesterday".

Other assertions

> **Note:** The `?` character may also be used as a quantifier.

Characters	Meaning
`x(?=y)`	**Lookahead assertion:** Matches "x" only if "x" is followed by "y". For example, `/Jack(?=Sprat)/` matches "Jack" only if it is followed by "Sprat". `/Jack(?=Sprat|Frost)/` matches "Jack" only if it is followed by "Sprat" or "Frost". However, neither "Sprat" nor "Frost" is part of the match results.
`x(?!y)`	**Negative lookahead assertion:** Matches "x" only if "x" is not followed by "y". For example, `/\d+(?!\.)/` matches a number only if it is not followed by a decimal point. `/\d+(?!\.)/.exec('3.141')` matches "141" but not "3".
`(?<=y)x`	**Lookbehind assertion:** Matches "x" only if "x" is preceded by "y". For example, `/(?<=Jack)Sprat/` matches "Sprat" only if it is preceded by "Jack". `/(?<=Jack|Tom)Sprat/` matches "Sprat" only if it is preceded by "Jack" or "Tom". However, neither "Jack" nor "Tom" is part of the match results.
`(?<!y)x`	**Negative lookbehind assertion:** Matches "x" only if "x" is not preceded by "y". For example, `/(?<!-)\d+/` matches a number only if it is not preceded by a minus sign. `/(?<!-)\d+/.exec('3')` matches "3". `/(?<!-)\d+/.exec('-3')` match is not found because the number is preceded by the minus sign.

Examples

General boundary-type overview example

```js
// Using Regex boundaries to fix buggy string.
buggyMultiline = `tey, ihe light-greon apple
tangs on ihe greon traa`;

// 1) Use ^ to fix the matching at the beginning of the string, and right after newline.
buggyMultiline = buggyMultiline.replace(/^t/gim, "h");
console.log(1, buggyMultiline); // fix 'tey' => 'hey' and 'tangs' => 'hangs' but do not
touch 'traa'.

// 2) Use $ to fix matching at the end of the text.
buggyMultiline = buggyMultiline.replace(/aa$/gim, "ee.");
console.log(2, buggyMultiline); // fix 'traa' => 'tree.'.

// 3) Use \b to match characters right on border between a word and a space.
buggyMultiline = buggyMultiline.replace(/\bi/gim, "t");
console.log(3, buggyMultiline); // fix 'ihe' => 'the' but do not touch 'light'.

// 4) Use \B to match characters inside borders of an entity.
fixedMultiline = buggyMultiline.replace(/\Bo/gim, "e");
console.log(4, fixedMultiline); // fix 'greon' => 'green' but do not touch 'on'.
```

Matching the beginning of input using a ^ control character

Use ^ for matching at the beginning of input. In this example, we can get the fruits that start with 'A' by a `/^A/` regex. For selecting appropriate fruits we can use the `filter` method with an arrow function.

```js
const fruits = ["Apple", "Watermelon", "Orange", "Avocado", "Strawberry"];

// Select fruits started with 'A' by /^A/ Regex.
// Here '^' control symbol used only in one role: Matching beginning of an input.

const fruitsStartsWithA = fruits.filter((fruit) => /^A/.test(fruit));
console.log(fruitsStartsWithA); // [ 'Apple', 'Avocado' ]
```

In the second example ^ is used both for matching at the beginning of input and for creating negated or complemented character class when used within character classes.

```js
const fruits = ["Apple", "Watermelon", "Orange", "Avocado", "Strawberry"];

// Selecting fruits that do not start by 'A' with a /^[^A]/ regex.
// In this example, two meanings of '^' control symbol are represented:
// 1) Matching beginning of the input
// 2) A negated or complemented character class: [^A]
// That is, it matches anything that is not enclosed in the square brackets.

const fruitsStartsWithNotA = fruits.filter((fruit) => /^[^A]/.test(fruit));

console.log(fruitsStartsWithNotA); // [ 'Watermelon', 'Orange', 'Strawberry' ]
```

See more examples in the input boundary assertion reference.

Matching a word boundary

In this example, we match fruit names containing a word that ends in "en" or "ed".

```js
const fruitsWithDescription = ["Red apple", "Orange orange", "Green Avocado"];

// Select descriptions that contains 'en' or 'ed' words endings:
const enEdSelection = fruitsWithDescription.filter((description) =>
  /(?:en|ed)\b/.test(description),
);

console.log(enEdSelection); // [ 'Red apple', 'Green Avocado' ]
```

See more examples in the word boundary assertion reference.

Lookahead assertion

In this example, we match the word "First" only if it is followed by the word "test", but we do not include "test" in the match results.

```js
const regex = /First(?= test)/g;

console.log("First test".match(regex)); // [ 'First' ]
console.log("First peach".match(regex)); // null
console.log("This is a First test in a year.".match(regex)); // [ 'First' ]
console.log("This is a First peach in a month.".match(regex)); // null
```

See more examples in the lookahead assertion reference.

Basic negative lookahead assertion

For example, `/\d+(?!\.)/` matches a number only if it is not followed by a decimal point. `/\d+(?!\.)/.exec('3.141')` matches "141" but not "3".

```js
console.log(/\d+(?!\.)/g.exec("3.141")); // [ '141', index: 2, input: '3.141' ]
```

See more examples in the lookahead assertion reference.

Different meaning of '?!' combination usage in assertions and character classes

The `?!` combination has different meanings in assertions like `/x(?!y)/` and character classes like `[^?!]`.

```js
const orangeNotLemon =
  "Do you want to have an orange? Yes, I do not want to have a lemon!";

// Different meaning of '?!' combination usage in Assertions /x(?!y)/ and Ranges /[^?!]/
const selectNotLemonRegex = /[^?!]+have(?! a lemon)[^?!]+[?!]/gi;
console.log(orangeNotLemon.match(selectNotLemonRegex)); // [ 'Do you want to have an
orange?' ]

const selectNotOrangeRegex = /[^?!]+have(?! an orange)[^?!]+[?!]/gi;
console.log(orangeNotLemon.match(selectNotOrangeRegex)); // [ ' Yes, I do not want to have a
lemon!' ]
```

Lookbehind assertion

In this example, we replace the word "orange" with "apple" only if it is preceded by the word "ripe".

```js
const oranges = ["ripe orange A", "green orange B", "ripe orange C"];

const newFruits = oranges.map((fruit) =>
  fruit.replace(/(?<=ripe )orange/, "apple"),
);
console.log(newFruits); // ['ripe apple A', 'green orange B', 'ripe apple C']
```

See more examples in the lookbehind assertion reference.

See also

- Regular expressions guide
- Character classes guide
- Quantifiers guide
- Groups and backreferences guide
- `RegExp`
- Regular expressions reference
- Input boundary assertion: `^`, `$`
- Lookahead assertion: `(?=...)`, `(?!...)`
- Lookbehind assertion: `(?<=...)`, `(?<!...)`
- Word boundary assertion: `\b`, `\B`

JavaScript › Guide › Regular expressions

Character classes

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions/Character_classes · MDN page last modified 2026-08-27

Character classes distinguish kinds of characters such as, for example, distinguishing between letters and digits.

Try it

▷ Interactive example on MDN: "JavaScript Demo: RegExp Character classes". It runs live in the browser there; its source code follows.

js · interactive example source

```
const chessStory = "He played the King in a8 and she moved her Queen in c2.";
const regexpCoordinates = /\w\d/g;
console.log(chessStory.match(regexpCoordinates));
// Expected output: Array [ 'a8', 'c2']

const moods = "happy ☺, confused ☹, sad ☹";
const regexpEmoticons = /[\u{1F600}-\u{1F64F}]/gu;
console.log(moods.match(regexpEmoticons));
// Expected output: Array ['☺', '☹', '☹']
```

Types

Characters	Meaning
`[xyz]` `[a-c]`	**Character class:** Matches any one of the enclosed characters. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first or last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[abcd]` is the same as `[a-d]`. They match the "b" in "brisket", and the "c" in "chop". For example, `[abcd-]` and `[-abcd]` match the "b" in "brisket", the "c" in "chop", and the "-" (hyphen) in "non-profit". For example, `[\w-]` is the same as `[A-Za-z0-9_-]`. They both match the "b" in "brisket", the "c" in "chop", and the "n" in "non-profit". When the `unicodeSets` (`v`) flag is enabled, the character class has some additional features. See the character class reference for more information.
`[^xyz]` `[^a-c]`	**Negated character class:** Matches anything that is not enclosed in the square brackets. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first character after the `^` or the last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[^abc]` is the same as `[^a-c]`. They initially match "o" in "bacon" and "h" in "chop". **Note:** The `^` character may also indicate the beginning of input.

Characters	Meaning
`.`	**Wildcard:** Matches any single character *except* line terminators: `\n`, `\r`, `\u2028` or `\u2029`. For example, `/.y/` matches "my" and "ay", but not "yes", in "yes make my day", as there is no character before "y" in "yes". If the `dotAll` (s) flag is enabled, also matches line terminators. Inside a character class, the dot loses its special meaning and matches a literal dot.
`\d`	**Digit character class escape:** Matches any digit (Arabic numeral). Equivalent to `[0-9]`. For example, `/\d/` or `/[0-9]/` matches "2" in "B2 is the suite number".
`\D`	**Non-digit character class escape:** Matches any character that is not a digit (Arabic numeral). Equivalent to `[^0-9]`. For example, `/\D/` or `/[^0-9]/` matches "B" in "B2 is the suite number".
`\w`	**Word character class escape:** Matches any alphanumeric character from the basic Latin alphabet, including the underscore. Equivalent to `[A-Za-z0-9_]`. For example, `/\w/` matches "a" in "apple", "5" in "$5.28", "3" in "3D" and "m" in "Émanuel".
`\W`	**Non-word character class escape:** Matches any character that is not a word character from the basic Latin alphabet. Equivalent to `[^A-Za-z0-9_]`. For example, `/\W/` or `/[^A-Za-z0-9_]/` matches "%" in "50%" and "É" in "Émanuel".
`\s`	**White space character class escape:** Matches a single white space character, including space, tab, form feed, line feed, and other Unicode spaces. Equivalent to `[\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\s\w*/` matches " bar" in "foo bar".
`\S`	**Non-white space character class escape:** Matches a single character other than white space. Equivalent to `[^\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\S\w*/` matches "foo" in "foo bar".
`\t`	Matches a horizontal tab.
`\r`	Matches a carriage return.
`\n`	Matches a linefeed.
`\v`	Matches a vertical tab.
`\f`	Matches a form-feed.
`[\b]`	Matches a backspace. If you're looking for the word-boundary assertion (`\b`), see Assertions.
`\0`	Matches a NUL character. Do not follow this with another digit.
`\c`*X*	Matches a control character using caret notation, where "X" is a letter from A–Z or a–z (corresponding to code points `U+0001`–`U+001A`). For example, `/\cM\cJ/` matches "\r\n".
`\x`*hh*	**Hex escape:** Matches the character with the code *hh* (two hexadecimal digits).
`\u`*HHHH*	**Unicode escape:** Matches a UTF-16 code-unit with the value *HHHH* (four hexadecimal digits).
`\u{`*H…H*`}`	**Unicode code point escape:** Matches the character with the Unicode value `U+`*H…H* (1 to 6 hexadecimal digits). Only valid in Unicode-aware mode.
`\p{`*UnicodeProperty*`}`, `\P{`*UnicodeProperty*`}`	**Unicode character class escape:** Matches a character based on its Unicode character properties: for example, emoji characters, or Japanese *katakana* characters, or Chinese/Japanese Han/Kanji characters, etc.). Only valid in Unicode-aware mode.

Characters	Meaning
\	Indicates that the following character should be treated specially, or "escaped". It behaves one of two ways. • For characters that are usually treated literally, indicates that the next character is special and not to be interpreted literally. For example, `/b/` matches the character "b". By placing a backslash in front of "b", that is by using `/\b/`, the character becomes special to mean match a word boundary. • For characters that are usually treated specially, indicates that the next character is not special and should be interpreted literally. For example, "*" is a special character that means 0 or more occurrences of the preceding character should be matched; for example, `/a*/` means match 0 or more "a"s. To match * literally, precede it with a backslash; for example, `/a\*/` matches "a*". **Note:** To match this character literally, escape it with itself. In other words to search for \ use `/\\/`.
x|*y*	**Disjunction:** Matches either "x" or "y". Each component, separated by a pipe (|), is called an *alternative*. For example, `/green|red/` matches "green" in "green apple" and "red" in "red apple". **Note:** A disjunction is another way to specify "a set of choices", but it's not a character class. Disjunctions are not atoms — you need to use a group to make it part of a bigger pattern. `[abc]` is functionally equivalent to `(?:a|b|c)`.

Examples

Looking for a series of digits

In this example, we match a sequence of 4 digits with `\d{4}`. `\b` indicates a word boundary (i.e., do not start or end matching in the middle of a number sequence).

```js
const randomData = "015 354 8787 687351 3512 8735";
const regexpFourDigits = /\b\d{4}\b/g;

console.table(randomData.match(regexpFourDigits));
// ['8787', '3512', '8735']
```

See more examples in the character class escape reference.

Looking for a word (from the latin alphabet) starting with A

In this example, we match a word starting with the letter A. `\b` indicates a word boundary (i.e., do not start matching in the middle of a word). `[aA]` indicates the letter "a" or "A". `\w+` indicates any character *from the Latin alphabet*, multiple times (+ is a quantifier). Note that because we already match until there are no more word characters, an end `\b` boundary is not necessary.

```js
const aliceExcerpt =
  "I'm sure I'm not Ada,' she said, 'for her hair goes in such long ringlets, and mine
doesn't go in ringlets at all.";
const regexpWordStartingWithA = /\b[aA]\w+/g;

console.table(aliceExcerpt.match(regexpWordStartingWithA));
// ['Ada', 'and', 'at', 'all']
```

See more examples in the character class escape reference.

Looking for a word (from Unicode characters)

Instead of the Latin alphabet, we can use a range of Unicode characters to identify a word (thus being able to deal with text in other languages like Russian or Arabic). The "Basic Multilingual Plane" of Unicode contains most of the characters used around the world and we can use character classes and ranges to match words written with those characters.

```js
const nonEnglishText = "Приключения Алисы в Стране чудес";
const regexpBMPWord = /([\u0000-\u0019\u0021-\uFFFF])+/gu;
// BMP goes through U+0000 to U+FFFF but space is U+0020

console.table(nonEnglishText.match(regexpBMPWord));
["Приключения", "Алисы", "в", "Стране", "чудес"];
```

See more examples in the Unicode character class escape reference.

Counting vowels

In this example, we count the number of vowels (A, E, I, O, U, Y) in a text. The `g` flag is used to match all occurrences of the pattern in the text. The `i` flag is used to make the pattern case-insensitive, so it matches both uppercase and lowercase vowels.

```js
const aliceExcerpt =
  "There was a long silence after this, and Alice could only hear whispers now and then.";
const regexpVowels = /[aeiouy]/gi;

console.log("Number of vowels:", aliceExcerpt.match(regexpVowels).length);
// Number of vowels: 26
```

See also

- Regular expressions guide
- Assertions guide
- Quantifiers guide
- Groups and backreferences guide
- `RegExp`
- Regular expressions reference
- Character class: `[...]`, `[^...]`
- Character class escape: `\d`, `\D`, `\w`, `\W`, `\s`, `\S`
- Character escape: `\n`, `\u{...}`
- Disjunction: `|`

- Unicode character class escape: `\p{...}`, `\P{...}`
- Wildcard: `.`

JavaScript › Guide › Regular expressions

Groups and backreferences

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions/Groups_and_backreferences · MDN page last modified 2026-01-08

Groups group multiple patterns as a whole, and capturing groups provide extra submatch information when using a regular expression pattern to match against a string. Backreferences refer to a previously captured group in the same regular expression.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp Groups and backreferences". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Groups
const imageDescription = "This image has a resolution of 1440×900 pixels.";
const regexpSize = /(\d+)×(\d+)/;
const match = imageDescription.match(regexpSize);
console.log(`Width: ${match[1]} / Height: ${match[2]}.`);
// Expected output: "Width: 1440 / Height: 900."

// Backreferences
const findDuplicates = "foo foo bar";
const regex = /\b(\w+)\s+\1\b/g;
console.log(findDuplicates.match(regex));
// Expected output: Array ["foo foo"]
```

Types

Characters	Meaning
`(x)`	**Capturing group:** Matches *x* and remembers the match. For example, `/(foo)/` matches and remembers "foo" in "foo bar". A regular expression may have multiple capturing groups. In results, matches to capturing groups typically in an array whose members are in the same order as the left parentheses in the capturing group. This is usually just the order of the capturing groups themselves. This becomes important when capturing groups are nested. Matches are accessed using the index of the result's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`). Capturing groups have a performance penalty. If you don't need the matched substring to be recalled, prefer non-capturing parentheses (see below). `String.prototype.match()` won't return groups if the `/.../g` flag is set. However, you can still use `String.prototype.matchAll()` to get all matches.
`(?<Name>x)`	**Named capturing group:** Matches "x" and stores it on the groups property of the returned matches under the name specified by `<Name>`. The angle brackets (< and >) are required for group name. For example, to extract the United States area code from a phone number, we could use `/\((?<area>\d\d\d)\)/`. The resulting number would appear under `matches.groups.area`.
`(?:x)`	**Non-capturing group:** Matches "x" but does not remember the match. The matched substring cannot be recalled from the resulting array's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`).

Characters	Meaning
`(?flags:x)`, `(?flags-flags:x)`	**Modifier:** Enables or disables the specified flags only to the enclosed pattern. Only the `i`, `m`, and `s` flags can be used in a modifier.
`\n`	**Backreference:** Where "n" is a positive integer. Matches the same substring matched by the nth capturing group in the regular expression (counting left parentheses). For example, `/apple(,)\sorange\1/` matches "apple, orange," in "apple, orange, cherry, peach".
`\k<Name>`	**Named backreference:** A back reference to the last substring matching the **Named capture group** specified by `<Name>`. For example, `/(?<title>\w+), yes \k<title>/` matches "Sir, yes Sir" in "Do you copy? Sir, yes Sir!". **Note:** `\k` is used literally here to indicate the beginning of a back reference to a Named capture group.

Examples

Using groups

In this example, we match two words in a structured format by using capturing groups to remember them. `\w+` matches one or more word characters, and the parentheses `()` create a capturing group. The `g` flag is used to match all occurrences.

```js
const personList = `First_Name: John, Last_Name: Doe
First_Name: Jane, Last_Name: Smith`;

const regexpNames = /First_Name: (\w+), Last_Name: (\w+)/g;
for (const match of personList.matchAll(regexpNames)) {
  console.log(`Hello ${match[1]} ${match[2]}`);
}
```

See more examples in the capturing group reference.

Using named groups

This example is the same as above, but we use named capturing groups to remember the matched words instead. This way, we can access the matched words by their meanings.

```js
const personList = `First_Name: John, Last_Name: Doe
First_Name: Jane, Last_Name: Smith`;

const regexpNames =
  /First_Name: (?<firstName>\w+), Last_Name: (?<lastName>\w+)/g;
for (const match of personList.matchAll(regexpNames)) {
  console.log(`Hello ${match.groups.firstName} ${match.groups.lastName}`);
}
```

See more examples in the named capturing group reference.

Using groups and back references

In this example, we first match a single or double quote character with `['"]`, remember it, match an arbitrary number of characters with `.*?` (`*?` is a non-greedy quantifier), until we match the remembered quote character again with `\1`. The `\1` is a backreference to the first capturing group, which matches the same type of quote. The result will therefore be two strings: `"'"` and `'"'`.

```js
const quote = `Single quote "'" and double quote '"'`;
const regexpQuotes = /(['"]).*?\1/g;
for (const match of quote.matchAll(regexpQuotes)) {
  console.log(match[0]);
}
```

See more examples in the backreference reference.

Using groups and match indices

By providing the `d` flag, the indices of each capturing group is returned. This is especially useful if you are correlating each matched group with the original text — for example, to provide compiler diagnostics.

```js
const code = `function add(x, y) {
  return x + y;
}`;
const functionRegexp =
  /(function\s+)(?<name>[$_\p{ID_Start}][$\p{ID_Continue}]*)/du;
const match = functionRegexp.exec(code);
const lines = code.split("\n");
lines.splice(
  1,
  0,
  " ".repeat(match.indices[1][1] - match.indices[1][0]) +
    "^".repeat(match.indices.groups.name[1] - match.indices.groups.name[0]),
);
console.log(lines.join("\n"));
// function add(x, y) {
//          ^^^
//   return x + y;
// }
```

See also

- Regular expressions guide
- Character classes guide
- Assertions guide
- Quantifiers guide
- RegExp
- Regular expressions reference
- Backreference: `\1`, `\2`
- Capturing group: `(...)`
- Named backreference: `\k<name>`
- Named capturing group: `(?<name>...)`

- Non-capturing group: `(?:...)`

JavaScript › Guide › Regular expressions

Quantifiers

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions/Quantifiers · MDN page last modified 2025-12-08

Quantifiers indicate numbers of characters or expressions to match.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: RegExp quantifiers". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const ghostSpeak = "booh booooooooh";
const regexpSpooky = /bo{3,}h/;
console.log(ghostSpeak.match(regexpSpooky));
// Expected output: Array ["boooooooh"]

const modifiedQuote = "[He] ha[s] to go read this novel [Alice in Wonderland].";
const regexpModifications = /\[.*?\]/g;
console.log(modifiedQuote.match(regexpModifications));
// Expected output: Array ["[He]", "[s]", "[Alice in Wonderland]"]

const regexpTooGreedy = /\[.*\]/g;
console.log(modifiedQuote.match(regexpTooGreedy));
// Expected output: Array ["[He] ha[s] to go read this novel [Alice in Wonderland]"]
```

Types

> **Note:** In the following, *item* refers not only to singular characters, but also includes character classes and groups and backreferences.

Characters	Meaning
*x**	Matches the preceding item "x" 0 or more times. For example, `/bo*/` matches "boooo" in "A ghost booooed" and "b" in "A bird warbled", but nothing in "A goat grunted".
x+	Matches the preceding item "x" 1 or more times. Equivalent to `{1,}`. For example, `/a+/` matches the "a" in "candy" and all the "a"'s in "caaaaaaandy".
x?	Matches the preceding item "x" 0 or 1 times. For example, `/e?le?/` matches the "el" in "angel" and the "le" in "angle." If used immediately after any of the quantifiers `*`, `+`, `?`, or `{}`, makes the quantifier non-greedy (matching the minimum number of times), as opposed to the default, which is greedy (matching the maximum number of times).
x{*n*}	Where "n" is a non-negative integer, matches exactly "n" occurrences of the preceding item "x". For example, `/a{2}/` doesn't match the "a" in "candy", but it matches all of the "a"'s in "caandy", and the first two "a"'s in "caaandy".
x{*n*,}	Where "n" is a non-negative integer, matches at least "n" occurrences of the preceding item "x". For example, `/a{2,}/` doesn't match the "a" in "candy", but matches all of the a's in "caandy" and in "caaaaaaandy".

Characters	Meaning
x{*n*,*m*}	Where "n" and "m" are non-negative integers and `m >= n`, matches at least "n" and at most "m" occurrences of the preceding item "x". For example, `/a{1,3}/` matches nothing in "cndy", the "a" in "candy", the two "a"'s in "caandy", and the first three "a"'s in "caaaaaaandy". Notice that when matching "caaaaaaandy", the match is "aaa", even though the original string had more "a"s in it.
*x**? *x*+? *x*?? *x*{n}? *x*{n,}? *x*{n,m}?	By default quantifiers like * and + are "greedy", meaning that they try to match as many times as possible. The ? character after the quantifier makes the quantifier "non-greedy": meaning that it will stop as soon as it finds the minimum number of matches. For example, given a string like "some <foo> <bar> new </bar> </foo> thing": • `/<.*>/` will match "<foo> <bar> new </bar> </foo>" • `/<.*?>/` will match "<foo>" **Note:** Adding `?` after `{n}` is syntactically valid but practically useless. Since `{n}` always matches exactly n times, `x{n}?` behaves the same as `x{n}`.

Examples

Repeated pattern

In this example, we match one or more word characters with `\w+`, then one or more characters "a" with `a+`, and finally end at a word boundary with `\b`.

```js
const wordEndingWithAs = /\w+a+\b/;
const delicateMessage = "This is Spartaaaaaaa";

console.table(delicateMessage.match(wordEndingWithAs)); // [ "Spartaaaaaaa" ]
```

Counting characters

In this example, we match words that have a single letter, words that have between 2 and 6 letters, and words that have 13 or more letters.

```js
const singleLetterWord = /\b\w\b/g;
const notSoLongWord = /\b\w{2,6}\b/g;
const longWord = /\b\w{13,}\b/g;

const sentence = "Why do I have to learn multiplication table?";

console.table(sentence.match(singleLetterWord)); // ["I"]
console.table(sentence.match(notSoLongWord)); // [ "Why", "do", "have", "to", "learn",
"table" ]
console.table(sentence.match(longWord)); // ["multiplication"]
```

Optional character

In this example, we match words that either end with "our" or "or".

```js
const britishText = "He asked his neighbour a favour.";
const americanText = "He asked his neighbor a favor.";

const regexpEnding = /\w+ou?r/g;
// \w+ One or several letters
// o   followed by an "o",
// u?  optionally followed by a "u"
// r   followed by an "r"

console.table(britishText.match(regexpEnding));
// ["neighbour", "favour"]

console.table(americanText.match(regexpEnding));
// ["neighbor", "favor"]
```

Greedy versus non-greedy

In this example, we match one or more word characters or spaces with `[\w ]+` and `[\w ]+?`. The first one is greedy and the second one is non-greedy. Note how the second one stops as soon as it meets the minimal requirement.

```js
const text = "I must be getting somewhere near the center of the earth.";
const greedyRegexp = /[\w ]+/;

console.log(text.match(greedyRegexp)[0]);
// "I must be getting somewhere near the center of the earth"
// almost all of the text matches (leaves out the dot character)

const nonGreedyRegexp = /[\w ]+?/; // Notice the question mark
console.log(text.match(nonGreedyRegexp));
// "I"
// The match is the smallest one possible
```

See also

- Regular expressions guide
- Character classes guide
- Assertions guide
- Groups and backreferences guide
- `RegExp`
- Regular expressions reference
- Quantifier: `*`, `+`, `?`, `{n}`, `{n,}`, `{n,m}`

JavaScript › Guide › Regular expressions

Regular expression syntax cheat sheet

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions/Cheatsheet · MDN page last modified 2026-08-27

This page provides an overall cheat sheet of all the capabilities of `RegExp` syntax by aggregating the content of the articles in the `RegExp` guide. If you need more information on a specific topic, please follow the link on the corresponding heading to access the full article or head to the guide.

Character classes

Character classes distinguish kinds of characters such as, for example, distinguishing between letters and digits.

Characters	Meaning
`[xyz]` `[a-c]`	**Character class:** Matches any one of the enclosed characters. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first or last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[abcd]` is the same as `[a-d]`. They match the "b" in "brisket", and the "c" in "chop". For example, `[abcd-]` and `[-abcd]` match the "b" in "brisket", the "c" in "chop", and the "-" (hyphen) in "non-profit". For example, `[\w-]` is the same as `[A-Za-z0-9_-]`. They both match the "b" in "brisket", the "c" in "chop", and the "n" in "non-profit".
`[^xyz]` `[^a-c]`	**Negated character class:** Matches anything that is not enclosed in the square brackets. You can specify a range of characters by using a hyphen, but if the hyphen appears as the first character after the `^` or the last character enclosed in the square brackets, it is taken as a literal hyphen to be included in the character class as a normal character. For example, `[^abc]` is the same as `[^a-c]`. They initially match "o" in "bacon" and "h" in "chop". **Note:** The `^` character may also indicate the beginning of input.
`.`	**Wildcard:** Matches any single character *except* line terminators: `\n`, `\r`, `\u2028` or `\u2029`. For example, `/.y/` matches "my" and "ay", but not "yes", in "yes make my day", as there is no character before "y" in "yes". If the `dotAll` (`s`) flag is enabled, also matches line terminators. Inside a character class, the dot loses its special meaning and matches a literal dot.
`\d`	**Digit character class escape:** Matches any digit (Arabic numeral). Equivalent to `[0-9]`. For example, `/\d/` or `/[0-9]/` matches "2" in "B2 is the suite number".
`\D`	**Non-digit character class escape:** Matches any character that is not a digit (Arabic numeral). Equivalent to `[^0-9]`. For example, `/\D/` or `/[^0-9]/` matches "B" in "B2 is the suite number".
`\w`	**Word character class escape:** Matches any alphanumeric character from the basic Latin alphabet, including the underscore. Equivalent to `[A-Za-z0-9_]`. For example, `/\w/` matches "a" in "apple", "5" in "$5.28", "3" in "3D" and "m" in "Émanuel".

Characters	Meaning
`\W`	**Non-word character class escape:** Matches any character that is not a word character from the basic Latin alphabet. Equivalent to `[^A-Za-z0-9_]`. For example, `/\W/` or `/[^A-Za-z0-9_]/` matches "%" in "50%" and "É" in "Émanuel".
`\s`	**White space character class escape:** Matches a single white space character, including space, tab, form feed, line feed, and other Unicode spaces. Equivalent to `[\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\s\w*/` matches " bar" in "foo bar".
`\S`	**Non-white space character class escape:** Matches a single character other than white space. Equivalent to `[^\f\n\r\t\v\u0020\u00a0\u1680\u2000-\u200a\u2028\u2029\u202f\u205f\u3000\ufeff]`. For example, `/\S\w*/` matches "foo" in "foo bar".
`\t`	Matches a horizontal tab.
`\r`	Matches a carriage return.
`\n`	Matches a linefeed.
`\v`	Matches a vertical tab.
`\f`	Matches a form-feed.
`[\b]`	Matches a backspace. If you're looking for the word-boundary assertion (`\b`), see Assertions.
`\0`	Matches a NUL character. Do not follow this with another digit.
`\c`*`X`*	Matches a control character using caret notation, where "X" is a letter from A–Z or a–z (corresponding to code points `U+0001`–`U+001A`). For example, `/\cM\cJ/` matches "\r\n".
`\x`*`hh`*	**Hex escape:** Matches the character with the code *hh* (two hexadecimal digits).
`\u`*`HHHH`*	**Unicode escape:** Matches a UTF-16 code-unit with the value *HHHH* (four hexadecimal digits).
`\u{`*`H…H`*`}`	**Unicode code point escape:** Matches the character with the Unicode value `U+`*`H…H`* (1 to 6 hexadecimal digits). Only valid in Unicode-aware mode.
`\p{`*`UnicodeProperty`*`}`, `\P{`*`UnicodeProperty`*`}`	**Unicode character class escape:** Matches a character based on its Unicode character properties: for example, emoji characters, or Japanese *katakana* characters, or Chinese/Japanese Han/Kanji characters, etc.). Only valid in Unicode-aware mode.
`\`	Indicates that the following character should be treated specially, or "escaped". It behaves one of two ways. • For characters that are usually treated literally, indicates that the next character is special and not to be interpreted literally. For example, `/b/` matches the character "b". By placing a backslash in front of "b", that is by using `/\b/`, the character becomes special to mean match a word boundary. • For characters that are usually treated specially, indicates that the next character is not special and should be interpreted literally. For example, "*" is a special character that means 0 or more occurrences of the preceding character should be matched; for example, `/a*/` means match 0 or more "a"s. To match * literally, precede it with a backslash; for example, `/a\*/` matches "a*". **Note:** To match this character literally, escape it with itself. In other words to search for `\` use `/\\/`.

Characters	Meaning
x|*y*	**Disjunction:** Matches either "x" or "y". Each component, separated by a pipe (|), is called an *alternative*. For example, `/green|red/` matches "green" in "green apple" and "red" in "red apple". **Note:** A disjunction is another way to specify "a set of choices", but it's not a character class. Disjunctions are not atoms — you need to use a group to make it part of a bigger pattern. `[abc]` is functionally equivalent to `(?:a|b|c)`.

Assertions

Assertions include boundaries, which indicate the beginnings and endings of lines and words, and other patterns indicating in some way that a match is possible (including look-ahead, look-behind, and conditional expressions).

Boundary-type assertions

Characters	Meaning
^	**Input boundary beginning assertion:** Matches the beginning of input. If the `multiline` (m) flag is enabled, also matches immediately after a line break character. For example, `/^A/` does not match the "A" in "an A", but does match the first "A" in "An A". **Note:** This character has a different meaning when it appears at the start of a character class.
$	**Input boundary end assertion:** Matches the end of input. If the `multiline` (m) flag is enabled, also matches immediately before a line break character. For example, `/t$/` does not match the "t" in "eater", but does match it in "eat".
\A	**Buffer boundary start assertion:** Matches the start of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
\z	**Buffer boundary end assertion:** Matches the end of the entire string regardless of the presence of the `m` flag. Only valid in Unicode-aware mode.
\Z	**Buffer boundary end assertion with optional newline:** Matches the end of the entire string, but allows an optional trailing newline character sequence (either a line terminator or a `\r\n` sequence). Only valid in Unicode-aware mode.

Characters	Meaning
`\b`	**Word boundary assertion:** Matches a word boundary. This is the position where a word character is not followed or preceded by another word-character, such as between a letter and a space. Note that a matched word boundary is not included in the match. In other words, the length of a matched word boundary is zero. Examples: • `/\bm/` matches the "m" in "moon". • `/oo\b/` does not match the "oo" in "moon", because "oo" is followed by "n" which is a word character. • `/oon\b/` matches the "oon" in "moon", because "oon" is the end of the string, thus not followed by a word character. • `/\w\b\w/` will never match anything, because a word character can never be followed by both a non-word and a word character. To match a backspace character (`[\b]`), see Character Classes.
`\B`	**Non-word-boundary assertion:** Matches a non-word boundary. This is a position where the previous and next character are of the same type: Either both must be words, or both must be non-words, for example between two letters or between two spaces. The beginning and end of a string are considered non-words. Same as the matched word boundary, the matched non-word boundary is also not included in the match. For example, `/\Bon/` matches "on" in "at noon", and `/ye\B/` matches "ye" in "possibly yesterday".

Other assertions

> **Note:** The `?` character may also be used as a quantifier.

Characters	Meaning
`x(?=y)`	**Lookahead assertion:** Matches "x" only if "x" is followed by "y". For example, `/Jack(?=Sprat)/` matches "Jack" only if it is followed by "Sprat". `/Jack(?=Sprat|Frost)/` matches "Jack" only if it is followed by "Sprat" or "Frost". However, neither "Sprat" nor "Frost" is part of the match results.
`x(?!y)`	**Negative lookahead assertion:** Matches "x" only if "x" is not followed by "y". For example, `/\d+(?!\.)/` matches a number only if it is not followed by a decimal point. `/\d+(?!\.)/.exec('3.141')` matches "141" but not "3".
`(?<=y)x`	**Lookbehind assertion:** Matches "x" only if "x" is preceded by "y". For example, `/(?<=Jack)Sprat/` matches "Sprat" only if it is preceded by "Jack". `/(?<=Jack|Tom)Sprat/` matches "Sprat" only if it is preceded by "Jack" or "Tom". However, neither "Jack" nor "Tom" is part of the match results.
`(?<!y)x`	**Negative lookbehind assertion:** Matches "x" only if "x" is not preceded by "y". For example, `/(?<!-)\d+/` matches a number only if it is not preceded by a minus sign. `/(?<!-)\d+/.exec('3')` matches "3". `/(?<!-)\d+/.exec('-3')` match is not found because the number is preceded by the minus sign.

Groups and backreferences

Groups and backreferences indicate groups of expression characters.

Characters	Meaning
`(x)`	**Capturing group:** Matches *x* and remembers the match. For example, `/(foo)/` matches and remembers "foo" in "foo bar". A regular expression may have multiple capturing groups. In results, matches to capturing groups typically in an array whose members are in the same order as the left parentheses in the capturing group. This is usually just the order of the capturing groups themselves. This becomes important when capturing groups are nested. Matches are accessed using the index of the result's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`). Capturing groups have a performance penalty. If you don't need the matched substring to be recalled, prefer non-capturing parentheses (see below). `String.prototype.match()` won't return groups if the `/.../g` flag is set. However, you can still use `String.prototype.matchAll()` to get all matches.
`(?<Name>x)`	**Named capturing group:** Matches "x" and stores it on the groups property of the returned matches under the name specified by `<Name>`. The angle brackets (`<` and `>`) are required for group name. For example, to extract the United States area code from a phone number, we could use `/\((?<area>\d\d\d)\)/`. The resulting number would appear under `matches.groups.area`.
`(?:x)`	**Non-capturing group:** Matches "x" but does not remember the match. The matched substring cannot be recalled from the resulting array's elements (`[1], …, [n]`) or from the predefined `RegExp` object's properties (`$1, …, $9`).
`(?flags:x)`, `(?flags-flags:x)`	**Modifier:** Enables or disables the specified flags only to the enclosed pattern. Only the `i`, `m`, and `s` flags can be used in a modifier.
`\n`	**Backreference:** Where "n" is a positive integer. Matches the same substring matched by the nth capturing group in the regular expression (counting left parentheses). For example, `/apple(,)\sorange\1/` matches "apple, orange," in "apple, orange, cherry, peach".
`\k<Name>`	**Named backreference:** A back reference to the last substring matching the **Named capture group** specified by `<Name>`. For example, `/(?<title>\w+), yes \k<title>/` matches "Sir, yes Sir" in "Do you copy? Sir, yes Sir!". > **Note:** `\k` is used literally here to indicate the beginning of a back reference to a Named capture group.

Quantifiers

Quantifiers indicate numbers of characters or expressions to match.

> **Note:** In the following, *item* refers not only to singular characters, but also includes character classes and groups and backreferences.

Characters	Meaning
`x*`	Matches the preceding item "x" 0 or more times. For example, `/bo*/` matches "boooo" in "A ghost booooed" and "b" in "A bird warbled", but nothing in "A goat grunted".
`x+`	Matches the preceding item "x" 1 or more times. Equivalent to `{1,}`. For example, `/a+/` matches the "a" in "candy" and all the "a"'s in "caaaaaaandy".

Characters	Meaning
x?	Matches the preceding item "x" 0 or 1 times. For example, `/e?le?/` matches the "el" in "angel" and the "le" in "angle." If used immediately after any of the quantifiers *, +, ?, or {}, makes the quantifier non-greedy (matching the minimum number of times), as opposed to the default, which is greedy (matching the maximum number of times).
x{*n*}	Where "n" is a non-negative integer, matches exactly "n" occurrences of the preceding item "x". For example, `/a{2}/` doesn't match the "a" in "candy", but it matches all of the "a"'s in "caandy", and the first two "a"'s in "caaandy".
x{*n*,}	Where "n" is a non-negative integer, matches at least "n" occurrences of the preceding item "x". For example, `/a{2,}/` doesn't match the "a" in "candy", but matches all of the a's in "caandy" and in "caaaaaaandy".
x{*n*,*m*}	Where "n" and "m" are non-negative integers and `m >= n`, matches at least "n" and at most "m" occurrences of the preceding item "x". For example, `/a{1,3}/` matches nothing in "cndy", the "a" in "candy", the two "a"'s in "caandy", and the first three "a"'s in "caaaaaaandy". Notice that when matching "caaaaaaandy", the match is "aaa", even though the original string had more "a"s in it.
*x**? *x*+? *x*?? *x*{n}? *x*{n,}? *x*{n,m}?	By default quantifiers like * and + are "greedy", meaning that they try to match as many times as possible. The ? character after the quantifier makes the quantifier "non-greedy": meaning that it will stop as soon as it finds the minimum number of matches. For example, given a string like "some <foo> <bar> new </bar> </foo> thing": • `/<.*>/` will match "<foo> <bar> new </bar> </foo>" • `/<.*?>/` will match "<foo>" **Note:** Adding ? after {n} is syntactically valid but practically useless. Since {n} always matches exactly n times, x{n}? behaves the same as x{n}.

JavaScript › Guide

Indexed collections

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Indexed_collections · MDN page last modified 2026-09-04

This chapter introduces collections of data which are ordered by an index value. This includes arrays and array-like constructs such as `Array` objects and `TypedArray` objects.

An *array* is an ordered list of values that you refer to with a name and an index.

For example, consider an array called `emp`, which contains employees' names indexed by their numerical employee number. So `emp[0]` would be employee number zero, `emp[1]` employee number one, and so on.

JavaScript does not have an explicit array data type. However, you can use the predefined `Array` object and its methods to work with arrays in your applications. The `Array` object has methods for manipulating arrays in various ways, such as joining, reversing, and sorting them. It has a property for determining the array length and other properties for use with regular expressions.

We will be focusing on arrays in this article, but many of the same concepts apply to typed arrays as well, since arrays and typed arrays share many similar methods. For more information on typed arrays, see the typed array guide.

Creating an array

The following statements create equivalent arrays:

```js
const arr1 = new Array(element0, element1, /* …, */ elementN);
const arr2 = Array(element0, element1, /* …, */ elementN);
const arr3 = [element0, element1, /* …, */ elementN];
```

`element0, element1, …, elementN` is a list of values for the array's elements. When these values are specified, the array is initialized with them as the array's elements. The array's `length` property is set to the number of arguments.

The bracket syntax is called an "array literal" or "array initializer." It's shorter than other forms of array creation, and so is generally preferred. See Array literals for details.

To create an array with non-zero length, but without any items, either of the following can be used:

```js
// This...
const arr1 = new Array(arrayLength);

// … results in the same array as this
const arr2 = Array(arrayLength);

// This has exactly the same effect
const arr3 = [];
arr3.length = arrayLength;
```

> **Note:** In the above code, `arrayLength` must be a `Number`. Otherwise, an array with a single element (the provided value) will be created. Calling `arr.length` will return `arrayLength`, but the array doesn't contain any elements. A `for...in` loop will not find any property on the array.

In addition to a newly defined variable as shown above, arrays can also be assigned as a property of a new or an existing object:

```js
const obj = {};
// …
obj.prop = [element0, element1, /* …, */ elementN];

// OR
const obj = { prop: [element0, element1, /* …, */ elementN] };
```

If you wish to initialize an array with a single element, and the element happens to be a `Number`, you must use the bracket syntax. When a single `Number` value is passed to the `Array()` constructor or function, it is interpreted as an `arrayLength`, not as a single element.

This creates an array with only one element: the number 42.

```js
const arr = [42];
```

This creates an array with no elements and `arr.length` set to 42.

```js
const arr = Array(42);
```

This is equivalent to:

```js
const arr = [];
arr.length = 42;
```

Calling `Array(N)` results in a `RangeError`, if `N` is a non-whole number whose fractional portion is non-zero. The following example illustrates this behavior.

```js
const arr = Array(9.3); // RangeError: Invalid array length
```

If your code needs to create arrays with single elements of an arbitrary data type, it is safer to use array literals. Alternatively, create an empty array first before adding the single element to it.

You can also use the `Array.of` static method to create arrays with single element.

```js
const arr = Array.of(9.3); // arr contains only one element 9.3
```

Referring to array elements

Because elements are also properties, you can access them using property accessors. Suppose you define the following array:

```js
const myArray = ["Wind", "Rain", "Fire"];
```

You can refer to the first element of the array as `myArray[0]`, the second element of the array as `myArray[1]`, etc. The index of the elements begins with zero.

Note: You can also use property accessors to access other properties of the array, like with an object.

```js
const arr = ["one", "two", "three"];
arr[2]; // three
arr["length"]; // 3
```

Populating an array

You can populate an array by assigning values to its elements. For example:

```js
const emp = [];
emp[0] = "Casey Jones";
emp[1] = "Phil Lesh";
emp[2] = "August West";
```

Note: If you supply a non-integer value to the array operator in the code above, a property will be created in the object representing the array, instead of an array element.

```js
const arr = [];
arr[3.4] = "Oranges";
console.log(arr.length); // 0
console.log(Object.hasOwn(arr, 3.4)); // true
```

You can also populate an array when you create it:

```js
const myArray = new Array("Hello", myVar, 3.14159);
// OR
const myArray = ["Mango", "Apple", "Orange"];
```

Understanding length

At the implementation level, JavaScript's arrays actually store their elements as standard object properties, using the array index as the property name.

The `length` property is special. Its value is always a positive integer greater than the index of the last element if one exists. (In the example below, `'Dusty'` is indexed at `30`, so `cats.length` returns `30 + 1`).

Remember, JavaScript Array indexes are 0-based: they start at `0`, not `1`. This means that the `length` property will be one more than the highest index stored in the array:

```js
const cats = [];
cats[30] = ["Dusty"];
console.log(cats.length); // 31
```

You can also assign to the `length` property.

Writing a value that is shorter than the number of stored items truncates the array. Writing `0` empties it entirely:

```js
const cats = ["Dusty", "Misty", "Twiggy"];
console.log(cats.length); // 3

cats.length = 2;
console.log(cats); // [ 'Dusty', 'Misty' ] - Twiggy has been removed

cats.length = 0;
console.log(cats); // []; the cats array is empty

cats.length = 3;
console.log(cats); // [ <3 empty items> ]
```

Iterating over arrays

A common operation is to iterate over the values of an array, processing each one in some way, as follows:

```js
const colors = ["red", "green", "blue"];
for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}
```

If you know that none of the elements in your array evaluate to `false` in a boolean context—if your array consists only of DOM nodes, for example—you can use a more efficient idiom:

```js
const divs = document.getElementsByTagName("div");
for (let i = 0, div; (div = divs[i]); i++) {
  /* Process div in some way */
}
```

This avoids the overhead of checking the length of the array, and ensures that the `div` variable is reassigned to the current item each time around the loop for added convenience.

The `forEach()` method provides another way of iterating over an array:

```js
const colors = ["red", "green", "blue"];
colors.forEach((color) => console.log(color));
// red
// green
// blue
```

The function passed to `forEach` is executed once for every item in the array, with the array item passed as the argument to the function. Unassigned values are not iterated in a `forEach` loop.

Note that the elements of an array that are omitted when the array is defined are not listed when iterating by `forEach`, but *are* listed when `undefined` has been manually assigned to the element:

```js
const sparseArray = ["first", "second", , "fourth"];
```

(code continues on the next page)

(code continued from the previous page)

```
sparseArray.forEach((element) => {
  console.log(element);
});
// Logs:
// first
// second
// fourth

if (sparseArray[2] === undefined) {
  console.log("sparseArray[2] is undefined"); // true
}

const nonsparseArray = ["first", "second", undefined, "fourth"];

nonsparseArray.forEach((element) => {
  console.log(element);
});
// Logs:
// first
// second
// undefined
// fourth
```

Since JavaScript array elements are saved as standard object properties, it is not advisable to iterate through JavaScript arrays using `for...in` loops, because normal elements and all enumerable properties will be listed.

Array methods

The `Array` object has the following methods:

The `concat()` method joins two or more arrays and returns a new array.

js

```
let myArray = ["1", "2", "3"];
myArray = myArray.concat("a", "b", "c");
// myArray is now ["1", "2", "3", "a", "b", "c"]
```

The `join()` method joins all elements of an array into a string.

js

```
const myArray = ["Wind", "Rain", "Fire"];
const list = myArray.join(" - "); // list is "Wind - Rain - Fire"
```

The `push()` method adds one or more elements to the end of an array and returns the resulting `length` of the array.

js

```
const myArray = ["1", "2"];
myArray.push("3"); // myArray is now ["1", "2", "3"]
```

The `pop()` method removes the last element from an array and returns that element.

js

```
const myArray = ["1", "2", "3"];
const last = myArray.pop();
// myArray is now ["1", "2"], last = "3"
```

The `shift()` method removes the first element from an array and returns that element.

```js
const myArray = ["1", "2", "3"];
const first = myArray.shift();
// myArray is now ["2", "3"], first is "1"
```

The `unshift()` method adds one or more elements to the front of an array and returns the new length of the array.

```js
const myArray = ["1", "2", "3"];
myArray.unshift("4", "5");
// myArray becomes ["4", "5", "1", "2", "3"]
```

The `slice()` method extracts a section of an array and returns a new array.

```js
let myArray = ["a", "b", "c", "d", "e"];
myArray = myArray.slice(1, 4); // [ "b", "c", "d"]
// starts at index 1 and extracts all elements
// until index 3
```

The `at()` method returns the element at the specified index in the array, or `undefined` if the index is out of range. It's notably used for negative indices that access elements from the end of the array.

```js
const myArray = ["a", "b", "c", "d", "e"];
myArray.at(-2); // "d", the second-last element of myArray
```

The `splice()` method removes elements from an array and (optionally) replaces them. It returns the items which were removed from the array.

```js
const myArray = ["1", "2", "3", "4", "5"];
myArray.splice(1, 3, "a", "b", "c", "d");
// myArray is now ["1", "a", "b", "c", "d", "5"]
// This code started at index one (or where the "2" was),
// removed 3 elements there, and then inserted all consecutive
// elements in its place.
```

The `reverse()` method transposes the elements of an array, in place: the first array element becomes the last and the last becomes the first. It returns a reference to the array.

```js
const myArray = ["1", "2", "3"];
myArray.reverse();
// transposes the array so that myArray = ["3", "2", "1"]
```

The `flat()` method returns a new array with all sub-array elements concatenated into it recursively up to the specified depth.

```js
let myArray = [1, 2, [3, 4]];
myArray = myArray.flat();
// myArray is now [1, 2, 3, 4], since the [3, 4] subarray is flattened
```

The `sort()` method sorts the elements of an array in place, and returns a reference to the array.

```js
const myArray = ["Wind", "Rain", "Fire"];
myArray.sort();
// sorts the array so that myArray = ["Fire", "Rain", "Wind"]
```

`sort()` can also take a callback function to determine how array elements are compared. The callback function is called with two arguments, which are two values from the array. The function compares these two values and returns a positive number, negative number, or zero, indicating the order of the two values. For instance, the following will sort the array by the last letter of a string:

```js
const sortFn = (a, b) => {
  if (a[a.length - 1] < b[b.length - 1]) {
    return -1; // Negative number => a < b, a comes before b
  } else if (a[a.length - 1] > b[b.length - 1]) {
    return 1; // Positive number => a > b, a comes after b
  }
  return 0; // Zero => a = b, a and b keep their original order
};
myArray.sort(sortFn);
// sorts the array so that myArray = ["Wind","Fire","Rain"]
```

- if `a` is less than `b` by the sorting system, return `-1` (or any negative number)
- if `a` is greater than `b` by the sorting system, return `1` (or any positive number)
- if `a` and `b` are considered equivalent, return `0`.

The `indexOf()` method searches the array for `searchElement` and returns the index of the first match.

```js
const a = ["a", "b", "a", "b", "a"];
console.log(a.indexOf("b")); // 1

// Now try again, starting from after the last match
console.log(a.indexOf("b", 2)); // 3
console.log(a.indexOf("z")); // -1, because 'z' was not found
```

The `lastIndexOf()` method works like `indexOf`, but starts at the end and searches backwards.

```js
const a = ["a", "b", "c", "d", "a", "b"];
console.log(a.lastIndexOf("b")); // 5

// Now try again, starting from before the last match
console.log(a.lastIndexOf("b", 4)); // 1
console.log(a.lastIndexOf("z")); // -1
```

The `forEach()` method executes `callback` on every array item and returns `undefined`.

```js
const a = ["a", "b", "c"];
a.forEach((element) => {
  console.log(element);
});
// Logs:
// a
// b
// c
```

The `forEach` method (and others below) that take a callback are known as *iterative methods*, because they iterate over the entire array in some fashion. Each one takes an optional second argument called `thisArg`. If provided, `thisArg` becomes the value of the `this` keyword inside the body of the callback function. If not provided, as with other cases where a function is invoked outside of an explicit object context, `this` will refer to the global object (`window`, `globalThis`, etc.) when the function is not strict, or `undefined` when the function is strict.

> **Note:** The `sort()` method introduced above is not an iterative method, because its callback function is only used for comparison and may not be called in any particular order based on element order. `sort()` does not accept the `thisArg` parameter either.

The `map()` method returns a new array of the return value from executing `callback` on every array item.

```js
const a1 = ["a", "b", "c"];
const a2 = a1.map((item) => item.toUpperCase());
console.log(a2); // ['A', 'B', 'C']
```

The `flatMap()` method runs `map()` followed by a `flat()` of depth 1.

```js
const a1 = ["a", "b", "c"];
const a2 = a1.flatMap((item) => [item.toUpperCase(), item.toLowerCase()]);
console.log(a2); // ['A', 'a', 'B', 'b', 'C', 'c']
```

The `filter()` method returns a new array containing the items for which `callback` returned `true`.

```js
const a1 = ["a", 10, "b", 20, "c", 30];
const a2 = a1.filter((item) => typeof item === "number");
console.log(a2); // [10, 20, 30]
```

The `find()` method returns the first item for which `callback` returned `true`.

```js
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.find((item) => typeof item === "number");
console.log(i); // 10
```

The `findLast()` method returns the last item for which `callback` returned `true`.

```js
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.findLast((item) => typeof item === "number");
console.log(i); // 30
```

The `findIndex()` method returns the index of the first item for which `callback` returned `true`.

```js
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.findIndex((item) => typeof item === "number");
console.log(i); // 1
```

The `findLastIndex()` method returns the index of the last item for which `callback` returned `true`.

```js
const a1 = ["a", 10, "b", 20, "c", 30];
const i = a1.findLastIndex((item) => typeof item === "number");
console.log(i); // 5
```

The `every()` method returns `true` if `callback` returns `true` for every item in the array.

```js
function isNumber(value) {
  return typeof value === "number";
}
const a1 = [1, 2, 3];
console.log(a1.every(isNumber)); // true
const a2 = [1, "2", 3];
console.log(a2.every(isNumber)); // false
```

The `some()` method returns `true` if `callback` returns `true` for at least one item in the array.

```js
function isNumber(value) {
  return typeof value === "number";
}
const a1 = [1, 2, 3];
console.log(a1.some(isNumber)); // true
const a2 = [1, "2", 3];
console.log(a2.some(isNumber)); // true
const a3 = ["1", "2", "3"];
console.log(a3.some(isNumber)); // false
```

The `reduce()` method applies `callback(accumulator, currentValue, currentIndex, array)` for each value in the array for the purpose of reducing the list of items down to a single value. The `reduce` function returns the final value returned by `callback` function.

If `initialValue` is specified, then `callback` is called with `initialValue` as the first parameter value and the value of the first item in the array as the second parameter value.

If `initialValue` is *not* specified, then `callback`'s first two parameter values will be the first and second elements of the array. On *every* subsequent call, the first parameter's value will be whatever `callback` returned on the previous call, and the second parameter's value will be the next value in the array.

If `callback` needs access to the index of the item being processed, or access to the entire array, they are available as optional parameters.

```js
const a = [10, 20, 30];
const total = a.reduce(
  (accumulator, currentValue) => accumulator + currentValue,
  0,
);
console.log(total); // 60
```

The `reduceRight()` method works like `reduce()`, but starts with the last element.

`reduce` and `reduceRight` are the least obvious of the iterative array methods. They should be used for algorithms that combine two values recursively in order to reduce a sequence down to a single value.

Array transformations

You can transform back and forth between arrays and other data structures.

Grouping the elements of an array

The `Object.groupBy()` method can be used to group the elements of an array, using a test function that returns a string indicating the group of the current element.

Here we have an inventory array that contains "food" objects that have a `name` and a `type`.

```js
const inventory = [
  { name: "asparagus", type: "vegetables" },
  { name: "bananas", type: "fruit" },
  { name: "goat", type: "meat" },
  { name: "cherries", type: "fruit" },
  { name: "fish", type: "meat" },
];
```

To use `Object.groupBy()`, you supply a callback function that is called with the current element, and optionally the current index and array, and returns a string indicating the group of the element.

The code below uses an arrow function to return the `type` of each array element (this uses object destructuring syntax for function arguments to unpack the `type` element from the passed object). The result is an object that has properties named after the unique strings returned by the callback. Each property is assigned an array containing the elements in the group.

```js
const result = Object.groupBy(inventory, ({ type }) => type);
console.log(result);
// Logs
// {
//   vegetables: [{ name: 'asparagus', type: 'vegetables' }],
//   fruit: [
//     { name: 'bananas', type: 'fruit' },
//     { name: 'cherries', type: 'fruit' }
//   ],
//   meat: [
//     { name: 'goat', type: 'meat' },
//     { name: 'fish', type: 'meat' }
//   ]
// }
```

Note that the returned object references the *same* elements as the original array (not deep copies). Changing the internal structure of these elements will be reflected in both the original array and the returned object.

If you can't use a string as the key, for example, if the information to group is associated with an object that might change, then you can instead use `Map.groupBy()`. This is very similar to `Object.groupBy()` except that it groups the elements of the array into a `Map` that can use an arbitrary value (object or primitive) as a key.

Sparse arrays

Arrays can contain "empty slots", which are not the same as slots filled with the value `undefined`. Empty slots can be created in one of the following ways:

```js
// Array constructor:
const a = Array(5); // [ <5 empty items> ]

// Consecutive commas in array literal:
const b = [1, 2, , , 5]; // [ 1, 2, <2 empty items>, 5 ]

// Directly setting a slot with index greater than array.length:
const c = [1, 2];
c[4] = 5; // [ 1, 2, <2 empty items>, 5 ]

// Elongating an array by directly setting .length:
const d = [1, 2];
d.length = 5; // [ 1, 2, <3 empty items> ]

// Deleting an element:
const e = [1, 2, 3, 4, 5];
delete e[2]; // [ 1, 2, <1 empty item>, 4, 5 ]
```

In some operations, empty slots behave as if they are filled with `undefined`.

```js
const arr = [1, 2, , , 5]; // Create a sparse array

// Indexed access
console.log(arr[2]); // undefined

// For...of
for (const i of arr) {
  console.log(i);
}
// Logs: 1 2 undefined undefined 5

// Spreading
const another = [...arr]; // "another" is [ 1, 2, undefined, undefined, 5 ]
```

But in others (most notably array iteration methods), empty slots are skipped.

```js
const mapped = arr.map((i) => i + 1); // [ 2, 3, <2 empty items>, 6 ]
arr.forEach((i) => console.log(i)); // 1 2 5
const filtered = arr.filter(() => true); // [ 1, 2, 5 ]
const hasFalsy = arr.some((k) => !k); // false

// Property enumeration
const keys = Object.keys(arr); // [ '0', '1', '4' ]
for (const key in arr) {
  console.log(key);
}
// Logs: '0' '1' '4'
// Spreading into an object uses property enumeration, not the array's iterator
const objectSpread = { ...arr }; // { '0': 1, '1': 2, '4': 5 }
```

For a complete list of how array methods behave with sparse arrays, see the `Array` reference page.

Multi-dimensional arrays

Arrays can be nested, meaning that an array can contain another array as an element. Using this characteristic of JavaScript arrays, multi-dimensional arrays can be created.

The following code creates a two-dimensional array.

```js
const a = new Array(4);
for (let i = 0; i < 4; i++) {
  a[i] = new Array(4);
  for (let j = 0; j < 4; j++) {
    a[i][j] = `[${i}, ${j}]`;
  }
}
```

This example creates an array with the following rows:

```
Row 0: [0, 0] [0, 1] [0, 2] [0, 3]
Row 1: [1, 0] [1, 1] [1, 2] [1, 3]
Row 2: [2, 0] [2, 1] [2, 2] [2, 3]
Row 3: [3, 0] [3, 1] [3, 2] [3, 3]
```

Using arrays to store other properties

Arrays can also be used like objects, to store related information.

```js
const arr = [1, 2, 3];
arr.property = "value";
console.log(arr.property); // "value"
```

For example, when an array is the result of a match between a regular expression and a string, the array returns properties and elements that provide information about the match. An array is the return value of `RegExp.prototype.exec()`, `String.prototype.match()`, and `String.prototype.split()`. For information on using arrays with regular expressions, see Regular Expressions.

Working with array-like objects

Some JavaScript objects, such as the `NodeList` returned by `document.getElementsByTagName()` or the `arguments` object made available within the body of a function, look and behave like arrays on the surface but do not share all of their methods. The `arguments` object provides a `length` attribute but does not implement array methods like `forEach()`.

Array methods cannot be called directly on array-like objects.

js · ✗ bad example

```
function printArguments() {
  arguments.forEach((item) => {
    console.log(item);
  }); // TypeError: arguments.forEach is not a function
}
```

But you can call them indirectly using `Function.prototype.call()`.

js · ✓ good example

```
function printArguments() {
  Array.prototype.forEach.call(arguments, (item) => {
    console.log(item);
  });
}
```

Array prototype methods can be used on strings as well, since they provide sequential access to their characters in a similar way to arrays:

js

```
Array.prototype.forEach.call("a string", (chr) => {
  console.log(chr);
});
```

JavaScript › Guide

Keyed collections

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Keyed_collections · MDN page last modified 2026-08-11

This chapter introduces collections of data which are indexed by a key; `Map` and `Set` objects contain elements which are iterable in the order of insertion.

Maps

Map object

A `Map` object is a key/value map that can iterate its elements in insertion order.

The following code shows some basic operations with a `Map`. See also the `Map` reference page for more examples and the complete API. You can use a `for...of` loop to return an array of `[key, value]` for each iteration.

```js
const sayings = new Map();
sayings.set("dog", "woof");
sayings.set("cat", "meow");
sayings.set("elephant", "toot");
sayings.size; // 3
sayings.get("dog"); // woof
sayings.get("fox"); // undefined
sayings.has("bird"); // false
sayings.delete("dog");
sayings.has("dog"); // false

for (const [key, value] of sayings) {
  console.log(`${key} goes ${value}`);
}
// "cat goes meow"
// "elephant goes toot"

sayings.clear();
sayings.size; // 0
```

Object and Map compared

Traditionally, objects have been used to map strings to values. Objects allow you to set keys to values, retrieve those values, delete keys, and detect whether something is stored at a key. `Map` objects, however, have a few more advantages that make them better maps.

- The keys of an `Object` are strings or symbols, whereas they can be of any value for a `Map`.
- You can get the `size` of a `Map` easily, while you have to manually keep track of size for an `Object`.
- The iteration of maps is in insertion order of the elements.
- An `Object` has a prototype, so there are default keys in the map. (This can be bypassed using `map = Object.create(null)`.)

These three tips can help you to decide whether to use a `Map` or an `Object`:

- Use maps over objects when keys are unknown until run time, especially if the keys come from external input.

- Maps support keys and values of any type and do not require keys to be serializable to strings or symbols.
- Use objects when the shape is known ahead of time and all keys are expressible as strings.

WeakMap object

A `WeakMap` is a collection of key/value pairs whose keys must be objects or non-registered symbols, with values of any arbitrary JavaScript type, and which does not create strong references to its keys. That is, an object's presence as a key in a `WeakMap` does not prevent the object from being garbage collected. Once an object used as a key has been collected, its corresponding values in any `WeakMap` become candidates for garbage collection as well — as long as they aren't strongly referred to elsewhere. The only primitive type that can be used as a `WeakMap` key is symbol — more specifically, non-registered symbols — because non-registered symbols are guaranteed to be unique and cannot be re-created.

The `WeakMap` API is essentially the same as the `Map` API. However, a `WeakMap` doesn't allow observing the liveness of its keys, which is why it doesn't allow enumeration. So there is no method to obtain a list of the keys in a `WeakMap`. If there were, the list would depend on the state of garbage collection, introducing non-determinism.

For more information and example code, see also "Why WeakMap?" on the `WeakMap` reference page.

One use case of `WeakMap` objects is to store private data for an object, or to hide implementation details. In the following example, the private data and methods belong inside the object and are stored in the `privates` object, which is a `WeakMap`. Everything exposed on the instance and prototype is public; everything else is inaccessible from the outside world because `privates` is not exported from the module.

```js
const privates = new WeakMap();

export default function Public() {
  const me = {
    // Private data goes here
  };
  privates.set(this, me);
}

Public.prototype.method = function () {
  const me = privates.get(this);
  // Do stuff with private data in `me`
  // …
};
```

> **Note:** This use case can now be implemented with classes and private fields.

Sets

Set object

`Set` objects are collections of unique values. You can iterate its elements in insertion order. A value in a `Set` may only occur once; it is unique in the `Set`'s collection.

The following code shows some basic operations with a `Set`. See also the `Set` reference page for more examples and the complete API.

```js
const mySet = new Set();
mySet.add(1);
mySet.add("some text");
mySet.add("foo");

mySet.has(1); // true
mySet.delete("foo");
mySet.size; // 2

for (const item of mySet) {
  console.log(item);
}
// 1
// "some text"
```

Converting between Array and Set

You can create an `Array` from a Set using `Array.from` or the spread syntax. Also, the `Set` constructor accepts an `Array` to convert in the other direction.

> **Note:** `Set` objects store *unique values*—so any duplicate elements from an Array are deleted when converting!

```js
Array.from(mySet);
[...mySet2];

mySet2 = new Set([1, 2, 3, 4]);
```

Array and Set compared

Traditionally, a set of elements has been stored in arrays in JavaScript in a lot of situations. The `Set` object, however, has some advantages:

- Deleting Array elements by value (`arr.splice(arr.indexOf(val), 1)`) is very slow.
- `Set` objects let you delete elements by their value. With an array, you would have to `splice` based on an element's index.
- The value `NaN` cannot be found with `indexOf` in an array.
- `Set` objects store unique values. You don't have to manually keep track of duplicates.

WeakSet object

`WeakSet` objects are collections of garbage-collectable values, including objects and non-registered symbols. A value in the `WeakSet` may only occur once. It is unique in the `WeakSet`'s collection.

The main differences to the `Set` object are:

- In contrast to `Sets`, `WeakSets` are **collections of *objects or symbols only***, and not of arbitrary values of any type.
- The `WeakSet` is *weak*: References to objects in the collection are held weakly. If there is no other reference to an object stored in the `WeakSet`, they can be garbage collected. That also means that there is no list of current objects stored in the collection.
- `WeakSets` are not enumerable.

The use cases of `WeakSet` objects are limited. They will not leak memory, so it can be safe to use DOM elements as a key and mark them for tracking purposes, for example.

Key and value equality of Map and Set

Both the key equality of `Map` objects and the value equality of `Set` objects are based on the SameValueZero algorithm:

- Equality works like the identity comparison operator `===`.
- `-0` and `+0` are considered equal.
- `NaN` is considered equal to itself (contrary to `===`).

JavaScript › Guide

Working with objects

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects · MDN page last modified 2026-08-21

JavaScript is designed on an object-based paradigm. An object is a collection of properties, and a property is an association between a name (or *key*) and a value. A property's value can be a function, in which case the property is known as a method.

Objects in JavaScript, just as in many other programming languages, can be compared to objects in real life. In JavaScript, an object is a standalone entity, with properties and type. Compare it with a cup, for example. A cup is an object with properties. A cup has a color, a design, weight, a material it is made of, etc. In the same way, JavaScript objects can have properties, which define their characteristics.

In addition to objects that are predefined in the browser, you can define your own objects. This chapter describes how to use objects, properties, and methods, and how to create your own objects.

Creating new objects

You can create an object using an object initializer. Alternatively, you can first create a constructor function and then instantiate an object by invoking that function with the `new` operator.

Using object initializers

Object initializers are also called *object literals*. "Object initializer" is consistent with the terminology used by C++.

The syntax for an object using an object initializer is:

```js
const obj = {
  property1: value1, // property name may be an identifier
  2: value2, // or a number
  "property n": value3, // or a string
};
```

Each property name before the colon is either an identifier, a number literal, or a string literal, and each `valueN` is an expression whose value is assigned to the property name. The property name can also be an expression; computed keys need to be wrapped in square brackets. The object initializer reference contains a more detailed explanation of the syntax.

In this example, the newly created object is assigned to a variable `obj` — this is optional. If you do not need to refer to this object elsewhere, you do not need to assign it to a variable. (Note that you may need to wrap the object literal in parentheses if the object appears where a statement is expected, so as not to have the literal be confused with a block statement.)

Object initializers are expressions, and each object initializer results in a new object being created whenever the statement in which it appears is executed. Identical object initializers create distinct objects that do not compare to each other as equal.

The following statement creates an object and assigns it to the variable `x` if and only if the expression `cond` is true:

```js
let x;
if (cond) {
  x = { greeting: "hi there" };
}
```

The following example creates `myHonda` with three properties. Note that the `engine` property is also an object with its own properties.

```js
const myHonda = {
  color: "red",
  wheels: 4,
  engine: { cylinders: 4, size: 2.2 },
};
```

Objects created with initializers are called *plain objects*, because they are instances of `Object`, but not any other object type. Some object types have special initializer syntaxes — for example, array initializers and regex literals.

Using a constructor function

Alternatively, you can create an object with these two steps:

1. Define the object type by writing a constructor function. There is a strong convention, with good reason, to use a capital initial letter.
2. Create an instance of the object with `new`.

To define an object type, create a function for the object type that specifies its name, properties, and methods. For example, suppose you want to create an object type for cars. You want this type of object to be called `Car`, and you want it to have properties for make, model, and year. To do this, you would write the following function:

```js
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
```

Notice the use of `this` to assign values to the object's properties based on the values passed to the function.

Now you can create an object called `myCar` as follows:

```js
const myCar = new Car("Eagle", "Talon TSi", 1993);
```

This statement creates `myCar` and assigns it the specified values for its properties. Then the value of `myCar.make` is the string `"Eagle"`, `myCar.model` is the string `"Talon TSi"`, `myCar.year` is the integer `1993`, and so on. The order of arguments and parameters should be the same.

You can create any number of `Car` objects by calls to `new`. For example,

```js
const randCar = new Car("Nissan", "300ZX", 1992);
const kenCar = new Car("Mazda", "Miata", 1990);
```

An object can have a property that is itself another object. For example, suppose you define an object called `Person` as follows:

```js
function Person(name, age, sex) {
  this.name = name;
  this.age = age;
  this.sex = sex;
}
```

and then instantiate two new `Person` objects as follows:

```js
const rand = new Person("Rand McKinnon", 33, "M");
const ken = new Person("Ken Jones", 39, "M");
```

Then, you can rewrite the definition of `Car` to include an `owner` property that takes a `Person` object, as follows:

```js
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
  this.year = year;
  this.owner = owner;
}
```

To instantiate the new objects, you then use the following:

```js
const car1 = new Car("Eagle", "Talon TSi", 1993, rand);
const car2 = new Car("Nissan", "300ZX", 1992, ken);
```

Notice that instead of passing a literal string or integer value when creating the new objects, the above statements pass the objects `rand` and `ken` as the arguments for the owners. Then if you want to find out the name of the owner of `car2`, you can access the following property:

```js
car2.owner.name;
```

You can always add a property to a previously defined object. For example, the statement

```js
car1.color = "black";
```

adds a property `color` to `car1`, and assigns it a value of `"black"`. However, this does not affect any other objects. To add the new property to all objects of the same type, you have to add the property to the definition of the `Car` object type.

You can also use the `class` syntax instead of the `function` syntax to define a constructor function. For more information, see the class guide.

Using the Object.create() method

Objects can also be created using the `Object.create()` method. This method can be very useful, because it allows you to choose the prototype object for the object you want to create, without having to define a constructor function.

```js
// Animal properties and method encapsulation
const animalProto = {
  type: "Invertebrates", // Default value of properties
  displayType() {
    // Method which will display the type of animal
    console.log(this.type);
  },
};

// Create a new animal type called `animal`
const animal = Object.create(animalProto);
animal.displayType(); // Logs: Invertebrates

// Create a new animal type called fish
const fish = Object.create(animalProto);
fish.type = "Fishes";
fish.displayType(); // Logs: Fishes
```

Objects and properties

A JavaScript object has properties associated with it. Object properties are basically the same as variables, except that they are associated with objects, not scopes. The properties of an object define the characteristics of the object.

For example, this example creates an object named `myCar`, with properties named `make`, `model`, and `year`, with their values set to `"Ford"`, `"Mustang"`, and `1969`:

```js
const myCar = {
  make: "Ford",
  model: "Mustang",
  year: 1969,
};
```

Like JavaScript variables, property names are case sensitive. Property names can only be strings or Symbols — all keys are converted to strings unless they are Symbols. Array indices are, in fact, properties with string keys that contain integers.

Accessing properties

You can access a property of an object by its property name. Property accessors come in two syntaxes: *dot notation* and *bracket notation*. For example, you could access the properties of the `myCar` object as follows:

```js
// Dot notation
myCar.make = "Ford";
myCar.model = "Mustang";
myCar.year = 1969;

// Bracket notation
myCar["make"] = "Ford";
myCar["model"] = "Mustang";
myCar["year"] = 1969;
```

An object property name can be any JavaScript string or symbol, including an empty string. However, you cannot use dot notation to access a property whose name is not a valid JavaScript identifier. For example, a property name that has a space or a hyphen, that starts with a number, or that is held inside a variable can only be accessed using the bracket notation. This notation is also very useful when property names are to be dynamically determined, i.e., not determinable until runtime. Examples are as follows:

```js
const myObj = {};
const str = "myString";
const rand = Math.random();
const anotherObj = {};

// Create additional properties on myObj
myObj.type = "Dot syntax for a key named type";
myObj["date created"] = "This key has a space";
myObj[str] = "This key is in variable str";
myObj[rand] = "A random number is the key here";
myObj[anotherObj] = "This key is object anotherObj";
myObj[""] = "This key is an empty string";

console.log(myObj);
// {
//   type: 'Dot syntax for a key named type',
//   'date created': 'This key has a space',
//   myString: 'This key is in variable str',
//   '0.6398914448618778': 'A random number is the key here',
//   '[object Object]': 'This key is object anotherObj',
//   '': 'This key is an empty string'
// }
console.log(myObj.myString); // 'This key is in variable str'
```

In the above code, the key `anotherObj` is an object, which is neither a string nor a symbol. When it is added to the `myObj`, JavaScript calls the `toString()` method of `anotherObj`, and use the resulting string as the new key.

You can also access properties with a string value stored in a variable. The variable must be passed in bracket notation. In the example above, the variable `str` held `"myString"` and it is `"myString"` that is the property name. Therefore, `myObj.str` will return as undefined.

```js
str = "myString";
myObj[str] = "This key is in variable str";

console.log(myObj.str); // undefined

console.log(myObj[str]); // 'This key is in variable str'
console.log(myObj.myString); // 'This key is in variable str'
```

This allows accessing any property as determined at runtime:

```js
let propertyName = "make";
myCar[propertyName] = "Ford";

// access different properties by changing the contents of the variable
propertyName = "model";
myCar[propertyName] = "Mustang";

console.log(myCar); // { make: 'Ford', model: 'Mustang' }
```

However, beware of using square brackets to access properties whose names are given by external input. This may make your code susceptible to object injection attacks.

Nonexistent properties of an object have the value `undefined` (and not `null`).

```js
myCar.nonexistentProperty; // undefined
```

Enumerating properties

There are three native ways to list/traverse object properties:

- `for...in` loops. This method traverses all of the enumerable string properties of an object as well as its prototype chain.
- `Object.keys()`. This method returns an array with only the enumerable own string property names ("keys") in the object `myObj`, but not those in the prototype chain.
- `Object.getOwnPropertyNames()`. This method returns an array containing all the own string property names in the object `myObj`, regardless of if they are enumerable or not.

You can use the bracket notation with `for...in` to iterate over all the enumerable properties of an object. To illustrate how this works, the following function displays the properties of the object when you pass the object and the object's name as arguments to the function:

```js
function showProps(obj, objName) {
  let result = "";
  for (const i in obj) {
    // Object.hasOwn() is used to exclude properties from the object's
    // prototype chain and only show "own properties"
    if (Object.hasOwn(obj, i)) {
      result += `${objName}.${i} = ${obj[i]}\n`;
    }
  }
  console.log(result);
}
```

The term "own property" refers to the properties of the object, but excluding those of the prototype chain. So, the function call `showProps(myCar, 'myCar')` would print the following:

```
myCar.make = Ford
myCar.model = Mustang
myCar.year = 1969
```

The above is equivalent to:

```js
function showProps(obj, objName) {
  let result = "";
  Object.keys(obj).forEach((i) => {
    result += `${objName}.${i} = ${obj[i]}\n`;
  });
  console.log(result);
}
```

There is no native way to list all inherited properties, including non-enumerable ones. However, this can be achieved with the following function:

```js
function listAllProperties(myObj) {
  let objectToInspect = myObj;
  let result = [];

  while (objectToInspect !== null) {
    result = result.concat(Object.getOwnPropertyNames(objectToInspect));
    objectToInspect = Object.getPrototypeOf(objectToInspect);
  }

  return result;
}
```

For more information, see Enumerability and ownership of properties.

Deleting properties

You can remove a non-inherited property using the `delete` operator. The following code shows how to remove a property.

```js
// Creates a new object, myObj, with two properties, a and b.
const myObj = { a: 5, b: 12 };

// Removes the a property, leaving myObj with only the b property.
delete myObj.a;
console.log("a" in myObj); // false
```

Inheritance

All objects in JavaScript inherit from at least one other object. The object being inherited from is known as the prototype, and the inherited properties can be found in the `prototype` object of the constructor. See Inheritance and the prototype chain for more information.

Defining properties for all objects of one type

You can add a property to all objects created through a certain constructor using the `prototype` property. This defines a property that is shared by all objects of the specified type, rather than by just one instance of the object. The following code adds a `color` property to all objects of type `Car`, and then reads the property's value from an instance `car1`.

```js
Car.prototype.color = "red";
console.log(car1.color); // "red"
```

Defining methods

A *method* is a function associated with an object, or, put differently, a method is a property of an object that is a function. Methods are defined the way normal functions are defined, except that they have to be assigned as the property of an object. See also method definitions for more details. An example is:

```js
objectName.methodName = functionName;

const myObj = {
  myMethod: function (params) {
    // do something
  },

  // this works too!
  myOtherMethod(params) {
    // do something else
  },
};
```

where `objectName` is an existing object, `methodName` is the name you are assigning to the method, and `functionName` is the name of the function.

You can then call the method in the context of the object as follows:

```js
objectName.methodName(params);
```

Methods are typically defined on the `prototype` object of the constructor, so that all objects of the same type share the same method. For example, you can define a function that formats and displays the properties of the previously-defined `Car` objects.

```js
Car.prototype.displayCar = function () {
  const result = `A Beautiful ${this.year} ${this.make} ${this.model}`;
  console.log(result);
};
```

Notice the use of `this` to refer to the object to which the method belongs. Then you can call the `displayCar` method for each of the objects as follows:

```js
car1.displayCar();
car2.displayCar();
```

Using this for object references

JavaScript has a special keyword, `this`, that you can use within a method to refer to the current object. For example, suppose you have 2 objects, `manager` and `intern`. Each object has its own `name`, `age` and `job`. In the function `sayHi()`, notice the use of `this.name`. When added to the 2 objects, the same function will print the message with the name of the respective object it's attached to.

```js
const manager = {
  name: "Karina",
  age: 27,
  job: "Software Engineer",
```

(code continues on the next page)

(code continued from the previous page)

```
};
const intern = {
  name: "Tyrone",
  age: 21,
  job: "Software Engineer Intern",
};

function sayHi() {
  console.log(`Hello, my name is ${this.name}`);
}

// Add sayHi function to both objects
manager.sayHi = sayHi;
intern.sayHi = sayHi;

manager.sayHi(); // Hello, my name is Karina
intern.sayHi(); // Hello, my name is Tyrone
```

`this` is a "hidden parameter" of a function call that's passed in by specifying the object before the function that was called. For example, in `manager.sayHi()`, `this` is the `manager` object, because `manager` comes before the function `sayHi()`. If you access the same function from another object, `this` will change as well. If you use other methods to call the function, like `Function.prototype.call()` or `Reflect.apply()`, you can explicitly pass the value of `this` as an argument.

Defining getters and setters

A getter is a function associated with a property that gets the value of a specific property. A setter is a function associated with a property that sets the value of a specific property. Together, they can indirectly represent the value of a property.

Getters and setters can be either

- defined within object initializers, or
- added later to any existing object.

Within object initializers, getters and setters are defined like regular methods, but prefixed with the keywords `get` or `set`. The getter method must not expect a parameter, while the setter method expects exactly one parameter (the new value to set). For instance:

js

```
const myObj = {
  a: 7,
  get b() {
    return this.a + 1;
  },
  set c(x) {
    this.a = x / 2;
  },
};

console.log(myObj.a); // 7
console.log(myObj.b); // 8, returned from the get b() method
myObj.c = 50; // Calls the set c(x) method
console.log(myObj.a); // 25
```

The `myObj` object's properties are:

- `myObj.a` — a number
- `myObj.b` — a getter that returns `myObj.a` plus 1
- `myObj.c` — a setter that sets the value of `myObj.a` to half of the value `myObj.c` is being set to

Getters and setters can also be added to an object at any time after creation using the `Object.defineProperties()` method. This method's first parameter is the object on which you want to define the getter or setter. The second parameter is an object whose property names are the getter or setter names, and whose property values are objects for defining the getter or setter functions. Here's an example that defines the same getter and setter used in the previous example:

```js
const myObj = { a: 0 };

Object.defineProperties(myObj, {
  b: {
    get() {
      return this.a + 1;
    },
  },
  c: {
    set(x) {
      this.a = x / 2;
    },
  },
});

myObj.c = 10; // Runs the setter, which assigns 10 / 2 (5) to the 'a' property
console.log(myObj.b); // Runs the getter, which yields a + 1 or 6
```

Which of the two forms to choose depends on your programming style and task at hand. If you can change the definition of the original object, you will probably define getters and setters through the original initializer. This form is more compact and natural. However, if you need to add getters and setters later — maybe because you did not write the particular object — then the second form is the only possible form. The second form better represents the dynamic nature of JavaScript, but it can make the code hard to read and understand.

Comparing objects

In JavaScript, objects are a reference type. Two distinct objects are never equal, even if they have the same properties. Only comparing the same object reference with itself yields true.

```js
// Two variables, two distinct objects with the same properties
const fruit = { name: "apple" };
const anotherFruit = { name: "apple" };

fruit == anotherFruit; // return false
fruit === anotherFruit; // return false
```

```js
// Two variables, a single object
const fruit = { name: "apple" };
const anotherFruit = fruit; // Assign fruit object reference to anotherFruit

// Here fruit and anotherFruit are pointing to same object
fruit == anotherFruit; // return true
fruit === anotherFruit; // return true

fruit.name = "grape";
console.log(anotherFruit); // { name: "grape" }; not { name: "apple" }
```

For more information about comparison operators, see equality operators.

See also

- Inheritance and the prototype chain
- Classes

JavaScript › Guide

Using classes

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_classes · MDN page last modified 2026-10-03

JavaScript is a prototype-based language — an object's behaviors are specified by its own properties and its prototype's properties. However, with the addition of classes, the creation of hierarchies of objects and the inheritance of properties and their values are much more in line with other object-oriented languages such as Java. In this section, we will demonstrate how objects can be created from classes.

In many other languages, *classes*, or constructors, are clearly distinguished from *objects*, or instances. In JavaScript, classes are mainly an abstraction over the existing prototypical inheritance mechanism — all patterns are convertible to prototype-based inheritance. Classes themselves are normal JavaScript values as well, and have their own prototype chains. In fact, most plain JavaScript functions can be used as constructors — you use the `new` operator with a constructor function to create a new object.

We will be playing with the well-abstracted class model in this tutorial, and discuss what semantics classes offer. If you want to dive deep into the underlying prototype system, you can read the Inheritance and the prototype chain guide.

This chapter assumes that you are already somewhat familiar with JavaScript and that you have used ordinary objects.

Overview of classes

If you have some hands-on experience with JavaScript, or have followed along with the guide, you probably have already used classes, even if you haven't created one. For example, this may seem familiar to you:

```js
const bigDay = new Date(2019, 6, 19);
console.log(bigDay.toLocaleDateString());
if (bigDay.getTime() < Date.now()) {
  console.log("Once upon a time...");
}
```

On the first line, we created an instance of the class `Date`, and called it `bigDay`. On the second line, we called a method `toLocaleDateString()` on the `bigDay` instance, which returns a string. Then, we compared two numbers: one returned from the `getTime()` method, the other directly called from the `Date` class *itself*, as `Date.now()`.

`Date` is a built-in class of JavaScript. From this example, we can get some basic ideas of what classes do:

- Classes create objects through the `new` operator.
- Each object has some properties (data or method) added by the class.
- The class stores some properties (data or method) itself, which are usually used to interact with instances.

These correspond to the three key features of classes:

- Constructor;
- Instance methods and instance fields;
- Static methods and static fields.

Declaring a class

Classes are usually created with *class declarations*.

```js
class MyClass {
  // class body...
}
```

Within a class body, there are a range of features available.

```js
class MyClass {
  // Constructor
  constructor() {
    // Constructor body
  }
  // Instance field
  myField = "foo";
  // Instance method
  myMethod() {
    // myMethod body
  }
  // Static field
  static myStaticField = "bar";
  // Static method
  static myStaticMethod() {
    // myStaticMethod body
  }
  // Static block
  static {
    // Static initialization code
  }
  // Fields, methods, static fields, and static methods all have
  // "private" forms
  #myPrivateField = "bar";
}
```

If you came from a pre-ES6 world, you may be more familiar with using functions as constructors. The pattern above would roughly translate to the following with function constructors:

```js
function MyClass() {
  this.myField = "foo";
  // Constructor body
}
MyClass.myStaticField = "bar";
MyClass.myStaticMethod = function () {
  // myStaticMethod body
};
MyClass.prototype.myMethod = function () {
  // myMethod body
};

(function () {
  // Static initialization code
})();
```

> **Note:** Private fields and methods are new features in classes with no trivial equivalent in function constructors.

Constructing a class

After a class has been declared, you can create instances of it using the new operator.

```js
const myInstance = new MyClass();
console.log(myInstance.myField); // 'foo'
myInstance.myMethod();
```

Typical function constructors can both be constructed with new and called without new. However, attempting to "call" a class without new will result in an error.

```js
const myInstance = MyClass(); // TypeError: Class constructor MyClass cannot be invoked
without 'new'
```

Class declaration hoisting

Unlike function declarations, class declarations are not hoisted (or, in some interpretations, hoisted but with the temporal dead zone restriction), which means you cannot use a class before it is declared.

```js
new MyClass(); // ReferenceError: Cannot access 'MyClass' before initialization

class MyClass {}
```

This behavior is similar to variables declared with let and const.

Class expressions

Similar to functions, class declarations also have their expression counterparts.

```js
const MyClass = class {
  // Class body...
};
```

Class expressions can have names as well. The expression's name is only visible to the class's body.

```js
const MyClass = class MyClassLongerName {
  // Class body. Here MyClass and MyClassLongerName point to the same class.
};
new MyClassLongerName(); // ReferenceError: MyClassLongerName is not defined
```

Constructor

Perhaps the most important job of a class is to act as a "factory" for objects. For example, when we use the Date constructor, we expect it to give a new object which represents the date data we passed in — which we can then manipulate with other methods the instance exposes. In classes, the instance creation is done by the constructor.

As an example, we would create a class called `Color`, which represents a specific color. Users create colors through passing in an RGB triplet.

```js
class Color {
  constructor(r, g, b) {
    // Assign the RGB values as a property of `this`.
    this.values = [r, g, b];
  }
}
```

Open your browser's devtools, paste the above code into the console, and then create an instance:

```js
const red = new Color(255, 0, 0);
console.log(red);
```

You should see some output like this:

```
Object { values: (3) […] }
  values: Array(3) [ 255, 0, 0 ]
```

You have successfully created a `Color` instance, and the instance has a `values` property, which is an array of the RGB values you passed in. That is pretty much equivalent to the following:

```js
function createColor(r, g, b) {
  return {
    values: [r, g, b],
  };
}
```

The constructor's syntax is exactly the same as a normal function — which means you can use other syntaxes, like rest parameters:

```js
class Color {
  constructor(...values) {
    this.values = values;
  }
}

const red = new Color(255, 0, 0);
// Creates an instance with the same shape as above.
```

Each time you call `new`, a different instance is created.

```js
const red = new Color(255, 0, 0);
const anotherRed = new Color(255, 0, 0);
console.log(red === anotherRed); // false
```

Within a class constructor, the value of `this` points to the newly created instance. You can assign properties to it, or read existing properties (especially methods — which we will cover next).

The `this` value will be automatically returned as the result of `new`. You are advised to not return any value from the constructor — because if you return a non-primitive value, it will become the value of

the `new` expression, and the value of `this` is dropped. (You can read more about what `new` does in its description.)

```js
class MyClass {
  constructor() {
    this.myField = "foo";
    return {};
  }
}

console.log(new MyClass().myField); // undefined
```

Instance methods

If a class only has a constructor, it is not much different from a `createX` factory function which just creates plain objects. However, the power of classes is that they can be used as "templates" which automatically assign methods to instances.

For example, for `Date` instances, you can use a range of methods to get different information from a single date value, such as the year, month, day of the week, etc. You can also set those values through the `setX` counterparts like `setFullYear`.

For our own `Color` class, we can add a method called `getRed` which returns the red value of the color.

```js
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  getRed() {
    return this.values[0];
  }
}

const red = new Color(255, 0, 0);
console.log(red.getRed()); // 255
```

Without methods, you may be tempted to define the function within the constructor:

```js
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
    this.getRed = function () {
      return this.values[0];
    };
  }
}
```

This also works. However, a problem is that this creates a new function every time a `Color` instance is created, even when they all do the same thing!

```js
console.log(new Color().getRed === new Color().getRed); // false
```

In contrast, if you use a method, it will be shared between all instances. A function can be shared between all instances, but still have its behavior differ when different instances call it, because the value of `this` is different. If you are curious *where* this method is stored in — it's defined on the prototype of all instances, or `Color.prototype`, which is explained in more detail in Inheritance and the prototype chain.

Similarly, we can create a new method called `setRed`, which sets the red value of the color.

```js
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  getRed() {
    return this.values[0];
  }
  setRed(value) {
    this.values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.setRed(0);
console.log(red.getRed()); // 0; of course, it should be called "black" at this stage!
```

Private fields

You might be wondering: why do we want to go to the trouble of using `getRed` and `setRed` methods, when we can directly access the `values` array on the instance?

```js
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
}

const red = new Color(255, 0, 0);
red.values[0] = 0;
console.log(red.values[0]); // 0
```

There is a philosophy in object-oriented programming called "encapsulation". This means you should not access the underlying implementation of an object, but instead use well-abstracted methods to interact with it. For example, if we suddenly decided to represent colors as HSL instead:

```js
class Color {
  constructor(r, g, b) {
    // values is now an HSL array!
    this.values = rgbToHSL([r, g, b]);
  }
  getRed() {
    return hslToRGB(this.values)[0];
  }
  setRed(value) {
    const rgb = hslToRGB(this.values);
    rgb[0] = value;
    this.values = rgbToHSL(rgb);
```

(code continues on the next page)

(code continued from the previous page)

```
  }
}

const red = new Color(255, 0, 0);
console.log(red.values[0]); // 0; It's not 255 anymore, because the H value for pure red is
0
```

The user assumption that `values` means the RGB value suddenly collapses, and it may cause their logic to break. So, if you are an implementor of a class, you would want to hide the internal data structure of your instance from your user, both to keep the API clean and to prevent the user's code from breaking when you do some "harmless refactors". In classes, this is done through *private fields*.

A private field is an identifier prefixed with # (the hash symbol). The hash is an integral part of the field's name, which means a private field can never clash names with a public field or method. In order to refer to a private field anywhere in the class, you must *declare* it in the class body (you can't create a private element on the fly). Apart from this, a private field is pretty much equivalent to a normal property.

js

```
class Color {
  // Declare: every Color instance has a private field called #values.
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  getRed() {
    return this.#values[0];
  }
  setRed(value) {
    this.#values[0] = value;
  }
}

const red = new Color(255, 0, 0);
console.log(red.getRed()); // 255
```

Accessing private fields outside the class is an early syntax error. The language can guard against this because `#privateField` is a special syntax, so it can do some static analysis and find all usage of private fields before even evaluating the code.

js · ✗ bad example

```
console.log(red.#values); // SyntaxError: Private field '#values' must be declared in an
enclosing class
```

> **Note:** Code run in the Chrome console can access private elements outside the class. This is a DevTools-only relaxation of the JavaScript syntax restriction.

Private fields in JavaScript are *hard private*: if the class does not implement methods that expose these private fields, there's absolutely no mechanism to retrieve them from outside the class. This means you are safe to do any refactors to your class's private fields, as long as the behavior of exposed methods stay the same.

After we've made the `values` field private, we can add some more logic in the `getRed` and `setRed` methods, instead of making them simple pass-through methods. For example, we can add a check in `setRed` to see if it's a valid R value:

```js
class Color {
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  getRed() {
    return this.#values[0];
  }
  setRed(value) {
    if (value < 0 || value > 255) {
      throw new RangeError("Invalid R value");
    }
    this.#values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.setRed(1000); // RangeError: Invalid R value
```

If we leave the `values` property exposed, our users can easily circumvent that check by assigning to `values[0]` directly, and create invalid colors. But with a well-encapsulated API, we can make our code more robust and prevent logic errors downstream.

A class method can read the private fields of other instances, as long as they belong to the same class.

```js
class Color {
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  redDifference(anotherColor) {
    // #values doesn't necessarily need to be accessed from this:
    // you can access private fields of other instances belonging
    // to the same class.
    return this.#values[0] - anotherColor.#values[0];
  }
}

const red = new Color(255, 0, 0);
const crimson = new Color(220, 20, 60);
red.redDifference(crimson); // 35
```

However, if `anotherColor` is not a Color instance, `#values` won't exist. (Even if another class has an identically named `#values` private field, it's not referring to the same thing and cannot be accessed here.) Accessing a nonexistent private element throws an error instead of returning `undefined` like normal properties do. If you don't know if a private field exists on an object and you wish to access it without using `try/catch` to handle the error, you can use the `in` operator.

```js
class Color {
  #values;
  constructor(r, g, b) {
    this.#values = [r, g, b];
  }
  redDifference(anotherColor) {
    if (!(#values in anotherColor)) {
      throw new TypeError("Color instance expected");
    }
    return this.#values[0] - anotherColor.#values[0];
  }
}
```

> **Note:** Keep in mind that the `#` is a special identifier syntax, and you can't use the field name as if it's a string. `"#values" in anotherColor` would look for a property name literally called `"#values"`, instead of a private field.

There are some limitations in using private elements: the same name can't be declared twice in a single class, and they can't be deleted. Both lead to early syntax errors.

js · ✗ bad example

```
class BadIdeas {
  #firstName;
  #firstName; // syntax error occurs here
  #lastName;
  constructor() {
    delete this.#lastName; // also a syntax error
  }
}
```

Methods, getters, and setters can be private as well. They're useful when you have something complex that the class needs to do internally but no other part of the code should be allowed to call.

For example, imagine creating HTML custom elements that should do something somewhat complicated when clicked/tapped/otherwise activated. Furthermore, the somewhat complicated things that happen when the element is clicked should be restricted to this class, because no other part of the JavaScript will (or should) ever access it.

```js
class Counter extends HTMLElement {
  #xValue = 0;
  constructor() {
    super();
    this.onclick = this.#clicked.bind(this);
  }
  get #x() {
    return this.#xValue;
  }
  set #x(value) {
    this.#xValue = value;
    window.requestAnimationFrame(this.#render.bind(this));
  }
  #clicked() {
    this.#x++;
  }
  #render() {
```

(code continues on the next page)

(code continued from the previous page)

```
    this.textContent = this.#x.toString();
  }
  connectedCallback() {
    this.#render();
  }
}

customElements.define("num-counter", Counter);
```

In this case, pretty much every field and method is private to the class. Thus, it presents an interface to the rest of the code that's essentially just like a built-in HTML element. No other part of the program has the power to affect any of the internals of `Counter`.

Accessor fields

`color.getRed()` and `color.setRed()` allow us to read and write to the red value of a color. If you come from languages like Java, you will be very familiar with this pattern. However, using methods to simply access a property is still somewhat unergonomic in JavaScript. *Accessor fields* allow us to manipulate something as if it is an "actual property".

js

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  get red() {
    return this.values[0];
  }
  set red(value) {
    this.values[0] = value;
  }
}

const red = new Color(255, 0, 0);
red.red = 0;
console.log(red.red); // 0
```

It looks as if the object has a property called `red` — but actually, no such property exists on the instance! There are only two methods, but they are prefixed with `get` and `set`, which allows them to be manipulated as if they were properties.

If a field only has a getter but no setter, it will be effectively read-only.

js

```
class Color {
  constructor(r, g, b) {
    this.values = [r, g, b];
  }
  get red() {
    return this.values[0];
  }
}

const red = new Color(255, 0, 0);
red.red = 0;
console.log(red.red); // 255
```

In strict mode, the `red.red = 0` line will throw a type error: "Cannot set property red of #<Color> which has only a getter". In non-strict mode, the assignment is silently ignored.

Public fields

Private fields also have their public counterparts, which allow every instance to have a property. Fields are usually designed to be independent of the constructor's parameters.

```js
class MyClass {
  luckyNumber = Math.random();
}
console.log(new MyClass().luckyNumber); // 0.5
console.log(new MyClass().luckyNumber); // 0.3
```

Public fields are almost equivalent to assigning a property to `this`. For example, the above example can also be converted to:

```js
class MyClass {
  constructor() {
    this.luckyNumber = Math.random();
  }
}
```

Static properties

With the `Date` example, we have also encountered the `Date.now()` method, which returns the current date. This method does not belong to any date instance — it belongs to the class itself. However, it's put on the `Date` class instead of being exposed as a global `DateNow()` function, because it's mostly useful when dealing with date instances.

> **Note:** Prefixing utility methods with what they deal with is called "namespacing" and is considered a good practice. For example, in addition to the older, unprefixed `parseInt()` method, JavaScript also later added the prefixed `Number.parseInt()` method to indicate that it's for dealing with numbers.

Static properties are a group of class features that are defined on the class itself, rather than on individual instances of the class. These features include:

- Static methods
- Static fields
- Static getters and setters

Everything also has private counterparts. For example, for our `Color` class, we can create a static method that checks whether a given triplet is a valid RGB value:

```js
class Color {
  static isValid(r, g, b) {
    return r >= 0 && r <= 255 && g >= 0 && g <= 255 && b >= 0 && b <= 255;
  }
}

Color.isValid(255, 0, 0); // true
Color.isValid(1000, 0, 0); // false
```

Static properties are very similar to their instance counterparts, except that:

- They are all prefixed with `static`, and
- They are not accessible from instances.

```js
console.log(new Color(0, 0, 0).isValid); // undefined
```

There is also a special construct called a *static initialization block*, which is a block of code that runs when the class is first loaded.

```js
class MyClass {
  static {
    MyClass.myStaticProperty = "foo";
  }
}

console.log(MyClass.myStaticProperty); // 'foo'
```

Static initialization blocks are almost equivalent to immediately executing some code after a class has been declared. The only difference is that they have access to static private elements.

Extends and inheritance

A key feature that classes bring about (in addition to ergonomic encapsulation with private fields) is *inheritance*, which means one object can "borrow" a large part of another object's behaviors, while overriding or enhancing certain parts with its own logic.

For example, suppose our `Color` class now needs to support transparency. We may be tempted to add a new field that indicates its transparency:

```js
class Color {
  #values;
  constructor(r, g, b, a = 1) {
    this.#values = [r, g, b, a];
  }
  get alpha() {
    return this.#values[3];
  }
  set alpha(value) {
    if (value < 0 || value > 1) {
      throw new RangeError("Alpha value must be between 0 and 1");
    }
    this.#values[3] = value;
  }
}
```

However, this means every instance — even the vast majority which aren't transparent (those with an alpha value of 1) — will have to have the extra alpha value, which is not very elegant. Plus, if the features keep growing, our `Color` class will become very bloated and hard to maintain.

Instead, in object-oriented programming, we would create a *derived class*. The derived class has access to all public properties of the parent class. In JavaScript, derived classes are declared with an `extends` clause, which indicates the class it extends from.

```js
class ColorWithAlpha extends Color {
  #alpha;
  constructor(r, g, b, a) {
    super(r, g, b);
    this.#alpha = a;
  }
  get alpha() {
    return this.#alpha;
  }
  set alpha(value) {
    if (value < 0 || value > 1) {
      throw new RangeError("Alpha value must be between 0 and 1");
    }
    this.#alpha = value;
  }
}
```

There are a few things that have immediately come to attention. First is that in the constructor, we are calling `super(r, g, b)`. It is a language requirement to call `super()` before accessing `this`. The `super()` call calls the parent class's constructor to initialize `this` — here it's roughly equivalent to `this = new Color(r, g, b)`. You can have code before `super()`, but you cannot access `this` before `super()` — the language prevents you from accessing the uninitialized `this`.

After the parent class is done with modifying `this`, the derived class can do its own logic. Here we added a private field called `#alpha`, and also provided a pair of getter/setters to interact with them.

A derived class inherits all methods from its parent. For example, consider the `get red()` accessor we added to the `Color` in the Accessor fields section—even though we haven't declared one in `ColorWithAlpha`, we can still access `red` because this behavior is specified by the parent class:

```js
const color = new ColorWithAlpha(255, 0, 0, 0.5);
console.log(color.red); // 255
```

Derived classes can also override methods from the parent class. For example, all classes implicitly inherit the `Object` class, which defines some basic methods like `toString()`. However, the base `toString()` method is notoriously useless, because it prints `[object Object]` in most cases:

```js
console.log(red.toString()); // [object Object]
```

Instead, our class can override it to print the color's RGB values:

```js
class Color {
  #values;
  // …
  toString() {
    return this.#values.join(", ");
  }
}

console.log(new Color(255, 0, 0).toString()); // '255, 0, 0'
```

Within derived classes, you can access the parent class's methods by using `super`. This allows you to build enhancement methods and avoid code duplication.

```js
class ColorWithAlpha extends Color {
  #alpha;
  // …
  toString() {
    // Call the parent class's toString() and build on the return value
    return `${super.toString()}, ${this.#alpha}`;
  }
}

console.log(new ColorWithAlpha(255, 0, 0, 0.5).toString()); // '255, 0, 0, 0.5'
```

When you use `extends`, the static methods inherit from each other as well, so you can also override or enhance them.

```js
class ColorWithAlpha extends Color {
  // …
  static isValid(r, g, b, a) {
    // Call the parent class's isValid() and build on the return value
    return super.isValid(r, g, b) && a >= 0 && a <= 1;
  }
}

console.log(ColorWithAlpha.isValid(255, 0, 0, -1)); // false
```

Derived classes don't have access to the parent class's private fields — this is another key aspect to JavaScript private fields being "hard private". Private fields are scoped to the class body itself and do not grant access to *any* outside code.

js · ✗ bad example

```js
class ColorWithAlpha extends Color {
  log() {
    console.log(this.#values); // SyntaxError: Private field '#values' must be declared in
an enclosing class
  }
}
```

A class can only extend from one class. This prevents problems in multiple inheritance like the diamond problem. However, due to the dynamic nature of JavaScript, it's still possible to achieve the effect of multiple inheritance through class composition and mixins.

Instances of derived classes are also instances of the base class.

```js
const color = new ColorWithAlpha(255, 0, 0, 0.5);
console.log(color instanceof Color); // true
console.log(color instanceof ColorWithAlpha); // true
```

Why classes?

The guide has been pragmatic so far: we are focusing on *how* classes can be used, but there's one question unanswered: *why* would one use a class? The answer is: it depends.

Classes introduce a *paradigm*, or a way to organize your code. Classes are the foundations of object-oriented programming, which is built on concepts like inheritance and polymorphism (especially

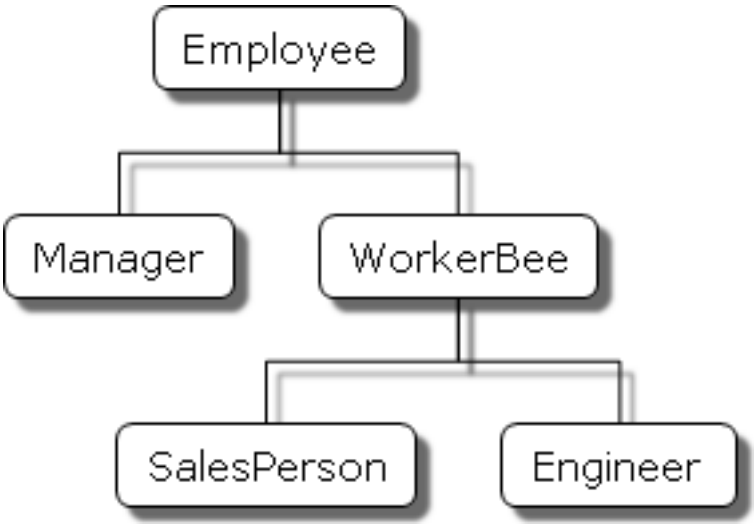
subtype polymorphism). However, many people are philosophically against certain OOP practices and don't use classes as a result.

For example, one thing that makes `Date` objects infamous is that they're *mutable*.

```js
function incrementDay(date) {
  return new Date(date.setDate(date.getDate() + 1));
}
const date = new Date(); // 2019-06-19
const newDay = incrementDay(date);
console.log(newDay); // 2019-06-20
// The old date is modified as well!?
console.log(date); // 2019-06-20
```

Mutability and internal state are important aspects of object-oriented programming, but often make code hard to reason with — because any seemingly innocent operation may have unexpected side effects and change the behavior in other parts of the program.

In order to reuse code, we usually resort to extending classes, which can create big hierarchies of inheritance patterns.



However, it is often hard to describe inheritance cleanly when one class can only extend one other class. Often, we want the behavior of multiple classes. In Java, this is done through interfaces; in JavaScript, it can be done through mixins. But at the end of the day, it's still not very convenient.

On the brighter side, classes are a very powerful way to organize our code on a higher level. For example, without the `Color` class, we may need to create a dozen of utility functions:

```js
function isRed(color) {
  return color.red === 255;
}
function isValidColor(color) {
  return (
    color.red >= 0 &&
    color.red <= 255 &&
    color.green >= 0 &&
    color.green <= 255 &&
    color.blue >= 0 &&
    color.blue <= 255
  );
}
// …
```

But with classes, we can congregate them all under the `Color` namespace, which improves readability. In addition, the introduction of private fields allows us to hide certain data from downstream users, creating a clean API.

In general, you should consider using classes when you want to create objects that store their own internal data and expose a lot of behavior. Take built-in JavaScript classes as examples:

- The `Map` and `Set` classes store a collection of elements and allow you to access them by key using `get()`, `set()`, `has()`, etc.
- The `Date` class stores a date as a Unix timestamp (a number) and allows you to format, update, and read individual date components.
- The `Error` class stores information about a particular exception, including the error message, stack trace, cause, etc. It's one of the few classes that come with a rich inheritance structure: there are multiple built-in classes like `TypeError` and `ReferenceError` that extend `Error`. In the case of errors, this inheritance allows refining the semantics of errors: each error class represents a specific type of error, which can be easily checked with `instanceof`.

JavaScript offers the mechanism to organize your code in a canonical object-oriented way, but whether and how to use it is entirely up to the programmer's discretion.

JavaScript › Guide

Using promises

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises · MDN page last modified 2026-09-22

A `Promise` is an object representing the eventual completion or failure of an asynchronous operation. Since most people are consumers of already-created promises, this guide will explain consumption of returned promises before explaining how to create them.

Essentially, a promise is a returned object to which you attach callbacks, instead of passing callbacks into a function. Imagine a function, `createAudioFileAsync()`, which asynchronously generates a sound file given a configuration record and two callback functions: one called if the audio file is successfully created, and the other called if an error occurs.

Here's some code that uses `createAudioFileAsync()`:

```js
function successCallback(result) {
  console.log(`Audio file ready at URL: ${result}`);
}

function failureCallback(error) {
  console.error(`Error generating audio file: ${error}`);
}

createAudioFileAsync(audioSettings, successCallback, failureCallback);
```

If `createAudioFileAsync()` were rewritten to return a promise, you would attach your callbacks to it instead:

```js
createAudioFileAsync(audioSettings).then(successCallback, failureCallback);
```

This convention has several advantages. We will explore each one.

Chaining

A common need is to execute two or more asynchronous operations back to back, where each subsequent operation starts when the previous operation succeeds, with the result from the previous step. In the old days, doing several asynchronous operations in a row would lead to the classic callback hell:

```js
doSomething(function (result) {
  doSomethingElse(result, function (newResult) {
    doThirdThing(newResult, function (finalResult) {
      console.log(`Got the final result: ${finalResult}`);
    }, failureCallback);
  }, failureCallback);
}, failureCallback);
```

With promises, we accomplish this by creating a promise chain. The API design of promises makes this great, because callbacks are attached to the returned promise object, instead of being passed into a function.

Here's the magic: the `then()` function returns a **new promise**, different from the original:

```js
const promise = doSomething();
const promise2 = promise.then(successCallback, failureCallback);
```

This second promise (`promise2`) represents the completion not just of `doSomething()`, but also of the `successCallback` or `failureCallback` you passed in — which can be other asynchronous functions returning a promise. When that's the case, any callbacks added to `promise2` get queued behind the promise returned by either `successCallback` or `failureCallback`.

> **Note:** If you want a working example to play with, you can use the following template to create any function returning a promise:
>
> ```js
> function doSomething() {
>   return new Promise((resolve) => {
>     setTimeout(() => {
>       // Other things to do before completion of the promise
>       console.log("Did something");
>       // The fulfillment value of the promise
>       resolve("https://example.com/");
>     }, 200);
>   });
> }
> ```
>
> The implementation is discussed in the Creating a Promise around an old callback API section below.

With this pattern, you can create longer chains of processing, where each promise represents the completion of one asynchronous step in the chain. In addition, the arguments to `then` are optional, and `catch(failureCallback)` is short for `then(null, failureCallback)` — so if your error handling code is the same for all steps, you can attach it to the end of the chain:

```js
doSomething()
  .then(function (result) {
    return doSomethingElse(result);
  })
  .then(function (newResult) {
    return doThirdThing(newResult);
  })
  .then(function (finalResult) {
    console.log(`Got the final result: ${finalResult}`);
  })
  .catch(failureCallback);
```

You might see this expressed with arrow functions instead:

```js
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => {
    console.log(`Got the final result: ${finalResult}`);
  })
  .catch(failureCallback);
```

> **Note:** Arrow function expressions can have an implicit return; so, `() => x` is short for `() => { return x; }`.

`doSomethingElse` and `doThirdThing` can return any value — if they return promises, that promise is first waited until it settles, and the next callback receives the fulfillment value, not the promise itself. It is important to always return promises from `then` callbacks, even if the promise always resolves to `undefined`. If the previous handler started a promise but did not return it, there's no way to track its settlement anymore, and the promise is said to be "floating".

js · ✗ bad example

```
doSomething()
  .then((url) => {
    // Missing `return` keyword in front of fetch(url).
    fetch(url);
  })
  .then((result) => {
    // result is undefined, because nothing is returned from the previous
    // handler. There's no way to know the return value of the fetch()
    // call anymore, or whether it succeeded at all.
  });
```

By returning the result of the `fetch` call (which is a promise), we can both track its completion and receive its value when it completes.

js · ✓ good example

```
doSomething()
  .then((url) => {
    // `return` keyword added
    return fetch(url);
  })
  .then((result) => {
    // result is a Response object
  });
```

Floating promises could be worse if you have race conditions — if the promise from the last handler is not returned, the next `then` handler will be called early, and any value it reads may be incomplete.

js · ✗ bad example

```
const listOfIngredients = [];

doSomething()
  .then((url) => {
    // Missing `return` keyword in front of fetch(url).
    fetch(url)
      .then((res) => res.json())
      .then((data) => {
        listOfIngredients.push(data);
      });
  })
  .then(() => {
    console.log(listOfIngredients);
    // listOfIngredients will always be [], because the fetch request hasn't completed yet.
  });
```

Therefore, as a rule of thumb, whenever your operation encounters a promise, return it and defer its handling to the next `then` handler.

js · ✓ good example

```
const listOfIngredients = [];

doSomething()
  .then((url) => {
    // `return` keyword now included in front of fetch call.
    return fetch(url)
      .then((res) => res.json())
      .then((data) => {
        listOfIngredients.push(data);
      });
  })
  .then(() => {
    console.log(listOfIngredients);
    // listOfIngredients will now contain data from fetch call.
  });
```

Even better, you can flatten the nested chain into a single chain, which is simpler and makes error handling easier. The details are discussed in the Nesting section below.

js

```
doSomething()
  .then((url) => fetch(url))
  .then((res) => res.json())
  .then((data) => {
    listOfIngredients.push(data);
  })
  .then(() => {
    console.log(listOfIngredients);
  });
```

Using `async/await` can help you write code that's more intuitive and resembles synchronous code. Below is the same example using `async/await`:

js

```
async function logIngredients() {
  const url = await doSomething();
  const res = await fetch(url);
  const data = await res.json();
  listOfIngredients.push(data);
  console.log(listOfIngredients);
}
```

Note how the code looks exactly like synchronous code, except for the `await` keywords in front of promises. One of the only tradeoffs is that it may be easy to forget the `await` keyword, which can only be fixed when there's a type mismatch (e.g., trying to use a promise as a value).

`async/await` builds on promises — for example, `doSomething()` is the same function as before, so there's minimal refactoring needed to change from promises to `async/await`. You can read more about the `async/await` syntax in the async functions and `await` references.

> **Note:** `async/await` has the same concurrency semantics as normal promise chains. `await` within one async function does not stop the entire program, only the parts that depend on its value, so other async jobs can still run while the `await` is pending.

Error handling

You might recall seeing `failureCallback` three times in the pyramid of doom earlier, compared to only once at the end of the promise chain:

```js
doSomething()
  .then((result) => doSomethingElse(result))
  .then((newResult) => doThirdThing(newResult))
  .then((finalResult) => console.log(`Got the final result: ${finalResult}`))
  .catch(failureCallback);
```

If there's an exception, the browser will look down the chain for `.catch()` handlers or `onRejected`. This is very much modeled after how synchronous code works:

```js
try {
  const result = syncDoSomething();
  const newResult = syncDoSomethingElse(result);
  const finalResult = syncDoThirdThing(newResult);
  console.log(`Got the final result: ${finalResult}`);
} catch (error) {
  failureCallback(error);
}
```

This symmetry with asynchronous code culminates in the `async/await` syntax:

```js
async function foo() {
  try {
    const result = await doSomething();
    const newResult = await doSomethingElse(result);
    const finalResult = await doThirdThing(newResult);
    console.log(`Got the final result: ${finalResult}`);
  } catch (error) {
    failureCallback(error);
  }
}
```

Promises solve a fundamental flaw with the callback pyramid of doom, by catching all errors, even thrown exceptions and programming errors. This is essential for functional composition of asynchronous operations. All errors are now handled by the `catch()` method at the end of the chain, and you should almost never need to use `try/catch` without using `async/await`.

Nesting

In the examples above involving `listOfIngredients`, the first one has one promise chain nested in the return value of another `then()` handler, while the second one uses an entirely flat chain. Simple promise chains are best kept flat without nesting, as nesting can be a result of careless composition.

Nesting is a control structure to limit the scope of `catch` statements. Specifically, a nested `catch` only catches failures in its scope and below, not errors higher up in the chain outside the nested scope. When used correctly, this gives greater precision in error recovery:

```js
doSomethingCritical()
  .then((result) =>
    doSomethingOptional(result)
      .then((optionalResult) => doSomethingExtraNice(optionalResult))
      .catch((e) => {}),
  ) // Ignore if optional stuff fails; proceed.
  .then(() => moreCriticalStuff())
  .catch((e) => console.error(`Critical failure: ${e.message}`));
```

Note that the optional steps here are nested — with the nesting caused not by the indentation, but by the placement of the outer (and) parentheses around the steps.

The inner error-silencing `catch` handler only catches failures from `doSomethingOptional()` and `doSomethingExtraNice()`, after which the code resumes with `moreCriticalStuff()`. Importantly, if `doSomethingCritical()` fails, its error is caught by the final (outer) `catch` only, and does not get swallowed by the inner `catch` handler.

In `async/await`, this code looks like:

```js
async function main() {
  try {
    const result = await doSomethingCritical();
    try {
      const optionalResult = await doSomethingOptional(result);
      await doSomethingExtraNice(optionalResult);
    } catch (e) {
      // Ignore failures in optional steps and proceed.
    }
    await moreCriticalStuff();
  } catch (e) {
    console.error(`Critical failure: ${e.message}`);
  }
}
```

> **Note:** If you don't have sophisticated error handling, you very likely don't need nested `then` handlers. Instead, use a flat chain and put the error handling logic at the end.

Chaining after a catch

It's possible to chain *after* a failure, i.e., a `catch`, which is useful to accomplish new actions even after an action failed in the chain. Read the following example:

```js
doSomething()
  .then(() => {
    throw new Error("Something failed");

    console.log("Do this");
  })
  .catch(() => {
    console.error("Do that");
  })
  .then(() => {
    console.log("Do this, no matter what happened before");
  });
```

This will output the following text:

```
Do that
Do this, no matter what happened before
```

> **Note:** The text "Do this" is not displayed because the "Something failed" error caused a rejection.

In `async/await`, this code looks like:

```js
async function main() {
  try {
    await doSomething();
    throw new Error("Something failed");
    console.log("Do this");
  } catch (e) {
    console.error("Do that");
  }
  console.log("Do this, no matter what happened before");
}
```

Promise rejection events

If a promise rejection event is not handled by any handler, it bubbles to the top of the call stack, and the host needs to surface it. On the web, whenever a promise is rejected, one of two events is sent to the global scope (generally, this is either the `window` or, if being used in a web worker, it's the `Worker` or other worker-based interface). The two events are:

`unhandledrejection`
: Sent when a promise is rejected but there is no rejection handler available.

`rejectionhandled`
: Sent when a handler is attached to a rejected promise that has already caused an `unhandledrejection` event.

In both cases, the event (of type `PromiseRejectionEvent`) has as members a `promise` property indicating the promise that was rejected, and a `reason` property that provides the reason given for the promise to be rejected.

These make it possible to offer fallback error handling for promises, as well as to help debug issues with your promise management. These handlers are global per context, so all errors will go to the same event handlers, regardless of source.

In Node.js, handling promise rejection is slightly different. You capture unhandled rejections by adding a handler for the Node.js `unhandledRejection` event (notice the difference in capitalization of the name), like this:

```js
process.on("unhandledRejection", (reason, promise) => {
  // Add code here to examine the "promise" and "reason" values
});
```

For Node.js, to prevent the error from being logged to the console (the default action that would otherwise occur), adding that `process.on()` listener is all that's necessary; there's no need for an equivalent of the browser runtime's `preventDefault()` method.

However, if you add that `process.on` listener but don't also have code within it to handle rejected promises, they will just be dropped on the floor and silently ignored. So ideally, you should add code within that listener to examine each rejected promise and make sure it was not caused by an actual code bug.

Composition

There are four composition tools for running asynchronous operations concurrently: `Promise.all()`, `Promise.allSettled()`, `Promise.any()`, and `Promise.race()`.

We can start operations at the same time and wait for them all to finish like this:

```js
Promise.all([func1(), func2(), func3()]).then(([result1, result2, result3]) => {
  // use result1, result2 and result3
});
```

If one of the promises in the array rejects, `Promise.all()` immediately rejects the returned promise. The other operations continue to run, but their outcomes are not available via the return value of `Promise.all()`. This may cause unexpected state or behavior. `Promise.allSettled()` is another composition tool that ensures all operations are complete before resolving.

These methods all run promises concurrently — a sequence of promises are started simultaneously and do not wait for each other. Sequential composition is possible using some clever JavaScript:

```js
[func1, func2, func3]
  .reduce((p, f) => p.then(f), Promise.resolve())
  .then((result3) => {
    /* use result3 */
  });
```

In this example, we reduce an array of asynchronous functions down to a promise chain. The code above is equivalent to:

```js
Promise.resolve()
  .then(func1)
  .then(func2)
  .then(func3)
  .then((result3) => {
    /* use result3 */
  });
```

This can be made into a reusable compose function, which is common in functional programming:

```js
const applyAsync = (acc, val) => acc.then(val);
const composeAsync =
  (...funcs) =>
  (x) =>
    funcs.reduce(applyAsync, Promise.resolve(x));
```

The `composeAsync()` function accepts any number of functions as arguments and returns a new function that accepts an initial value to be passed through the composition pipeline:

```js
const transformData = composeAsync(func1, func2, func3);
const result3 = transformData(data);
```

Sequential composition can also be done more succinctly with async/await:

```js
let result;
for (const f of [func1, func2, func3]) {
  result = await f(result);
}
/* use last result (i.e. result3) */
```

However, before you compose promises sequentially, consider if it's really necessary — it's always better to run promises concurrently so that they don't unnecessarily block each other unless one promise's execution depends on another's result.

Cancellation

`Promise` itself has no first-class protocol for cancellation, but you may be able to directly cancel the underlying asynchronous operation, typically using `AbortController`.

Creating a Promise around an old callback API

A `Promise` can be created from scratch using its constructor. This should be needed only to wrap old APIs.

In an ideal world, all asynchronous functions would already return promises. Unfortunately, some APIs still expect success and/or failure callbacks to be passed in the old way. The most obvious example is the `setTimeout()` function:

```js
setTimeout(() => saySomething("10 seconds passed"), 10 * 1000);
```

Mixing old-style callbacks and promises is problematic. If `saySomething()` fails or contains a programming error, nothing catches it. This is intrinsic to the design of `setTimeout()`.

Luckily we can wrap `setTimeout()` in a promise. The best practice is to wrap the callback-accepting functions at the lowest possible level, and then never call them directly again:

```js
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait(10 * 1000)
  .then(() => saySomething("10 seconds"))
  .catch(failureCallback);
```

The promise constructor takes an executor function that lets us resolve or reject a promise manually. Since `setTimeout()` doesn't really fail, we left out reject in this case. For more information on how the executor function works, see the `Promise()` reference.

Timing

Lastly, we will look into the more technical details, about when the registered callbacks get called.

Guarantees

In the callback-based API, when and how the callback gets called depends on the API implementor. For example, the callback may be called synchronously or asynchronously:

js · ✗ bad example

```
function doSomething(callback) {
  if (Math.random() > 0.5) {
    callback();
  } else {
    setTimeout(() => callback(), 1000);
  }
}
```

The above design is strongly discouraged because it leads to the so-called "state of Zalgo". In the context of designing asynchronous APIs, this means a callback is called synchronously in some cases but asynchronously in other cases, creating ambiguity for the caller. For further background, see the article Designing APIs for Asynchrony, where the term was first formally presented. This API design makes side effects hard to analyze:

js

```
let value = 1;
doSomething(() => {
  value = 2;
});
console.log(value); // 1 or 2?
```

On the other hand, promises are a form of inversion of control — the API implementor does not control when the callback gets called. Instead, the job of maintaining the callback queue and deciding when to call the callbacks is delegated to the promise implementation, and both the API user and API developer automatically get strong semantic guarantees, including:

- Callbacks added with `then()` will never be invoked before the completion of the current run of the JavaScript event loop.
- These callbacks will be invoked even if they were added *after* the success or failure of the asynchronous operation that the promise represents.
- Multiple callbacks may be added by calling `then()` several times. They will be invoked one after another, in the order in which they were inserted.

To avoid surprises, functions passed to `then()` will never be called synchronously, even with an already-resolved promise:

js

```
Promise.resolve().then(() => console.log(2));
console.log(1);
// Logs: 1, 2
```

Instead of running immediately, the passed-in function is put on a microtask queue, which means it runs later (only after the function which created it exits, and when the JavaScript execution stack is empty), just before control is returned to the event loop; i.e., pretty soon:

```js
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

wait(0).then(() => console.log(4));
Promise.resolve()
  .then(() => console.log(2))
  .then(() => console.log(3));
console.log(1); // 1, 2, 3, 4
```

Task queues vs. microtasks

Promise callbacks are handled as a microtask whereas `setTimeout()` callbacks are handled as task queues.

```js
const promise = new Promise((resolve, reject) => {
  console.log("Promise callback");
  resolve();
}).then((result) => {
  console.log("Promise callback (.then)");
});

setTimeout(() => {
  console.log("event-loop cycle: Promise (fulfilled)", promise);
}, 0);

console.log("Promise (pending)", promise);
```

The code above will output:

```
Promise callback
Promise (pending) Promise {<pending>}
Promise callback (.then)
event-loop cycle: Promise (fulfilled) Promise {<fulfilled>}
```

For more details, refer to Tasks vs. microtasks.

When promises and tasks collide

If you run into situations in which you have promises and tasks (such as events or callbacks) which are firing in unpredictable orders, it's possible you may benefit from using a microtask to check status or balance out your promises when promises are created conditionally.

If you think microtasks may help solve this problem, see the microtask guide to learn more about how to use `queueMicrotask()` to enqueue a function as a microtask.

See also

- `Promise`
- `async function`
- `await`
- Promises/A+ specification
- We have a problem with promises on pouchdb.com (2015)

JavaScript › Guide

JavaScript typed arrays

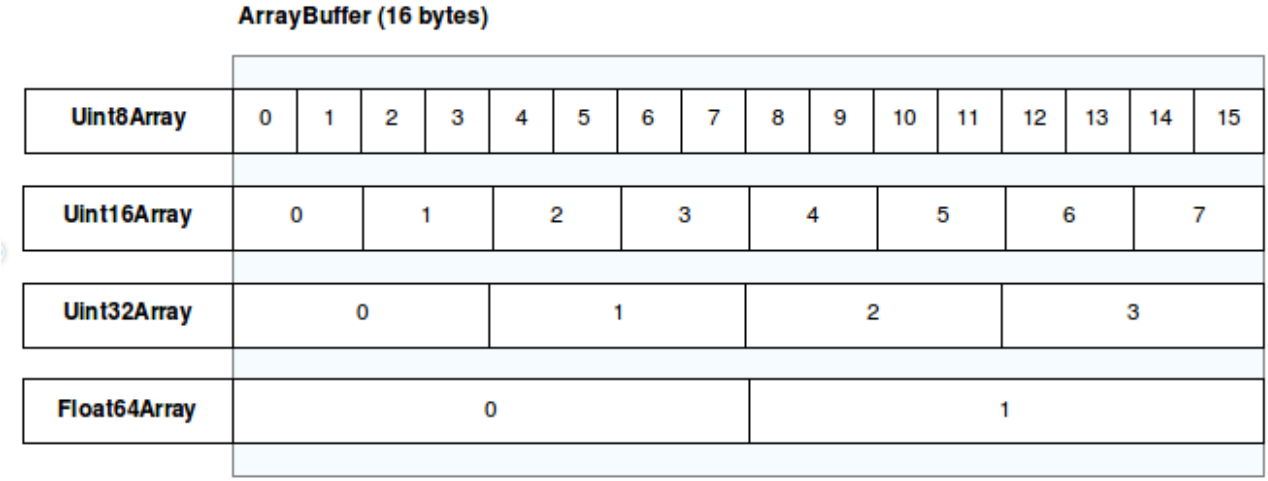
Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Typed_arrays · MDN page last modified 2025-07-08

JavaScript typed arrays are array-like objects that provide a mechanism for reading and writing raw binary data in memory buffers.

Typed arrays are not intended to replace arrays for any kind of functionality. Instead, they provide developers with a familiar interface for manipulating binary data. This is useful when interacting with platform features, such as audio and video manipulation, access to raw data using WebSockets, and so forth. Each entry in a JavaScript typed array is a raw binary value in one of a number of supported formats, from 8-bit integers to 64-bit floating-point numbers.

Typed array objects share many of the same methods as arrays with similar semantics. However, typed arrays are *not* to be confused with normal arrays, as calling `Array.isArray()` on a typed array returns `false`. Moreover, not all methods available for normal arrays are supported by typed arrays (e.g., push and pop).

To achieve maximum flexibility and efficiency, JavaScript typed arrays split the implementation into *buffers* and *views*. A buffer is an object representing a chunk of data; it has no format to speak of, and offers no mechanism for accessing its contents. In order to access the memory contained in a buffer, you need to use a view. A view provides a *context* — that is, a data type, starting offset, and number of elements.



Buffers

There are two types of buffers: `ArrayBuffer` and `SharedArrayBuffer`. Both are low-level representations of a memory span. They have "array" in their names, but they don't have much to do with arrays — you cannot read or write to them directly. Instead, buffers are generic objects that just contain raw data. In order to access the memory represented by a buffer, you need to use a view.

Buffers support the following actions:

- *Allocate*: As soon as a new buffer is created, a new memory span is allocated and initialized to `0`.
- *Copy*: Using the `slice()` method, you can efficiently copy a portion of the memory without creating views to manually copy each byte.

- *Transfer*: Using the `transfer()` and `transferToFixedLength()` methods, you can transfer ownership of the memory span to a new buffer object. This is useful when transferring data between different execution contexts without copying. After the transfer, the original buffer is no longer usable. A `SharedArrayBuffer` cannot be transferred (as the buffer is already shared by all execution contexts).
- *Resize*: Using the `resize()` method, you can resize the memory span (either claim more memory space, as long as it doesn't pass the pre-set `maxByteLength` limit, or release some memory space). `SharedArrayBuffer` can only be grown but not shrunk.

The difference between `ArrayBuffer` and `SharedArrayBuffer` is that the former is always owned by a single execution context at a time. If you pass an `ArrayBuffer` to a different execution context, it is *transferred* and the original `ArrayBuffer` becomes unusable. This ensures that only one execution context can access the memory at a time. A `SharedArrayBuffer` is not transferred when passed to a different execution context, so it can be accessed by multiple execution contexts at the same time. This may introduce race conditions when multiple threads access the same memory span, so operations such as `Atomics` methods become useful.

Views

There are currently two main kinds of views: typed array views and `DataView`. Typed arrays provide utility methods that allow you to conveniently transform binary data. `DataView` is more low-level and allows granular control of how data is accessed. The ways to read and write data using the two views are very different.

Both kinds of views cause `ArrayBuffer.isView()` to return `true`. They both have the following properties:

`buffer`
: The underlying buffer that the view references.

`byteOffset`
: The offset, in bytes, of the view from the start of its buffer.

`byteLength`
: The length, in bytes, of the view.

Both constructors accept the above three as separate arguments, although typed array constructors accept `length` as the number of elements rather than the number of bytes.

Typed array views

Typed array views have self-descriptive names and provide views for all the usual numeric types like `Int8`, `Uint32`, `Float64` and so forth. There is one special typed array view, `Uint8ClampedArray`, which clamps the values between `0` and `255`. This is useful for Canvas data processing, for example.

Type	Value Range	Size in bytes	Web IDL type
`Int8Array`	-128 to 127	1	`byte`
`Uint8Array`	0 to 255	1	`octet`
`Uint8ClampedArray`	0 to 255	1	`octet`
`Int16Array`	-32768 to 32767	2	`short`
`Uint16Array`	0 to 65535	2	`unsigned short`
`Int32Array`	-2147483648 to 2147483647	4	`long`

Type	Value Range	Size in bytes	Web IDL type
Uint32Array	0 to 4294967295	4	unsigned long
Float16Array	-65504 to 65504	2	N/A
Float32Array	-3.4e38 to 3.4e38	4	unrestricted float
Float64Array	-1.8e308 to 1.8e308	8	unrestricted double
BigInt64Array	-2^{63} to $2^{63} - 1$	8	bigint
BigUint64Array	0 to $2^{64} - 1$	8	bigint

All typed array views have the same methods and properties, as defined by the TypedArray class. They only differ in the underlying data type and the size in bytes. This is discussed in more detail in Value encoding and normalization.

Typed arrays are, in principle, fixed-length, so array methods that may change the length of an array are not available. This includes pop, push, shift, splice, and unshift. In addition, flat is unavailable because there are no nested typed arrays, and related methods including concat and flatMap do not have great use cases so are unavailable. As splice is unavailable, so too is toSpliced. All other array methods are shared between Array and TypedArray.

On the other hand, TypedArray has the extra set and subarray methods that optimize working with multiple typed arrays that view the same buffer. The set() method allows setting multiple typed array indices at once, using data from another array or typed array. If the two typed arrays share the same underlying buffer, the operation may be more efficient as it's a fast memory move. The subarray() method creates a new typed array view that references the same buffer as the original typed array, but with a narrower span.

There's no way to directly change the length of a typed array without changing the underlying buffer. However, when the typed array views a resizable buffer and does not have a fixed byteLength, it is *length-tracking*, and will automatically resize to fit the underlying buffer as the resizable buffer is resized. See Behavior when viewing a resizable buffer for details.

Similar to regular arrays, you can access typed array elements using bracket notation. The corresponding bytes in the underlying buffer are retrieved and interpreted as a number. Any property access using a number (or the string representation of a number, since numbers are always converted to strings when accessing properties) will be proxied by the typed array — they never interact with the object itself. This means, for example:

- Out-of-bounds index access always returns undefined, without actually accessing the property on the object.
- Any attempt to write to such an out-of-bounds property has no effect: it does not throw an error but doesn't change the buffer or typed array either.
- Typed array indices appear to be configurable and writable, but any attempt to change their attributes will fail.

```js
const uint8 = new Uint8Array([1, 2, 3]);
console.log(uint8[0]); // 1
```

(code continues on the next page)

(code continued from the previous page)

```
// For illustrative purposes only. Not for production code.
uint8[-1] = 0;
uint8[2.5] = 0;
uint8[NaN] = 0;
console.log(Object.keys(uint8)); // ["0", "1", "2"]
console.log(uint8[NaN]); // undefined

// Non-numeric access still works
uint8[true] = 0;
console.log(uint8[true]); // 0

Object.freeze(uint8); // TypeError: Cannot freeze array buffer views with elements
```

DataView

The `DataView` is a low-level interface that provides a getter/setter API to read and write arbitrary data to the buffer. This is useful when dealing with different types of data, for example. Typed array views are in the native byte-order (see Endianness) of your platform. With a `DataView`, the byte-order can be controlled. By default, it's big-endian—the bytes are ordered from most significant to least significant. This can be reversed, with the bytes ordered from least significant to most significant (little-endian), using getter/setter methods.

`DataView` does not require alignment; multi-byte read and write can be started at any specified offset. The setter methods work the same way.

The following example uses a `DataView` to get the binary representation of any number:

js

```
function toBinary(
  x,
  { type = "Float64", littleEndian = false, separator = " ", radix = 16 } = {},
) {
  const bytesNeeded = globalThis[`${type}Array`].BYTES_PER_ELEMENT;
  const dv = new DataView(new ArrayBuffer(bytesNeeded));
  dv[`set${type}`](0, x, littleEndian);
  const bytes = Array.from({ length: bytesNeeded }, (_, i) =>
    dv
      .getUint8(i)
      .toString(radix)
      .padStart(8 / Math.log2(radix), "0"),
  );
  return bytes.join(separator);
}

console.log(toBinary(1.1)); // 3f f1 99 99 99 99 99 9a
console.log(toBinary(1.1, { littleEndian: true })); // 9a 99 99 99 99 99 f1 3f
console.log(toBinary(20, { type: "Int8", radix: 2 })); // 00010100
```

Web APIs using typed arrays

These are some examples of APIs that make use of typed arrays; there are others, and more are being added all the time.

`FileReader.prototype.readAsArrayBuffer()`

The `FileReader.prototype.readAsArrayBuffer()` method starts reading the contents of the specified `Blob` or `File`.

`fetch()`

The `body` option to `fetch()` can be a typed array or `ArrayBuffer`, enabling you to send these objects as the payload of a `POST` request.

`ImageData.data`

Is a `Uint8ClampedArray` representing a one-dimensional array containing the data in the RGBA order, with integer values between `0` and `255` inclusive.

Examples

Using views with buffers

First of all, we will need to create a buffer, here with a fixed length of 16-bytes:

```js
const buffer = new ArrayBuffer(16);
```

At this point, we have a chunk of memory whose bytes are all pre-initialized to 0. There's not a lot we can do with it, though. For example, we can confirm that the buffer is the right size:

```js
if (buffer.byteLength === 16) {
  console.log("Yes, it's 16 bytes.");
} else {
  console.log("Oh no, it's the wrong size!");
}
```

Before we can really work with this buffer, we need to create a view. Let's create a view that treats the data in the buffer as an array of 32-bit signed integers:

```js
const int32View = new Int32Array(buffer);
```

Now we can access the fields in the array just like a normal array:

```js
for (let i = 0; i < int32View.length; i++) {
  int32View[i] = i * 2;
}
```

This fills out the 4 entries in the array (4 entries at 4 bytes each makes 16 total bytes) with the values `0`, `2`, `4`, and `6`.

Multiple views on the same data

Things start to get really interesting when you consider that you can create multiple views onto the same data. For example, given the code above, we can continue like this:

```js
const int16View = new Int16Array(buffer);

for (let i = 0; i < int16View.length; i++) {
  console.log(`Entry ${i}: ${int16View[i]}`);
}
```

Here we create a 16-bit integer view that shares the same buffer as the existing 32-bit view and we output all the values in the buffer as 16-bit integers. Now we get the output `0`, `0`, `2`, `0`, `4`, `0`, `6`, `0` (assuming

little-endian encoding):

```
Int16Array  |   0  |  0   |   2  |  0   |   4  |  0   |   6  |  0   |
Int32Array  |      0      |      2      |      4      |      6      |
ArrayBuffer | 00 00 00 00 | 02 00 00 00 | 04 00 00 00 | 06 00 00 00 |
```

You can go a step farther, though. Consider this:

```js
int16View[0] = 32;
console.log(`Entry 0 in the 32-bit array is now ${int32View[0]}`);
```

The output from this is `"Entry 0 in the 32-bit array is now 32"`.

In other words, the two arrays are indeed viewed on the same data buffer, treating it as different formats.

```
Int16Array  |  32  |  0   |   2  |  0   |   4  |  0   |   6  |  0   |
Int32Array  |     32      |      2      |      4      |      6      |
ArrayBuffer | 20 00 00 00 | 02 00 00 00 | 04 00 00 00 | 06 00 00 00 |
```

You can do this with any view type, although if you set an integer and then read it as a floating-point number, you will probably get a strange result because the bits are interpreted differently.

```js
const float32View = new Float32Array(buffer);
console.log(float32View[0]); // 4.484155085839415e-44
```

Reading text from a buffer

Buffers don't always represent numbers. For example, reading a file can give you a text data buffer. You can read this data out of the buffer using a typed array.

The following reads UTF-8 text using the `TextDecoder` web API:

```js
const buffer = new ArrayBuffer(8);
const uint8 = new Uint8Array(buffer);
// Data manually written here, but pretend it was already in the buffer
uint8.set([228, 189, 160, 229, 165, 189]);
const text = new TextDecoder().decode(uint8);
console.log(text); // "你好"
```

The following reads UTF-16 text using the `String.fromCharCode()` method:

```js
const buffer = new ArrayBuffer(8);
const uint16 = new Uint16Array(buffer);
// Data manually written here, but pretend it was already in the buffer
uint16.set([0x4f60, 0x597d]);
const text = String.fromCharCode(...uint16);
console.log(text); // "你好"
```

Working with complex data structures

By combining a single buffer with multiple views of different types, starting at different offsets into the buffer, you can interact with data objects containing multiple data types. This lets you, for example, interact with complex data structures from WebGL or data files.

Consider this C structure:

```c
struct someStruct {
    unsigned long id;
    char username[16];
    float amountDue;
};
```

You can access a buffer containing data in this format like this:

```js
const buffer = new ArrayBuffer(24);

// … read the data into the buffer …

const idView = new Uint32Array(buffer, 0, 1);
const usernameView = new Uint8Array(buffer, 4, 16);
const amountDueView = new Float32Array(buffer, 20, 1);
```

Then you can access, for example, the amount due with `amountDueView[0]`.

> **Note:** The data structure alignment in a C structure is platform-dependent. Take precautions and considerations for these padding differences.

Conversion to normal arrays

After processing a typed array, it is sometimes useful to convert it back to a normal array in order to benefit from the `Array` prototype. This can be done using `Array.from()`:

```js
const typedArray = new Uint8Array([1, 2, 3, 4]);
const normalArray = Array.from(typedArray);
```

as well as the spread syntax:

```js
const typedArray = new Uint8Array([1, 2, 3, 4]);
const normalArray = [...typedArray];
```

See also

- Faster Canvas Pixel Manipulation with Typed Arrays on hacks.mozilla.org (2011)
- Typed arrays - Binary data in the browser on web.dev (2012)
- Endianness
- `ArrayBuffer`
- `DataView`
- `TypedArray`
- `SharedArrayBuffer`

JavaScript › Guide

Iterators and generators

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Iterators_and_generators · MDN page last modified 2026-08-21

Iterators and Generators bring the concept of iteration directly into the core language and provide a mechanism for customizing the behavior of `for...of` loops.

For details, see also:

- Iteration protocols
- `for...of`
- `function*` and `Generator`
- `yield` and `yield*`

Iterators

In JavaScript an **iterator** is an object which defines a sequence and potentially a return value upon its termination.

Specifically, an iterator is any object which implements the Iterator protocol by having a `next()` method that returns an object with two properties:

`value`
The next value in the iteration sequence.

`done`
This is `true` if the last value in the sequence has already been consumed. If `value` is present alongside `done`, it is the iterator's return value.

Once created, an iterator object can be iterated explicitly by repeatedly calling `next()`. Iterating over an iterator is said to consume the iterator, because it is generally only possible to do once. After a terminating value has been yielded additional calls to `next()` should continue to return `{done: true}`.

The most common iterator in JavaScript is the Array iterator, which returns each value in the associated array in sequence.

While it is easy to imagine that all iterators could be expressed as arrays, this is not true. Arrays must be allocated in their entirety, but iterators are consumed only as necessary. Because of this, iterators can express sequences of unlimited size, such as the range of integers between `0` and `Infinity`.

Here is an example which can do just that. It allows creation of a range iterator which defines a sequence of integers from `start` (inclusive) to `end` (exclusive) spaced `step` apart. Its final return value is the size of the sequence it created, tracked by the variable `iterationCount`.

```js
function makeRangeIterator(start = 0, end = Infinity, step = 1) {
  let nextIndex = start;
  let iterationCount = 0;

  const rangeIterator = {
    next() {
      let result;
      if (nextIndex < end) {
        result = { value: nextIndex, done: false };
        nextIndex += step;
        iterationCount++;
```

(code continues on the next page)

(code continued from the previous page)

```
        return result;
      }
      return { value: iterationCount, done: true };
    },
  };
  return rangeIterator;
}
```

Using the iterator then looks like this:

js

```
const iter = makeRangeIterator(1, 10, 2);

let result = iter.next();
while (!result.done) {
  console.log(result.value); // 1 3 5 7 9
  result = iter.next();
}

console.log("Iterated over sequence of size:", result.value); // [5 numbers returned, that
took interval in between: 0 to 10]
```

> **Note:** It is not possible to know reflectively whether a particular object is an iterator. If you need to do this, use Iterables.

Generator functions

While custom iterators are a useful tool, their creation requires careful programming due to the need to explicitly maintain their internal state. **Generator functions** provide a powerful alternative: they allow you to define an iterative algorithm by writing a single function whose execution is not continuous. Generator functions are written using the `function*` syntax.

When called, generator functions do not initially execute their code. Instead, they return a special type of iterator, called a **Generator**. When a value is consumed by calling the generator's `next` method, the Generator function executes until it encounters the `yield` keyword.

The function can be called as many times as desired, and returns a new Generator each time. Each Generator may only be iterated once.

We can now adapt the example from above. The behavior of this code is identical, but the implementation is much easier to write and read.

js

```
function* makeRangeIterator(start = 0, end = Infinity, step = 1) {
  let iterationCount = 0;
  for (let i = start; i < end; i += step) {
    iterationCount++;
    yield i;
  }
  return iterationCount;
}
```

Iterables

An object is **iterable** if it defines its iteration behavior, such as what values are looped over in a `for...of` construct. Some built-in types, such as `Array` or `Map`, have a default iteration behavior, while other types (such as `Object`) do not.

In order to be **iterable**, an object must implement the `[Symbol.iterator]()` method. This means that the object (or one of the objects up its prototype chain) must have a property with a `Symbol.iterator` key.

It may be possible to iterate over an iterable more than once, or only once. It is up to the programmer to know which is the case.

Iterables which can iterate only once (such as Generators) customarily return `this` from their `[Symbol.iterator]()` method, whereas iterables which can be iterated many times must return a new iterator on each invocation of `[Symbol.iterator]()`.

```js
function* makeIterator() {
  yield 1;
  yield 2;
}

const iter = makeIterator();

for (const itItem of iter) {
  console.log(itItem);
}

console.log(iter[Symbol.iterator]() === iter); // true

// This example shows us generator(iterator) is an iterable object,
// which has the [Symbol.iterator]() method return the `iter` (itself),
// and consequently, the `iter` object can iterate only _once_.

// If we change the [Symbol.iterator]() method of `iter` to a function/generator
// which returns a new iterator/generator object, `iter`
// can iterate many times

iter[Symbol.iterator] = function* () {
  yield 2;
  yield 1;
};
```

User-defined iterables

You can make your own iterables like this:

```js
const myIterable = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  },
};
```

User-defined iterables can be used in `for...of` loops or the spread syntax as usual.

```js
for (const value of myIterable) {
  console.log(value);
}
// 1
// 2
// 3

[...myIterable]; // [1, 2, 3]
```

Built-in iterables

`String`, `Array`, `TypedArray`, `Map` and `Set` are all built-in iterables, because their prototype objects all have a `Symbol.iterator` method.

Syntaxes expecting iterables

Some statements and expressions expect iterables. For example: the `for...of` loops, spread syntax, `yield*`, and destructuring syntax.

```js
for (const value of ["a", "b", "c"]) {
  console.log(value);
}
// "a"
// "b"
// "c"

[..."abc"];
// ["a", "b", "c"]

function* gen() {
  yield* ["a", "b", "c"];
}

gen().next();
// { value: "a", done: false }

[a, b, c] = new Set(["a", "b", "c"]);
a;
// "a"
```

Advanced generators

Generators compute their `yield`ded values *on demand*, which allows them to efficiently represent sequences that are expensive to compute (or even infinite sequences, as demonstrated above).

The `next()` method also accepts a value, which can be used to modify the internal state of the generator. A value passed to `next()` will be received by `yield`.

> **Note:** A value passed to the *first* invocation of `next()` is always ignored.

Here is the fibonacci generator using `next(x)` to restart the sequence:

```js
function* fibonacci() {
  let current = 0;
  let next = 1;
  while (true) {
    const reset = yield current;
    [current, next] = [next, next + current];
    if (reset) {
      current = 0;
      next = 1;
    }
  }
}

const sequence = fibonacci();
console.log(sequence.next().value); // 0
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
console.log(sequence.next().value); // 3
console.log(sequence.next().value); // 5
console.log(sequence.next().value); // 8
console.log(sequence.next(true).value); // 0
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 1
console.log(sequence.next().value); // 2
```

You can force a generator to throw an exception by calling its `throw()` method and passing the exception value it should throw. This exception will be thrown from the current suspended context of the generator, as if the `yield` that is currently suspended were instead a `throw value` statement.

If the exception is not caught from within the generator, it will propagate up through the call to `throw()`, and subsequent calls to `next()` will result in the `done` property being `true`.

Generators have a `return()` method that returns the given value and finishes the generator itself.

JavaScript › Guide

JavaScript resource management

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Resource_management · MDN page last modified 2026-09-11

This guide talks about how to do *resource management* in JavaScript. Resource management is not exactly the same as memory management, which is a more advanced topic and usually handled automatically by JavaScript. Resource management is about managing resources that are *not* automatically cleaned up by JavaScript. Sometimes, it's okay to have some unused objects in memory, because they don't interfere with application logic, but resource leaks often lead to things not working, or a lot of excess memory usage. Therefore, this is not an optional feature about optimization, but a core feature to write correct programs!

> **Note:** While memory management and resource management are two separate topics, sometimes you can hook into the memory management system to do resource management, as a last resort. For example, if you have a JavaScript object representing a handle of an external resource, you can create a `FinalizationRegistry` to clean up the resource when the handle is garbage collected, because there is definitely no way to access the resource afterwards. However, there is no guarantee that the finalizer will run, so it's not a good idea to rely on it for critical resources.

Problem

Let's first look at a few examples of resources that need to be managed:

- **File handles**: A file handle is used to read and write bytes in a file. When you are done with it, you must call `fileHandle.close()`, otherwise the file will remain open, even when the JS object is no longer accessible. As the linked Node.js docs say:

  > If a `<FileHandle>` is not closed using the `fileHandle.close()` method, it will try to automatically close the file descriptor and emit a process warning, helping to prevent memory leaks. Please do not rely on this behavior because it can be unreliable and the file may not be closed. Instead, always explicitly close `<FileHandle>`s. Node.js may change this behavior in the future.

- **Network connections**: Some connections, such as `WebSocket` and `RTCPeerConnection`, need to be closed if no messages are transmitted. Otherwise, the connection remains open, and connection pools are often very limited in size.
- **Stream readers**: If you don't call `ReadableStreamDefaultReader.releaseLock()`, the stream will be locked and does not permit another reader to consume it.

Here is one concrete example, using a readable stream:

```js
const stream = new ReadableStream({
  start(controller) {
    controller.enqueue("a");
    controller.enqueue("b");
    controller.enqueue("c");
    controller.close();
  },
});
```

(code continues on the next page)

(code continued from the previous page)

```
async function readUntil(stream, text) {
  const reader = stream.getReader();
  let chunk = await reader.read();

  while (!chunk.done && chunk.value !== text) {
    console.log(chunk);
    chunk = await reader.read();
  }
  // We forgot to release the lock here
}

readUntil(stream, "b").then(() => {
  const anotherReader = stream.getReader();
  // TypeError: ReadableStreamDefaultReader constructor can only
  // accept readable streams that are not yet locked to a reader
});
```

Here, we have a stream that emits three chunks of data. We read from the stream until we find the letter "b". When `readUntil` returns, the stream is only partially consumed, so we should be able to continue to read from it using another reader. However, we forgot to release the lock, so although `reader` is no longer available, the stream is still locked and we cannot create another reader.

The solution in this case is straightforward: call `reader.releaseLock()` at the end of `readUntil`. But, a few issues still remain:

- Inconsistency: different resources have different ways to release them. For example, we have `close()`, `releaseLock()`, `disconnect()`, etc. The pattern does not generalize.
- Error handling: what happens if the `reader.read()` call fails? Then `readUntil` would terminate and never get to the `reader.releaseLock()` call. We can fix this using `try...finally`:

  js

  ```
  async function readUntil(stream, text) {
    const reader = stream.getReader();
    try {
      let chunk = await reader.read();

      while (!chunk.done && chunk.value !== text) {
        console.log(chunk);
        chunk = await reader.read();
      }
    } finally {
      reader.releaseLock();
    }
  }
  ```

  But you have to remember to do this every time you have some important resource to release.
- Scoping: in the above example, `reader` is already closed when we exit the `try...finally` statement, but it continues to be available in its scope. This means you may accidentally use it after it's closed.
- Multiple resources: if we have two readers on different streams, we have to remember to release both of them. This is a respectable attempt to do so:

```js
const reader1 = stream1.getReader();
const reader2 = stream2.getReader();
try {
  // do something with reader1 and reader2
} finally {
  reader1.releaseLock();
  reader2.releaseLock();
}
```

However, this introduces more error handling troubles. If `stream2.getReader()` throws, then `reader1` does not get released; if `reader1.releaseLock()` throws an error, then `reader2` does not get released. This means we actually have to wrap each resource acquisition-release pair in its own `try...finally`:

```js
const reader1 = stream1.getReader();
try {
  const reader2 = stream2.getReader();
  try {
    // do something with reader1 and reader2
  } finally {
    reader2.releaseLock();
  }
} finally {
  reader1.releaseLock();
}
```

You see how a seemingly benign task of calling `releaseLock` can quickly lead to nested boilerplate code. This is why JavaScript provides integrated language support for resource management.

The `using` and `await using` declarations

The solution we have is two special kinds of variable declaration: `using` and `await using`. They are similar to `const`, but they automatically release the resource when the variable goes out of scope as long as the resource is *disposable*. Using the same example as above, we can rewrite it as:

```js
{
  using reader1 = stream1.getReader();
  using reader2 = stream2.getReader();

  // do something with reader1 and reader2

  // Before we exit the block, reader1 and reader2 are automatically released
}
```

> **Note:** At the time of writing, `ReadableStreamDefaultReader` does not implement the disposable protocol. This is a hypothetical example.

First, notice the extra braces around the code. This creates a new block scope for the `using` declarations. Resources declared with `using` are automatically freed when they go out of the scope of `using`, which, in this case, is whenever we are exiting the block, either because all statements have executed, or because an error or `return`/`break`/`continue` was encountered somewhere.

This means `using` can only be used in a scope that has a clear lifetime—namely, it cannot be used at the top level of a script, because variables at the top level of a script are in scope for all future scripts on the page, which practically means the resource can never be freed if the page never unloads. However, you can use it at the top level of a module, because the module scope ends when the module finishes executing.

Now we know *when* `using` does cleanup. But *how* is it done? `using` requires the resource to implement the *disposable* protocol. An object is disposable if it has the `[Symbol.dispose]()` method. This method is called with no arguments to perform cleanup. For example, in the reader case, the `[Symbol.dispose]` property can be a simple alias or wrapper of `releaseLock`:

```js
// For demonstration
class MyReader {
  // A wrapper
  [Symbol.dispose]() {
    this.releaseLock();
  }
  releaseLock() {
    // Logic to release resources
  }
}

// OR, an alias
MyReader.prototype[Symbol.dispose] = MyReader.prototype.releaseLock;
```

Through the disposable protocol, `using` can dispose all resources in a consistent fashion without understanding what type of resource it is.

Every scope has a list of resources associated with it, in the order they were declared. When the scope exits, the resources are disposed in reverse order, by calling their `[Symbol.dispose]()` method. For example, in the example above, `reader1` is declared before `reader2`, so `reader2` is disposed first, then `reader1`. Errors thrown when attempting to dispose one resource will not prevent disposal of other resources. This is consistent with the `try...finally` pattern, and respects possible dependencies between resources.

`await using` is very similar to `using`. The syntax tells you that an `await` happens somewhere—not when the resource is declared, but actually when it's disposed. `await using` requires the resource to be *async disposable*, which means it has a `[Symbol.asyncDispose]()` method. This method is called with no arguments and returns a promise that resolves when the cleanup is done. This is useful when the cleanup is asynchronous, such as `fileHandle.close()`, in which case the result of the disposal can only be known asynchronously.

```js
{
  await using fileHandle = open("file.txt", "w");
  await fileHandle.write("Hello");

  // fileHandle.close() is called and awaited
}
```

Because `await using` requires doing an `await`, it is only permitted in contexts where `await` is, which includes inside `async` functions and top-level `await` in modules.

Resources are cleaned up sequentially, not concurrently: the return value of one resource's `[Symbol.asyncDispose]()` method will be `awaited` before the next resource's `[Symbol.asyncDispose]()` method is called.

Some things to note:

- `using` and `await using` are *opt in*. If you declare your resource using `let`, `const`, or `var`, no automatic disposal happens, just like any other non-disposable values.
- `using` and `await using` require the resource to be disposable (or async disposable). If the resource does not have the `[Symbol.dispose]()` or `[Symbol.asyncDispose]()` method respectively, you will get a `TypeError` at the line of declaration. The resource can be `null` or `undefined`, however, allowing you to conditionally acquire resources.
- Like `const`, `using` and `await using` variables cannot be re-assigned, although the properties of the objects they hold can be changed. However, the `[Symbol.dispose]()`/`[Symbol.asyncDispose]()` method is already saved at the time of declaration, so changing the method after the declaration does not affect the cleanup.
- There are some gotchas when conflating scopes with resource lifetime. See `using` for a few examples.

The `DisposableStack` and `AsyncDisposableStack` objects

`using` and `await using` are special syntaxes. Syntaxes are convenient and hide a lot of the complexity, but sometimes you need to do things manually.

For one common example: what if you don't want to dispose the resource at the end of *this* scope, but at some *later* scope? Consider this:

```js
let reader;
if (someCondition) {
  reader = stream.getReader();
} else {
  reader = stream.getReader({ mode: "byob" });
}
```

As we said, `using` is like `const`: it must be initialized and can't be reassigned, so you may attempt this:

```js
if (someCondition) {
  using reader = stream.getReader();
} else {
  using reader = stream.getReader({ mode: "byob" });
}
```

However, this means all logic has to be written inside the `if` or `else`, causing a lot of duplication. What we want to do is to acquire and register the resource in one scope but dispose it in another. We can use a `DisposableStack` for this purpose, which is an object which holds a collection of disposable resources and which itself is disposable:

```js
{
  using disposer = new DisposableStack();
  let reader;
  if (someCondition) {
    reader = disposer.use(stream.getReader());
  } else {
    reader = disposer.use(stream.getReader({ mode: "byob" }));
  }
  // Do something with reader
  // Before scope exit, disposer is disposed, which disposes reader
}
```

You may have a resource that does not yet implement the disposable protocol, so it will be rejected by `using`. In this case, you can use `adopt()`.

```js
{
  using disposer = new DisposableStack();
  // Suppose reader does not have the [Symbol.dispose]() method,
  // then it cannot be used with using.
  // However, we can manually pass a disposer function to disposer.adopt
  const reader = disposer.adopt(stream.getReader(), (reader) =>
    reader.releaseLock(),
  );
  // Do something with reader
  // Before scope exit, disposer is disposed, which disposes reader
}
```

You may have a disposal action to perform but it's not "tethered" to any resource in particular. Maybe you just want to log a message saying "All database connections closed" when there are multiple connections open simultaneously. In this case, you can use `defer()`.

```js
{
  using disposer = new DisposableStack();
  disposer.defer(() => console.log("All database connections closed"));
  const connection1 = disposer.use(openConnection());
  const connection2 = disposer.use(openConnection());
  // Do something with connection1 and connection2
  // Before scope exit, disposer is disposed, which first disposes connection1
  // and connection2 and then logs the message
}
```

You may want to do *conditional* disposal—for example, only dispose claimed resources when an error occurred. In this case, you can use `move()` to preserve the resources which would otherwise be disposed.

```js
class MyResource {
  #resource1;
  #resource2;
  #disposables;
  constructor() {
    using disposer = new DisposableStack();
    this.#resource1 = disposer.use(getResource1());
    this.#resource2 = disposer.use(getResource2());
    // If we made it here, then there were no errors during construction and
```

(code continues on the next page)

(code continued from the previous page)

```
    // we can safely move the disposables out of `disposer` and into `#disposables`.
    this.#disposables = disposer.move();
    // If construction failed, then `disposer` would be disposed before reaching
    // the line above, disposing `#resource1` and `#resource2`.
  }
  [Symbol.dispose]() {
    this.#disposables.dispose(); // Dispose `#resource2` and `#resource1`.
  }
}
```

`AsyncDisposableStack` is like `DisposableStack`, but for use with async disposable resources. Its `use()` method expects an async disposable, its `adopt()` method expects an async cleanup function, and its `dispose()` method expects an async callback. It provides a `[Symbol.asyncDispose]()` method. You can still pass it sync resources if you have a mix of both sync and async.

The reference for `DisposableStack` contains more examples and details.

Error handling

A major use case of the resource management feature is to ensure that resources are always disposed, even when an error occurs. Let us investigate some complex error handling scenarios.

We start with the following code, which, by using `using`, is robust against errors:

js

```
async function readUntil(stream, text) {
  // Use `using` instead of `await using` because `releaseLock` is synchronous
  using reader = stream.getReader();
  let chunk = await reader.read();

  while (!chunk.done && chunk.value !== text) {
    console.log(chunk.toUpperCase());
    chunk = await reader.read();
  }
}
```

Suppose that `chunk` turns out to be `null`. Then `!chunk.done` will throw a `TypeError`, causing the function to terminate. Before the function exits, `stream[Symbol.dispose]()` is called, which releases the lock on the stream.

js

```
const stream = new ReadableStream({
  start(controller) {
    controller.enqueue("a");
    controller.enqueue(null);
    controller.enqueue("b");
    controller.enqueue("c");
    controller.close();
  },
});

readUntil(stream, "b")
  .catch((e) => console.error(e)) // TypeError: chunk.toUpperCase is not a function
  .then(() => {
    const anotherReader = stream.getReader();
    // Successfully creates another reader
  });
```

So, `using` does not swallow any errors: all errors that occur are still thrown, but the resources get closed right before that. Now, what happens if the resource cleanup itself also throws an error? Let's use a more contrived example:

```js
class MyReader {
  [Symbol.dispose]() {
    throw new Error("Failed to release lock");
  }
}

function doSomething() {
  using reader = new MyReader();
  throw new Error("Failed to read");
}

try {
  doSomething();
} catch (e) {
  console.error(e); // SuppressedError: An error was suppressed during disposal
}
```

There are two errors generated in the `doSomething()` call: an error thrown during `doSomething`, and an error thrown during disposal of `reader` because of the first error. Both errors are thrown together, so what you caught is a `SuppressedError`. This is a special error that wraps two errors: the `error` property contains the later error, and the `suppressed` property contains the earlier error, which gets "suppressed" by the later error.

If we have more than one resource, and *both* of them throw an error during disposal (this should be exceedingly rare–it's already rare for disposal to fail!), then each earlier error is suppressed by the later error, forming a chain of suppressed errors.

```js
class MyReader {
  [Symbol.dispose]() {
    throw new Error("Failed to release lock on reader");
  }
}

class MyWriter {
  [Symbol.dispose]() {
    throw new Error("Failed to release lock on writer");
  }
}

function doSomething() {
  using reader = new MyReader();
  using writer = new MyWriter();
  throw new Error("Failed to read");
}
```

(code continues on the next page)

(code continued from the previous page)

```
try {
  doSomething();
} catch (e) {
  console.error(e); // SuppressedError: An error was suppressed during disposal
  console.error(e.suppressed); // SuppressedError: An error was suppressed during disposal
  console.error(e.error); // Error: Failed to release lock on reader
  console.error(e.suppressed.suppressed); // Error: Failed to read
  console.error(e.suppressed.error); // Error: Failed to release lock on writer
}
```

- The `reader` is released last, so its error is the latest and therefore suppresses everything else: it shows up as `e.error`.
- The `writer` is released first, so its error is later than the original exiting error, but earlier than the `reader` error: it shows up as `e.suppressed.error`.
- The original error about "Failed to read" is the earliest error, so it shows up as `e.suppressed.suppressed`.

Examples

Automatically releasing object URLs

In the following example, we create an object URL to a blob (in a real application, this blob would be fetched from somewhere, such as a file or a fetch response) so we can download the blob as a file. In order to prevent a resource leak, we must free the object URL with `URL.revokeObjectURL()` when it is no longer needed (that is, when the download has successfully started). Because the URL itself is just a string and therefore doesn't implement the disposable protocol, we cannot directly declare `url` with `using`; therefore, we create a `DisposableStack` to serve as the disposer for `url`. The object URL is revoked as soon as `disposer` goes out of scope, which is when either `link.click()` finishes or an error occurs somewhere.

js

```
const downloadButton = document.getElementById("download-button");
const exampleBlob = new Blob(["example data"]);

downloadButton.addEventListener("click", () => {
  using disposer = new DisposableStack();
  const link = document.createElement("a");
  const url = disposer.adopt(
    URL.createObjectURL(exampleBlob),
    URL.revokeObjectURL,
  );

  link.href = url;
  link.download = "example.txt";
  link.click();
});
```

Automatically canceling in-progress requests

In the following example, we fetch a list of resources concurrently using `Promise.all()`. `Promise.all()` fails and rejects the resulting promise as soon as one request failed; however, the other pending requests continue to run, despite their results being inaccessible to the program. To avoid these remaining requests needlessly consuming resources, we need to automatically cancel in-progress requests whenever `Promise.all()` settles. We implement cancellation with an `AbortController`, and

pass its `signal` to every `fetch()` call. If `Promise.all()` fulfills, then the function returns normally and the controller aborts, which is harmless because there's no pending request to cancel; if `Promise.all()` rejects and the function throws, then the controller aborts and cancels all pending requests.

```js
async function getAllData(urls) {
  using disposer = new DisposableStack();
  const { signal } = disposer.adopt(new AbortController(), (controller) =>
    controller.abort(),
  );

  // Fetch all URLs in parallel
  // Automatically cancel any incomplete requests if any request fails
  const pages = await Promise.all(
    urls.map((url) =>
      fetch(url, { signal }).then((response) => {
        if (!response.ok)
          throw new Error(
            `Response error: ${response.status} - ${response.statusText}`,
          );
        return response.text();
      }),
    ),
  );
  return pages;
}
```

Pitfalls

The resource disposal syntax offers a lot of strong error handling guarantees that ensure the resources are always cleaned up no matter what happens, but there are some pitfalls you may still encounter:

- Forgetting to use `using` or `await using`. The resource management syntax is only there to help you when you know you need it, but there's nothing to yell at you if you forget to use it! Unfortunately, there's no good way to prevent this before-the-fact, because there are no syntactic clues that something is a disposable resource, and even for disposable resources, you may want to declare them without automatic disposal. You probably need a type checker combined with a linter to catch these issues, such as typescript-eslint (which is still planning to work on this feature).
- Use-after-free. Generally, the `using` syntax ensures that a resource is freed when it goes out of scope, but there are many ways to persist a value beyond its binding variable. JavaScript does not have an ownership mechanism like Rust, so you can declare an alias that doesn't use `using`, or preserve the resource in a closure, etc. The `using` reference contains many examples of such pitfalls. Again, there's no good way to properly detect this in a complicated control flow, so you need to be careful.

The resource management feature is not a silver bullet. It is definitely an improvement over manually invoking the disposal methods, but it is not smart enough to prevent all resource management bugs. You still need to be careful and understand the semantics of the resources you are using.

Conclusion

Here are the key components of the resource management system:

- `using` and `await using` declarations for automatic resource disposal.

- The *disposable* and *async disposable* protocols, using the `Symbol.dispose` and `Symbol.asyncDispose` respectively, for resources to implement.
- The `DisposableStack` and `AsyncDisposableStack` objects for cases where `using` and `await using` are not suitable.

With proper usage of these APIs, you can create systems interacting with external resources that remain strong and robust against all error conditions without lots of boilerplate code.

JavaScript › Guide

Internationalization

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Internationalization · MDN page last modified 2025-09-24

The `Intl` object is the namespace for the ECMAScript Internationalization API, which provides a wide range of locale- and culture-sensitive data and operations.

Overview

The `Intl` object is very use-case driven. It provides a separate object for each use case that requires locale-specific logic. Currently, it provides the following functionalities:

- Getting information about a locale using `Intl.Locale`.
- Formatting data using `Intl.DateTimeFormat`, `Intl.DurationFormat`, `Intl.ListFormat`, `Intl.NumberFormat`, and `Intl.RelativeTimeFormat`.
- Collation (i.e., comparing strings for sorting or searching) using `Intl.Collator`.
- Selecting plural forms using `Intl.PluralRules`.
- Segmenting text into units such as words, sentences, or graphemes using `Intl.Segmenter`.
- Getting the displayed name for currencies, languages, scripts, regions, and time zones using `Intl.DisplayNames`.

Most `Intl` APIs share a similar design (`Intl.Locale` is the only exception). You start by constructing an instance with the desired locale and options. This defines a set of rules for the desired operation (formatting, collation, segmentation, etc.). Then, when you call the method on the instance, such as `format()`, `compare()`, or `segment()`, the object applies the specified rule to the passed data.

```js
// 1. Construct a formatter object, specifying the locale and formatting options:
const price = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
});

// 2. Use the `format` method of the formatter object to format a number:
console.log(price.format(5.259)); // $5.26
```

The general signature of the constructor is:

```js
new Intl.SomeObject(locales, options)
```

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object containing properties that customize specific aspects of the operation, which is key to understanding how to use each `Intl` object.

Locale information

Locales underlie all behaviors of `Intl`. A *locale* is a set of conventions, represented in the `Intl` API by the `Intl.Locale` object. All `Intl` constructors that accept language tags also accept `Intl.Locale` objects.

Each locale is primarily defined by four things: a `language`, a `script`, a `region`, and sometimes some `variants`. When connected together by `-` in that order, they form a BCP 47 language tag.

- The language is the most important part of the locale and is mandatory. When given a single language, like `en` or `fr`, there are algorithms to infer the rest of the information (see `Intl.Locale.prototype.maximize()`).
- However, you often want to specify the region as well, because conventions can differ drastically between regions that speak the same language. For example, the date format in the US is MM/DD/YYYY, whereas in the UK it is DD/MM/YYYY, so specifying `en-US` or `en-GB` is important.
- You can also specify a script. The script is the writing system, or the characters used to transcribe the language. In practice, the script is often unnecessary, because the language used in a certain region is only written in one script. However, there are exceptions such as the Serbian language, which can be written in both the Latin and Cyrillic scripts (`sr-Latn` and `sr-Cyrl`), or the Chinese language, which can be written in both the Simplified and Traditional scripts (`zh-Hans` and `zh-Hant`).
- The variants are rarely used. Usually, they denote different orthographies; for example, German has the `1901` and `1996` orthography variants, which are written as `de-1901` and `de-1996`.

```js
// These two are equivalent when passed to other Intl APIs
const locale1 = new Intl.Locale("en-US");
const locale2 = new Intl.Locale("en-Latn-US");

console.log(locale1.language, locale1.script, locale1.region); // "en", undefined, "US"
console.log(locale2.language, locale2.script, locale2.region); // "en", "Latn", "US"
```

A locale also contains a set of conventions used by that particular culture:

Use case	Property	Description	Extension subtag
Date/time formatting	`calendar`	Used to group days into years, months, and weeks, and to assign names to them. For example, the `gregory` date "2022-01-01" becomes "28 Tevet 5782" in the `hebrew` calendar.	`ca`
	`hourCycle`	Decides if times are displayed in 12-hour or 24-hour format, and whether the smallest hour number is 0 or 1.	`hc`
Number formatting, including dates, times, durations, etc.	`numberingSystem`	Transforms numbers into a locale-specific notation. The regular `0123456789` system is called `latn` (Latin). Often, each script has a numbering system that is just a digit-by-digit translation, but some scripts have more than one numbering system, some may not typically write numbers in that script (for example, Chinese has its own `hanidec` numbering system, but most texts use the standard `latn` system), and others may require special conversion algorithms (for example, Roman numerals — `roman`).	`nu`

Use case	Property	Description	Extension subtag
Collation	`collation`	Defines the generic collation algorithm. For example, if you use the German phonebk collation, then "ä" is treated as "ae", and will be sorted between "ad" and "af".	`co`
	`caseFirst`	Decides whether uppercase or lowercase letters should be sorted first, or if casing is ignored.	`kf`
	`numeric`	Decides whether numbers are sorted as numbers or as strings. For example, if true, "10" will be sorted after "2".	`kn`

You can explicitly specify these properties when constructing the `Intl.Locale` or passing language tags to other `Intl` constructors. There are two ways to do this — append them to the language tag or specify them as options.

- To append them to the language tag, you first append the string `-u` (meaning "Unicode extension"), then the extension subtag as given above, then the value.
- To specify them as options, you just add the property name as given above, together with its value, to the `options` object.

Using `Intl.DateTimeFormat` as an example, both of the following lines create a formatter that formats dates in the Hebrew calendar:

```js
const df1 = new Intl.DateTimeFormat("en-US-u-ca-hebrew");
const df2 = new Intl.DateTimeFormat("en-US", { calendar: "hebrew" });
```

Unrecognized properties are ignored, so you can use the same syntax as above with `Intl.NumberFormat`, but it won't do anything different than just passing `en-US`, because number formatting doesn't use the `calendar` property.

It is tricky to get the default values of these locale conventions. `new Intl.Locale("en-US").calendar` returns `undefined`, because the `Locale` object only contains the information you passed to it. The default calendar is theoretically dependent on what API you are using the calendar with, so to get the default calendar of `en-US` as used by `Intl.DateTimeFormat`, you can use its `resolvedOptions()` method. The same goes for other properties.

```js
const locale = new Intl.Locale("en-US");
console.log(locale.calendar); // undefined; it's not provided
console.log(new Intl.DateTimeFormat(locale).resolvedOptions().calendar); // "gregory"
```

`Intl.Locale` objects do two things simultaneously: they represent a parsed BCP 47 language tag (as demonstrated above), and they provide information about that locale. All of its properties, like `calendar`, are extracted from the input only, without querying any data source about default values. On the other hand, it has a group of methods for querying real-world information about the locale. For example, the `getCalendars()`, `getHourCycles()`, `getNumberingSystems()`, and `getCollations()` methods complement the `calendar`, `hourCycle`, `numberingSystem`, and `collation` properties, and each returns an array of preferred values for that property.

```js
const locale = new Intl.Locale("ar-EG");
console.log(locale.getCalendars()); // ['gregory', 'coptic', 'islamic', 'islamic-civil',
'islamic-tbla']
```

`Intl.Locale` instances also contain other methods that expose useful information, such as `getTextInfo()`, `getTimeZones()`, and `getWeekInfo()`.

Figuring out the locale

A shared concern for internationalization is: how do I know what locale to use?

The most obvious answer is "what the user prefers." Browsers expose the user's language preferences through the `navigator.languages` property. This is an array of language identifiers that can be directly passed to the formatter constructor—more on this later. The user can configure this list in their browser settings. You can also pass an empty array or `undefined`, which both cause the browser's default locale to be used.

```js
const numberFormatter = new Intl.NumberFormat(navigator.languages);
console.log(numberFormatter.format(1234567.89));

const numberFormatter2 = new Intl.NumberFormat([]);
```

However, this may not always provide the most desirable result. Strings formatted by `Intl` formatters represent a tiny fraction of text displayed on your site; most localized content is provided by you, the site developer. For example, suppose your site is only offered in two languages: English and French. If a Japanese user visits your site and expects to use your site in English, they will be baffled when they see the English text interleaved with numbers and dates in Japanese!

Usually, you don't want to use the browser's default language. Rather, you want to use the same language that the rest of your site is offered in. Suppose your site has a language switcher that stores the user's choice somewhere — you could directly use that.

```js
// Suppose this can be changed by some site-wide control
const userSettings = {
  locale: "en-US",
  colorMode: "dark",
};
const numberFormatter = new Intl.NumberFormat(userSettings.locale);
console.log(numberFormatter.format(1234567.89));
```

If your site has a backend that dynamically selects the language based on the user's `Accept-Language` header and sends back different HTML based on that, you could also use the HTML element's `HTMLElement.lang` property: `new Intl.NumberFormat(document.documentElement.lang)`.

If your site is only offered in one language, you could also hardcode the locale in your code: `new Intl.NumberFormat("en-US")`.

As previously mentioned, you can also pass an array of locales to the constructor, representing a list of fallback choices. The first example using `navigator.languages` is an example of this: if the first user-configured locale is not supported for the particular operation, the next one is tried, and so on, until we find a requested locale for which the runtime has data. You can do this manually as well. In the example below, we specify a list of locales in decreasing order of specificity which all represent languages likely to be understood by a Hong Kong Chinese speaker, so the formatter picks the most specific one that it supports.

```js
const numberFormatter = new Intl.NumberFormat([
  "yue-Hant",
  "zh-Hant-HK",
  "zh-Hant",
  "zh",
]);
```

There is no API for listing all supported locales, but there are a few methods for handling the locale list:

- `Intl.getCanonicalLocales()`: This function takes a list of locale identifiers and returns a list of canonicalized locale identifiers. This is useful for understanding the canonicalization process for each `Intl` constructor.
- The `supportedLocalesOf()` static method on each `Intl` object (like `Intl.DateTimeFormat.supportedLocalesOf()`): This method takes the same arguments as the constructor (`locales` and `options`) and returns a subset of the given locale tags that match the given data. This is useful for understanding which locales are supported by the runtime for a particular operation, for example, to show a language switcher containing only the supported languages.

Understanding the return value

The second shared concern for all objects is "what does the method return?" This is a hard question to answer beyond the structure or type of the returned value because there's no normative specification saying what *exactly* should be returned. Most of the time, the result of a method is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string returned by `format()` may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of any `Intl` method to hardcoded constants; they should only be displayed to users.

Of course, this answer seems unsatisfactory, because most developers do wish to control what the output looks like—at least, you don't want your user to be confused by nonsensical output. Here are some guidelines if you do want to conduct testing, whether automated or manual:

- Test all locales that your user may use. This is easier if you have a fixed set of supported locales (for example via a language switcher). If you are using whatever the user prefers, you may pick a few common ones for your users, but bear in mind that what the user sees could vary. You can usually mock user preference via the test runner's configuration or mocking the `Intl` constructors.
- Test on multiple JavaScript engines. The `Intl` API is directly implemented by the JavaScript engine, so for example you should expect Node.js and Chrome (which both use V8) to have the same output, while Firefox (which uses SpiderMonkey) may have a different output. Although all engines likely use the CLDR data, they usually postprocess it in different ways. Some browser build settings (for reducing the install size, for example) may also affect what locales and options are supported.
- Don't assume the output. This means you shouldn't write the output by hand, such as `expect(result).toBe("foo")`. Instead, use snapshot testing or copy the string value from the output of a test run.

Formatting data

A major use case of `Intl` is to output locale-specific texts representing structured data. This is similar to translation software, but instead of letting you translate arbitrary text, it takes data such as dates, numbers, and lists, and formats them according to locale-specific rules.

The `Intl.DateTimeFormat`, `Intl.DurationFormat`, `Intl.ListFormat`, `Intl.NumberFormat`, and `Intl.RelativeTimeFormat` objects each format one kind of data. Each instance provides two methods:

- `format()`: Takes a piece of data and returns a string using the formatting rule as determined by the locale and options.
- `formatToParts()`: Takes the same data and returns the same string, but broken down into parts, each part being an object with a `type` and a `value`. This is useful for more advanced use cases, such as interleaving the formatted text with other texts.

For example, here's a typical use of the `Intl.NumberFormat` object:

```js
// 1. Construct a formatter object, specifying the locale and formatting options:
const price = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
});

// 2. Use the `format` method of the formatter object to format a number:
console.log(price.format(5.259)); // $5.26

// Or, use the `formatToParts` method to get the formatted number
// broken down into parts:
console.table(price.formatToParts(5.259));
// |   | type       | value |
// | 0 | "currency" | "$"   |
// | 1 | "integer"  | "5"   |
// | 2 | "decimal"  | "."   |
// | 3 | "fraction" | "26"  |
```

You don't always have to construct a formatter object in order to format strings. For casual use, you can also directly call the `toLocaleString()` method on the data, passing the locale and options as arguments. The `toLocaleString()` method is implemented by `Temporal.PlainDate.prototype.toLocaleString()`, `Temporal.Duration.prototype.toLocaleString()`, `Number.prototype.toLocaleString()`, and so on. Read the documentation for the data you are formatting to see if it supports `toLocaleString()`, and what formatter options it corresponds to.

```js
console.log(
  (5.259).toLocaleString("en-US", {
    style: "currency",
    currency: "USD",
  }),
); // $5.26
```

Note that `toLocaleString()` is potentially less efficient than using a formatter object, because every time `toLocaleString` is called, it has to perform a search in a big database of localization strings. When the method is called many times with the same arguments, it is better to create a formatter object and

use its `format()` method, because a formatter object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Date and time formatting

`Intl.DateTimeFormat` formats dates and times, as well as ranges of dates and times. The `DateTimeFormat` object takes date/time inputs in one of the following forms: `Date`, `Temporal.PlainDateTime`, `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay`.

> **Note:** You can't pass a `Temporal.ZonedDateTime` object directly, because the time zone is already fixed in the object. You should use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object first.

Common use cases of localized date and time formatting are as follows:

- Output the same date and time in another calendar system, such as the Islamic, Hebrew, or Chinese calendar.
- Output the same real-world time (instant) but in another time zone.
- Selectively output certain components of the date and time, such as only the year and month, and the specific representation of them (such as "Thursday" or "Thu").
- Output the date according to locale-specific conventions, such as MM/DD/YYYY as in the U.S., DD/MM/YYYY as in the U.K., or YYYY/MM/DD as in Japan.
- Output the time according to locale-specific conventions, such as 12-hour or 24-hour clock.

To decide what the formatted string looks like, you first choose the calendar (which affects the year, month, week, and day calculation) and time zone (which affects the exact time as well as possibly the date). This is done using the aforementioned `calendar` option (or the `-ca-` extension key in the locale identifier), and the `timeZone` option.

- `Date` objects represent a unique instant in the user's time zone and in the ISO 8601 calendar (as reported by methods like `Date.prototype.getHours()` and `Date.prototype.getMonth()`). They are converted to the given `calendar` and `timeZone` by preserving the instant, so the date and time components may change.
- The various `Temporal` objects already have a calendar built-in, so the `calendar` option must be consistent with the calendar of the object—unless the date's calendar is `"iso8601"`, in which case it is converted to the requested `calendar`. These objects don't have a time zone, so they are directly displayed in the given `timeZone` without conversion.

Here we demonstrate how the combination of `calendar` and `timeZone` configurations result in different representations of the same instant.

```js
// Assume that the local time zone is UTC
const targetDate = new Date(2022, 0, 1); // 2022-01-01 midnight in the local time zone
const results = [];

for (const calendar of ["gregory", "hebrew"]) {
  for (const timeZone of ["America/New_York", "Asia/Tokyo"]) {
    const df = new Intl.DateTimeFormat("en-US", {
      calendar,
      timeZone,
```

(code continues on the next page)

(code continued from the previous page)

```
      // More on these later
      dateStyle: "full",
      timeStyle: "full",
    });
    results.push({ calendar, timeZone, output: df.format(targetDate) });
  }
}

console.table(results);
```

The output looks like this:

calendar	timeZone	output
'gregory'	'America/New_York'	'Friday, December 31, 2021 at 7:00:00 PM Eastern Standard Time'
'gregory'	'Asia/Tokyo'	'Saturday, January 1, 2022 at 9:00:00 AM Japan Standard Time'
'hebrew'	'America/New_York'	'Friday, 27 Tevet 5782 at 7:00:00 PM Eastern Standard Time'
'hebrew'	'Asia/Tokyo'	'Saturday, 28 Tevet 5782 at 9:00:00 AM Japan Standard Time'

A date/time is comprised of the following components: `weekday`, `era`, `year`, `month`, `day`, `dayPeriod`, `hour`, `minute`, `second`, `fractionalSecondDigits`, and `timeZoneName`. Your next decision is what components to include in the output, and what forms they should take. You have two choices:

- You can manually configure each component, using options with the same name as the component. Only the components you specify will be included in the output, with the specified form.
- You can use the shortcuts `dateStyle` and `timeStyle`, which are predefined sets of components. They expand to a set of component options depending on the locale.

You should pick one of these two methods, because they are mutually exclusive. Using both methods simultaneously will result in an error.

Underlyingly, after requesting a combination of components, the `DateTimeFormat` object looks for a "template" that matches the requested components, so it only needs to fill in the values one by one. Not every combination of components has a predefined template. `DateTimeFormat` has a `formatMatcher` option that decides how to negotiate, by making components longer or shorter than requested, or by omitting or adding components. It gets quite technical, so you should read the `Intl.DateTimeFormat()` reference to better understand how it handles this.

Here, we demonstrate a few common ways to format the components:

js

```
const df1 = new Intl.DateTimeFormat("en-US", {
  // Include all components (usually)
  dateStyle: "full",
  timeStyle: "full",
});

const df2 = new Intl.DateTimeFormat("en-US", {
  // Display the calendar date
  era: "short",
  year: "numeric",
  month: "long",
  day: "numeric",
});
```

(code continues on the next page)

(code continued from the previous page)

```
const df3 = new Intl.DateTimeFormat("en-US", {
  // Display a time like on a digital clock
  hour: "2-digit",
  minute: "2-digit",
  second: "2-digit",
  timeZoneName: "shortOffset",
});

const targetDate = new Date(2022, 0, 1, 12, 34, 56); // 2022-01-01 12:34:56 in the local
time zone
console.log(df1.format(targetDate));
// Saturday, January 1, 2022 at 12:34:56 PM Coordinated Universal Time
// January 1, 2022 AD
// 12:34:56 PM GMT
```

There are other customization options. For example, you can use the `hourCycle` option to display the time in 12-hour or 24-hour and show midnight/noon as 12:00 or 0:00. You can also use the `numberingSystem` option to display any numbers in another numbering system.

Apart from `format()`, there's a second important method, `formatRange()`, which formats a range of dates or times. It takes two date-times of the same type, formats each one, joins them with a range separator (like the en-dash), and deduplicates the common parts.

js

```
const springBreak = {
  start: new Date(2023, 2, 10),
  end: new Date(2023, 2, 26),
};

const df = new Intl.DateTimeFormat("en-US", { dateStyle: "long" });
console.log(df.formatRange(springBreak.start, springBreak.end));
// March 10 – 26, 2023
```

Number formatting

Number formatting is done with the `Intl.NumberFormat` object. The `NumberFormat` object accepts inputs in the form of numbers, strings, or `BigInt` values. Passing a string or `BigInt` instead of a number allows you to format numbers that are too large or too small to be represented precisely as a JavaScript number.

Common use cases of localized number formatting are as follows:

- Output the number in another numbering system (script), such as Chinese, Arabic, or Roman.
- Output the number with locale-specific conventions, such as the decimal symbol ("." in English, but "," in many European cultures), or digit grouping (3 digits in English, but may be 4 or 2 in other cultures, and may use ",", " ", or ".").
- Output the number with exponential notation such as "3.7 million" or "2 thousand".
- Output the number as a currency, applying specific currency symbols and rounding rules. For example, monetary values less than one cent in the U.S. or less than one yen in Japan may not be meaningful to display.
- Output the number as a percentage, applying locale-specific conversion and formatting rules.
- Output the number with units, such as "meters" or "liters", with translated unit names.

To decide what the formatted string looks like, you first choose the numbering system (which affects the characters used for the digits). The purpose of a numbering system is already discussed in locale

information. Another option you need to decide is the `style`, which sets the context for what the number represents, and potentially affects other options' default values. It is one of `"decimal"`, `"percent"`, `"currency"`, or `"unit"`. If you want to format currencies, then you also need to provide the `currency` option. If you want to format units, then you also need to provide the `unit` option.

```js
const results = [];
for (const options of [
  { style: "decimal" }, // Format the number as a dimensionless decimal
  { style: "percent" }, // Format the number as a percentage; it is multiplied by 100
  { style: "currency", currency: "USD" }, // Format the number as a US dollar amount
  { style: "unit", unit: "meter" }, // Format the number as a length in meters
]) {
  const nf = new Intl.NumberFormat("en-US", options);
  results.push({ style: options.style, output: nf.format(1234567.89) });
}
console.table(results);
```

The output looks like:

style	output
'decimal'	'1,234,567.89'
'percent'	'123,456,789%'
'currency'	'$1,234,567.89'
'unit'	'1,234,567.89 m'

The next group of options all specify what the numeric part should look like. First, you may want to represent extremely large values in a more readable way. You can set the `notation` option to `"scientific"` or `"engineering"`, which both use the `1.23e+6` notation. The difference is that the latter uses multiples of 3 for the exponent, keeping the mantissa (the part before the `e` symbol) between 1 and 1000, while the former can use any integer for the exponent, keeping the mantissa between 1 and 10. You can also set `notation` to `"compact"` to use a more human-readable notation.

```js
const results = [];
for (const options of [
  { notation: "scientific" },
  { notation: "engineering" },
  { notation: "compact", compactDisplay: "short" }, // "short" is default
  { notation: "compact", compactDisplay: "long" },
]) {
  const nf = new Intl.NumberFormat("en-US", options);
  results.push({
    notation: options.compactDisplay
      ? `${options.notation}-${options.compactDisplay}`
      : options.notation,
    output: nf.format(12000),
  });
}
console.table(results);
```

The output looks like:

notation	output
'scientific'	'1.2E4'

notation	output
'engineering'	'12E3'
'compact-short'	'12K'
'compact-long'	'12 thousand'

Then, you may want to round the number (if you specified `notation`, then just the mantissa part), so you don't display a number that's too long. These are the digit options, which include:

- `minimumIntegerDigits`
- `minimumFractionDigits`
- `maximumFractionDigits`
- `minimumSignificantDigits`
- `maximumSignificantDigits`
- `roundingPriority`
- `roundingIncrement`
- `roundingMode`

The exact interaction of these options is quite complex and not worth covering here. You should read the digit options reference for more detail. Nevertheless, the general idea is straightforward: we first find the number of decimal places we want to keep, then we round excess decimal places off, either rounding down or up, depending on the value of the last digit.

```js
const results = [];
for (const options of [
  { minimumFractionDigits: 4, maximumFractionDigits: 4 },
  { minimumSignificantDigits: 4, maximumSignificantDigits: 4 },
  { minimumFractionDigits: 0, maximumFractionDigits: 0, roundingMode: "floor" },
  {
    minimumFractionDigits: 0,
    maximumFractionDigits: 0,
    roundingMode: "floor",
    roundingIncrement: 10,
  },
]) {
  const nf = new Intl.NumberFormat("en-US", options);
  results.push({
    options,
    output: nf.format(1234.56789),
  });
}
console.table(results);
```

The output looks like this:

options	output
`{ minimumFractionDigits: 4, maximumFractionDigits: 4 }`	'1,234.5679'
`{ minimumSignificantDigits: 4, maximumSignificantDigits: 4 }`	'1,235'
`{ minimumFractionDigits: 0, maximumFractionDigits: 0, roundingMode: "floor" }`	'1,234'
`{ minimumFractionDigits: 0, maximumFractionDigits: 0, roundingMode: "floor", roundingIncrement: 10 }`	'1,230'

There are other customization options. For example, you can use the `useGrouping` and `signDisplay` options to customize whether and how to show the group separators (like "," in "1,234,567.89") and the sign. However, note that the characters used for the group separator, decimal point, and sign are locale-specific, so you can't customize them directly.

Apart from `format()`, there's a second important method, `formatRange()`, which formats a range of numbers. It takes two number representations, formats each one, joins them with a range separator (like the en-dash), and potentially deduplicates the common parts.

js

```
const heightRange = {
  min: 1.63,
  max: 1.95,
};

const nf = new Intl.NumberFormat("en-US", { style: "unit", unit: "meter" });
console.log(nf.formatRange(heightRange.min, heightRange.max));
// 1.63–1.95 m
```

List formatting

You may already have written code that does this:

js · ✗ bad example

```
const fruits = ["apple", "banana", "cherry"];
console.log(`I like ${fruits.join(", ")}.`);
// I like apple, banana, cherry.
```

This code is not internationalized. In some languages, the list separator is not a comma. In *most* languages (including English), you need a conjunction before the last item. But even just adding an "and" manually doesn't make it correct among all English speakers, because there's the debate of Oxford commas in English: "apple, banana, and cherry" vs. "apple, banana and cherry".

The `Intl.ListFormat` object solves this problem. It takes an array of strings and joins them in a locale-specific way such that the result represents a conjunction (and), disjunction (or), or a list of units.

js

```
const fruits = ["apple", "banana", "cherry"];
const lf = new Intl.ListFormat("en-US", { style: "long", type: "conjunction" });
console.log(`I like ${lf.format(fruits)}.`);
// I like apple, banana, and cherry.

const lf = new Intl.ListFormat("en-US", { style: "long", type: "disjunction" });
console.log(`I can give you ${lf.format(fruits)}.`);
// I can give you apple, banana, or cherry.
```

Check `Intl.ListFormat()` for more examples and options.

Relative time formatting

`Intl.RelativeTimeFormat` formats a time difference. The `RelativeTimeFormat` object takes relative times in the form of two arguments: a number (with any sign) and a time unit, such as `"day"`, `"hour"`, or `"minute"`.

It does several things at once:

- It localizes and pluralizes the time unit, such as "1 day" vs. "2 days", like in number formatting.

- It chooses the appropriate phrase for past and future times, such as "in 1 day" vs. "1 day ago".
- It may select a special phrase for some time units, such as "1 day ago" vs. "yesterday".

```js
const rtf = new Intl.RelativeTimeFormat("en-US", { numeric: "auto" });
console.log(rtf.format(1, "day")); // tomorrow
console.log(rtf.format(2, "day")); // in 2 days
console.log(rtf.format(-1, "hour")); // 1 hour ago
```

Check `Intl.RelativeTimeFormat()` for more examples and options.

Duration formatting

`Intl.DurationFormat` provides duration formatting, such as "3 hours, 4 minutes, 5 seconds". It is not a primitive operation with its own formatter: it uses `Intl.NumberFormat` and `Intl.ListFormat` internally to format each duration component, then joins them with a list separator. The `DurationFormat` object takes durations in the form of a `Temporal.Duration` object, or a plain object with the same properties.

Apart from customizing the numbering system, the duration formatting options decides whether or not to show each component, and how long they should be.

```js
console.log(
  new Intl.DurationFormat("en-US", {
    style: "long",
  }).format({ hours: 3, minutes: 4, seconds: 5 }),
);
// 3 hours, 4 minutes, and 5 seconds
```

Check `Intl.DurationFormat()` for more examples and options.

Collation

The `Intl.Collator` object is useful for comparing and sorting strings. It takes two strings and returns a number indicating their relative order, in the same way as the `compareFn` argument of the `Array.prototype.sort` method.

There are many reasons why you should not use JavaScript operators like === or > to compare user-facing strings:

- Irrelevant orthographic variants: For example, in English, "naïve" and "naive" are just alternative spellings of the same word and should be treated as equal.
- Ignoring case: Often, you want to ignore case when comparing strings. For example, "apple" and "Apple" should be treated as equal.
- Unicode code point order doesn't make sense: Comparison operators like > compare by Unicode code point order, which is not the same as the order of characters in a dictionary. For example, "ï" comes after "z" in code point order, but you would want it to be ordered next to "i" in a dictionary.
- Unicode normalization: The same character may have multiple representations in Unicode. For example, "ñ" can be represented as a single character or as "n" followed by a combining tilde. (See `String.prototype.normalize()`.) These should be treated as equal.
- Number comparison: Numbers in strings should be compared as numbers, not as strings. For example, you would want "test-10" to come after "test-2".

There are two distinct use cases for collation: **sorting** and **searching**. Sorting is when you have a list of strings and you want to order them according to some rule. Searching is when you have a list of strings and you want to find a string that matches a query. When searching, you should only pay attention to whether the comparison result is zero (equal) or not, not the sign of the result.

There are many different ways to sort, even within the same locale. For example, there are two different sort orders in German, *phonebook* and *dictionary*. Phonebook sort emphasizes sound — as if "ä", "ö", and so on were expanded to "ae", "oe", and so on before sorting.

js

```
const names = ["Hochberg", "Hönigswald", "Holzman"];

const germanPhonebook = new Intl.Collator("de-DE-u-co-phonebk");

// as if sorting ["Hochberg", "Hoenigswald", "Holzman"]:
console.log(names.sort(germanPhonebook.compare));
// ['Hochberg', 'Hönigswald', 'Holzman']
```

Some German words conjugate with extra umlauts, so in dictionaries it's sensible to order ignoring umlauts (except when ordering words differing *only* by umlauts: *schon* before *schön*).

js

```
const germanDictionary = new Intl.Collator("de-DE-u-co-dict");

// as if sorting ["Hochberg", "Honigswald", "Holzman"]:
console.log(names.sort(germanDictionary.compare).join(", "));
// "Hochberg, Holzman, Hönigswald"
```

Plural rules

The `Intl.PluralRules` object is useful for selecting the correct plural form of a word. It doesn't automatically pluralize words for you (for example, you can't pass it "apple" and expect "apples" back), but it tells you which plural form to use based on a number. You may already be doing this:

js

```
function formatMessage(n) {
  return `You have ${n} ${n === 1 ? "apple" : "apples"}.`;
}
```

But this is hard to generalize across languages, especially those with many plural forms. You can see `Intl.PluralRules` for a general introduction to plural rules. Here, we just demonstrate a few common use cases.

js

```
const prCard = new Intl.PluralRules("en-US");
const prOrd = new Intl.PluralRules("en-US", { type: "ordinal" });

const englishOrdinalSuffixes = {
  one: "st",
  two: "nd",
  few: "rd",
  other: "th",
};
```

(code continues on the next page)

(code continued from the previous page)

```
const catPlural = {
  one: "cat",
  other: "cats",
};

function formatMessage(n1, n2) {
  return `The ${n1}${englishOrdinalSuffixes[prOrd.select(n1)]} U.S. president had ${n2}
${catPlural[prCard.select(n2)]}.`;
}

console.log(formatMessage(42, 1)); // The 42nd U.S. president had 1 cat.
console.log(formatMessage(45, 0)); // The 45th U.S. president had 0 cats.
```

Segmentation

The `Intl.Segmenter` object is useful for breaking a string into segments. Without `Intl`, you are already able to split a string by UTF-16 code units and Unicode code points:

js

```
const str = "USCNRUGBFR";
console.log(str.split(""));
// Array(20) ['\uD83C', '\uDDFA', '\uD83C', ...]
console.log([...str]);
// Array(10) ['U', 'S', 'C', 'N', 'R', 'U', 'G', 'B', 'F', 'R']
```

But as you can see, Unicode code points are not the same as what human users perceive as discrete characters. This often happens with emojis, where a single emoji can be represented by multiple code points. When the user interacts with text, a grapheme is the smallest unit of text they can manipulate, such as delete or select. The `Segmenter` object enables grapheme-level segmentation, which is useful for counting characters, measuring text width, and so on. It takes a string and returns an iterable segments object, each element of which has a `segment` property representing the text of the segment.

js

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "grapheme" });
console.log([...segmenter.segment("USCNRUGBFR")].map((s) => s.segment));
// ['US', 'CN', 'RU', 'GB', 'FR']
```

The segmenter can also do higher-level segmentation, including word-level and sentence-level splitting. These use cases are necessarily language-specific. For example, the following is a very poor implementation of word-counting:

js · ✗ bad example

```
const wordCount = (str) => str.split(/\s+/).length;
console.log(wordCount("Hello, world!")); // 2
```

There are several problems with this: not all languages use spaces to separate words, not all spaces are word-separating, and not all words are separated by spaces. To solve this, use `Segmenter` with `granularity: "word"`. The result is the input string, split into segments of words and non-words. If you are counting words, you should filter out the non-words by checking each segment's `isWordLike` property.

js

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "word" });
const str = "It can even split non-space-separated words";
console.table([...segmenter.segment(str)]);
// ┌──────────────┬────────┬──────────────┐
```

(code continues on the next page)

(code continued from the previous page)

```
// │ segment     │ index │ isWordLike │
// ├─────────────┼───────┼────────────┤
// │ 'It'        │ 0     │ true       │
// │ ' '         │ 2     │ false      │
// │ 'can'       │ 3     │ true       │
// │ ' '         │ 6     │ false      │
// │ 'even'      │ 7     │ true       │
// │ ' '         │ 11    │ false      │
// │ 'split'     │ 12    │ true       │
// │ ' '         │ 17    │ false      │
// │ 'non'       │ 18    │ true       │
// │ '-'         │ 21    │ false      │
// │ 'space'     │ 22    │ true       │
// │ '-'         │ 27    │ false      │
// │ 'separated' │ 28    │ true       │
// │ ' '         │ 37    │ false      │
// │ 'words'     │ 38    │ true       │
// └─────────────┴───────┴────────────┘

console.log(
  [...segmenter.segment(str)].filter((s) => s.isWordLike).map((s) => s.segment),
);
// ['It', 'can', 'even', 'split', 'non', 'space', 'separated', 'words']
```

Word segmentation works for character-based languages too. For example, in Chinese, several characters can represent a single word, but there's no space between them. The segmenter implements the same behavior as the browser's built-in word segmentation, triggered by double-clicking a word.

js

```
const segmenter = new Intl.Segmenter("zh-Hans", { granularity: "word" });
console.log([...segmenter.segment("我是这篇文档的作者")].map((s) => s.segment));
// ['我是', '这', '篇', '文', '档', '的', '作者']
```

Sentence segmentation is similarly complex. For example, in English, there are many punctuation marks that could mark the end of a sentence (".", "!", "?", and so on).

js

```
const segmenter = new Intl.Segmenter("en-US", { granularity: "sentence" });
console.log(
  [...segmenter.segment("I ate a sandwich. Then I went to bed.")].map(
    (s) => s.segment,
  ),
);
// ['I ate a sandwich. ', 'Then I went to bed.']
```

Note that the segmenter doesn't remove any characters. It just splits the string into segments, each of which is a sentence. You can then remove the punctuation marks if you want. Also, the current implementation of the segmenter doesn't support sentence segmentation suppressions (preventing sentence breaks after periods like "Mr." or "Approx."), but there's ongoing work to support this.

Display names

After introducing so many options and behaviors, you may wonder how to present them to the user. `Intl` comes with two useful APIs for building user interfaces: `Intl.supportedValuesOf()` and `Intl.DisplayNames`.

The `Intl.supportedValuesOf()` function returns an array of supported values for a given option. For example, you can use it to populate a dropdown list of supported calendars, from which users can select to display dates.

js

```
const supportedCal = Intl.supportedValuesOf("calendar");
console.log(supportedCal);
// ['buddhist', 'chinese', 'coptic', 'dangi', ...]
```

But often, these identifiers are not user-friendly. For example, you may want to show the calendars in the user's language, or unabbreviate them. The `Intl.DisplayNames` object is useful for this. It's like a formatter, but it's not template-based. Instead, it is a direct mapping from language-agnostic identifiers to localized names. It supports formatting languages, regions, scripts (the three subfields of a BCP 47 language tag), currency, calendar, and date-time fields.

Try the demo below:

html

```
<select id="lang"></select>
<select id="calendar"></select>
<output id="output"></output>
```

css · hidden on the MDN page (supporting code for the live sample)

```
output {
  display: block;
  margin: 1em;
  font-size: x-large;
}

rt {
  font-weight: bold;
}

ruby {
  border-bottom: 1px dotted;
  ruby-position: under;
  ruby-align: center;
}
```

js

```
const langSelect = document.getElementById("lang");
const calSelect = document.getElementById("calendar");
const fieldset = document.querySelector("fieldset");
const output = document.getElementById("output");

// A few examples
const langs = [
  "en-US",
  "zh-Hans-CN",
  "ja-JP",
  "ar-EG",
  "ru-RU",
  "es-ES",
  "fr-FR",
  "de-DE",
  "hi-IN",
  "pt-BR",
  "bn-BD",
```

(code continues on the next page)

(code continued from the previous page)

```
  "he-IL",
];
const calendars = Intl.supportedValuesOf("calendar");

for (const lang of langs) {
  const option = document.createElement("option");
  option.value = lang;
  option.textContent = new Intl.DisplayNames(lang, { type: "language" }).of(
    lang,
  );
  langSelect.appendChild(option);
}

function renderCalSelect() {
  const lang = langSelect.value;
  calSelect.innerHTML = "";
  const dn = new Intl.DisplayNames(lang, { type: "calendar" });
  const preferredCalendars = new Intl.Locale(lang).getCalendars?.() ?? [
    "gregory",
  ];
  for (const cal of [
    ...preferredCalendars,
    ...calendars.filter((c) => !preferredCalendars.includes(c)),
  ]) {
    const option = document.createElement("option");
    option.value = cal;
    option.textContent = dn.of(cal);
    calSelect.appendChild(option);
  }
}

function renderFieldInputs() {
  const lang = langSelect.value;
  fieldset.querySelectorAll("label").forEach((label) => label.remove());
  const dn = new Intl.DisplayNames(lang, { type: "dateTimeField" });
  for (const field of fields) {
    const label = document.createElement("label");
    label.textContent = dn.of(field);
    const input = document.createElement("input");
    input.type = "checkbox";
    input.value = field;
    label.appendChild(input);
    fieldset.appendChild(label);
  }
}

function renderTime() {
  const lang = langSelect.value;
  const cal = calSelect.value;
  const df = new Intl.DateTimeFormat(lang, {
    calendar: cal,
    dateStyle: "full",
    timeStyle: "full",
  });
  const now = new Date();
  const dn = new Intl.DisplayNames(lang, { type: "dateTimeField" });
  output.innerHTML = "";
  for (const component of df.formatToParts(now)) {
    const text = document.createElement("span");
    text.textContent = component.value;
```

(code continues on the next page)

(code continued from the previous page)

```
    if (
      ![
        "era",
        "year",
        "quarter",
        "month",
        "weekOfYear",
        "weekday",
        "day",
        "dayPeriod",
        "hour",
        "minute",
        "second",
        "timeZoneName",
      ].includes(component.type)
    ) {
      output.appendChild(text);
      continue;
    }
    const title = dn.of(component.type);
    const field = document.createElement("ruby");
    field.appendChild(text);
    const rt = document.createElement("rt");
    rt.textContent = title;
    field.appendChild(rt);
    output.appendChild(field);
  }
}

renderCalSelect();
renderTime();
langSelect.addEventListener("change", renderCalSelect);
langSelect.addEventListener("change", renderTime);
calSelect.addEventListener("change", renderTime);
setInterval(renderTime, 500);
```

▷ Live sample output ("display names sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Internationalization#display_names .

JavaScript modules

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules · MDN page last modified 2026-11-09

This guide gives you all you need to get started with JavaScript module syntax.

Module philosophy

Before we look at what syntax is available inside modules, let's first talk at a high level about what modules are, how they differ from traditional scripts, and why you might need them.

A background on modules

JavaScript programs started off pretty small — most of its usage in the early days was to do isolated scripting tasks, providing a bit of interactivity to your web pages where needed, so large scripts were generally not needed. Fast forward a few years and we now have complete applications being run in browsers with a lot of JavaScript, as well as JavaScript being used in other contexts (Node.js, for example).

Complex projects necessitate a mechanism for splitting JavaScript programs into separate modules: units of code that can be imported when needed. Node.js has had this ability for a long time, and there are a number of JavaScript libraries and tools that enable module usage (for example, AMD loaders like RequireJS, bundlers like webpack, and compilers like Babel).

All modern browsers support module features natively without needing transpilation. It can only be a good thing — browsers can automatically handle the module loading process, without requiring a library. It does not obsolete bundlers like webpack, though — bundlers still do a good job at partitioning code into reasonably sized chunks, and are able to do other optimizations like minification, dead code elimination, and tree shaking.

What's a module?

JavaScript code can be evaluated in two ways: as a *script* (also called "classic script" or "traditional script") or as a *module*. There are two main differences:

1. At parsing time, modules are parsed with a slightly different syntax. Namely, they are automatically in strict mode, and you can only use `import` and `export` statements in modules.
2. At runtime, modules are executed in their own scope, not in the global scope. This means that variables, functions, classes, etc. declared in a module are not visible outside the module unless they are either explicitly exported (so they can be imported in other modules), or are made available globally by attaching them to the global object (e.g., `window` in a browser). Uncaught errors from modules still show in the DevTools, but you won't be able to access module-scoped variables in the JavaScript console.

There are other differences, but they are specific to modules on the web. Find them in Using modules on the web.

Here is how scripts are traditionally attached to web pages:

html

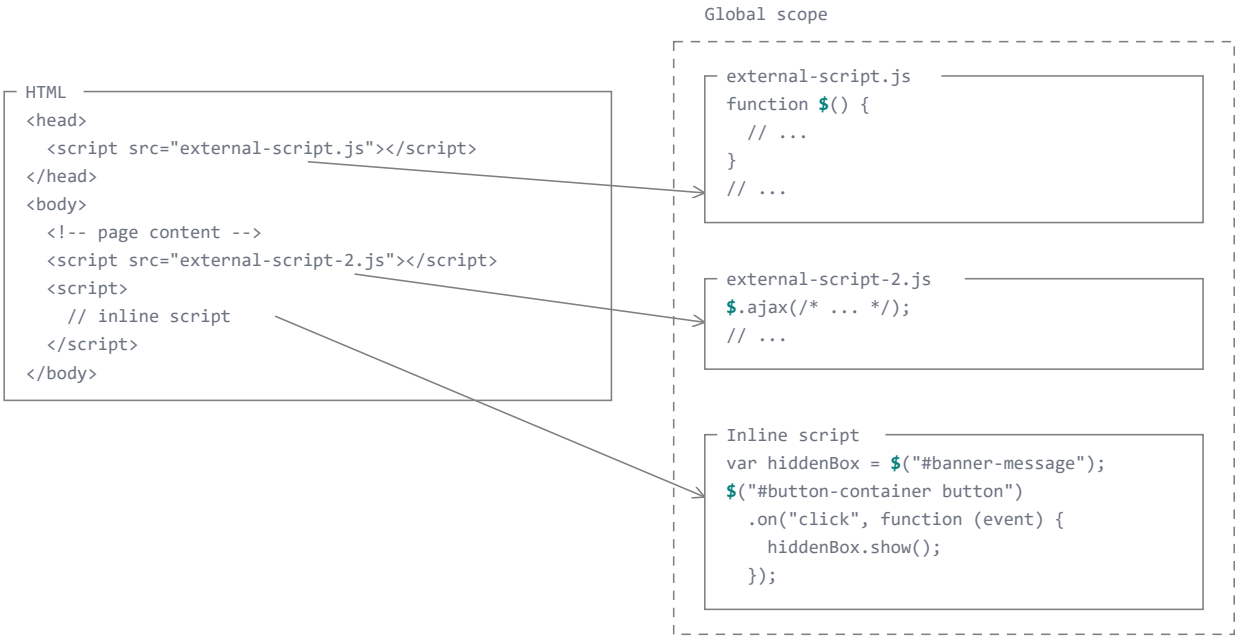
```
<head>
  <script src="external-script.js"></script>
</head>
<body>
  <!-- page content -->
  <script src="external-script-2.js"></script>
  <script>
    // inline script
  </script>
</body>
```

js

```
// -- external-script.js --
function $() {
  // …
}
// …
```

js

```
// -- external-script-2.js --
$.ajax(/* … */);
// …
```



Most notably, all classic scripts attached to the webpage are executed under the same scope—the global scope. Any variables, unless they are contained within other functions or blocks, can be accessed from anywhere later in the page. This can be both convenient and dangerous. Shown in this figure is how the second and third scripts are able to access the `$` variable declared in the first script, thanks to them all being in the global scope.

This pattern is very convenient, but it gets unwieldy at scale:

- Scope pollution and identifier conflicts become a big concern. Popular libraries like jQuery and Underscore put everything under special identifiers like `$` and `_` to minimize the risk, but many third-party libraries compete for common names like `map`, `merge`, `debounce`, etc.

- All dependencies must be explicitly declared and managed by the author. If library A imports library B, then it's the application author's job to add a `<script>` element that loads library B and ensure that it happens before A.
- It's hard to load libraries on demand: everything is loaded and initialized as soon as the page loads.

Now, let us consider how this same code may look with modules. The first script, `external-script.js`, wants to share the variable `$` with the rest of the scripts. It is done above by putting the variable in the global scope. In the module version, we can export the variable and import it in the other scripts.

html

```
<head>
  <script type="module" src="external-script-2.js"></script>
  <script type="module">
    import { $ } from "./external-script.js";
    // inline script
  </script>
</head>
<body>
  <!-- page content -->
</body>
```

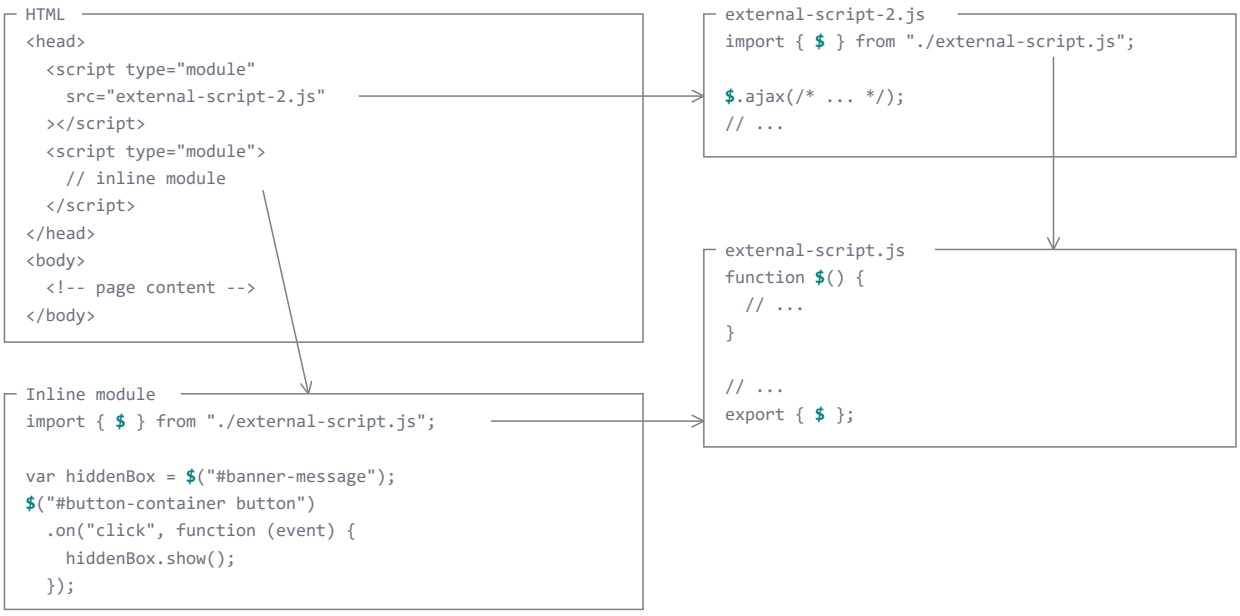
js

```
// -- external-script.js --
function $() {
  // …
}
// …
export { $ };
```

js

```
// -- external-script-2.js --
import { $ } from "./external-script.js";

$.ajax(/* … */);
// …
```



Whereas the script version formed a *linear* order of dependencies where each piece of code has access to all of the code coming earlier, the module version forms a *dependency graph* where each module explicitly declares the modules it relies on, and what variables it needs from those dependencies.

As we said, modules are parsed differently from scripts—this means that the JavaScript engine needs to know whether to apply the module or script parsing rules. Hosts commonly specify this using an *out-of-band signaling mechanism*, where the behavior of the code is configured by information outside of the code itself. (This is different from strict mode, where the interpretation mode is declared inside the script with `"use strict"`.) There are many ways to give such information:

- If this module is referenced from a `<script>` tag, you can use the `type="module"` attribute. Also see Using modules on the web.
- If you are using Node.js, you can use the `.mjs` file extension, or add `"type": "module"` to the closest `package.json` file. Also see Modules across platforms.
- If this module is to be used as a worker, you can pass `type: "module"` when calling the `Worker()` constructor.

> **Note:** Certain runtimes, like Node.js, also have implicit module type detection if there's no explicit indication, based on whether the code uses syntax like `import/export` that's unavailable in non-modules. Browsers don't offer this.

Modules still have access to global variables. For example, a module can use a variable declared by a classic script:

html

```
<script>
  var text = "Hello";
</script>
<script type="module" src="render.js"></script>
```

js

```
// -- render.js --
document.body.textContent = text;
```

Aside — .mjs versus .js

Throughout this article, we've used `.js` extensions for our module files, but in other resources you may see the `.mjs` extension used instead. V8's documentation recommends this, for example. The reasons given are:

- You can tell if a file is a module or a script directly from the file, without looking at contextual clues like the HTML tag or the package.json.
- Outside of browsers, it's almost universally accepted as directly entering module mode, including by runtimes such as Node.js and build tools such as Babel.

For browsers, the difference is purely cosmetic. The file extension doesn't matter on the web—JavaScript is identified by the `Content-Type` header, and modules are identified by the `type="module"` attribute in the `<script>` tag. To get modules to work correctly in a browser, you need to make sure that your server is serving them with a `Content-Type` header that contains a JavaScript MIME type such as `text/javascript`. If you don't, you'll get a strict MIME type checking error along the lines of "The server responded with a non-JavaScript MIME type" and the browser won't run your JavaScript.

We decided to keep to `.js`. The main benefit is that the `.js` extension is much more widely understood than `.mjs`, and although the latter is catching up, you might always run into certain software that doesn't understand `.mjs`.

- Most servers automatically set the correct type for `.js` files, but some don't for `.mjs` files. Servers that already serve `.mjs` files correctly include GitHub Pages and `http-server` for Node.js. This is OK if you are using such an environment already, or if you aren't but you have access to configure your server to set the correct `Content-Type` for `.mjs` files. It could however cause confusion if you don't control the server you are serving files from, or are publishing files for public use, as we are here.
- The file extension is used by many tools other than the HTTP server. It may be used by your editor, the operating system, static analysis tools, formatters, and more. `.mjs` may be less well-supported by tooling at large. For example, some operating systems might not recognize it, or try to replace it with something else, such as implicitly appending a `.js` extension when you try to open it.

If you really value the clarity of using `.mjs` for modules versus using `.js` for "normal" JavaScript files, but don't want to run into the problem described above, you could always use `.mjs` during development and convert them to `.js` during your build step.

Working with examples

To demonstrate usage of modules, we've created a simple set of examples that you can find on GitHub. These examples demonstrate a simple set of modules that create a `<canvas>` element on a webpage, and then draw (and report information about) different shapes on the canvas.

These are fairly trivial, but have been kept deliberately simple to demonstrate modules clearly.

In our first example (see basic-modules) we have a file structure as follows:

```
index.html
main.js
modules/
    canvas.js
    square.js
```

> **Note:** All of the examples in this guide have essentially this same structure.

The `index.html` file only declares the DOM and applies the `main.js` module; all code we demonstrate subsequently concerns only the `.js` files.

The `modules` directory's two modules are described below:

- `canvas.js` — contains functions related to setting up the canvas:
  - `create()` — creates a canvas with a specified `width` and `height` inside a wrapper `<div>` with a specified ID, which is itself appended inside a specified parent element. Returns an object containing the canvas's 2D context and the wrapper's ID.
  - `createReportList()` — creates an unordered list appended inside a specified wrapper element, which can be used to output report data into. Returns the list's ID.
- `square.js` — contains:
  - `name` — a constant containing the string 'square'.
  - `draw()` — draws a square on a specified canvas, with a specified size, position, and color. Returns an object containing the square's size, position, and color.
  - `reportArea()` — writes a square's area to a specific report list, given its length.

- `reportPerimeter()` — writes a square's perimeter to a specific report list, given its length.

After you've downloaded these examples, start a local server and visit the `index.html` file.

Named exports

Unlike scripts, where all variables declared at the top level are implicitly accessible to subsequent scripts, you must explicitly `export` variables for them to be accessible in other modules. You can place this keyword in front of any declaration—including `var`, `let`, `const`, `function`, `class`, and more. They need to be top-level items: for example, you can't use `export` inside a function.

```js
export const name = "square";

export function draw(ctx, length, x, y, color) {
  ctx.fillStyle = color;
  ctx.fillRect(x, y, length, length);

  return { length, x, y, color };
}
```

Equivalently, you can export things you already declared elsewhere inside this module by using `export` followed by a comma-separated list of variable names within curly braces:

```js
export { name, draw, reportArea, reportPerimeter };
```

Named imports

Once you've exported some variables out of your module, you need to import them into another module to use them. You use the `import` statement, followed by a comma-separated list of the names you want to import wrapped in curly braces, followed by the keyword `from`, followed by the *module specifier*.

```js
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";
```

You can see such lines in action in `main.js`.

> **Note:** The *module specifier* is a string that points to the module that exports these variables. In a browser, this is usually the URL relative to the URL of the current module, similar to the `href` attribute of `<a>` elements. For example, if our module above is served at `https://example.com/basic-modules/main.js`, and the `square.js` module is served at `https://example.com/basic-modules/modules/square.js`, then all of the following are equivalent:
>
> ```
> "https://example.com/basic-modules/modules/square.js"
> "/basic-modules/modules/square.js"
> "./modules/square.js"
> ```
>
> Different runtimes (browsers, Node, bundler, etc.) have different rules for the specifier format. You can read more about these differences in Using modules on the web and Modules across platforms.

Once you've imported the features into your script, you can use them just like they were defined inside the same file. The following is found in `main.js`, below the import lines:

```js
const myCanvas = create("myCanvas", document.body, 480, 320);
const reportList = createReportList(myCanvas.id);

const square = draw(myCanvas.ctx, 50, 50, 100, "blue");
reportArea(square.length, reportList);
reportPerimeter(square.length, reportList);
```

> **Note:** The imported values are read-only views of the features that were exported. Similar to `const` variables, you cannot re-assign the variable that was imported, but you can still modify properties of object values. The value can only be re-assigned by the module exporting it. See the `import` reference for an example.

Default exports and imports

The examples above export and import variables by name. The names for each variable have to match between the exporter and the importer. You can write functional module code using just named imports and exports. However, if you want to import from non-ESM modules (like CommonJS or non-JavaScript files), these systems do not use named exports. In these systems, each module corresponds to exactly one JavaScript value. To represent these types of modules, JavaScript provides a *default export*. This has other use cases, such as modules that are built around one main value—a single class or function—where the default export lets each importer pick its own local name without knowing what the exporting module calls it.

Unlike named exports, default exports have no names associated with them. Each module can have up to one default export. To create a default export, instead of exporting a *declaration* like `const` or `function`, you export an *expression* instead, by prepending `export default` to it. For example, all of the following would work:

```js
// An object literal is an expression
export default {
  some: "object",
};

// An addition is an expression
export default 1 + 1;

// An identifier is also an expression
export default randomSquare;
```

You can see the last version in our basic-modules `square.js`, which exports a function called `randomSquare()` that creates a square with a random color, size, and position. Note the lack of curly braces.

As a special case, functions and classes are exported as *declarations*, not expressions, and these declarations can be anonymous. So you can also default-export a function directly when you declare it:

```js
export default function randomSquare(ctx) {
  // …
}
```

```js
// -- main.js --
import randomSquare from "./modules/square.js";
```

Again, note the lack of curly braces. This is because there is only one default export allowed per module, and we know that `randomSquare` is it. You can name this import however you want—it doesn't have to match the function name on the exporting side.

Named exports and default export can coexist in the same module.

Avoiding naming conflicts

So far, our canvas shape drawing modules seem to be working OK. But what happens if we try to add a module that deals with drawing another shape, like a circle or triangle? These shapes would probably have associated functions like `draw()`, `reportArea()`, etc. too; if we tried to import different functions of the same name into the same top-level module file, we'd end up with conflicts and errors.

Let's look at a real example. In our renaming directory you'll see the same module system as in the previous example, except that we've added `circle.js` and `triangle.js` modules to draw and report on circles and triangles.

Inside each of these modules, we've got features with the same names being exported, and therefore each has the same `export` statement at the bottom:

```js
export { name, draw, reportArea, reportPerimeter };
```

When importing these into `main.js`, if we tried to use

```js
import { name, draw, reportArea, reportPerimeter } from "./modules/square.js";
import { name, draw, reportArea, reportPerimeter } from "./modules/circle.js";
import { name, draw, reportArea, reportPerimeter } from "./modules/triangle.js";
```

The browser would throw an error such as "SyntaxError: redeclaration of import name" (Firefox).

Fortunately there are a number of ways to get around this. We'll look at these in the following sections.

Renaming imports and exports

Inside your `import` and `export` statement's curly braces, you can use the keyword `as` along with a new feature name, to change the identifying name you will use for a feature inside the top-level module.

So in our renaming example, we can rename the imports so that they are unique:

```js
import {
  name as squareName,
  draw as drawSquare,
  reportArea as reportSquareArea,
  reportPerimeter as reportSquarePerimeter,
} from "./modules/square.js";
```

(code continues on the next page)

(code continued from the previous page)

```
import {
  name as circleName,
  draw as drawCircle,
  reportArea as reportCircleArea,
  reportPerimeter as reportCirclePerimeter,
} from "./modules/circle.js";

import {
  name as triangleName,
  draw as drawTriangle,
  reportArea as reportTriangleArea,
  reportPerimeter as reportTrianglePerimeter,
} from "./modules/triangle.js";
```

Note that you could solve the problem in the module files instead, e.g.

js

```
// -- square.js --
export {
  name as squareName,
  draw as drawSquare,
  reportArea as reportSquareArea,
  reportPerimeter as reportSquarePerimeter,
};
```

js

```
// -- main.js --
import {
  squareName,
  drawSquare,
  reportSquareArea,
  reportSquarePerimeter,
} from "./modules/square.js";
```

And it would work just the same. What style you use is up to you, however it arguably makes more sense to leave your module code alone, and make the changes in the imports. This especially makes sense when you are importing from third party modules that you don't have any control over.

By the way, you can use the same syntax for default exports too, by referring to the export name `default` (this is a reserved word, not an actually valid identifier name). The difference with `export default randomSquare;` is that this exports the existing *live binding*, instead of capturing the value of the expression. If you re-assign `randomSquare` in the exporter, this form allows the importer to see the new value. Named default function and class declarations, such as `export default function randomSquare() {}`, also export live bindings.

js

```
import { default as randomSquare } from "./modules/square.js";

export { randomSquare as default };
```

Creating a module namespace object

The above method works OK, but it's a little messy and long-winded. An even better solution is to import each module's features inside a *module namespace object*. The following syntax form does that:

js

```
import * as Module from "./modules/module.js";
```

This grabs all the exports available inside `module.js`, and makes them available as members of an object `Module`, effectively giving it its own namespace. So for example:

```js
Module.function1();
Module.function2();
```

Again, let's look at a real example. If you go to our module-objects directory, you'll see the same example again, but rewritten to take advantage of this new syntax. In the modules, the exports are all in the following simple form:

```js
export { name, draw, reportArea, reportPerimeter };
```

The imports on the other hand look like this:

```js
import * as Canvas from "./modules/canvas.js";

import * as Square from "./modules/square.js";
import * as Circle from "./modules/circle.js";
import * as Triangle from "./modules/triangle.js";
```

In each case, you can now access the module's imports underneath the specified object name, for example:

```js
const square = Square.draw(myCanvas.ctx, 50, 50, 100, "blue");
Square.reportArea(square.length, reportList);
Square.reportPerimeter(square.length, reportList);
```

So you can now write the code just the same as before (as long as you include the object names where needed), and the imports are much neater.

Modules and classes

As we hinted at earlier, you can also export and import classes; this is another option for avoiding conflicts in your code, and is especially useful if you've already got your module code written in an object-oriented style.

You can see an example of our shape drawing module rewritten with ES classes in our classes directory. As an example, the `square.js` file now contains all its functionality in a single class:

```js
class Square {
  constructor(ctx, listId, length, x, y, color) {
    // …
  }

  draw() {
    // …
  }

  // …
}
```

which we then export:

```js
export { Square };
```

```js
// -- main.js --
import { Square } from "./modules/square.js";
```

And then use the class to draw our square:

```js
const square = new Square(myCanvas.ctx, myCanvas.listId, 50, 50, 100, "blue");
square.draw();
square.reportArea();
square.reportPerimeter();
```

Aggregating modules

We now have multiple modules, each doing a similar thing. If someone wants to use several of these, they have to write a separate import declaration for each:

```js
import { Square } from "./modules/square.js";
import { Circle } from "./modules/circle.js";
import { Triangle } from "./modules/triangle.js";
```

But it would be better if we could put all these exports into the same module, so no matter how many of these exports users need, they can grab all of them with one `import`.

```js
import { Square, Circle, Triangle } from "./modules/shapes.js";
```

In our module-aggregation example (based on our earlier classes example) we've got an extra module called `shapes.js`, which aggregates all the functionality from `circle.js`, `square.js`, and `triangle.js` together. This is called a "barrel module" because it doesn't do anything other than aggregating exports from all other modules. We've also moved our submodules inside a subdirectory inside the `modules` directory called `shapes`. So the module structure in this example is:

```
modules/
  canvas.js
  shapes.js
  shapes/
    circle.js
    square.js
    triangle.js
```

```js
// -- shapes.js --
import { Circle } from "./shapes/circle.js";
import { Square } from "./shapes/square.js";
import { Triangle } from "./shapes/triangle.js";

export { Circle, Square, Triangle };
```

These grab the exports from the individual submodules and effectively make them available from the `shapes.js` module. But there's no need to `import` something just to `export` it. You can use the `export`

`... from` syntax to directly import and re-export in one line.

```js
export { Square } from "./shapes/square.js";
export { Triangle } from "./shapes/triangle.js";
export { Circle } from "./shapes/circle.js";
```

> **Note:** The exports referenced in `shapes.js` basically get redirected through the file and don't really exist there, so you won't be able to write any useful related code inside the same file.

Alternatively, you can replace `shapes.js` with the special `export * from` syntax:

```js
export * from "./shapes/square.js";
export * from "./shapes/triangle.js";
export * from "./shapes/circle.js";
```

This re-exports *all* exports (other than the default) from the target module as named exports of the current module, so if you export something else from `square.js`, you don't have to modify `shapes.js` as well. Only do this if you actually want `shapes.js` to re-export everything from `square.js`, or you may accidentally expose things.

Almost all `import` syntaxes we've introduced have `export ... from` equivalents. The only exception is the default import `import x from "mod"`, for which `export x from "mod"` does not exist, and you must use the special `export { default } from` syntax.

```js
export { x } from "mod";
export { x as v } from "mod";
export * as ns from "mod";
export { default } from "mod";
```

Importing a module for its side effects

Sometimes you want to run a module's initialization code without importing any of its exports, for example, to install a polyfill. Use an import with only a module specifier:

```js
import "./modules/polyfills.js";
```

See the `import` reference for more information.

Import declarations are hoisted

Import declarations are hoisted. In this case, it means that the imported values are available in the module's code even before the place that declares them, and that the imported module's side effects are produced before the rest of the module's code starts running.

So for example, in `main.js`, importing `Canvas` in the middle of the code would still work:

```js
// …
const myCanvas = new Canvas("myCanvas", document.body, 480, 320);
myCanvas.create();
import { Canvas } from "./modules/canvas.js";
myCanvas.createReportList();
// …
```

However, it is considered good practice to put all your imports at the top of the code, which makes it easier to analyze dependencies.

Importing JSON modules

We have seen how to import from JavaScript modules, where data is exported with `export` statements. You can also import values from modules written in other languages, as long as the runtime environment knows how to interpret them. The specification only defines two other resource types: JSON modules and text modules. Here, we look at JSON modules, which are more common.

A JSON module is basically a standalone JSON file. When imported, it provides a single default export containing the parsed JSON value. You import it like this:

```js
import data from "./data.json" with { type: "json" };
```

Notice the extra `with { type: "json" }` at the end. This is an import attribute that tells the runtime environment to validate that the loaded file is indeed JSON. We will talk more about them in the Using modules on the web guide, because their semantics are not defined in the core language. You might be able to import JSON modules without `with { type: "json" }`. The only requirement is that if `with { type: "json" }` is specified, then the loaded module must be parsed as JSON. It is good practice to always declare the type of the module you are importing so it can work everywhere.

Text modules work similarly and result in strings.

```js
import text from "./text.txt" with { type: "text" };
```

Dynamic module loading

You can also dynamically load modules only when they are needed, rather than having to load everything up front. This has some obvious performance advantages; let's read on and see how it works.

The `import()` operator can be called with the path to the module as a parameter. It returns a `Promise`, which fulfills with a module namespace object (see Creating a module namespace object) giving you access to that object's exports. For example:

```js
import("./modules/myModule.js").then((module) => {
  // Do something with the module.
});
```

> **Note:** Dynamic import is permitted in the browser main thread, and in shared and dedicated workers. However `import()` will reject if called in a service worker or worklet.

Let's look at an example. In the dynamic-module-imports directory we've got another example based on our classes example. This time however we are not drawing anything on the canvas when the example loads. Instead, we include three buttons — "Circle", "Square", and "Triangle" — that, when pressed, dynamically load the required module and then use it to draw the associated shape.

In this example we've only made changes to our `index.html` and `main.js` files — the module exports remain the same as before.

Over in `main.js` we've grabbed a reference to each button using a `document.querySelector()` call, for example:

```js
const squareBtn = document.querySelector(".square");
```

We then attach an event listener to each button so that when pressed, the relevant module is dynamically loaded and used to draw the shape:

```js
squareBtn.addEventListener("click", () => {
  import("./modules/square.js").then((Module) => {
    const square = new Module.Square(
      myCanvas.ctx,
      myCanvas.listId,
      50,
      50,
      100,
      "blue",
    );
    square.draw();
    square.reportArea();
    square.reportPerimeter();
  });
});
```

Note that, because the promise fulfillment returns a module namespace object, the class is then made a subfeature of the object, hence we now need to access the constructor with `Module.` prepended to it, e.g., `Module.Square( /* … */ )`.

Another advantage of dynamic imports is that they are always available, even in script environments. Therefore, if you have an existing `<script>` tag in your HTML that doesn't have `type="module"`, you can still reuse code distributed as modules by dynamically importing it.

```html
<script>
  import("./modules/square.js").then((module) => {
    // Do something with the module.
  });
  // Other code that operates on the global scope and is not
  // ready to be refactored into modules yet.
  var btn = document.querySelector(".square");
</script>
```

Top-level await

Top-level `await` is a feature available within modules: the `await` keyword can be used at the top level, outside any async function. It allows a module to act as a big asynchronous function, meaning it can

finish asynchronous work before modules that depend on it execute, without blocking the evaluation of sibling modules that don't depend on it.

Let's take a look at an example. You can find all the files and code described in this section within the `top-level-await` directory, which extends from the previous examples.

Firstly we'll declare our color palette in a separate `colors.json` file:

json

```
{
  "yellow": "#F4D03F",
  "green": "#52BE80",
  "blue": "#5499C7",
  "red": "#CD6155",
  "orange": "#F39C12"
}
```

Then we'll create a module called `getColors.js` which uses a fetch request to load the `colors.json` file and return the data as an object.

js

```
// fetch request
const colors = fetch(
  "https://mdn.github.io/js-examples/module-examples/top-level-await/data/colors.json",
).then((response) => response.json());

export default await colors;
```

Notice the last export line here.

We're using the keyword `await` before specifying the constant `colors` to export. This means any other modules which include this one will wait until `colors` has been downloaded and parsed before using it.

js

```
// -- main.js --
import colors from "./modules/getColors.js";
import { Canvas } from "./modules/canvas.js";

const circleBtn = document.querySelector(".circle");

// …
```

We'll use `colors` instead of the previously used strings when calling our shape functions:

js

```
const square = new Module.Square(
  myCanvas.ctx,
  myCanvas.listId,
  50,
  50,
  100,
  colors.blue,
);
```

(code continues on the next page)

(code continued from the previous page)

```
const circle = new Module.Circle(
  myCanvas.ctx,
  myCanvas.listId,
  75,
  200,
  100,
  colors.green,
);

const triangle = new Module.Triangle(
  myCanvas.ctx,
  myCanvas.listId,
  100,
  75,
  190,
  colors.yellow,
);
```

This is useful because the code within `main.js` won't execute until the code in `getColors.js` has run. However, it doesn't hold up the evaluation of independent modules: our `canvas.js` module still evaluates while `colors` is being fetched.

Top-level `await` is not free; it has deep implications because it means a part of the module graph can only be evaluated asynchronously. See the module graph guide for more information.

Module metadata

Scripts are executed in the global context, so they can get information about their environment with global variables, such as `Window.document` or `Window.location`. Each module also has its own execution context which is accessed via a module-specific `import.meta` object. The properties of this object are defined by the host environment; the core language spec does not define any properties. See Using modules on the web for information about browsers, and Modules across platforms for information about Node.js, bundlers, and more.

Just as an example, in most web-like environments (browsers, Node.js, bundlers, etc.), `import.meta.url` provides the module's URL, which you can use to locate a resource relative to the module:

js

```
// -- modules/getColors.js --
const colorsURL = new URL("../data/colors.json", import.meta.url);
```

This keeps the resource URL relative to `getColors.js` even when the module is imported from a page or module in another directory.

Language versus host responsibilities

This article introduces modules as they are defined in the ECMAScript spec. JavaScript modules are inherently tied to the host environment. They need to adapt to different I/O conditions, different architectures, and different engineering needs. As such, the core language only defines the following:

- The syntax for module features, such as `import` and `export` declarations, `import.meta`, and the `import()` expression.
- Module graph building, linking, and evaluation, including cycle detection.
- The module namespace object's shape.

The core language does *not* care about the following:

- The concept of "files". Although each module is conventionally a separate file, it could as well be an in-memory object, a dynamically fetched string, or anything that can be represented as a JavaScript value.
- The structure of the module specifier string. It could be a URL, a file path, or any other identifier the host recognizes.
- The properties of `import.meta`. All properties, including `import.meta.url`, are host-defined.
- The module loading process. The host environment is responsible for fetching modules, including applying any import attributes, subject to the language's requirements, such as those for JSON modules.

In reality, runtime environments like browsers, Node.js, and Deno often end up implementing the same set of features, so that code is more likely to work across platforms. This guide walks through an example that's run in the browser, but we focus on core concepts that are applicable to all environments. In the Using modules on the web guide, we'll cover the specifics of module loading in browsers, especially import specifiers. Then, in the Modules across platforms guide, we will go further and discuss how the module system is integrated with other environments.

See also

- Using modules on the web
- Understanding the module graph
- Modules across platforms
- JavaScript modules on v8.dev (2018)
- ES modules: A cartoon deep-dive on hacks.mozilla.org (2018)
- ES6 in Depth: Modules on hacks.mozilla.org (2015)
- Exploring JS, Ch.16: Modules by Dr. Axel Rauschmayer

JavaScript › Guide › JavaScript modules

Using modules on the web

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules/Modules_on_the_web · MDN page last modified 2026-11-05

This page discusses how JavaScript modules are integrated with the web platform. Features discussed here are defined in the HTML spec and other web specifications, instead of by the core language, and they are generally only relevant to webpages. If you are writing modules for other environments, you may want to read the JavaScript modules and Modules across platforms guides instead.

Server configuration

The first difference you will encounter when migrating web pages to modules is that you can no longer view your HTML using the `file://` protocol. Serve the directory containing your HTML and modules through a local HTTP server, then open the page using the server's URL, such as `http://localhost:8000/`.

The reason for this is that module requests use CORS, and `file://` URLs are never same-origin. When importing from another origin, that server must allow your page's origin through its CORS response headers, such as `Access-Control-Allow-Origin`. By default, cross-origin module requests are sent without credentials such as cookies. Setting the `crossorigin="use-credentials"` attribute on the `<script>` element sends credentials, provided the server's CORS headers allow it (this applies to the whole graph fetched through that script).

The server must send JavaScript modules with a JavaScript MIME type, such as `Content-Type: text/javascript`. Check this for both `.js` and `.mjs` files. If an import fails with a MIME type error, inspect the response in your browser's network tools: a missing file or an HTML fallback page may have been returned instead of the module.

Applying modules to your HTML

To declare a script is a module in the `<script>` element, you need to include `type="module"`. For example, to import the `main.js` script, we use this:

html

```
<script type="module" src="main.js"></script>
```

Only the entry point needs a `<script type="module">` element: any modules it imports are loaded as modules automatically, so you don't need to add a `<script>` element just to declare something as a module.

You can also embed the module's script directly into the HTML file by placing the JavaScript code within the body of the `<script>` element:

html

```
<script type="module">
  /* JavaScript module code here */
</script>
```

You should generally define all your modules in separate files. Modules declared inline in HTML can only import other modules, but anything they export will not be accessible by other modules (because they don't have a URL).

You can only use `import` and `export` statements inside modules, not regular scripts. An error will be thrown if your `<script>` element doesn't have the `type="module"` attribute and attempts to import other modules. For example:

html · ✗ bad example

```
<script>
  import mod from "./mod.js"; // SyntaxError: import declarations may only appear at top
level of a module
  // ...
</script>
<script src="a-module-using-import-statements.js"></script>
<!-- SyntaxError: import declarations may only appear at top level of a module -->
```

Subresource integrity via the `integrity` attribute also works for module scripts, but the attribute only covers the entry module. Integrity metadata for its dependencies can be provided through an import map's `integrity` key.

> **Note:** Modules and their dependencies can be preloaded by specifying them in `<link>` elements with `rel="modulepreload"`. This can significantly reduce load time when the modules are used.

If you are already familiar with traditional `<script>` elements, here are some more behavior differences to look out for:

- Within the same environment, a module is only executed once, even if it has been imported multiple times or referenced in multiple `<script>` tags.
- There is no need to use the `defer` attribute (see `<script>` attributes) when loading a module script; module scripts declared in the document without `async` are deferred automatically.
- Adding the `async` attribute makes the module graph evaluate as soon as it has finished loading, without waiting for HTML parsing to complete and without any ordering guarantee relative to other scripts. Unlike for classic scripts, `async` works on inline module scripts too.

Modules in workers

Documents aren't the only place where modules run: workers can be modules too. You need to opt into modules by passing `type: "module"` to the `Worker()` or `SharedWorker()` constructor, or to `navigator.serviceWorker.register()`:

js

```
const worker = new Worker("./worker.js", { type: "module" });
```

The worker's file becomes the entry point of its own module graph.

You can only use `import` declarations or `import()` to import other modules inside a module worker. Calling `importScripts()` throws a `TypeError`. Note that dynamic `import()` is not available in service workers, so all of a module service worker's code must be reachable through static imports. Worklets load their code with `addModule()`, which is always a module.

Module specifiers on the web

Module specifiers are the strings coming after the `from` keyword:

js

```
import { name } from "./module.js";
```

In the Modules guide we already said that the module specifier format is runtime-dependent. Let's look at how they work in browsers.

First of all, everything on the web is URL-based. Each module has its URL, so you can import a module by its absolute URL directly:

```js
import { name } from "https://example.com/js/modules/module.js";
```

The browser sends an HTTP request to that given URL, and if the server returns a successful response with `Content-Type: text/javascript`, then the module loading process continues.

> **Note:** There's a popular belief that "ESM requires file extensions in module specifiers". This is true and false. The JavaScript runtime does not *add* an extension for you. If you write `import { name } from "https://example.com/modules/module";`, the browser still sends a request to the URL, without the `.js` extension. If that request succeeds with JavaScript content, the module is still loaded as usual. The only problem is that servers usually interpret extension-less requests to be requests for HTML files, so you almost always need extra configuration to get it serve a JavaScript file for that.

Other URL schemes that you can use as resource locations are supported too, such as `data:`. Read the `import` reference for more information.

Always importing from absolute URLs has the same problems as using absolute URLs for link targets: they are long, they don't work if you move your site to another domain, and they don't work if you move your entire JavaScript asset path. The module specifier can also be a relative URL, similar to the `href` attribute of `<a>` elements. The only difference is that all relative URLs must start with one of `/`, `./`, or `../`—i.e., you cannot use "bare" relative URLs like `modules/module.js`, but must write `./modules/module.js`. This restriction reserves bare specifiers so they can be given special meaning—the ecosystem convention, popularized by Node.js, was that a bare name like `"jquery"` refers to a package. On the web, that special meaning is now assigned by import maps. A bare specifier not remapped by an import map throws a `TypeError`.

The URL in the module specifier is resolved relative to the URL of the current module, not the URL of the HTML document. For example, within a module at `https://example.com/js/main.js`, to import `https://example.com/js/modules/module.js`, write:

```js
import { name } from "./modules/module.js";
```

Importing modules using import maps

In addition to relative and absolute URLs, import maps allow developers to specify almost any text they want in the module specifier when importing a module; the map provides a corresponding value that will replace the text when the module URL is resolved. The import map is defined using a JSON object inside a `<script>` element with the `type` attribute set to `importmap`.

> **Note:** An import map only applies to the document — the specification does not define how to apply an import map in a worker or worklet context.

For example, the `imports` key in the import map below defines a "module specifier map" JSON object where the property names can be used as module specifiers, and the corresponding values will be substituted when the browser resolves the module URL.

```html
<script type="importmap">
  {
    "imports": {
      "shapes": "./shapes/square.js",
      "shapes/square": "./modules/shapes/square.js",
      "https://example.com/shapes/square.js": "./shapes/square.js",
      "https://example.com/shapes/": "/shapes/square/",
      "../shapes/square": "./shapes/square.js"
    }
  }
</script>
```

> **Note:** From this point on, we'll omit the wrapping `<script type="importmap">` element and directly present import maps as JSON code. Just keep in mind that the JSON must be inlined in the HTML and cannot be an external file referenced with `src`.

With this map, you can now use bare specifiers like `shapes` and `shapes/square`, which would be rejected by default:

```js
// Bare module names as module specifiers
import { name as squareNameOne } from "shapes";
import { name as squareNameTwo } from "shapes/square";

// Remap a URL to another URL
import { name as squareNameThree } from "https://example.com/shapes/square.js";
```

There are many use cases of import maps:

- They allow modules to be imported using bare module names (as in Node.js), allowing modules that import npm packages to be directly deployed to the web (as long as the packages are also accessible at some URL). See Literal matching and Prefix matching.
- They allow modules imported using full URLs to be swapped out with other URLs as necessary, without changing the source code. See General URL remapping.
- They allow particular versions of a library to be imported, based on the path of the script that is importing the module. See The `scopes` object.

This can reduce the effort required to use the same JavaScript libraries in both browser and server.

In addition, the import map also allows module dependencies to have hash integrities (the `integrity` attribute on `<script>` only applies to the entrypoint module). This isn't introduced here because it's not relevant to specifier resolution. See Integrity metadata map.

Feature detection

You can check support for import maps using the `HTMLScriptElement.supports()` static method (which is itself broadly supported):

```js
if (HTMLScriptElement.supports?.("importmap")) {
  console.log("Browser supports import maps.");
}
```

The `imports` object

The `imports` property contains an object where each property value is a string. Other data types used as values will become `null`. There are many other conditions, introduced below, that will also cause the value to become `null`. When the resolver hits an entry whose value is `null`, the resolution for that specifier immediately fails.

Each key must be one of two kinds:

- The key does not end in `/`: it will be compared to the specifier as-is.
- The key ends in `/`: it will be compared as a prefix of the specifier. In this case, the value must also end in `/`, otherwise it becomes `null`. More on prefix matching later.

The value must be a URL, resolved relative to the base URL of the document containing the import map. If URL parsing failed, the value becomes `null`. (Note that relative URLs can be "bare"; only module specifiers aren't allowed to be bare.)

The property order of the `imports` object does not matter: internally, they are always sorted in descending lexicographic order by keys. During resolution, the properties are matched in order.

Literal matching

Suppose the following `<script>` element is included in the page at `https://example.com/welcome`.

```json
{
  "imports": {
    "square": "./shapes/square.js"
  }
}
```

Here, we use a relative URL as the value, which is resolved relative to the document's base URL, so it's equivalent to writing `https://example.com/welcome/shapes/square.js`. Had this `<script>` element been contained in the page `https://example.com/welcome/`, the value would be equivalent to `https://example.com/welcome/shapes/square.js` instead.

Now when the specifier resolver encounters a specifier like the following:

```js
import { name, draw, reportArea, reportPerimeter } from "square";
```

It will go through the `imports` entries one-by-one (of which there's only one), see that the `"square"` key is equal to the requested specifier, and use its value. The module at `https://example.com/shapes/square.js` will now be requested.

Prefix matching

If the key has a trailing slash `/` (in which case the value also needs a trailing slash), then in addition to literal matching, it can be matched as a prefix.

For example, consider the following import map:

```json
{
  "imports": {
    "shapes/": "./shapes/"
  }
}
```

Again, the relative URL value is resolved at the time the import map is parsed, relative to the document URL, so it's equivalent to writing `https://example.com/shapes/`.

Now when the specifier resolver encounters a specifier like the following:

```js
import { name as squareNameFour } from "shapes/square.js";
```

It will go through the `imports` entries one-by-one (of which there's only one), see that the `"shapes/"` key is a prefix of the requested specifier, and use its mapped value. The key prefix is stripped from the specifier, leaving `square.js`. This remainder is resolved using the mapped value (`https://example.com/shapes/`) as the base URL. The module at `https://example.com/shapes/square.js` will now be requested.

The remainder of the specifier is not allowed to "escape" the mapped value. For example, if you write the following:

```js
import { name as squareNameFour } from "shapes/../square.js";
```

Then the remainder is `../square.js`, which when resolved relative to `https://example.com/shapes/` will give `https://example.com/square.js`. The mapped value is no longer a prefix of the final URL. The resolver will throw an error in this case.

Keys ending in `/` can still be matched literally. For example:

```js
import { name as squareNameFour } from "shapes/";
```

The resolver will resolve this to `https://example.com/shapes/` and request that URL.

Once again, the entries of the `imports` object is always sorted such that if key A is a prefix of key B, then B comes before A, so with the following import map:

```json
{
  "imports": {
    "shapes/": "./shapes/",
    "shapes/square/": "./squares/"
  }
}
```

The module specifier `shapes/square/index.js` will still match the more-specific `shapes/square/` in priority to `shapes/`.

Mixing literal matching and prefix matching

In Node, you can import either a package itself, which imports the `main` file, or a file inside the package (assuming the package.json has no `exports` field):

```js
import _ from "lodash";
import fp from "lodash/fp.js";
```

In this case, you must provide separate entries for `lodash` and the `lodash/` prefix:

json

```
{
  "imports": {
    "lodash": "/node_modules/lodash-es/lodash.js",
    "lodash/": "/node_modules/lodash-es/"
  }
}
```

General URL remapping

The key does not need to be a bare name. It can also be an absolute or relative URL. Given the following import map:

json

```
{
  "imports": {
    "https://www.unpkg.com/lodash/": "/node_modules/lodash-es/"
  }
}
```

Now when the specifier resolver encounters a specifier like the following:

js

```
import { name as squareNameFour } from "https://www.unpkg.com/lodash/fp.js";
```

It will use the corresponding prefix entry and resolve this to `https://example.com/node_modules/lodash-es/fp.js`. This is especially useful for testing because you can mock a module with an alternative version.

URL remapping is also helpful for removing hashes from the module name. Script files used by websites often have hashed filenames to simplify caching. The downside of this approach is that if a module changes, any modules that import it using its hashed filename will also need to be updated/regenerated. This potentially results in a cascade of updates, which is wasteful of network resources.

Import maps provide a convenient solution to this problem. Rather than depending on specific hashed filenames, applications and scripts instead depend on an un-hashed version of the module name. An import map like the one below then provides a mapping to the actual script file.

json

```
{
  "imports": {
    "/node/srcs/application.js": "/node/srcs/application-fg7744e1b.js",
    "/node/srcs/dependency.js": "/node/srcs/dependency-3qn7e4b1q.js"
  }
}
```

If `dependency.js` changes, then its hash contained in the file name changes as well. In this case, we only need to update the import map to reflect the changed name of the module. The JavaScript source code continues to import from `"/node/srcs/dependency.js"`.

Unfortunately, import maps don't provide "suffix matching", so you can't say something like "replace `application.js` with `application-fg7744e1b.js`, whatever the path before it is". You must now always import from `/node/srcs/application.js` instead of `./application.js`, `https://example.com/node/srcs/application.js`, etc. Ultimately, you may still find that using a bundler to compile the code and rewrite the imports to be more convenient.

The scopes object

The `imports` object is global: the remapping is applied to all modules in the application, regardless of where they live. The same `lodash` specifier always resolves to the same module.

In real life, different modules may want to import different versions of the `lodash` package. For example, package A may depend on `lodash` v3, while package B may depend on `lodash` v4. In Node, this is achieved by giving each package its own `node_modules`:

```
https://example.com
└── node_modules
    ├── lodash          <-- v4; your application and package B will use this
    ├── package-a
    │   └── node_modules
    │       └── lodash  <-- v3; package A will use this
    └── package-b
```

To directly deploy this `node_modules` to your website without rewriting its source code, you must write the import map such that modules within `package-a` will have their `lodash` specifier resolve to a different address from `package-b`. You implement this with the `scopes` object, like the following:

json

```
{
  "imports": {
    "lodash": "/node_modules/lodash/lodash.js"
  },
  "scopes": {
    "/node_modules/package-a/": {
      "lodash": "/node_modules/package-a/lodash/lodash.js"
    }
  }
}
```

Within the `scopes` object, each key is a scope, and each value is a module specifier map, just like `imports`. Just like keys of `imports`:

- The scope can either not end in `/`, in which case it's matched literally against the importer's URL, or it can end in `/`, in which case it can additionally match as a prefix of the importer's URL.
- The scope keys are sorted in descending lexicographic order (after URL resolution), so more specific prefixes match in priority to less specific ones.

The only difference is that scopes must be valid URLs which are resolved relative to the document base URL.

The scopes are attempted in order: if a scope doesn't provide the corresponding import mapping for the requested specifier, the resolver moves to the next matching scope, if any. If no scope key matches the importer, or if all matching scopes don't provide the import mapping, then the global `imports` map is used. However, if it does hit an entry anywhere in this process, but the value is `null` (e.g., if the value isn't a valid URL), resolution fails immediately without falling back.

So for example, in the example above, the scope key is `/node_modules/package-a/`, which is resolved to `https://example/node_modules/package-a/`. If the module at `https://example.com/node_modules/package-a/index.js` contains:

js

```
import _ from "lodash";
```

The resolver will look at the `scopes` in priority, see that the importer's URL is a prefix of the scope key, and use its specifier map, which defines that `lodash` should map to `https://example.com/node_modules/package-a/lodash/lodash.js`, the v3 version.

On the other hand, if the module at `https://example.com/node_modules/package-b/index.js` contains the same line, then the resolver will check `scopes` and find no matching key, and fall back to the global `imports` map, which imports `https://example.com/node_modules/lodash/lodash.js`, the v4 version.

Note that the scope key is only used to match the importer's URL; it does not provide the base URL for resolving or validating any other URL. The final mapped URL does not have to match the scope path, and relative mapped URLs are still resolved to the base URL of the script that contains the import map.

Loading non-JavaScript resources

The Modules guide already introduces the import attributes syntax, which allows you to import non-JavaScript resources. The resource types specified by the core language are `type: "json"` and `type: "text"`.

```js
import colors from "./colors.json" with { type: "json" };
```

On the web, there is additionally `type: "css"`, which imports CSS files as `CSSStyleSheet` objects.

```js
import styles from "./styles.css" with { type: "css" };

document.adoptedStyleSheets = [styles];
```

While the core language spec does not say much about the `type` attribute in general (other than how `type: "json"` must result in a JSON module and `type: "text"` must result in a text module), the HTML spec requires browsers to implement strict module type checking for security reasons. If you specify `type: "json"` or `type: "css"`, the result must be served with the corresponding `Content-Type` (such as `application/json` or `text/css`). If you don't specify any type, the result also must be served as JavaScript or WebAssembly (`text/javascript` or `application/wasm`).

In the example above with `type: "css"`, if this file turns out to be JavaScript (it is served with a `Content-Type` of `text/javascript`), the import will fail. This makes sure that you don't accidentally execute code where you don't intend to. Read the import attribute reference for more information.

Other environments, including Node.js and Deno, often mimic web semantics, so they also require strict type matching.

On the web, the `type` attribute also changes the request's destination, so even with the same string specifier, you may end up with different module content.

```js
import code from "./module";
// This sends a request with Sec-Fetch-Dest: script
// The server may respond with JavaScript: `export default 1;`

import data from "./module" with { type: "json" };
// This sends a request with Sec-Fetch-Dest: json
// The server may respond with JSON: `{ "value": 1 }`
```

(code continues on the next page)

(code continued from the previous page)

```
import styles from "./module" with { type: "css" };
// This sends a request with Sec-Fetch-Dest: style
// The server may respond with CSS: `.value { color: green; }`

import text from "./module" with { type: "text" };
// This sends a request with Sec-Fetch-Dest: text
// The server may respond with text: `Hello world!`
```

Module caching

Module caching lets imports reuse a module and its state. On the web, the HTML specification defines which requests share a module.

In browsers, the module map uses the resolved request URL and the module type as its key. Different specifier strings can therefore identify the same module. For example, although the following two requests use different string specifiers, they point to the same module (assuming there's no import map that changes the resolution):

js

```
// -- https://example.com/main.js --
import * as config1 from "./config.js";
import * as config2 from "https://example.com/config.js";

console.log(config1 === config2); // true
```

The module type, selected through import attributes, is also part of the key, so requesting a URL with `{ type: "json" }` does not reuse the entry for that URL without a `type` (which is implicitly JavaScript). Separate documents and workers have separate module maps, so importing a module in a page and in its worker does not share the module's variables between them.

Module caching is separate from HTTP caching. HTTP caching can reuse response bytes; module caching reuses the module's identity and state. A new document can evaluate a module again even if its source comes from the HTTP cache. Conversely, `Cache-Control: no-store` does not make an already evaluated module execute again on each import.

Browsers treat URLs with different query strings or fragments as different module identities, even if they retrieve identical source code:

js

```
import * as config1 from "./config.js?version=1";
import * as config2 from "./config.js?version=2";

console.log(config1 === config2); // false
```

This creates two module instances from the same source code. Any further dependencies are still shared—if both instances import `"./logger.js"`, that specifier still resolves to the same URL from both, so they share `logger.js`.

For loading errors, the HTML specification requires failed fetches, including HTTP error responses, to be removed from the module map so a later import can retry. The HTTP cache may still supply a cached error response. Parse errors, however, are retained in the module map, so retrying the same module does not fetch corrected source.

The import.meta object

The Modules guide also introduced the `import.meta` object and mentioned that *all* of its properties are host-defined.

Use `import.meta.url` with the `URL()` constructor when a resource is located relative to your module:

```js
// -- modules/set-user.js --
const apiURL = new URL("../api/set-user", import.meta.url);

export async function setUser(name) {
  const response = await fetch(apiURL, {
    method: "POST",
    body: JSON.stringify({ name }),
  });
  if (!response.ok) {
    throw new Error(`Unable to set user: ${response.status}`);
  }
}
```

In a window, a relative URL passed directly to `fetch()`, such as `fetch("../api/set-user")`, is resolved against the document's base URL. It is not resolved relative to the JavaScript module containing the call. Constructing the URL explicitly makes the resource location independent of the page that imports the module.

To resolve a module specifier using the host's module resolution rules, use `import.meta.resolve()`. For example, in a browser with an import map that defines `"shapes"`, `import.meta.resolve("shapes")` returns its resolved URL. Unlike `import()`, this resolves the specifier without loading or evaluating the module.

See also

- HTML `<script>` element
- `<script type="importmap">`
- JavaScript modules

JavaScript › Guide › JavaScript modules

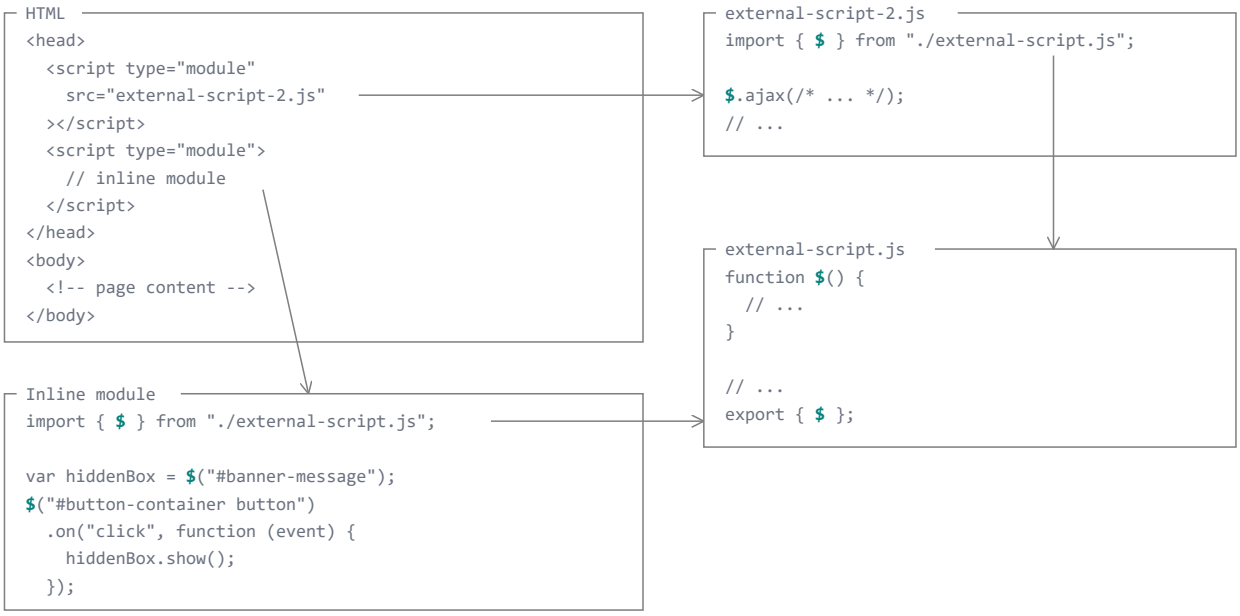
Understanding the module graph

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules/Module_graph · MDN page last modified 2026-11-05

Through `import` and `export` declarations, JavaScript modules form a complex graph of dependencies. This guide talks about various concepts related to how modules are loaded, and how that impacts the way you structure and author your code.

What is the module graph?

In the JavaScript modules guide, we already showed a basic example of a module graph:



Graphs are made of nodes connected by edges. In this module graph, the nodes are the different JavaScript files being imported (all modules here are JavaScript, but JSON, CSS, WebAssembly, etc. would all be valid). Each time you write `import ... from "module B"` (or `export ... from "module B"`) in `module A`, you create a directed edge from `module A` to `module B`. The HTML file itself is not part of the module graph, but it defines all the *entry points*.

Each module graph needs at least one entry point, from which the runtime starts discovering dependencies. In the example above, there are multiple entry points: each `<script>` element starts one. In Node.js (or other server-side runtimes), this entry point is the file you invoked `node` with. In workers, this is the file you passed to the `Worker()` constructor. Graphs from different entry points aren't necessarily disjoint: if they import the same module (or the entry point itself is already imported), they can be merged into one larger graph.

From module source to execution

Let's talk about how modules are loaded and evaluated, using the following example. We'll assume it is run in Node with `node main.js` (there's nothing Node-specific here, but modules are easier to spin up in Node).

```js
// -- main.js --
import { greet } from "./formatters.js";
import config from "./config.js";

greet("Josh", config.locale);
```

```js
// -- formatters.js --
import config from "./config.js";
import { log } from "./logger.js";

export function greet(name, localeName = config.defaultLocale) {
  const locale = new Intl.Locale(localeName);
  if (locale.language === "en") {
    log(`Hello, ${name}!`);
  } else if (locale.language === "fr") {
    log(`Bonjour, ${name} !`);
  } else {
    throw new Error(`Sorry, I don't speak ${locale}.`);
  }
}
```

```js
// -- config.js --
export default {
  locale: "en-US",
  defaultLocale: "en-US",
};
```

```js
// -- logger.js --
export function log(message) {
  console.log(`[${new Date().toISOString()}] ${message}`);
}
```

Loading the graph

Module loading requires a close collaboration between the host (Node) and the engine (V8). This happens recursively, where at each level of recursion, the host receives a *modules request* (containing the module specifier and import attributes, if any), and then:

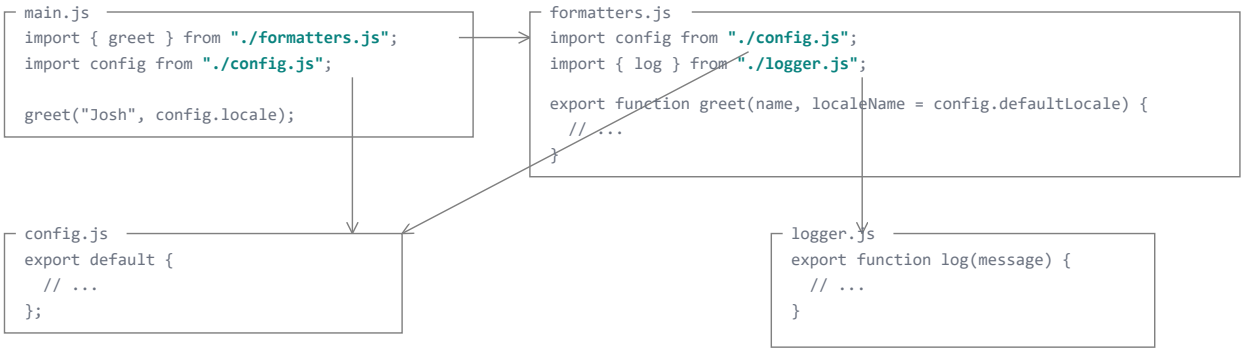
- The host loads the respective module's source code (such as by mapping it to a file system location and reading that file's content).
  - This step can be further broken down into two steps: specifier resolution and actually sending the request, which may be an HTTP request in browsers or a file system access in Node.
  - The entry point is slightly different: there's usually no specifier resolution. For example, the `main.js` command-line argument is directly read as a file path, and the `src` attribute of `<script>` elements is directly read as a URL; neither are module specifiers. The fetching process might also be slightly different.
- The host gives the engine the module source code. The engine parses the source code and collects all `import` and `export from` declarations, which create new module requests. (If the source is not JavaScript, the host may ask some other parser to process the source code instead.)

- All these module requests are given to the host and the host starts the whole process again, possibly handling multiple requests concurrently.
- If the host finds out that it has already loaded or is loading a module request (in browsers and Node.js, modules are cached by both resolved URLs and module types), it does not fetch it again and does not ask the engine to parse again. Instead, it supplies the existing module, waiting for the ongoing load to finish if necessary.

In this particular example:

- The host reads the `main.js` file and asks the engine to parse it.
- The engine finds two module requests, for `./formatters.js` and `./config.js`. It sends those module requests to the host.
- The host loads both of them by reading the corresponding files. Suppose `formatters.js` finishes loading first, so the host sends its source text to the engine first.
- Parsing `formatters.js` reveals another request for `./config.js`, as well as a new dependency, `./logger.js`. The engine sends both of them to the host.
- The host finds out that `./config.js` was already requested by `main.js` and skips it (although it's still loading), and starts loading `logger.js`.
- The loading for both `logger.js` and `config.js` finishes; neither contains further requests.

After this step, the module graph is already established. All modules have been loaded and parsed, but no JavaScript module bodies have been executed by this loading process.



Linking modules

After the graph has finished loading successfully, the host asks the engine to *link* the entry module. The main goal of linking is to set up the module environment, i.e., the various bindings, including the bindings to be exported. Each imported name is resolved to the place where it's actually defined and exported, potentially going through multiple layers of `export from` declarations. All modules in the graph need to be linked before they can be evaluated, so that importing a name that isn't exported on the other side can be caught early, before any code starts evaluating. It also makes subsequent evaluation easier because it avoids walking the module graph many times as each imported name gets used.

During the linking phase, the engine traverses the module graph using depth-first search (DFS). This happens recursively, where at each level of recursion, the engine visits a module (starting at the entry point), and then:

- If the module is already linked, or is already being linked (which happens with cyclic imports), the engine does not process it again. Otherwise, the engine marks the module as currently linking.

- The engine recursively links each of the module's dependencies using the same process. Unlike loading, linking is fully synchronous and deterministic: dependencies are visited in the order their module requests appear in the source.
- After all dependencies have been visited, the engine sets up the module's own environment. It creates a binding for each top-level declaration of the module. For each imported name (other than imports that resolve to a module namespace object), it resolves the name to the binding in the module that actually declares it, then creates an *indirect binding*: a name in the importing module that permanently refers to that other module's binding. Imports that resolve to a module namespace object, including named imports of `export * as ns from` exports, instead create local bindings initialized with that object. If some imported name cannot be resolved, linking fails.
- This module is marked as linked. (If there's a cycle, then they are all marked as linked together; again, we'll talk about this later.)

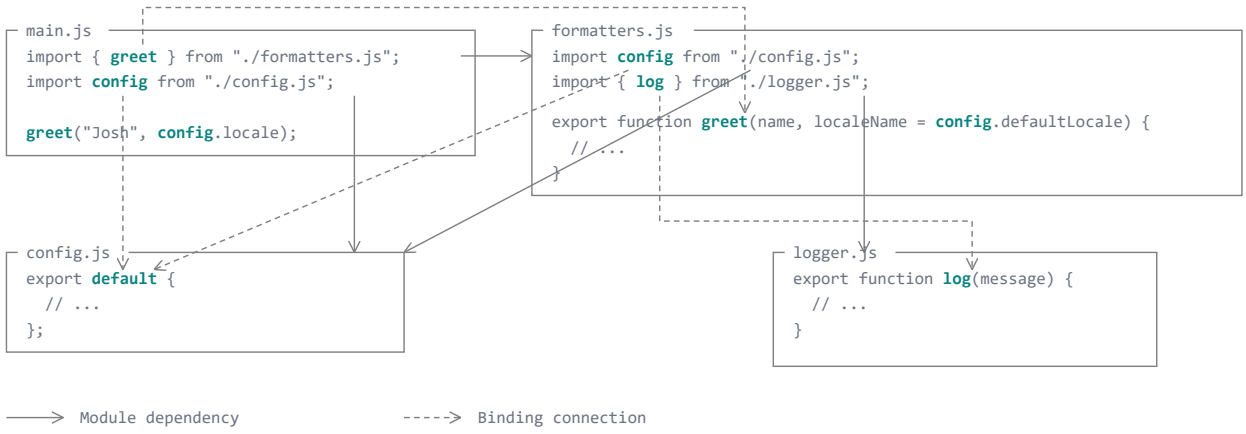
Just like normal declarations (see hoisting), exported bindings are also created and initialized in separate steps—function declarations are initialized during linking, `var` declarations are initialized to `undefined`, while `export default`, `let`, `const`, and `class` declarations remain uninitialized until evaluation reaches them.

In our example:

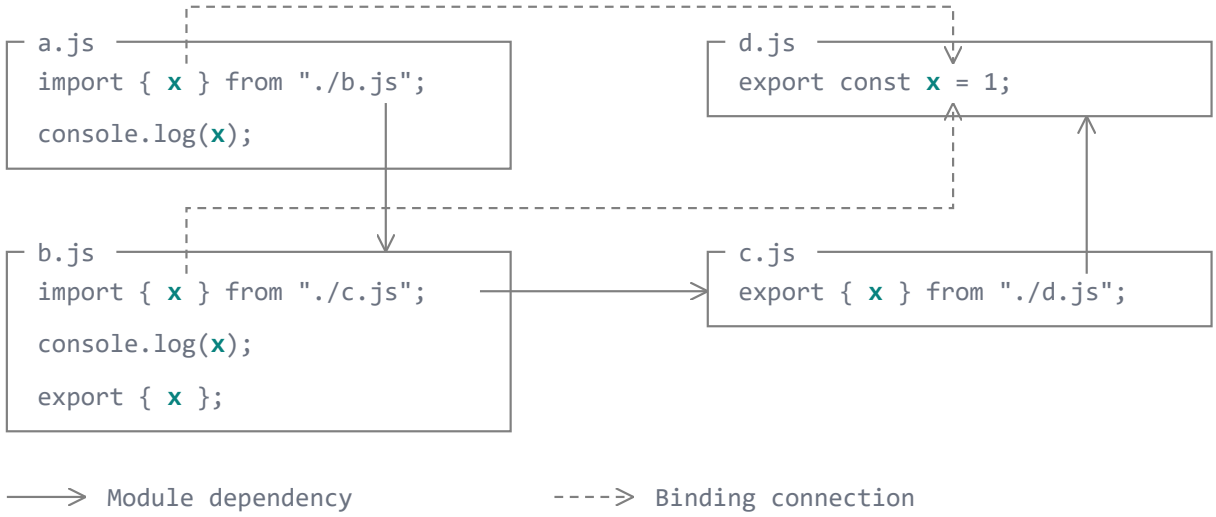
- The engine starts with `main.js`, follows its dependency on `formatters.js`, and then reaches `config.js`. It marks `main.js` and `formatters.js` as "linking".

  > **Note:** This is how the traversal is "depth-first": the engine first goes as far as possible until it hits a module with no further unprocessed requests, and works back from there.

- For `config.js`, the engine creates a binding for the default export, but does not yet evaluate the object literal—evaluation happens in the next phase. The binding remains uninitialized.
- Next, it links `logger.js`, the next dependency of `formatters.js`. The engine creates the `log` binding and initializes it with the function object.
- The engine returns to `formatters.js`. Its imported `config` binding refers to the default export of `config.js`, and its imported `log` binding refers to the `log` binding in `logger.js`. Its own `greet` binding is initialized with the function object.
- Finally, the engine returns to `main.js`. Its other dependency, `config.js`, has already been linked, so it is not linked again. The engine connects `main.js`'s `greet` and `config` imports to their respective exported bindings.



Most of the time, the imported name is declared directly in the imported module, as in the example here, so the binding connections follow the same paths as the module dependency graph. However, if you use `export from`, or export an identifier you imported, the linker will follow those layers until it finds the place where it's actually declared.



Here, although the import chain is `a.js → b.js → c.js → d.js`, every single `x` binding in this chain eventually resolves to the same `const` declaration in `d.js`, while `c.js` has no module-scoped binding. The binding connections go directly to `d.js`, while the module dependency graph still includes the entire import chain.

An `export * from` declaration re-exports all names from another module (except for `default`) as named exports of the current module. When resolving an imported name, the engine first checks for an explicit export of that name. If there is none, it follows the `export * from` declarations and recursively resolves the name in those modules.

Multiple `export * from` declarations may make the current module export the same name multiple times (again, `export * from` never collides with an explicit named export; the latter always takes priority). If these declarations eventually resolve to the same binding, then there's no ambiguity. If they resolve to different bindings, the name is ambiguous, even if those bindings happen to contain the same value. For example:

```js
// -- first.js --
export const x = 1;
```

```js
// -- second.js --
export const x = 1;
```

```js
// -- combined.js --
export * from "./first.js";
export * from "./second.js";
// Should "x" from this module resolve to first.js or second.js?
```

The star exports alone do not cause an error. However, `import { x } from "./combined.js"` or `export { x } from "./combined.js"` causes a SyntaxError during linking. A namespace import, `import * as combined from "./combined.js"`, succeeds, but the namespace object has no `x` property.

Adding `export { x } from "./first.js"` to `combined.js` resolves the ambiguity: explicit exports take precedence over star exports, regardless of declaration order.

Evaluating modules

Now that all modules have their environment prepared, they are finally ready for evaluation. The host asks the engine to evaluate the entry module. The engine traverses the module graph yet again, using depth-first search. For now, assume no module uses top-level `await`. This happens recursively, where at each level of recursion, the engine visits a module (starting at the entry point), and then:

- If the module has already been evaluated successfully, it does not execute it again. If a previous evaluation failed, the engine propagates the recorded error instead. If the module is already being evaluated, the engine does not visit its dependencies again, avoiding infinite recursion in a cycle. Otherwise, it marks the module as currently evaluating.
- The engine recursively evaluates each dependency using the same process, in the order their module requests appear in the source.
- After visiting all dependencies, the engine executes the module body. Variable initializers run, initializing the bindings prepared during linking, and other statements execute in source order. Imported names access the bindings connected during linking. In a cycle, a dependency's body may not have executed yet, so an imported binding may still be uninitialized; we'll discuss this later.
- After the module body finishes successfully, the module is marked as evaluated. In a cycle, modules in the same strongly connected component are marked as evaluated together once their bodies have finished.

In our example:

- The engine starts with `main.js`, follows its dependency on `formatters.js`, and then reaches `config.js`. It marks `main.js` and `formatters.js` as "evaluating".
- For `config.js`, the engine evaluates the object literal and initializes the default-export binding with the resulting object.
- Next, it evaluates `logger.js`, the next dependency of `formatters.js`. Its `log` function was already created during linking, and the function body does not execute until the function is called, so there's nothing else to evaluate here.
- The engine returns to `formatters.js`. Its `greet` function was also created during linking, so again nothing to evaluate here.
- Finally, the engine returns to `main.js`. Its other dependency, `config.js`, has already been evaluated, so it is not executed again. The engine calls `greet("Josh", config.locale)`, which calls `log` and prints a timestamp followed by `Hello, Josh!`.

Moving the `import` declarations below the `greet()` call, while keeping their relative order, would not change this order: dependencies are evaluated before the importing module's body starts, regardless of where its import declarations appear among other statements.

If evaluation throws an uncaught error, it propagates through the dependent modules being evaluated, and the entry module's evaluation promise rejects. Effects of code that already ran are not undone, and the error is retained so subsequent evaluation attempts do not execute the failed module again.

With top-level `await`, a module can suspend execution and delay modules that depend on it while other branches continue; we'll cover this in asynchronous evaluation.

Errors during the module pipeline

Any point in the module pipeline—load, link, and evaluate—could fail. The errors might leave the graph in a half-processed state.

Errors from loading

Loading errors for a module request may be for one of the following reasons (this step is performed by the host, so many reasons are host-specific):

- The specifier failed to resolve (for example, it's not a valid URL)
- The request contains invalid import attributes
- The request returned an error status or the file system generated an error
- The source code failed to parse

If a module request failed, the loading process for this graph stops immediately without processing any unprocessed module requests. Already in-progress requests may still complete normally and be used later.

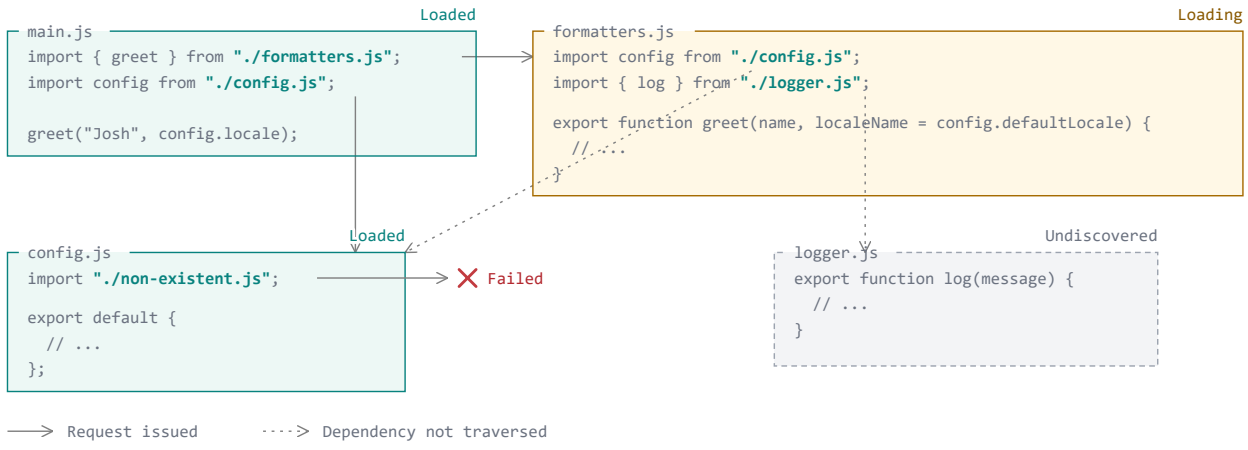
For example, suppose we add an import to `config.js` in the graph above:

```js
// -- config.js --
import "./non-existent.js";

export default {
  locale: "en-US",
  defaultLocale: "en-US",
};
```

1. `main.js` starts loading both `formatters.js` and `config.js`.
2. `config.js` finishes first.
3. Its request for `non-existent.js` fails (in Node.js, resolution throws `ERR_MODULE_NOT_FOUND`) while `formatters.js` is still loading.
4. The request for `formatters.js` can still finish, but this graph's loading process no longer traverses its dependencies, so `logger.js` is never requested by this traversal.

This assumes a fresh load without preloading or another entry point requesting these modules; if `formatters.js` had finished earlier, `logger.js` might already be loading too. No modules in this example are linked or evaluated.



Retry behavior depends on the host and the kind of failure. For browser-specific retry behavior, see Module caching on the web. Unlike module loading, `fetch()` does not reject merely because the response has an HTTP error status.

Linking is only performed if *all* modules in the graph completed loading. A graph partially loaded due to errors is never linked.

Errors from linking

Linking errors for an import may be for one of the following reasons:

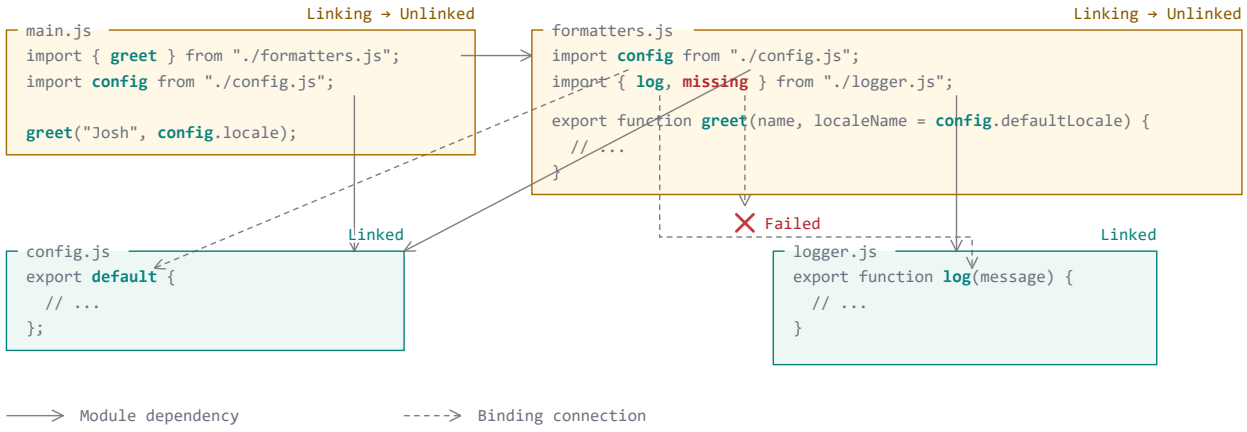
- The import points to a non-exported binding in the target module
- The import points to a binding that's cyclically exported and never actually defined (for example, `a.js` writes `export { x } from "./b.js"` and `b.js` writes `export { x } from "./a.js"`)
- The import points to a binding that's ambiguous between two `export * from` declarations

For example, starting from the original graph without the loading error, suppose we add `missing` to the import from `logger.js` in `formatters.js`:

```js
// -- formatters.js --
import config from "./config.js";
import { log, missing } from "./logger.js";

// The rest of the module is unchanged.
```

1. All four modules finish loading and are marked as unlinked.
2. Linking starts with `main.js`, then visits `formatters.js` and its dependencies.
3. `config.js` and `logger.js` are linked successfully.
4. While setting up the environment for `formatters.js`, the engine creates the imports for `config` and `log`, but cannot resolve `missing`, because `logger.js` does not export that name. It throws a `SyntaxError`.
5. `formatters.js` and `main.js`, which were still being linked, return to the unlinked state. `config.js` and `logger.js` remain linked. No module bodies are evaluated.



Note that linking is synchronous and side-effect-free. The host can do it after loading and before evaluation, and even retry it if it fails (although the same graph of JavaScript modules will encounter the same linking error). If a linking error occurs, this import does not start evaluation. Shared dependencies may already have been evaluated through another entry point.

Errors from evaluation

Errors from evaluation are the normal errors you get from executing JavaScript: using `throw` statements, making invalid function calls, accessing uninitialized variables, etc. When evaluation of a module throws, it may also leave the module graph in a partially-evaluated state.

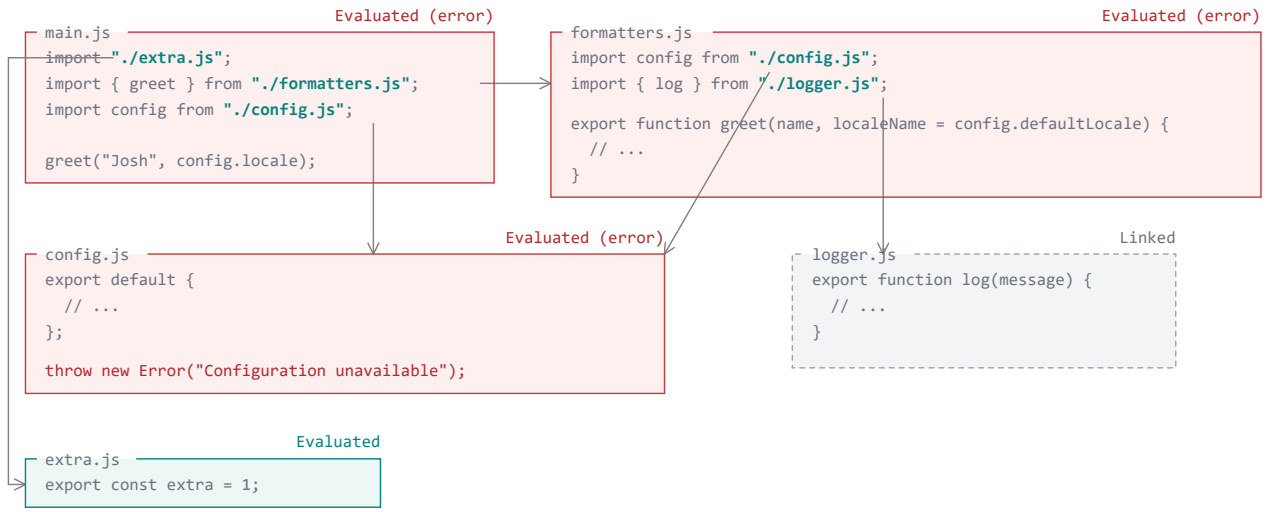
For example, starting from the original graph without the loading or linking errors, suppose we add a top-level `throw` to `config.js`:

```js
// -- config.js --
export default {
  locale: "en-US",
  defaultLocale: "en-US",
};

throw new Error("Configuration unavailable");
```

We also add one more dependency before the existing imports in `main.js`, just to show the case where a module finished evaluating successfully.

1. All five modules finish loading and linking successfully.
2. Evaluation starts with `main.js`, then visits its first dependency, `extra.js`, which finishes evaluating successfully.
3. The traversal visits `formatters.js`, then its first dependency, `config.js`, which creates the configuration object, then reaches the `throw` statement.
4. The error propagates through `formatters.js` to `main.js`. Neither module's body starts executing, so `greet("Josh", config.locale)` is never called.
5. `config.js`, `formatters.js`, and `main.js` are marked as evaluated, with the error recorded. `extra.js` remains successfully evaluated. The traversal never reaches `logger.js`, which remains linked but unevaluated.



The *evaluated* status means the evaluation attempt is finished, not necessarily that the module's body ran successfully—or at all. Because module evaluation has side effects, the engine guarantees that the same module is only ever evaluated once, whether or not it encountered an error (including when its dependency errored). Subsequent imports that reuse this module expose the same module namespace object on success, or propagate the recorded error on failure. The namespace object continues to

expose live bindings. A new environment, such as a new page or worker, can load and evaluate the source again. Any effects of code that ran before the error are not undone.

Cyclic imports

The module dependency graph is traversed using depth-first search (DFS) twice for linking and evaluation (loading can traverse branches concurrently in a non-deterministic order, and cycles do not affect it any more than normal shared dependencies do). This strategy works well for acyclic dependency graphs, like the example presented above. Even the case where two modules import the same module is minimally problematic—there's always a well-defined "end" (`config.js` and `logger.js`) from where we can work backwards. We just need to be careful not to link or evaluate the same module twice.

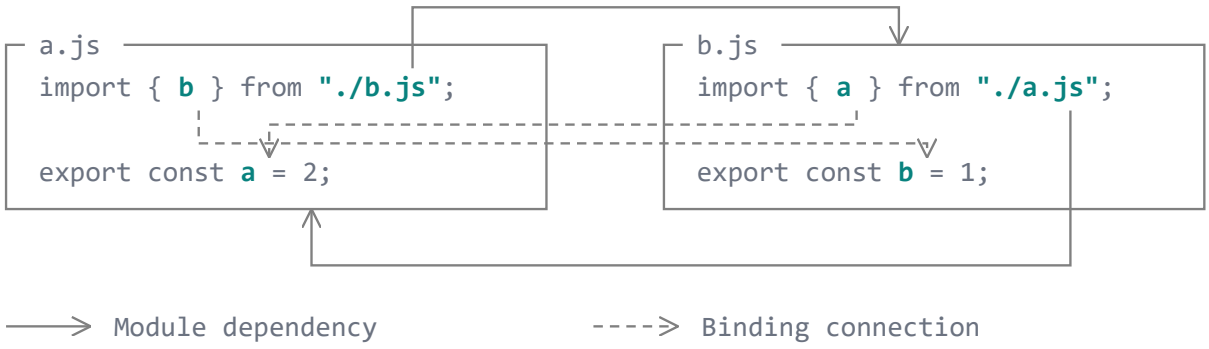
However, cycles are often inevitable. A cyclic import arises if module `a` imports module `b`, but `b` directly or indirectly depends on `a`. For example, consider the simplest two-module cycle:

```js
// -- a.js --
import { b } from "./b.js";

export const a = 2;
```

```js
// -- b.js --
import { a } from "./a.js";

export const b = 1;
```



Assuming no errors are ever thrown (including errors caused by accessing uninitialized imports, which we'll talk about very soon), the naïve DFS approach actually still works. Take the linking steps as an example:

- The engine visits `a.js` and marks it as currently linking. But before actually setting up its module environment, its dependencies must be visited first.
- The engine visits `b.js` and marks it as currently linking. But before actually setting up its module environment, its dependencies must be visited first.
- The engine visits `a.js` again, and sees that it's already marked as linking. There's no other downstream dependency.
- The engine links `b.js`, pointing `a` to the `export const a` in `a.js`.
- The engine links `a.js` by pointing `b` similarly.
- No upstream dependency is waiting for `a.js` and `b.js` to link; the whole graph finishes.

As long as each import can be traced back to some concrete definition and not a cyclic re-export (for example, `a.js` writes `export { x } from "./b.js"` and `b.js` writes `export { x } from "./a.js"`), linking does not require the definition to come from an already-linked module.

Similarly, for evaluation, `b.js` evaluates first because it's a dependency of `a.js`. It also depends on `a.js`, but `a.js` is already marked as evaluating, so the engine doesn't go back to `a.js`. The `b.js` module then evaluates the `const b` initialization. Finally, the `a.js` module evaluates and initializes `a = 2`.

As this example shows, cyclic imports don't always fail. The imported variable's value is only retrieved when the variable is actually used (hence allowing live bindings), and only if the variable remains uninitialized at that time will a `ReferenceError` be thrown. So for example, even if you use the value of `a` and `b`, you can prevent errors if you make sure the initialization happens before the usage:

```js
// -- a.js --
import { b } from "./b.js";

setTimeout(() => {
  console.log(b); // 1
}, 10);

export const a = 2;
```

```js
// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);

export const b = 1;
```

In this example, both `a` and `b` are used asynchronously. Therefore, at the time the module is evaluated, neither `b` nor `a` is actually read, so the rest of the code is executed as normal, and the two `export` declarations produce the values of `a` and `b`. Then, after the timeout, both `a` and `b` are available, so the two `console.log` statements also execute as normal.

If you change the code to use `a` synchronously, the module evaluation fails:

```js
// -- a.js (entry module) --
import { b } from "./b.js";

export const a = 2;
```

```js
// -- b.js --
import { a } from "./a.js";

console.log(a); // ReferenceError: Cannot access 'a' before initialization
export const b = 1;
```

This is because when JavaScript evaluates `a.js`, it needs to first evaluate `b.js`, the dependency of `a.js`. However, `b.js` uses `a`, which is not yet available.

On the other hand, if you change the code to use `b` synchronously but `a` asynchronously, the module evaluation succeeds:

```js
// -- a.js (entry module) --
import { b } from "./b.js";

console.log(b); // 1
export const a = 2;
```

```js
// -- b.js --
import { a } from "./a.js";

setTimeout(() => {
  console.log(a); // 2
}, 10);
export const b = 1;
```

This is because the evaluation of `b.js` completes normally, so the value of `b` is available when `a.js` is evaluated.

Function declarations are much more resilient in cycles, because they are initialized during linking, before any module body runs. Just like how hoisting allows two functions to call each other regardless of their declaration order, link-time initialization allows the same to happen across module boundaries.

```js
// -- a.js (entry module) --
import { isEven } from "./b.js";

export function isOdd(n) {
  return n !== 0 && isEven(n - 1);
}
```

```js
// -- b.js --
import { isOdd } from "./a.js";

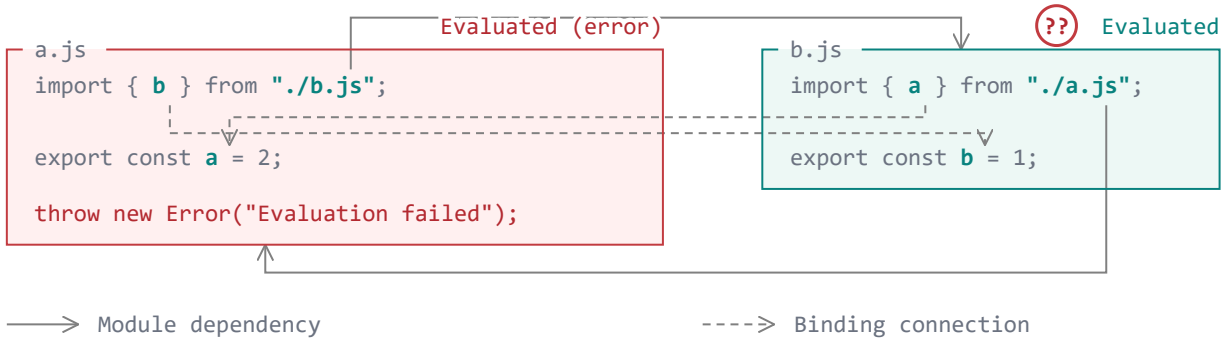
console.log(isEven(5)); // false

export function isEven(n) {
  return n === 0 || isOdd(n - 1);
}
```

Even though there's no asynchronous waiting, the `isOdd` binding in `a.js` was already initialized with its function object during linking, before `a.js`'s body has run. These modules only export function declarations, and their functions do not read any state initialized during module evaluation, so they can safely call each other. Reading an uninitialized `let`, `const`, or `class` binding would still throw, even if that binding were private to an exported function's module.

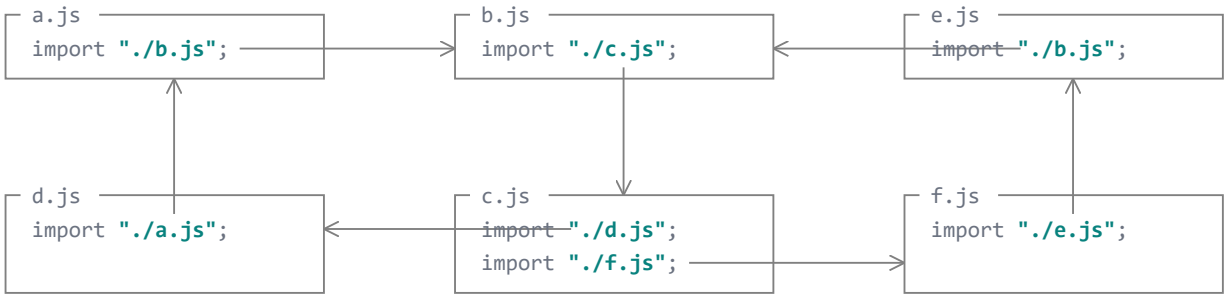
Errors with cyclic imports

The problem becomes more complicated when there's a linking or evaluation error. Suppose we add `throw new Error("Evaluation failed");` after `export const a = 2;` in the two-module cycle above. A naïve traversal would mark `b.js` as evaluated successfully before executing `a.js`, which then throws:



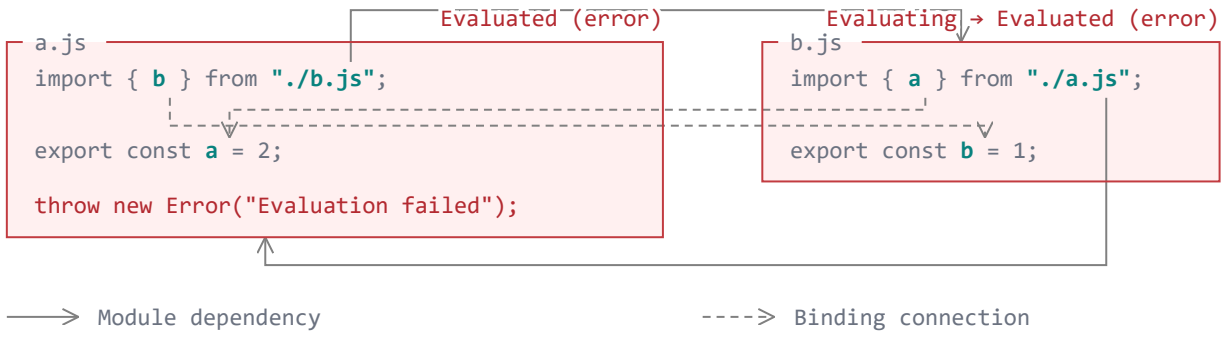
In this example, if `a.js` fails to link or evaluate, we don't want to mark `b.js` as successful even though it has finished its own linking and evaluating—because its success is contingent on the success of `a.js`, a dependency of `b.js`! Therefore, in a cycle, we must mark all modules as succeeding or failing together.

The precise term we are looking for is a *strongly connected component* (SCC). An SCC is a maximal group of modules where every module can reach every other module. An SCC can consist of a single cycle or a union of cycles that share vertices, as long as the group is maximal:



In graph theory, we know that any directed graph can be seen as an acyclic graph of SCCs (by viewing SCCs as nodes, we get a "condensation graph"). So if we process SCCs like we process single nodes (marking everything as succeeding or failing together), then we would end up with an acyclic graph, which we've established as easy to handle.

The end effect is that, in the case above, when `b.js` finishes evaluating, it is not marked as evaluated yet—it waits until `a.js` finishes.



Avoiding cyclic imports

You should usually avoid cyclic imports in your project, because they make your code more error-prone. Some common cycle-elimination techniques are:

- Merge the two modules into one.
- Move the shared code into a third module.
- Move some code from one module to the other.

However, cyclic imports can also occur if the libraries depend on each other, which is harder to fix.

Top-level await and asynchronous evaluation

Modules can use `await` at the top level, outside any function. This allows a module to finish asynchronous initialization before modules that depend on it execute. Previously, our evaluation was synchronous; now, we must make it asynchronous too, which has lots of implications about side-effect ordering and errors.

For example, suppose we change `config.js` to retrieve its configuration from a server, which returns the same object as in our original example:

```js
// -- config.js --
const response = await fetch("https://example.com/config.json");
if (!response.ok) {
  throw new Error(`Failed to fetch configuration: ${response.status}`);
}
export default await response.json();
```

The other modules don't need to change: the module system automatically waits for `config.js` to finish before executing their bodies and executing their dependents like `main.js`.

Previously, a module was either evaluated, unevaluated, or in an SCC and waiting for other modules to evaluate. Now, a module can also be *in progress*. This status covers both modules whose own asynchronous execution is in progress and modules waiting for asynchronous dependencies. A module can therefore have this status without containing an `await` expression.

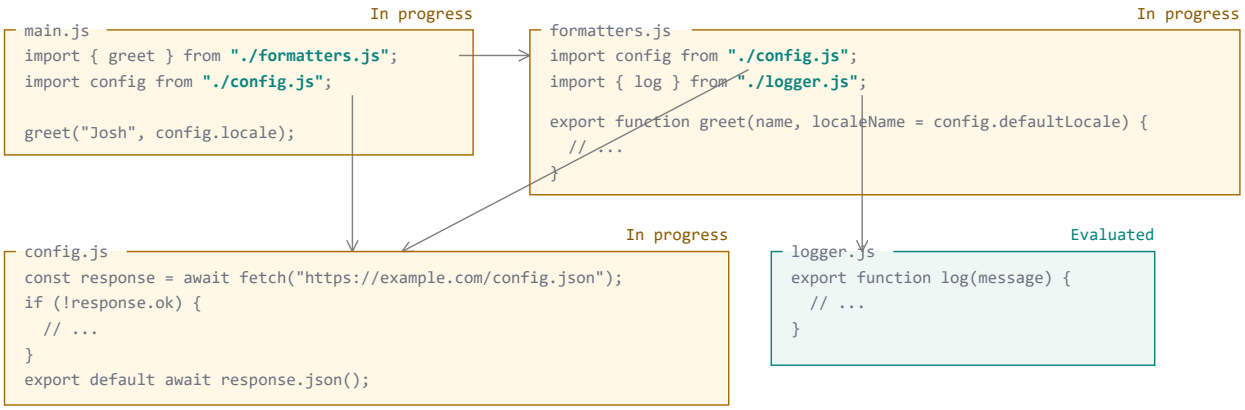
The engine still traverses dependencies depth-first, in the order their module requests appear in the source. However, it no longer needs to finish executing one branch before visiting another. For an acyclic graph, the process is:

- The engine visits a module's dependencies, as before. If a dependency is still in progress, the engine records that this module must wait for it, but continues visiting the other dependencies.
- Once all dependencies have been visited and none is in progress, the engine starts the module's body. A module with top-level `await` executes until it reaches an `await`, then suspends until the awaited value settles. It remains in progress until its entire body finishes, including any further `await` expressions.
- A module whose dependencies are still in progress is also in progress and waits without executing any of its body, even the code before its first `await`. This also applies to modules that have no top-level `await` themselves.
- When a module finishes successfully, the engine checks which waiting importers can now execute. An importer starts only after all the dependencies it was waiting for have finished successfully. If

several modules become ready together, the engine uses the evaluation order established by the depth-first traversal to decide which to start first.

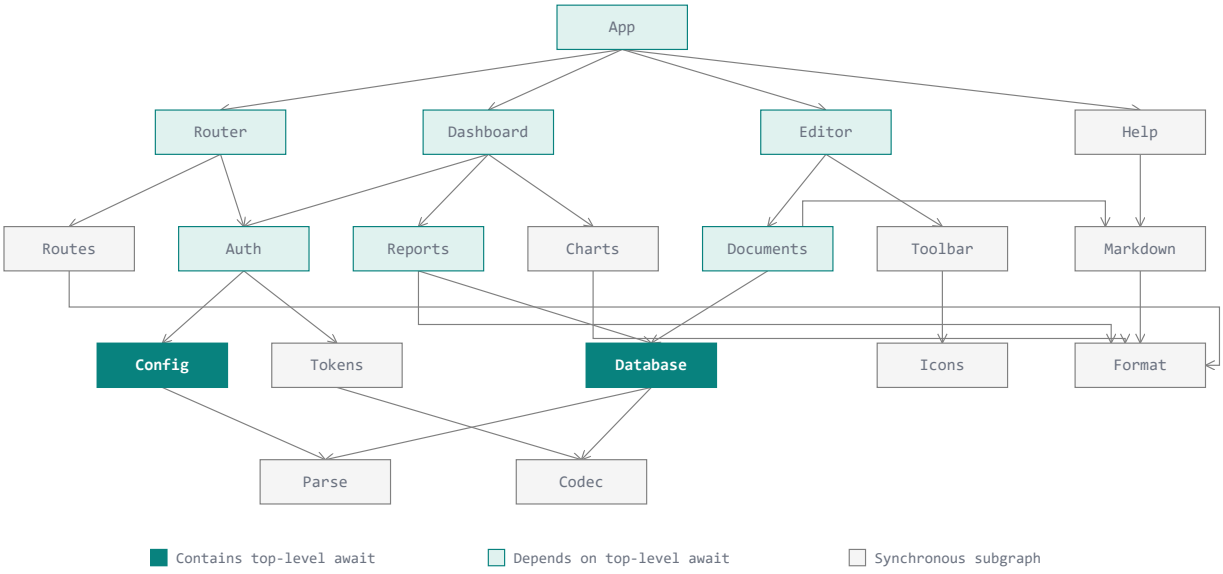
> **Note:** When multiple asynchronous modules are simultaneously executing (for example if `logger.js` also contains a top-level `await`), the behavior is the same as executing two async functions concurrently: each `await` yields out the current job and allows the other module to resume. See control flow effects of `await`.

In our example, the engine reaches `config.js` through `main.js` and `formatters.js`. It starts the fetch and suspends `config.js` at the first `await`. It then visits and evaluates `logger.js`, which doesn't depend on the configuration. `formatters.js` must wait for `config.js`, and `main.js` must wait for both `formatters.js` and `config.js`. After the response body has been parsed and `config.js` finishes, `formatters.js` executes, followed by `main.js`, which calls `greet()`.



Tainting effects of top-level await

Top-level `await` therefore affects more than the module that contains it: it makes the evaluation of every module that transitively imports it asynchronous too. In the following module graph, only `Config` and `Database` contain top-level `await`, but `Auth`, `Reports`, `Documents`, `Router`, `Dashboard`, `Editor`, and `App` all depend on their asynchronous completion. The other branches can still evaluate synchronously.



This matters when an API needs evaluation to finish synchronously. In Node.js, `require()` can load an ES module only if its entire dependency graph is synchronous. A module with top-level `await` "taints"

every module above it (in other words, everything that imports it), so they cannot be synchronously imported with `require()`. In the graph above, `require()` can load `Help`, but it cannot load `Router`, even though `Router` itself contains no `await`.

The `import defer` declaration (see Controlling import phases) is another example: the deferred evaluation is triggered by accessing a property on the namespace object, which must be able to finish synchronously, so it cannot entirely defer the evaluation of an asynchronous subgraph. However, in this case, it wouldn't just give up on deferral, or throw an error like `require()` does. Rather, it eagerly evaluates just those parts that prevent the graph from becoming synchronous—namely, the modules that use top-level await plus all their dependencies. For example, if another module uses `import defer` to import `App` in the graph above, then `Config` and `Database`, together with their dependencies `Parse` and `Codec`, evaluate before that importer's body runs. The bodies of `App` and its remaining dependencies can stay deferred. Essentially, the graph's dependencies on these asynchronous modules are replaced with results that are synchronously available, removing the taint and allowing the deferred evaluation to synchronously proceed later.

Adding top-level `await` to an existing module can be a breaking change for its importers, even if its exports remain unchanged. It delays their entire bodies, including statements written before their `import` declarations, while independent modules can continue executing. This can alter the execution order of sibling modules.

For example, these modules rely on the order of sibling imports to initialize a global variable before reading it:

```js
// -- entry.js --
import "./setup.js";
import "./display.js";
```

```js
// -- setup.js --
import { locale } from "./settings.js";

globalThis.appLocale = locale;
```

```js
// -- settings.js --
export const locale = "en-US";
```

```js
// -- display.js --
console.log(globalThis.appLocale); // "en-US"
```

Now suppose `settings.js` introduces top-level `await`:

```js
// -- settings.js --
export const locale = await Promise.resolve("en-US");
```

`setup.js` now waits for `settings.js` before executing the assignment to `globalThis.appLocale`. Meanwhile, `display.js` executes and logs `undefined`, assuming `appLocale` was not already defined. The order of imports in `entry.js` does not make `display.js` wait for `setup.js` to finish.

To preserve the required ordering, make `display.js` explicitly depend on `setup.js`. This is safe because a module is only evaluated once per application.

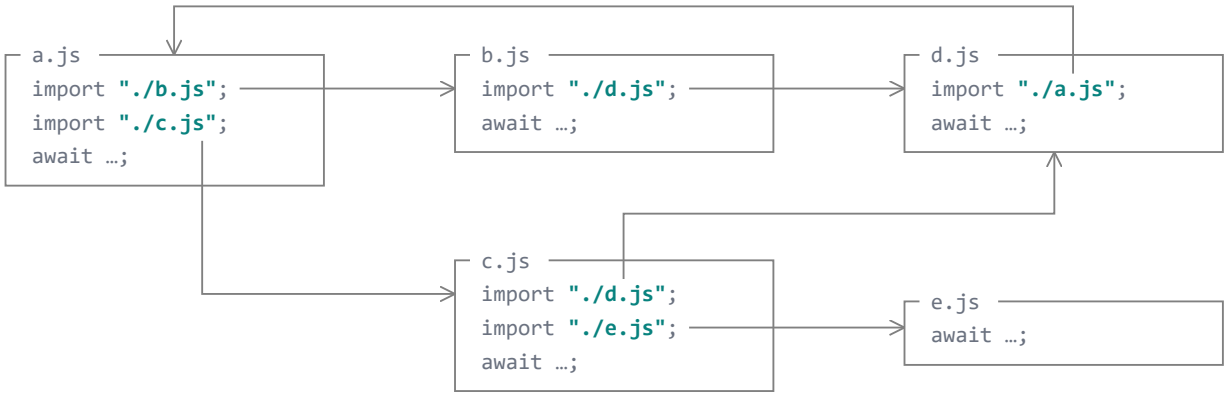
```js
// -- display.js --
import "./setup.js";

console.log(globalThis.appLocale); // "en-US"
```

The same issue can occur if `setup.js` and `display.js` are entry points of separate `<script type="module">` elements in that order. The explicit import ensures that `display.js` waits for setup in that case too.

Handling cycles with top-level await

Cycles require an additional distinction between the static dependency graph and the dependencies that evaluation actually waits for. Consider the asynchronous example in the specification, with `a.js` as the entry point and all five modules containing top-level `await`:



The modules `a.js`, `b.js`, `c.js`, and `d.js` form one SCC. The engine first visits `a.js`, `b.js`, and `d.js`. When it encounters `d.js`'s import of `a.js`, it breaks out of the cycle as before. At this point, `a.js` has not yet been recorded as awaiting asynchronous completion, so this edge does not make `d.js` wait for `a.js`. `d.js` starts executing. The engine then visits the other branch through `c.js`, finds that `d.js` is pending, and starts `e.js`.

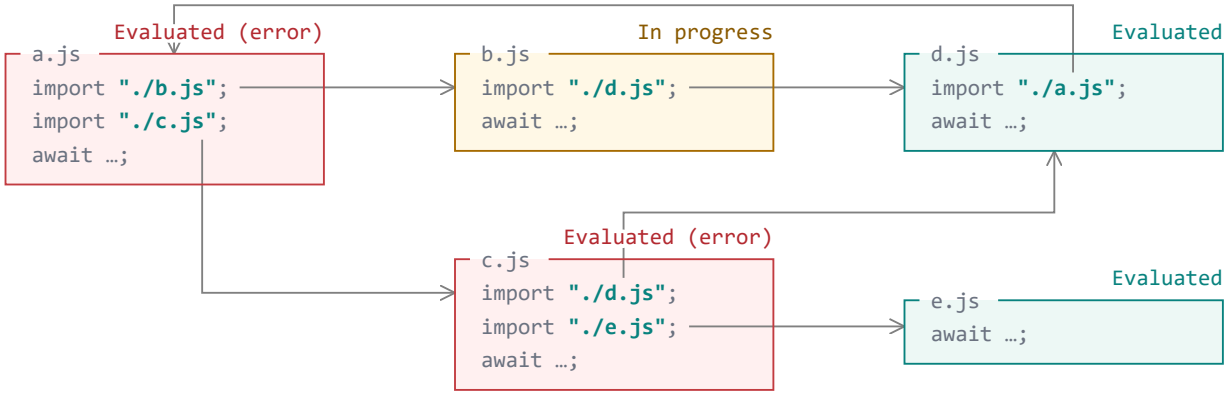
Suppose the asynchronous operations finish in the following order:

- `e.js` finishes. `c.js` still waits for `d.js`, so no other module starts yet.
- `d.js` finishes. Both `b.js` and `c.js` are now ready. `b.js` starts first, following the traversal's evaluation order, but when it suspends at `await`, `c.js` can start too.
- `c.js` finishes. `a.js` still waits for `b.js`.
- `b.js` finishes. `a.js` can now start executing its body.
- `a.js` finishes. The entry point's evaluation promise fulfills.

Unlike the synchronous cycles discussed earlier, the modules in an asynchronous SCC do not necessarily transition to *evaluated* together. Each can finish at a different time. To preserve the component's overall outcome, the engine associates its modules with a *cycle root*: the first module visited in that SCC, `a.js` in our case. Imports that encounter the component after its initial traversal use this root to wait for the component's completion and check for an evaluation error. A module's body finishing successfully does not, by itself, mean that the whole component can be imported successfully.

For example, suppose `c.js` fails while `b.js` is still awaiting. The error propagates to `a.js`, whose body never starts, and the entry point's evaluation promise rejects. `b.js` keeps executing. Nevertheless, a

later dynamic import of `b.js` rejects with the error recorded on the cycle root `a.js`. The successful execution of `b.js` does not erase the component's failure.



Deadlocks

You can have deadlocks with modules containing top-level await:

```js
// -- a.js --
import { resolve } from "./b.js";

resolve();
```

```js
// -- b.js --
const { promise, resolve } = Promise.withResolvers();
export { resolve };
await promise;
```

If evaluation starts at `a.js`, it waits for `b.js` to finish before executing its body. But `b.js` awaits a promise that only the call to `resolve()` in `a.js` can fulfill. Neither evaluation can complete. The module algorithm does not automatically reject the promises to break this deadlock.

The above may look contrived. A more realistic situation is when you wait on a dynamic import of a module that depends on the current module.

```js
// -- a.js --
await import("./b.js");
```

```js
// -- b.js --
import "./a.js";
```

Similarly in this case, `b.js` waits for the dynamic import in `a.js` to complete, but the dynamic import can only complete when `b.js` finishes executing.

Dynamic imports

Everything introduced so far uses static `import` with the entry point being the entry point to the whole application (the `node` command line, the HTML file, etc.), so the loading and linking happens before any user code is run. A dynamic import, `import()`, instead starts an import when user code execution

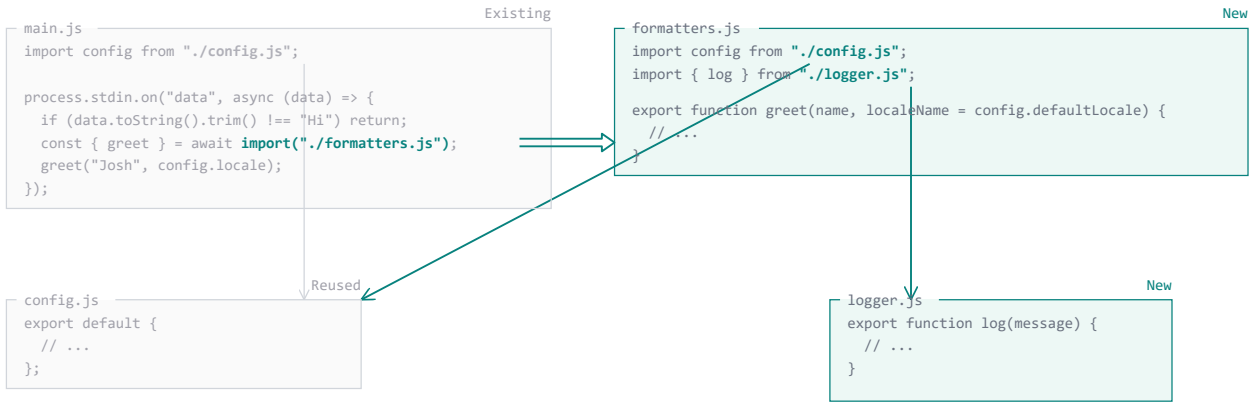
reaches the expression, with the requested module as the entry point. This allows you to dynamically *augment* the module graph after the initial graph is already up and running (or—with top-level `await`—*while* the initial graph is evaluating).

For example, we can change the original `main.js` to listen for data from Node.js's `process.stdin`, and import `formatters.js` only when the user types `Hi` and presses Enter:

```js
// -- main.js --
import config from "./config.js";

process.stdin.on("data", async (data) => {
  if (data.toString().trim() !== "Hi") return;
  const { greet } = await import("./formatters.js");
  greet("Josh", config.locale);
});
```

Initially, loading `main.js` discovers `config.js`. Registering the listener completes `main.js`'s evaluation without loading `formatters.js`. When the listener receives `Hi`, another loading process starts from `formatters.js` and discovers its imports of `config.js` and `logger.js`. In this case, `config.js` is already available and is reused, while `logger.js` needs to be loaded too, assuming nothing else has requested it. The new branch therefore joins the existing graph at `config.js`, reusing the same module.



Augmentation of the module graph can encounter existing modules in any state, and that state is preserved—loading modules keep loading, loaded modules aren't re-fetched, linked modules aren't re-linked, evaluated modules are never re-evaluated (including if the previous evaluation threw an error). In the example above, `config.js` has already evaluated, so only `logger.js` and `formatters.js` need to execute. The dynamically imported module itself may also already exist in the module graph and get reused.

Dynamic import is async, because the loading may need to fetch new modules and evaluation may need to wait for top-level `await`. The `import()` promise resolves when the requested module's evaluation finishes successfully.

The new traversal can also reach the module that called `import()` (`main.js`, for example). Because dynamic imports do not constitute dependency edges (they are only entry points), this does not create a cycle. However, if the caller uses top-level `await` to wait for that import, neither can finish, as shown in the deadlock example.

Module caching

Module caching means reusing a module's identity and state. Repeated successful requests from the same importer, with the same specifier and import attributes, must refer to the same module. Different requests can also refer to the same module, such as when two modules import the same dependency. The host determines how requests map to module identities; see Module caching on the web for browser-specific rules.

This means importers share the module's bindings, including changes made after evaluation. For example, this `counter.js` module keeps a count:

```js
export let count = 0;

export function increment() {
  count++;
}
```

Another module can access the same counter through both static and dynamic imports. The namespace object is shared too.

```js
import * as counter from "./counter.js";

counter.increment();

const sameCounter = await import("./counter.js");
console.log(sameCounter === counter); // true
console.log(sameCounter.count); // 1

sameCounter.increment();
console.log(counter.count); // 2
```

JavaScript provides no standard API for clearing the module cache. If callers need independent state, the module can export a function that creates it:

```js
export function createCounter() {
  let count = 0;
  return {
    increment() {
      return ++count;
    },
  };
}
```

Alternatively, you can add an arbitrary query or fragment to your URL specifier (see `import()`).

Controlling import phases

Ordinary `import` declarations trigger the whole load-link-evaluate process. *Import phase modifiers* let the importer pause the process at a certain stage. The later phases can be triggered at some point during execution only when actually needed.

With `import source`, the process is stopped partway through *loading*: the requested module itself is fetched and parsed, but its dependencies are not requested. The imported value is a *module source*

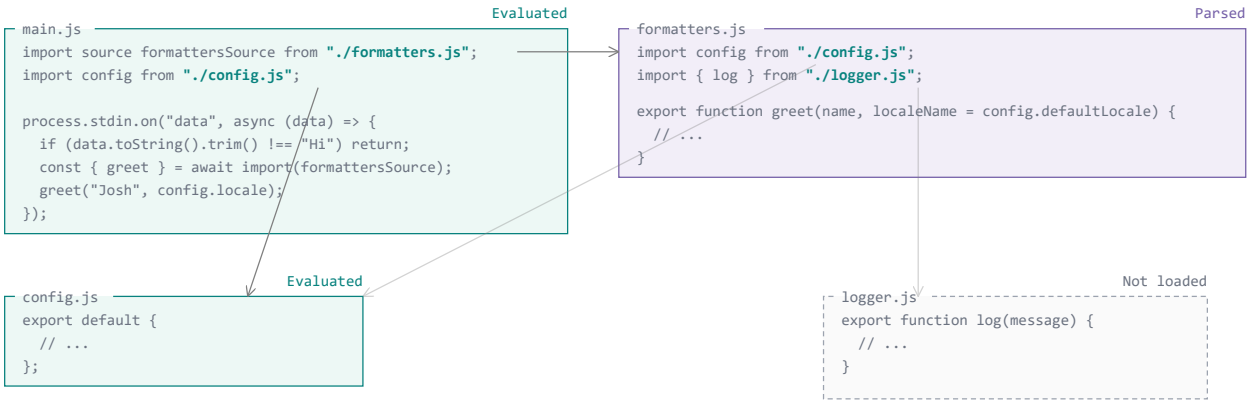
object, which can be linked and evaluated using other mechanisms. The following example uses JavaScript source imports:

> **Note:** JavaScript source imports and passing a module source object to `import()` are still proposed features, defined by the ECMAScript Module Phase Imports proposal.

```js
// -- main.js --
import source formattersSource from "./formatters.js";
import config from "./config.js";

process.stdin.on("data", async (data) => {
  if (data.toString().trim() !== "Hi") return;
  const { greet } = await import(formattersSource);
  greet("Josh", config.locale);
});
```

After `main.js` evaluates, `formatters.js` has been parsed, but its dependencies have not been traversed through the source import. `logger.js` has not been requested. `config.js` has evaluated because `main.js` imports it normally:



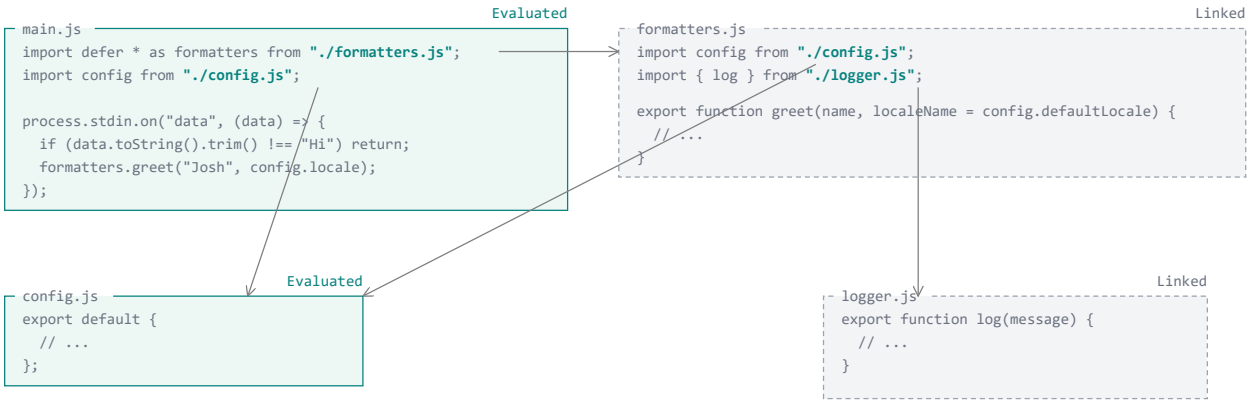
When the listener receives `Hi`, it passes the module source object to `import()`. This starts loading `formatters.js`'s dependencies, reusing `config.js` and loading `logger.js`, then links and evaluates the remaining modules. The promise fulfills with the namespace after evaluation succeeds. Repeated `Hi` inputs reuse the same module instance, without causing another evaluation.

With `import defer`, the process is stopped before *evaluation* (but asynchronous evaluation, if any, remains eager). The imported value is a *deferred module namespace object*. For example:

```js
// -- main.js --
import defer * as formatters from "./formatters.js";
import config from "./config.js";

process.stdin.on("data", (data) => {
  if (data.toString().trim() !== "Hi") return;
  formatters.greet("Josh", config.locale);
});
```

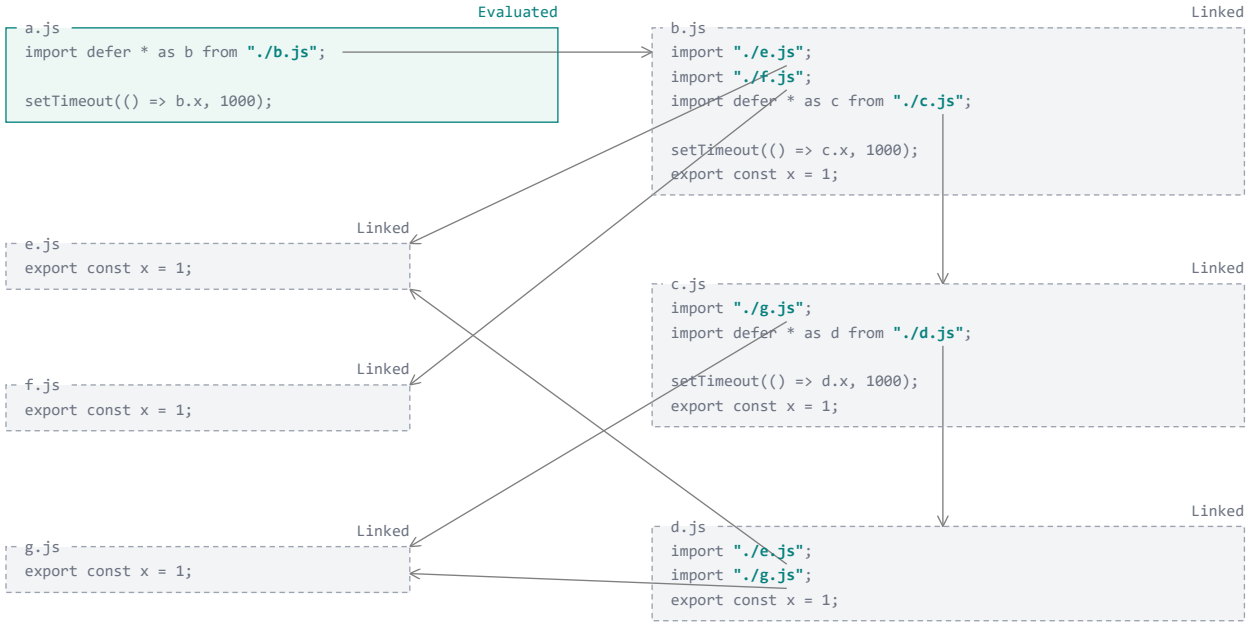
Assuming the original synchronous modules and no other imports, `main.js` can finish evaluating with `formatters.js` and `logger.js` still linked but unevaluated. `config.js` evaluates because `main.js` also imports it normally:



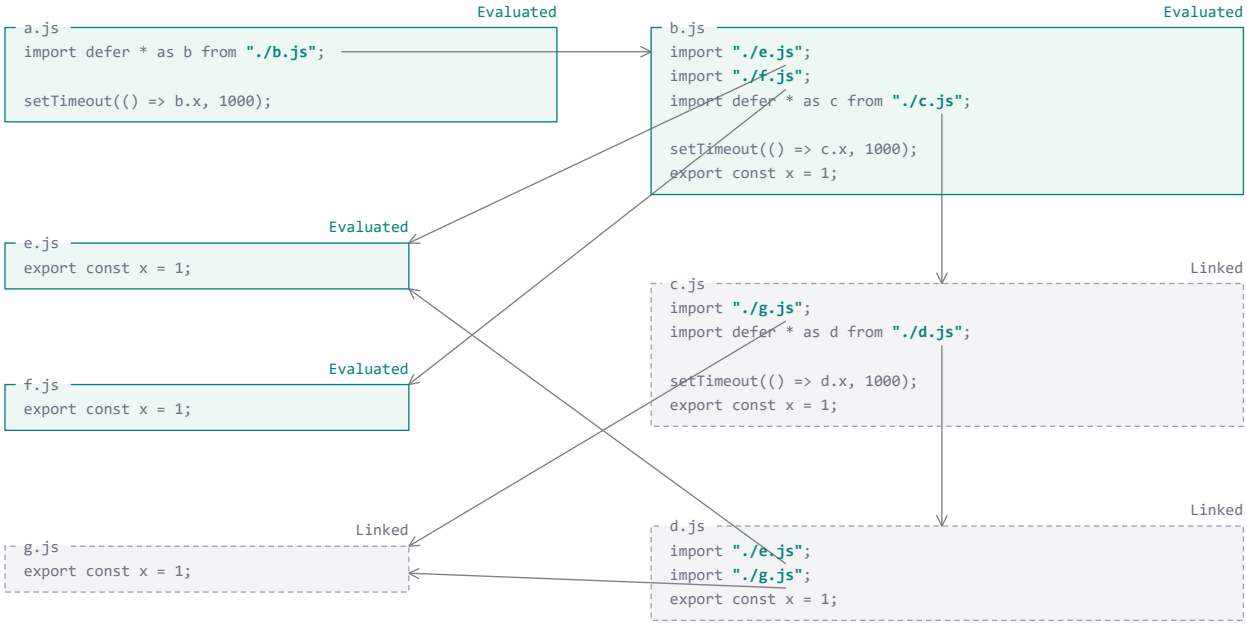
When the listener receives `Hi`, reading `formatters.greet` triggers synchronous evaluation of `logger.js` and then `formatters.js`, reusing the already evaluated `config.js`. If evaluation throws, the property access throws. Loading and linking errors, however, have already been checked before `main.js` executes.

Deferred imports can also be nested, creating a pausing boundary at each `defer`. In the following example, `a.js` defers `b.js`, which defers `c.js`, which in turn defers `d.js`. The other imports are ordinary imports. All modules are synchronous. Each timer reads a property of a deferred namespace, triggering the next stage of evaluation. Each timer has a one-second delay measured from when it is scheduled, so the stages occur roughly 1, 2, and 3 seconds after startup.

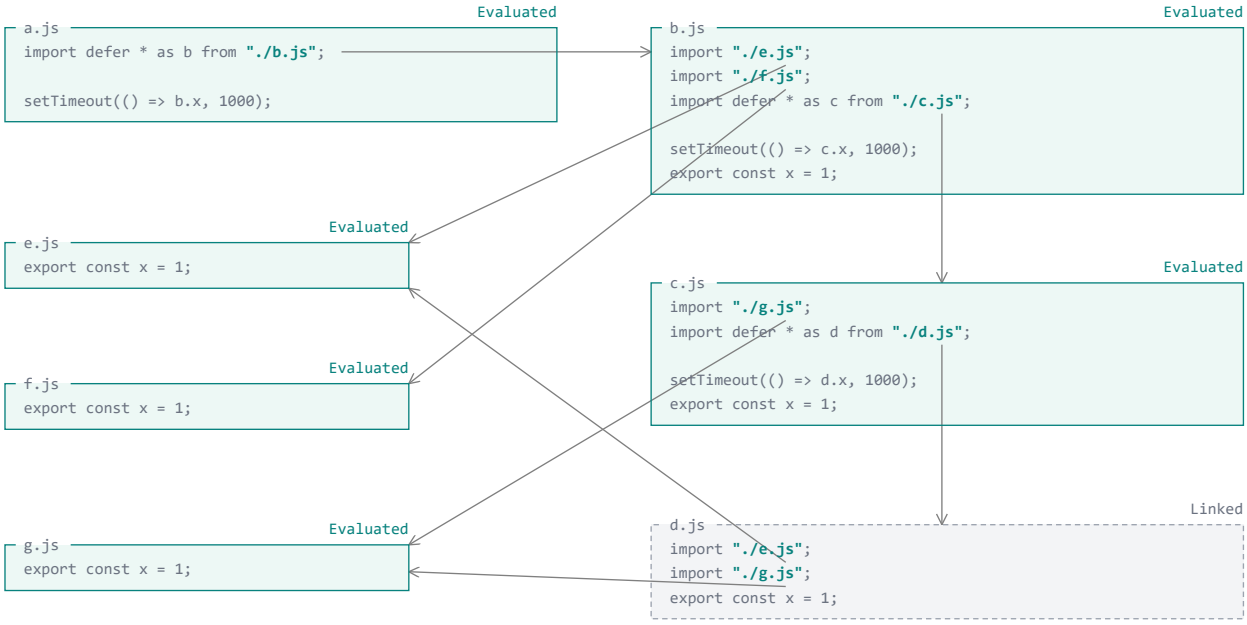
At startup, all seven modules are loaded and linked, but only `a.js` evaluates. It schedules a timer without accessing `b`'s properties:



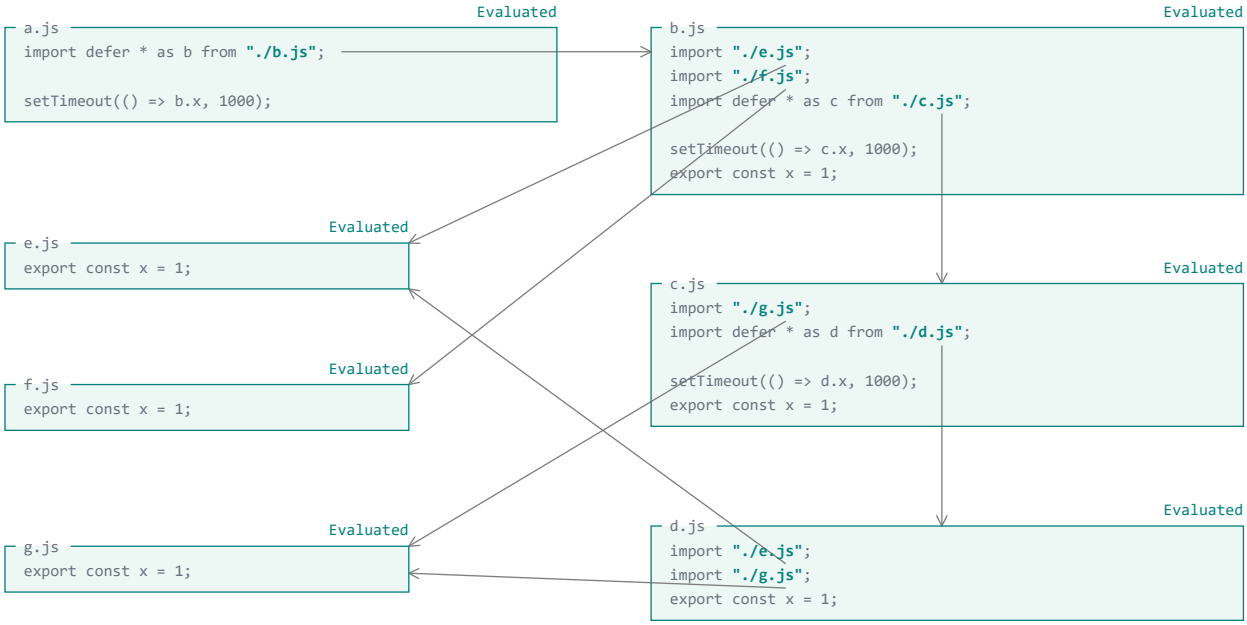
When the first timer runs, reading `b.x` evaluates `e.js`, `f.js`, and then `b.js`. Evaluating `b.js` schedules another timer. Its deferred import of `c.js` does not evaluate it, so `c.js`, `d.js`, and `g.js` remain linked:



When the second timer runs, reading `c.x` evaluates `g.js` and then `c.js`. Evaluating `c.js` schedules the third timer, without accessing `d`'s properties yet:



When the third timer runs, reading `d.x` evaluates `d.js`, reusing the already evaluated `e.js` and `g.js`. All modules have now evaluated:



As explained in Tainting effects of top-level await, modules containing top-level `await` and their dependencies must execute eagerly, immediately following the initial linking. Only the remaining synchronous execution can stay deferred.

For both kinds of phase modifiers, note that the modifier acts on the *import*, not on the *target module*. This means another ordinary import can cause the same module to evaluate earlier, and no separate cache entries are created for deferred and eager imports of the same module. For more information, see the caching semantics of `import defer` and `import source`.

Just like `import` is to `import()`, `import source` and `import defer` declarations also have dynamic `import.source()` and `import.defer()` forms, which start their requests when the expressions evaluate.

See also

- JavaScript modules
- Using modules on the web
- Modules across platforms

Modules across platforms

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules/Modules_across_platforms · MDN page last modified 2026-11-09

MDN focuses on JavaScript in the browser, but most developers will also work with other runtimes such as Node, Deno, or Bun. While it's easy to assume that the rules for the web apply everywhere, each of these environments (and many tools, such as bundlers and type checkers) has its own rules for resolving and interpreting modules. While we won't try to document every detail of non-browser environments, this section offers a mental model for thinking about modules across platforms, and practical guidance for writing portable module code.

Other module formats

ECMAScript modules (ESM) were standardized and implemented almost 20 years after the birth of JavaScript. By that time most applications were already complicated enough to need module systems, so various alternatives have been implemented by runtimes and bundlers.

In essence, every module system needs three parts:

- The module code, with its own scope to encapsulate variables, such as a wrapper function.
- Some protocol for letting the module declare its exports.
- An importing mechanism to identify another module and access its exports.

CommonJS

Node.js popularized the CommonJS system. In Node.js's implementation of CommonJS, each module is implicitly wrapped in a function. This function passes five things into the module scope: `exports`, `module`, `require`, `__dirname`, and `__filename`. Each module's exported value is a single JavaScript value contained in `module.exports`. You can obtain this value by calling `require()` with the module's ID.

```js
// -- rectangle.cjs --
module.exports = {
  area: (a, b) => a * b,
};
```

```js
// -- square.cjs --
const rectangle = require("./rectangle.cjs");
// `rectangle` contains the `module.exports` object of rectangle.cjs

module.exports = {
  area: (side) => rectangle.area(side, side),
};
```

Unlike ESM which leaves specifier resolution entirely host-defined, CommonJS defines some semantics for the module ID. It defines that relative IDs, starting with `.` or `..`, should be resolved relative to the current module, while top-level IDs starting with neither should be resolved relative to some "module name space root". Node.js picks up where the CommonJS spec left off and defines a formal resolution algorithm, which is used as the basis for resolution by many tools that recognize CJS, including bundlers and TypeScript. Most notably, the resolution logic of Node.js defines the `node_modules` folder and how to import paths pointing to directories (it uses the directory's `package.json` or `index.js`).

AMD

CommonJS works well in Node.js, but it's synchronous, which is detrimental for browsers because loading dependencies may involve network requests. Asynchronous Module Definition (AMD) is more popular in browsers because its imports are asynchronous. In AMD, each module calls `define()` with an array of dependency IDs and a factory function. The loader resolves the dependencies before calling the factory, passing their exported values as arguments in the same order. The factory can return a value to export from the module.

```js
// -- rectangle.js --
define([], function () {
  return {
    area: (a, b) => a * b,
  };
});
```

```js
// -- square.js --
define(["./rectangle"], function (rectangle) {
  // `rectangle` contains the value returned by rectangle.js's factory
  return {
    area: (side) => rectangle.area(side, side),
  };
});
```

The `define()` function is provided by an AMD loader, such as RequireJS, rather than by the runtime (browsers don't natively understand AMD). AMD supports relative and top-level module IDs, similar to CommonJS. How these IDs map to files depends on the loader and its configuration. For example, RequireJS normally resolves top-level IDs against a configured `baseUrl` and adds the `.js` extension, so module IDs usually omit it.

UMD

Before ESM, there was already the desire to make modules cross-platform! Universal Module Definition (UMD) is a set of wrapper patterns that allow the same file to work with different module systems. A typical UMD wrapper checks for an AMD loader, then for a CommonJS environment, and otherwise exposes the module as a global variable. Each branch uses the same factory function to create the module's exported value.

```js
// -- rectangle.js --
(function (root, factory) {
  if (typeof define === "function" && define.amd) {
    define([], factory);
  } else if (typeof module === "object" && module.exports) {
    module.exports = factory();
  } else {
    root.rectangle = factory();
  }
})(globalThis, function () {
  return {
    area: (a, b) => a * b,
  };
});
```

```js
// -- square.js --
(function (root, factory) {
  if (typeof define === "function" && define.amd) {
    define(["./rectangle"], factory);
  } else if (typeof module === "object" && module.exports) {
    module.exports = factory(require("./rectangle.js"));
  } else {
    root.square = factory(root.rectangle);
  }
})(globalThis, function (rectangle) {
  return {
    area: (side) => rectangle.area(side, side),
  };
});
```

Now this same file can be imported in both Node.js running CommonJS and browsers running AMD, without any modification to the source code. In the global-variable branch, you must load the dependencies yourself, in the correct order, just like good old global scripts.

UMD does not define its own dependency resolution rules; it's handled by the AMD/CommonJS branches. People rarely hand-write UMD; it is usually generated by bundlers and transpilers.

Using modules in Node.js

Node.js supports both ESM and CommonJS modules. When it attempts to execute a file, it must first decide if the file is CJS or ESM. For backwards compatibility, without any contextual cues, Node.js defaults to CJS (unless the file contains ESM-specific syntax like `import`/`export`, in which case it switches to ESM). To explicitly mark a file as ESM, use the `.mjs` extension, or use `.js` with `"type": "module"` in the nearest parent `package.json` file:

```json
{
  "type": "module"
}
```

To explicitly mark a file as CJS, use the `.cjs` extension, or use `"type": "commonjs"` in the nearest parent `package.json` file.

For example, create these two files in the same directory as `package.json` (which contains `"type": "module"`):

```js
// -- square.js --
export const area = (side) => side ** 2;
```

```js
// -- main.js --
import { area } from "./square.js";

console.log(area(4)); // 16
```

Run the entry module with `node main.js`. Node.js loads these files directly from the file system; no HTML page or HTTP server is needed.

Node ESM does not expose the CJS module-scoped variables: `__dirname`, `__filename`, `exports`, `module`, `require`. While you probably don't need `exports` and `module` because you have the `export` declarations, you might find `__dirname` and `__filename` useful for getting the path of the current module and `require` for synchronously loading a dependency when not at the top level. `import.meta` largely replaces `__dirname` and `__filename`, with Node-specific properties like `import.meta.dirname` and `import.meta.filename`. If you want to synchronously load dependencies, Node also provides the `createRequire` function to restore the `require` function within this module's context.

Node has different resolution logic for ESM and CJS module specifiers. Its ESM implementation is more consistent with that of browsers, which means you must keep the file extensions like `"./square.js"` instead of `"./square"`. To be compatible with the existing ecosystem, Node.js still resolves bare names to installed packages in `node_modules`. It provides built-in modules through the `node:` protocol.

```js
import { readFile } from "node:fs/promises"; // Import from built-in module
import _ from "lodash"; // Import from package in node_modules

const text = await readFile(new URL("./data.txt", import.meta.url), "utf8");
const wordCounts = _.countBy(_.words(text));
```

Whether the ESM or CJS specifier resolution rules are used depends on the importing method, not the target module. You use CJS module specifiers with `require` (including ones created by `createRequire` in ES modules) and ESM ones with `import` (including dynamic imports in CJS modules).

Node specifier resolution offers some more features, like import aliases using `"imports"` in `package.json`. See Node.js's ES module documentation for its loading rules and APIs.

ESM/CJS interop

In Node.js, an ES module can import a CJS module, which loads the `module.exports` value as the default export:

```js
// -- main.mjs --
import square from "./square.cjs";

console.log(square.area(4)); // 16
```

Conversely, CommonJS code can use asynchronous `import()` to load ES modules. Node.js also supports `require()` of synchronous ES module graphs:

```js
const square = require("./square.mjs");
// Assuming square.mjs and its dependencies do not use top-level await

console.log(square.area(4));

import("./module-with-top-level-await.mjs").then((mod) => {
  mod.foo();
});
```

The only restriction is that `require()` cannot load a graph that uses top-level `await`:

js · ✗ bad example

```
const mod = require("./module-with-top-level-await.mjs");
// Error [ERR_REQUIRE_ASYNC_MODULE]: require() cannot be used on an ESM graph with top-level
await. Use import() instead.
```

Many CJS packages are expected to export individual names, like this:

js

```
const { area } = require("./square.cjs");
```

This does *not* always translate to named imports in ESM:

js

```
import { area } from "./square.cjs";
```

This is because modules must be linked before evaluated, which means Node must be able to tell if an export called `area` exists before it runs `square.cjs`. Therefore, it can only look for certain code patterns, like `module.exports = { area }` or `module.exports.area = ...`, but it will not 100% succeed, especially if CJS code was hand-written and modifies `module.exports` in a convoluted way. Most common compiler-produced CJS idioms can be correctly analyzed.

In the worst case, you may need to import the whole `module.exports` object, and then destructure it:

js

```
import square from "./square.cjs";

const { area } = square;
```

Note that CJS has no concept of a "default export"; when you default-import a CJS module, you are in fact loading the whole `module.exports` object, within which there may be conceptual "named exports" as properties. Compilers would transpile `export default` declarations to CJS code that's essentially `module.exports.default = ...`, and this cannot be directly imported in an ES module (`import { default as area }` is the same as `import area`, which loads the whole `module.exports` object).

js

```
// -- square.js --
// This file will be compiled into CJS
export default function area(side) {
  return side ** 2;
}
// Compiled code:
// module.exports.default = function area(side) ...
```

js

```
// -- main.mjs --
import square from "./square.js";

// Can't avoid this!
const { default: area } = square;
```

For this reason, if you write ESM syntax that will get compiled to CJS and then imported by native ES modules, you should avoid default exports or provide a named export alias.

js

```
// -- square.js --
// Export the exact same value as the default export
export { area };
```

Packages

When publishing a package, its `"exports"` field defines the entry points consumers can import. Conditional exports can select different implementations for different environments or module systems. These package settings are separate from JavaScript `export` declarations, which determine the bindings each module exposes. The `"main"` field is now unnecessary and can be replaced by `"exports"`.

For a long time, Node did not allow CJS consumers to import ESM packages without using asynchronous `import()` (allowing `require()` to load ESM came much later). Therefore, packages that wish to publish in native ESM would need to publish a separate CJS copy for consumers still running CJS:

json

```
{
  "name": "dual-package",
  "type": "module",
  "exports": {
    ".": {
      "import": "./dist/esm/index.js",
      "require": "./dist/cjs/index.cjs",
      "default": "./dist/esm/index.js"
    }
  }
}
```

This practice, called dual packaging, is unnecessary in current Node.js as long as the package's dependency graph doesn't use top-level `await`. If different parts of your application loaded the same package in CJS and ESM forms, they may not share the same module state, creating subtle bugs.

Transpilers

Earlier sections mentioned transpilers in passing. They convert certain JavaScript syntaxes to other JavaScript syntaxes. Converting between module formats is, of course, a part of their job. Common transpilers, like Babel, SWC, and tsc, are all capable of taking modules written in native ESM syntax and emitting one of the aforementioned module formats.

For example, take the following source code:

js

```
// -- src/square.js --
import * as rectangle from "./rectangle";

export function area(side) {
  return rectangle.area(side, side);
}
```

And run a transpiler to produce CJS. It may output the following:

js

```
// -- dist/square.js --
"use strict";
Object.defineProperty(exports, "__esModule", {
  value: true,
});
Object.defineProperty(exports, "area", {
  enumerable: true,
  get: function () {
    return area;
```

(code continues on the next page)

(code continued from the previous page)

```
  },
});
const _rectangle = /*#__PURE__*/ _interop_require_wildcard(
  require("./rectangle"),
);
// ... definition for _interop_require_wildcard ...
function area(side) {
  return _rectangle.area(side, side);
}
```

Therefore, while you may feel like you are authoring ESM because you are writing `import` and `export` declarations, unless you configured your transpiler to emit native ESM, you may in fact still be writing CJS code (or some other module format). Transpilers typically work on single files only, preserving the module specifiers as-is. This is why code like `import * as rectangle from "./rectangle"` may still work despite the lack of extensions; it eventually gets compiled to `require("./rectangle")`, which is valid CJS.

The goal of transpilers is that you feel like you are writing native ESM as much as possible if both the target module and the importer module are transpiled. Specifically, most transpilers do not always convert `import rectangle from "./rectangle";` to load the whole `module.exports` object. In the output code above, notice the `__esModule` marker. The marker indicates that this CJS module was produced by transpiling ESM syntax. Most transpilers compile default-imports to read `module.exports.default` (i.e., the value exported by a transpiled `export default` declaration) if this marker is present, and `module.exports` otherwise.

The downside is that if only one side is transpiled while the other side is run in Node's native ESM, you may suddenly see rough edges (as mentioned in ESM/CJS interop). For example, a transpiled `export default` can no longer be imported using default-imports in native ESM.

You need to read the documentation for your transpiler or conduct experiments to understand specific rules about how different kinds of `import` and `export` declarations get transpiled.

Bundlers

While bundlers usually come with transpilers that convert module formats, they go one step further and actually try to understand the dependency graph in order to transform and optimize it.

Starting from one or more entry modules, a bundler follows their dependencies and combines code into output files. The most primitive bundler only outputs a single JavaScript file (the "bundle"), but multiple files may also be emitted, especially when dynamic `import()` is used. Even if you intend to run ESM directly, bundlers are still very useful, because they perform important transformations and optimizations. For example, Vite lets applications written in ESM use CommonJS and UMD dependencies through dependency pre-bundling. During development, Vite serves transformed modules using native ESM; its production build bundles the application for deployment. Bundlers are even useful outside of browsers: even if you are developing a Node library or application, you may still use a bundler to optimize your code size and/or use non-standard syntax.

Bundlers are not JavaScript hosts because they do not *execute* code. They only statically analyze files. Nevertheless, they have entire sets of rules for dealing with modules that are comparable to browsers or Node.

Module resolution

Unlike a transpiler that preserves module specifiers, a bundler needs to locate the dependencies it includes.

Most packages for frontend are also managed using Node package managers in the `node_modules` folder, so bundlers understand Node-style package imports by default. They also tend to support both ESM-style and CJS-style resolution simultaneously (i.e., directory imports and extension-less imports), although they may choose to adhere more to one of them if, for example, the module has an explicit `.mjs`/`.cjs` extension.

Bundlers often offer configurable resolution rules. For example, Vite's `resolve.alias` option lets you configure an alias:

```js
// -- vite.config.js --
import { fileURLToPath } from "node:url";
import { defineConfig } from "vite";

export default defineConfig({
  resolve: {
    alias: {
      "@shapes": fileURLToPath(new URL("./src/shapes", import.meta.url)),
    },
  },
});
```

Your source code can then use this alias:

```js
import { area } from "@shapes/square";
```

With this configuration, Vite can resolve the import to `src/shapes/square.js`. The browser doesn't need to understand this alias because Vite rewrites the import during development and resolves it during bundling. However, the original source cannot run directly in a browser without an import map providing an equivalent mapping.

Optimizing the module graph

Bundlers concatenate all files in the module graph together. If you import `lodash`, you copy the entire source code of Lodash—with over 100 utility functions—into your bundle, which is very costly. Bundlers can remove code that isn't used by the application, commonly called tree shaking. ESM's static imports and exports help the bundler determine which bindings are used.

> **Note:** This is a big reason why, if you are publishing a package that's intended to be consumed by bundlers and not run directly, you should publish in ESM syntax so bundlers can understand it more easily.

For example:

```js
// -- big-library --
export function map() {}

export function filter() {}

export function reduce() {}
```

```js
// -- my-app.js --
import { map } from "big-library";
```

The bundler will be able to see that the `filter` and `reduce` exports are never used by your app, and remove them from the final bundle. It can even go one step further and see that even though you imported `map`, you never called it (so it's "dead code") and therefore avoid bundling the entire `big-library`.

> **Note:** You may wonder why one would keep this import at all if it's unused. This is because it may be conditionally used, but the condition is known at build time. For example:
>
> ```js
> if (import.meta.env.PROD) {
>   map();
> } else {
>   someOtherCode();
> }
> ```
>
> The `import.meta.env.PROD` value is statically set by the bundler during the build, so if it's `false`, then the final bundle only contains the `someOtherCode()` branch, and the `import` becomes unused.

This doesn't mean the whole module can be discarded: executing the module may have side effects, such as registering an event listener or modifying a global object. So even after dead code elimination you may end up with a side-effect import like `import "big-library";`, and all unused declarations within that module are removed. Some bundlers recognize a `"sideEffects"` field in `package.json` that identifies files whose execution must be preserved. If your package doesn't have side effects on import (including any CSS files that must be applied), you can set `"sideEffects": false`, which allows bundlers to elide the whole `import`.

Bundling doesn't necessarily produce one file. With code splitting, a bundler can put dynamically imported code into separate files, called *chunks*, that are loaded when needed. This is useful because it minimizes the size of code to load on initialization and avoids loading code that may not be used by the user. Some bundlers may automatically split code, but generally you need to explicitly trigger code splitting using dynamic imports:

```js
button.addEventListener("click", async () => {
  const { showChart } = await import("./chart.js");
  showChart();
});
```

Depending on the build configuration, `chart.js` and its dependencies may be emitted separately from the initial application code. The bundler rewrites the import to load the generated chunk:

js

```
button.addEventListener("click", async () => {
  // A unique hash is appended for caching purposes
  const { showChart } = await import("./chart-ABCDEF.js");
  showChart();
});
```

You must deploy these additional files as well as the entry file.

The bundler must be able to discover the modules to include during the build. Native `import()` accepts an arbitrary string at runtime, but if you write `import(someVariable)` and the bundler cannot statically determine what values that `someVariable` may be, it may fail to bundle necessary files. Support for variable module specifiers varies between tools. For a known set of modules, explicitly listing the imports makes the dependencies clear:

js · ✗ bad example

```
// A bundler may need to bundle everything under shapes/
const shape = await import(`./shapes/${shapeName}.js`);
```

js

```
// The bundler knows exactly which files to bundle
const loaders = {
  square: () => import("./shapes/square.js"),
  circle: () => import("./shapes/circle.js"),
};

const shape = await loaders[shapeName]();
```

Bundlers can also leave dependencies *external*, meaning their code is not included in the bundle. Vite's library mode supports this when publishing a library whose consumers should provide a dependency themselves. If the output still contains `import` or `require()` calls for these dependencies, the runtime must be able to resolve them. A bare specifier left in browser ESM output still needs an import map, even if the bundler could locate that package during the build.

Non-JavaScript resources

Bundlers can interpret imports of non-JavaScript resources using built-in features or plugins. For example, Vite supports CSS imports and static asset imports:

js

```
import "./styles.css";
import logoURL from "./logo.svg";
```

Vite inserts the CSS into the page during development and normally extracts it into a separate stylesheet during the production build. The SVG import returns an asset URL, which may point to an emitted image or be an inlined data URL. These meanings come from the build tool (and you can create custom `module specifier → module object` mapping logic using custom loaders). They are not equivalent to native non-JavaScript module imports, which use import attributes and follow the runtime's module type rules.

Check your bundler's documentation when relying on its module resolution or resource handling. An import that works through a bundler may need different configuration when loaded directly by a runtime. For code intended to run directly in browsers, see Using modules on the web.

How TypeScript understands modules

TypeScript analyzes modules to check the types of imported values and, when compiling, produce JavaScript suitable for the runtime or bundler. To do this, it needs to understand both your TypeScript source and how the resulting JavaScript will be loaded. Its `module` and `moduleResolution` compiler options describe that environment's module format and resolution rules. These settings matter even when another tool produces the JavaScript and TypeScript only checks types.

In TypeScript, a file is either a script or a module (just like JavaScript itself, but this "module" is not necessarily an "ES module"; it may be CJS, etc.). The main difference is that variable declarations in scripts are automatically visible throughout the project, while modules create their own scope. By default, any file with a top-level `import` or `export` is a module, so sometimes adding a useless `export {}` can mark the file as a module and stop its variable declarations from becoming global. The `moduleDetection` option can force all non-declaration files to be treated as modules.

The module format

Files understood to be "modules" in the TypeScript sense may still be compiled to the various formats, most notably ESM and CJS. (AMD and UMD are less relevant to developers these days because you just let the bundler "do the right thing" on the TypeScript source code directly, and `tsc` is explicitly not a bundler, so it doesn't optimize your code for web deployment.)

`tsc` is a transpiler, so it can compile ESM syntax into CJS. The TypeScript docs now recommend the `verbatimModuleSyntax` option, which makes the module syntax easier to reason about (for example, you don't need to worry about what `import x from "mod"` will be compiled into). All subsequent documentation will assume this option is turned on.

TypeScript provides two sets of module syntaxes. There's the familiar ES syntax `import` and `export` (which deal with individual named or default exports), and there's the special CJS syntax (which deals with the whole `module.exports` object).

ts

```ts
import x = require("mod");
// Compiled to: const x = require("mod");

export = x;
// Compiled to: module.exports = x;
```

With `verbatimModuleSyntax`, TypeScript reports an error if ES import/export syntax would be compiled to CJS. Under Node.js module options, the `import = require()` syntax works in both CJS and ESM (generating a `createRequire` call in the latter case), while `export =` only works in CJS.

You declare the module type with the `module` option. While there are many options (comparable to a transpiler), you probably want to use one of:

- `"nodenext"`, if you are outputting code to run on Node.js-compatible runtimes. It uses the Node.js detection algorithm, like `.mts` (the `.ts` version of `.mjs`) and `"type": "module"`, for selecting between CJS and ESM. There are also versioned values like `"node16"`, but only use them if you specifically want to avoid new behavior. TypeScript also recommends this option for outputting code to run natively on browsers, because it recommends `"moduleResolution": "nodenext"` which requires this setting.
- `"preserve"`, if you are outputting code to run in bundlers or Bun. It outputs the module format as-is, with `import = require()` and `export =` compiled to CJS.

- `"esnext"`, if you want to enforce ESM syntax without transformations. It only outputs ESM, with `import = require()` and `export =` generating errors. There are also versioned values like `"es2020"`, but only use them if you specifically want to avoid new behavior.

The `module` option also decides if you can access `import.meta`, if you can use top-level `await`, if you can import other ES modules, etc. Just setting this option probably gives you sensible defaults in typical environments for most other options.

Specifier resolution

TypeScript re-implements the specifier resolution logic of whatever environment the generated JavaScript code is intended to run in, so you need to declare which resolver to use with `moduleResolution`. Currently the only recommended values are:

- `"nodenext"`, if you are outputting code to run on Node.js-compatible runtimes. It uses the Node.js resolution algorithm; there are also versioned values like `"node16"`, but only use them if you specifically want to avoid new behavior.
- `"bundler"`, if you are outputting code to run in bundlers or Bun. It uses the resolution algorithm that most bundlers can agree on, which is essentially just node resolution without requiring explicit extensions.

For code intended to run directly in browsers, TypeScript has no dedicated browser resolver. Its recommended approximation uses `"nodenext"` for `moduleResolution`, which simultaneously requires `module: "nodenext"` too. But both `"nodenext"` and `"bundler"` are supersets of what the browser is capable of, so you must be careful not to use Node-specific things that browsers don't natively understand, like package imports or importing CommonJS.

TypeScript does not read your bundler configuration or import maps to reproduce custom specifier resolution rules you may have. You may use the `paths` option; for the Vite alias shown above, make sure TypeScript and Vite both resolve `@shapes/square` to the same module. It also has other configuration options, all with the goal of making sure TypeScript can reproduce how specifiers are resolved at runtime.

Like transpilers, by default the specifiers are taken at face value without any rewriting—path aliases using `paths` are not rewritten (so only use this option if your path aliases are natively understood by the downstream consumer), and you still use `.js` to refer to a `.ts` file because that's the actual file you import at runtime.

ts

```ts
// -- src/main.ts --
import { area } from "./square.js";
// TypeScript can check this import against src/square.ts
// The emitted JavaScript still imports ./square.js
```

In fact, by default, it errors on `.ts` imports because they will likely not work when you compile to JavaScript. If you don't need to compile at all and intend to run TypeScript natively (either through a bundler or through a runtime that supports TypeScript), you can turn on `allowImportingTsExtensions` to silence these errors. There's also the `rewriteRelativeImportExtensions` option to opt into rewriting relative imports ending in `.ts`, `.tsx`, `.mts`, or `.cts` to their JavaScript extensions during emission. This option is generally only used if you intend to simultaneously run TypeScript natively and generate executable JavaScript.

Type import/export

Within a module, TypeScript lets you import and export types as well as JavaScript values. The `import type` and `export type` forms are erased from the emitted JavaScript. You can also mark individual bindings with `type` when importing types and values together, so that only those bindings are erased.

ts

```ts
import { area, type Shape } from "./shapes.js"; // Only "type Shape" removed
import type { Square } from "./square.js"; // Entire declaration removed
```

`import { type Shape }` leaves an `import {} from "./shapes.js"` declaration, which still loads and evaluates the module.

Module declarations

When you import things from local files, TypeScript reads the target module's TypeScript source directly to understand their types. This is costly. For published libraries, TypeScript recommends generating *declaration files* with the `.d.ts` extension, so it does not need to read the whole source file. The `.d.ts` file is usually produced by `tsc` and contains type declarations rather than executable implementations.

ts

```ts
// -- src/square.ts --
export function area(side: number) {
  return side ** 2;
}
```

ts

```ts
// -- dist/square.d.ts --
export function area(side: number): number;
```

Even if your project is JavaScript, you should still consider publishing a hand-written `.d.ts` file so your users can get the correct type definitions. Generally, TypeScript is able to automatically find the right `.d.ts` file when you import the package source code. Read the TypeScript declaration files guide to find out more.

"Isomorphic" modules

The introduction of modules encourages the JavaScript ecosystem to distribute and reuse code in a modular fashion. However, that doesn't necessarily mean a piece of JavaScript code can run in every environment. Suppose you discovered a module that generates SHA hashes of your users' passwords. Can you use it in the browser front end? Can you use it on your Node.js server? The answer is: it depends.

Modules still have access to global variables, as demonstrated in What's a module?. If the module references globals like `window`, it can run in the browser, but will throw an error in your Node.js server, because `window` is not available there. Similarly, if the code requires access to `process` to be functional, it can only be used in Node.js.

In order to maximize the reusability of a module, it is often advised to make the code "isomorphic" — that is, to make it behave the same way in every runtime. This is commonly achieved in three ways:

- Separate your modules into "core" and "binding". For the "core", focus on pure JavaScript logic like computing the hash, without any DOM, network, filesystem access, and expose utility functions. For

the "binding" part, you can read from and write to the global context. For example, the "browser binding" may choose to read the value from an input box, while the "Node binding" may read it from `process.env`, but values read from either place will be piped to the same core function and handled in the same way. The core can be imported in every environment and used in the same way, while only the binding, which is usually lightweight, needs to be platform-specific.

- Detect whether a particular global exists before using it. For example, the code below checks whether `process` is defined before reading from `process.env` (which is specific to Node.js and similar environments), and otherwise checks whether `window` is defined before reading from the DOM (which is specific to browsers and similar environments).

```js
// myModule.js
let password;
if (typeof process !== "undefined") {
  // We are running in Node.js; read it from `process.env`
  password = process.env.PASSWORD;
} else if (typeof window !== "undefined") {
  // We are running in the browser; read it from the input box
  password = document.getElementById("password").value;
}
```

  This is preferable if the two branches actually end up with the same behavior ("isomorphic"). If it's impossible to provide the same functionality, or if doing so involves loading significant amounts of code while a large part remains unused, it is better to use different "bindings" instead.

- Use a polyfill to provide a fallback for missing features. For example, if you want to use the `DOMParser` API to parse XML, which is available in browsers but not natively in Node.js, you can use a similar API, like the one provided by `@xmldom/xmldom`. You can do so conditionally through dynamic imports:

```js
// myModule.js
if (typeof DOMParser === "undefined") {
  // Use @xmldom/xmldom when no native DOMParser is available
  globalThis.DOMParser = (await import("@xmldom/xmldom")).DOMParser;
}
// …
```

  The `globalThis` variable is a global object that is available in every environment and is useful if you want to read or create global variables within modules.

These practices are not unique to modules. Still, with the trend of code reusability and modularization, you are encouraged to make your code cross-platform so that it can be enjoyed by as many people as possible. Runtimes like Node.js are also actively implementing web APIs where possible to improve interoperability with the web.

See also

- JavaScript modules
- Using modules on the web
- Understanding the module graph
- CommonJS module specification
- AMD specification

- UMD patterns
- RequireJS API
- Node.js: ECMAScript modules, CommonJS modules, and packages
- Modules in Deno
- Module resolution in Bun
- CommonJS module transformation in Babel
- Module configuration in SWC
- TypeScript: Modules theory, module reference, and choosing compiler options
- Vite: Dependency pre-bundling, features, and library mode

JavaScript › Guide

JavaScript language overview

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Language_overview · MDN page last modified 2025-10-30

JavaScript is a multi-paradigm, dynamic language with types and operators, standard built-in objects, and methods. Its syntax is based on the Java and C languages — many structures from those languages apply to JavaScript as well. JavaScript supports object-oriented programming with object prototypes and classes. It also supports functional programming since functions are first-class objects that can be easily created via expressions and passed around like any other object.

This page serves as a quick overview of various JavaScript language features, written for readers with background in other languages, such as C or Java.

Data types

Let's start off by looking at the building blocks of any language: the types. JavaScript programs manipulate values, and those values all belong to a type. JavaScript offers seven *primitive types*:

- Number: used for all number values (integer and floating point) except for *very* big integers.
- BigInt: used for arbitrarily large integers.
- String: used to store text.
- Boolean: `true` and `false` — usually used for conditional logic.
- Symbol: used for creating unique identifiers that won't collide.
- Undefined: indicating that a variable has not been assigned a value.
- Null: indicating a deliberate non-value.

Everything else is known as an Object. Common object types include:

- `Function`
- `Array`
- `Map`
- `RegExp`
- `Error`

Functions aren't special data structures in JavaScript — they are just a special type of object that can be called.

Numbers

JavaScript has two built-in numeric types: Number and BigInt.

The Number type is an IEEE 754 64-bit double-precision floating point value, which means integers can be safely represented between $-(2^{53} - 1)$ and $2^{53} - 1$ without loss of precision, and floating point numbers can be stored all the way up to 1.79×10^{308}. Within numbers, JavaScript does not distinguish between floating point numbers and integers.

```js
console.log(3 / 2); // 1.5, not 1
```

So an *apparent integer* is in fact *implicitly a float*. Because of IEEE 754 encoding, sometimes floating point arithmetic can be imprecise.

```js
console.log(0.1 + 0.2); // 0.30000000000000004
```

For operations that expect integers, such as bitwise operations, the number will be converted to a 32-bit integer.

Number literals can also have prefixes to indicate the base (binary, octal, decimal, or hexadecimal), or an exponent suffix.

```js
console.log(0b111110111); // 503
console.log(0o767); // 503
console.log(0x1f7); // 503
console.log(5.03e2); // 503
```

The BigInt type is an arbitrary length integer. Its behavior is similar to C's integer types (e.g., division truncates to zero), except it can grow indefinitely. BigInts are specified with a number literal and an `n` suffix.

```js
console.log(-3n / 2n); // -1n
```

The standard arithmetic operators are supported, including addition, subtraction, remainder arithmetic, etc. BigInts and numbers cannot be mixed in arithmetic operations.

The `Math` object provides standard mathematical functions and constants.

```js
Math.sin(3.5);
const circumference = 2 * Math.PI * r;
```

There are three ways to convert a string to a number:

- `parseInt()`, which parses the string for an integer.
- `parseFloat()`, which parses the string for a floating-point number.
- The `Number()` function, which parses a string as if it's a number literal and supports many different number representations.

You can also use the unary plus `+` as a shorthand for `Number()`.

Number values also include `NaN` (short for "Not a Number") and `Infinity`. Many "invalid math" operations will return `NaN` — for example, if attempting to parse a non-numeric string, or using `Math.log()` on a negative value. Division by zero produces `Infinity` (positive or negative).

`NaN` is contagious: if you provide it as an operand to any mathematical operation, the result will also be `NaN`. `NaN` is the only value in JavaScript that's not equal to itself (per IEEE 754 specification).

Strings

Strings in JavaScript are sequences of Unicode characters. This should be welcome news to anyone who has had to deal with internationalization. More accurately, they are UTF-16 encoded.

```js
console.log("Hello, world");
console.log("你好，世界！"); // Nearly all Unicode characters can be written literally in
string literals
```

Strings can be written with either single or double quotes — JavaScript does not have the distinction between characters and strings. If you want to represent a single character, you just use a string consisting of that single character.

```js
console.log("Hello"[1] === "e"); // true
```

To find the length of a string (in code units), access its `length` property.

Strings have utility methods to manipulate the string and access information about the string. Because all primitives are immutable by design, these methods return new strings.

The + operator is overloaded for strings: when one of the operands is a string, it performs string concatenation instead of number addition. A special template literal syntax allows you to write strings with embedded expressions more succinctly. Unlike Python's f-strings or C#'s interpolated strings, template literals use backticks (not single or double quotes).

```js
const age = 25;
console.log("I am " + age + " years old."); // String concatenation
console.log(`I am ${age} years old.`); // Template literal
```

Other types

JavaScript distinguishes between `null`, which indicates a deliberate non-value (and is only accessible through the `null` keyword), and `undefined`, which indicates absence of value. There are many ways to obtain `undefined`:

- A `return` statement with no value (`return;`) implicitly returns `undefined`.
- Accessing a nonexistent object property (`obj.iDontExist`) returns `undefined`.
- A variable declaration without initialization (`let x;`) will implicitly initialize the variable to `undefined`.

JavaScript has a Boolean type, with possible values `true` and `false` — both of which are keywords. Any value can be converted to a boolean according to the following rules:

1. `false`, `0`, empty strings (`""`), `NaN`, `null`, and `undefined` all become `false`.
2. All other values become `true`.

You can perform this conversion explicitly using the `Boolean()` function:

```js
Boolean(""); // false
Boolean(234); // true
```

However, this is rarely necessary, as JavaScript will silently perform this conversion when it expects a boolean, such as in an `if` statement (see Control structures). For this reason, we sometimes speak of "truthy" and "falsy", meaning values that become `true` and `false`, respectively, when used in boolean contexts.

Boolean operations such as `&&` (logical *and*), `||` (logical *or*), and `!` (logical *not*) are supported; see Operators.

The Symbol type is often used to create unique identifiers. Every symbol created with the `Symbol()` function is guaranteed to be unique. In addition, there are registered symbols, which are shared

constants, and well-known symbols, which are utilized by the language as "protocols" for certain operations. You can read more about them in the symbol reference.

Variables

Variables in JavaScript are declared using one of three keywords: `let`, `const`, or `var`.

`let` allows you to declare block-level variables. The declared variable is available from the *block* it is enclosed in.

```js
let a;
let name = "Simon";

// myLetVariable is *not* visible out here

for (let myLetVariable = 0; myLetVariable < 5; myLetVariable++) {
  // myLetVariable is only visible in here
}

// myLetVariable is *not* visible out here
```

`const` allows you to declare variables whose values are never intended to change. The variable is available from the *block* it is declared in.

```js
const Pi = 3.14; // Declare variable Pi
console.log(Pi); // 3.14
```

A variable declared with `const` cannot be reassigned.

js · ✗ bad example

```js
const Pi = 3.14;
Pi = 1; // will throw an error because you cannot change a constant variable.
```

`const` declarations only prevent *reassignments* — they don't prevent *mutations* of the variable's value, if it's an object.

```js
const obj = {};
obj.a = 1; // no error
console.log(obj); // { a: 1 }
```

`var` declarations can have surprising behaviors (for example, they are not block-scoped), and they are discouraged in modern JavaScript code.

If you declare a variable without assigning any value to it, its value is `undefined`. You can't declare a `const` variable without an initializer, because you can't change it later anyway.

`let` and `const` declared variables still occupy the entire scope they are defined in, and are in a region known as the temporal dead zone before the actual line of declaration. This has some interesting interactions with variable shadowing, which don't occur in other languages.

```js
function foo(x, condition) {
  if (condition) {
    console.log(x);
    const x = 2;
    console.log(x);
  }
}

foo(1, true);
```

In most other languages, this would log "1" and "2", because before the `const x = 2` line, `x` should still refer to the parameter `x` in the upper scope. In JavaScript, because each declaration occupies the entire scope, this would throw an error on the first `console.log`: "Cannot access 'x' before initialization". For more information, see the reference page of `let`.

JavaScript is *dynamically typed*. Types (as described in the previous section) are only associated with values, but not with variables. For `let`-declared variables, you can always change its type through reassignment.

```js
let a = 1;
a = "foo";
```

Operators

JavaScript's numeric operators include `+`, `-`, `*`, `/`, `%` (remainder), and `**` (exponentiation). Values are assigned using `=`. Each binary operator also has a compound assignment counterpart such as `+=` and `-=`, which extend out to `x = x operator y`.

```js
x += 5;
x = x + 5;
```

You can use `++` and `--` to increment and decrement respectively. These can be used as a prefix or postfix operators.

The + operator also does string concatenation:

```js
"hello" + " world"; // "hello world"
```

If you add a string to a number (or other value) everything is converted into a string first. This might trip you up:

```js
"3" + 4 + 5; // "345"
3 + 4 + "5"; // "75"
```

Adding an empty string to something is a useful way of converting it to a string itself.

Comparisons in JavaScript can be made using `<`, `>`, `<=` and `>=`, which work for both strings and numbers. For equality, the double-equals operator performs type coercion if you give it different types, with sometimes interesting results. On the other hand, the triple-equals operator does not attempt type coercion, and is usually preferred.

```js
123 == "123"; // true
1 == true; // true

123 === "123"; // false
1 === true; // false
```

The double-equals and triple-equals also have their inequality counterparts: `!=` and `!==`.

JavaScript also has bitwise operators and logical operators. Notably, logical operators don't work with boolean values only — they work by the "truthiness" of the value.

```js
const a = 0 && "Hello"; // 0 because 0 is "falsy"
const b = "Hello" || "world"; // "Hello" because both "Hello" and "world" are "truthy"
```

The `&&` and `||` operators use short-circuit logic, which means whether they will execute their second operand is dependent on the first. This is useful for checking for null objects before accessing their attributes:

```js
const name = o && o.getName();
```

Or for caching values (when falsy values are invalid):

```js
const name = cachedName || (cachedName = getName());
```

For a comprehensive list of operators, see the guide page or reference section. You may be especially interested in the operator precedence.

Grammar

JavaScript grammar is very similar to the C family. There are a few points worth mentioning:

- Identifiers can have Unicode characters, but they cannot be one of the reserved words.
- Comments are commonly `//` or `/* */`, while many other scripting languages like Perl, Python, and Bash use #.
- Semicolons are optional in JavaScript — the language automatically inserts them when needed. However, there are certain caveats to watch out for, since unlike Python, semicolons are still part of the syntax.

For an in-depth look at the JavaScript grammar, see the reference page for lexical grammar.

Control structures

JavaScript has a similar set of control structures to other languages in the C family. Conditional statements are supported by `if` and `else`; you can chain them together:

```js
let name = "kittens";
if (name === "puppies") {
  name += " woof";
} else if (name === "kittens") {
  name += " meow";
} else {
  name += "!";
}
name === "kittens meow";
```

JavaScript doesn't have `elif`, and `else if` is really just an `else` branch comprised of a single `if` statement.

JavaScript has `while` loops and `do...while` loops. The first is good for basic looping; the second is for loops where you wish to ensure that the body of the loop is executed at least once:

```js
while (true) {
  // an infinite loop!
}

let input;
do {
  input = get_input();
} while (inputIsNotValid(input));
```

JavaScript's `for` loop is the same as that in C and Java: it lets you provide the control information for your loop on a single line.

```js
for (let i = 0; i < 5; i++) {
  // Will execute 5 times
}
```

JavaScript also contains two other prominent for loops: `for...of`, which iterates over iterables, most notably arrays, and `for...in`, which visits all enumerable properties of an object.

```js
for (const value of array) {
  // do something with value
}

for (const property in object) {
  // do something with object property
}
```

The `switch` statement can be used for multiple branches based on equality checking.

```js
switch (action) {
  case "draw":
    drawIt();
    break;
  case "eat":
    eatIt();
    break;
  default:
    doNothing();
}
```

Similar to C, case clauses are conceptually the same as labels, so if you don't add a `break` statement, execution will "fall through" to the next level. However, they are not actually jump tables — any expression can be part of the `case` clause, not just string or number literals, and they would be evaluated one-by-one until one equals the value being matched. Comparison takes place between the two using the `===` operator.

Unlike some languages like Rust, control-flow structures are statements in JavaScript, meaning you can't assign them to a variable, like `const a = if (x) { 1 } else { 2 }`.

JavaScript errors are handled using the `try...catch` statement.

```js
try {
  buildMySite("./website");
} catch (e) {
  console.error("Building site failed:", e);
}
```

Errors can be thrown using the `throw` statement. Many built-in operations may throw as well.

```js
function buildMySite(siteDirectory) {
  if (!pathExists(siteDirectory)) {
    throw new Error("Site directory does not exist");
  }
}
```

In general, you can't tell the type of the error you just caught, because anything can be thrown from a `throw` statement. However, you can usually assume it's an `Error` instance, as is the example above. There are some subclasses of `Error` built-in, like `TypeError` and `RangeError`, that you can use to provide extra semantics about the error. There's no conditional catch in JavaScript — if you only want to handle one type of error, you need to catch everything, identify the type of error using `instanceof`, and then rethrow the other cases.

```js
try {
  buildMySite("./website");
} catch (e) {
  if (e instanceof RangeError) {
    console.error("Seems like a parameter is out of range:", e);
    console.log("Retrying...");
    buildMySite("./website");
  } else {
    // Don't know how to handle other error types; throw them so
    // something else up in the call stack may catch and handle it
    throw e;
  }
}
```

If an error is uncaught by any `try...catch` in the call stack, the program will exit.

For a comprehensive list of control flow statements, see the reference section.

Objects

JavaScript objects can be thought of as collections of key-value pairs. As such, they are similar to:

- Dictionaries in Python.
- Hashes in Perl and Ruby.
- Hash tables in C and C++.
- HashMaps in Java.
- Associative arrays in PHP.

JavaScript objects are hashes. Unlike objects in statically typed languages, objects in JavaScript do not have fixed shapes — properties can be added, deleted, re-ordered, mutated, or dynamically queried at any time. Object keys are always strings or symbols — even array indices, which are canonically integers, are actually strings under the hood.

Objects are usually created using the literal syntax:

```js
const obj = {
  name: "Carrot",
  for: "Max",
  details: {
    color: "orange",
    size: 12,
  },
};
```

Object properties can be accessed using dot (`.`) or square brackets (`[]`). When using the dot notation, the key must be a valid identifier. Square brackets, on the other hand, allow indexing the object with a dynamic key value.

```js
// Dot notation
obj.name = "Simon";
const name = obj.name;

// Bracket notation
obj["name"] = "Simon";
const name = obj["name"];

// Can use a variable to define a key
const userName = prompt("what is your key?");
obj[userName] = prompt("what is its value?");
```

Property access can be chained together:

```js
obj.details.color; // orange
obj["details"]["size"]; // 12
```

Objects are always references, so unless something is explicitly copying the object, mutations to an object would be visible to the outside.

```js
const obj = {};
function doSomething(o) {
  o.x = 1;
}
doSomething(obj);
console.log(obj.x); // 1
```

This also means two separately created objects will never be equal (`!==`), because they are different references. If you hold two references of the same object, mutating one would be observable through the other.

```js
const me = {};
const stillMe = me;
me.x = 1;
console.log(stillMe.x); // 1
```

For more on objects and prototypes, see the `Object` reference page. For more information on the object initializer syntax, see its reference page.

This page has omitted all details about object prototypes and inheritance because you can usually achieve inheritance with classes without touching the underlying mechanism (which you may have heard to be abstruse). To learn about them, see Inheritance and the prototype chain.

Arrays

Arrays in JavaScript are actually a special type of object. They work very much like regular objects (numerical properties can naturally be accessed only using `[]` syntax) but they have one magic property called `length`. This is always one more than the highest index in the array.

Arrays are usually created with array literals:

```js
const a = ["dog", "cat", "hen"];
a.length; // 3
```

JavaScript arrays are still objects — you can assign any properties to them, including arbitrary number indices. The only "magic" is that `length` will be automatically updated when you set a particular index.

```js
const a = ["dog", "cat", "hen"];
a[100] = "fox";
console.log(a.length); // 101
console.log(a); // ['dog', 'cat', 'hen', empty × 97, 'fox']
```

The array we got above is called a *sparse array* because there are uninhabited slots in the middle, and will cause the engine to deoptimize it from an array to a hash table. Make sure your array is densely populated!

Out-of-bounds indexing doesn't throw. If you query a non-existent array index, you'll get a value of `undefined` in return:

```js
const a = ["dog", "cat", "hen"];
console.log(typeof a[90]); // undefined
```

Arrays can have any elements and can grow or shrink arbitrarily.

```js
const arr = [1, "foo", true];
arr.push({});
// arr = [1, "foo", true, {}]
```

Arrays can be iterated with the `for` loop, as you can in other C-like languages:

```js
for (let i = 0; i < a.length; i++) {
  // Do something with a[i]
}
```

Or, since arrays are iterable, you can use the `for...of` loop, which is synonymous to C++/Java's `for (int x : arr)` syntax:

```js
for (const currentValue of a) {
  // Do something with currentValue
}
```

Arrays come with a plethora of array methods. Many of them would iterate the array — for example, `map()` would apply a callback to every array element, and return a new array:

```js
const babies = ["dog", "cat", "hen"].map((name) => `baby ${name}`);
// babies = ['baby dog', 'baby cat', 'baby hen']
```

Functions

Along with objects, functions are the core component in understanding JavaScript. The most basic function declaration looks like this:

```js
function add(x, y) {
  const total = x + y;
  return total;
}
```

A JavaScript function can take 0 or more parameters. The function body can contain as many statements as you like and can declare its own variables which are local to that function. The `return` statement can be used to return a value at any time, terminating the function. If no return statement is used (or an empty return with no value), JavaScript returns `undefined`.

Functions can be called with more or fewer parameters than it specifies. If you call a function without passing the parameters it expects, they will be set to `undefined`. If you pass more parameters than it expects, the function will ignore the extra parameters.

```js
add(); // NaN
// Equivalent to add(undefined, undefined)

add(2, 3, 4); // 5
// added the first two; 4 was ignored
```

There are a number of other parameter syntaxes available. For example, the rest parameter syntax allows collecting all the extra parameters passed by the caller into an array, similar to Python's `*args`. (Since JS doesn't have named parameters on the language level, there's no `**kwargs`.)

```js
function avg(...args) {
  let sum = 0;
  for (const item of args) {
    sum += item;
  }
  return sum / args.length;
}

avg(2, 3, 4, 5); // 3.5
```

In the above code, the variable `args` holds all the values that were passed into the function.

The rest parameter will store all arguments *after* where it's declared, but not before. In other words, `function avg(firstValue, ...args)` will store the first value passed into the function in the `firstValue` variable and the remaining arguments in `args`.

If a function accepts a list of arguments and you already hold them in an array, you can use the spread syntax in the function call to *spread* the array as a list of elements. For instance: `avg(...numbers)`.

We mentioned that JavaScript doesn't have named parameters. It's possible, though, to implement them using object destructuring, which allows objects to be conveniently packed and unpacked.

```js
// Note the { } braces: this is destructuring an object
function area({ width, height }) {
  return width * height;
}

// The { } braces here create a new object
console.log(area({ width: 2, height: 3 }));
```

There's also the *default parameter* syntax, which allows omitted parameters (or those passed as `undefined`) to have a default value.

```js
function avg(firstValue, secondValue, thirdValue = 0) {
  return (firstValue + secondValue + thirdValue) / 3;
}

avg(1, 2); // 1, instead of NaN
```

Anonymous functions

JavaScript lets you create anonymous functions — that is, functions without names. In practice, anonymous functions are typically used as arguments to other functions, immediately assigned to a variable that can be used to invoke the function, or returned from another function.

```js
// Note that there's no function name before the parentheses
const avg = function (...args) {
  let sum = 0;
  for (const item of args) {
    sum += item;
  }
  return sum / args.length;
};
```

That makes the anonymous function invocable by calling `avg()` with some arguments — that is, it's semantically equivalent to declaring the function using the `function avg() {}` declaration syntax.

There's another way to define anonymous functions — using an arrow function expression.

```js
// Note that there's no function name before the parentheses
const avg = (...args) => {
  let sum = 0;
  for (const item of args) {
    sum += item;
  }
  return sum / args.length;
};

// You can omit the `return` when simply returning an expression
const sum = (a, b, c) => a + b + c;
```

Arrow functions are not semantically equivalent to function expressions — for more information, see its reference page.

There's another way that anonymous functions can be useful: it can be simultaneously declared and invoked in a single expression, called an Immediately invoked function expression (IIFE):

```js
(function () {
  // …
})();
```

For use-cases of IIFEs, you can read emulating private methods with closures.

Recursive functions

JavaScript allows you to call functions recursively. This is particularly useful for dealing with tree structures, such as those found in the browser DOM.

```js
function countChars(elm) {
  if (elm.nodeType === 3) {
    // TEXT_NODE
    return elm.nodeValue.length;
  }
  let count = 0;
  for (let i = 0, child; (child = elm.childNodes[i]); i++) {
    count += countChars(child);
  }
  return count;
}
```

Function expressions can be named as well, which allows them to be recursive.

```js
const charsInBody = (function counter(elm) {
  if (elm.nodeType === 3) {
    // TEXT_NODE
    return elm.nodeValue.length;
  }
  let count = 0;
  for (let i = 0, child; (child = elm.childNodes[i]); i++) {
    count += counter(child);
  }
  return count;
})(document.body);
```

The name provided to a function expression as above is only available to the function's own scope. This allows more optimizations to be done by the engine and results in more readable code. The name also shows up in the debugger and some stack traces, which can save you time when debugging.

If you are used to functional programming, beware of the performance implications of recursion in JavaScript. Although the language specification specifies tail-call optimization, only JavaScriptCore (used by Safari) has implemented it, due to the difficulty of recovering stack traces and debuggability. For deep recursion, consider using iteration instead to avoid stack overflow.

Functions are first-class objects

JavaScript functions are first-class objects. This means that they can be assigned to variables, passed as arguments to other functions, and returned from other functions. In addition, JavaScript supports closures out-of-the-box without explicit capturing, allowing you to conveniently apply functional programming styles.

```js
// Function returning function
const add = (x) => (y) => x + y;
// Function accepting function
const babies = ["dog", "cat", "hen"].map((name) => `baby ${name}`);
```

Note that JavaScript functions are themselves objects — like everything else in JavaScript — and you can add or change properties on them just like we've seen earlier in the Objects section.

Inner functions

JavaScript function declarations are allowed inside other functions. An important detail of nested functions in JavaScript is that they can access variables in their parent function's scope:

```js
function parentFunc() {
  const a = 1;

  function nestedFunc() {
    const b = 4; // parentFunc can't use this
    return a + b;
  }
  return nestedFunc(); // 5
}
```

This provides a great deal of utility in writing more maintainable code. If a called function relies on one or two other functions that are not useful to any other part of your code, you can nest those utility functions inside it. This keeps the number of functions that are in the global scope down.

This is also a great counter to the lure of global variables. When writing complex code, it is often tempting to use global variables to share values between multiple functions, which leads to code that is hard to maintain. Nested functions can share variables in their parent, so you can use that mechanism to couple functions together without polluting your global namespace.

Classes

JavaScript offers the class syntax that's very similar to languages like Java.

```js
class Person {
  constructor(name) {
    this.name = name;
  }
  sayHello() {
    return `Hello, I'm ${this.name}!`;
  }
}

const p = new Person("Maria");
console.log(p.sayHello());
```

JavaScript classes are just functions that must be instantiated with the new operator. Every time a class is instantiated, it returns an object containing the methods and properties that the class specified. Classes don't enforce any code organization — for example, you can have functions returning classes, or you can have multiple classes per file. Here's an example of how ad hoc the creation of a class can be: it's just an expression returned from an arrow function. This pattern is called a mixin.

```js
const withAuthentication = (cls) =>
  class extends cls {
    authenticate() {
      // …
    }
  };

class Admin extends withAuthentication(Person) {
  // …
}
```

Static properties are created by prepending `static`. Private fields and methods are created by prepending a hash `#` (not `private`). The hash is an integral part of the element's name, and distinguishes it from a regular string-keyed property. (Think about `#` as `_` in Python.) Unlike most other languages, there's absolutely no way to read a private element outside the class body — not even in derived classes.

For a detailed guide on various class features, you can read the guide page.

Asynchronous programming

JavaScript is single-threaded by nature. There's no paralleling; only concurrency. Asynchronous programming is powered by an event loop, which allows a set of tasks to be queued and polled for completion.

There are three idiomatic ways to write asynchronous code in JavaScript:

- Callback-based (such as `setTimeout()`)
- `Promise`-based
- `async`/`await`, which is a syntactic sugar for Promises

For example, here's what a file-read operation might look like in JavaScript:

```js
// Callback-based
fs.readFile(filename, (err, content) => {
  // This callback is invoked when the file is read, which could be after a while
  if (err) {
    throw err;
  }
  console.log(content);
});
// Code here will be executed while the file is waiting to be read

// Promise-based
fs.readFile(filename)
  .then((content) => {
    // What to do when the file is read
    console.log(content);
  })
  .catch((err) => {
    throw err;
  });
// Code here will be executed while the file is waiting to be read
```

(code continues on the next page)

(code continued from the previous page)

```
// Async/await
async function readFile(filename) {
  const content = await fs.readFile(filename);
  console.log(content);
}
```

The core language doesn't specify any asynchronous programming features, but it's crucial when interacting with the external environment — from asking user permissions, to fetching data, to reading files. Keeping the potentially long-running operations async ensures that other processes can still run while this one waits — for example, the browser will not freeze while waiting for the user to click a button to grant permission.

If you have an async value, it's not possible to get its value synchronously. For example, if you have a promise, you can only access the eventual result via the `then()` method. Similarly, `await` can only be used in an async context, which is usually an async function or a module. Promises are *never blocking* — only the logic depending on the promise's result will be deferred; everything else continues to execute in the meantime. If you are a functional programmer, you may recognize promises as monads which can be mapped with `then()` (however, they are not *proper* monads because they auto-flatten; i.e., you can't have a `Promise<Promise<T>>`).

In fact, the single-threaded model has made Node.js a popular choice for server-side programming due to its non-blocking IO, making handling a large number of database or file-system requests very performant. However, CPU-bound (computationally intensive) tasks that are pure JavaScript will still block the main thread. To achieve real paralleling, you may need to use workers.

To learn more about asynchronous programming, you can read about using promises or follow the asynchronous JavaScript tutorial.

Modules

JavaScript also specifies a module system supported by most runtimes. A module is usually a file, identified by its file path or URL. You can use the `import` and `export` statements to exchange data between modules:

js

```
import { foo } from "./foo.js";

// Unexported variables are local to the module
const b = 2;

export const a = 1;
```

Unlike Haskell, Python, Java, etc., JavaScript module resolution is entirely host-defined — it's usually based on URLs or file paths, so relative file paths "just work" and are relative to the current module's path instead of some project root path.

However, the JavaScript language doesn't offer standard library modules — all core functionalities are powered by global variables like `Math` and `Intl` instead. This is due to the long history of JavaScript lacking a module system, and the fact that opting into the module system involves some changes to the runtime setup.

Different runtimes may use different module systems. For example, Node.js uses the package manager npm and is mostly file-system based, while Deno and browsers are fully URL-based and modules can be resolved from HTTP URLs.

For more information, see the modules guide page.

Language and runtime

Throughout this page, we've constantly mentioned that certain features are *language-level* while others are *runtime-level*.

JavaScript is a general-purpose scripting language. The core language specification focuses on pure computational logic. It doesn't deal with any input/output — in fact, without extra runtime-level APIs (most notably `console.log()`), a JavaScript program's behavior is entirely unobservable.

A runtime, or a host, is something that feeds data to the JavaScript engine (the interpreter), provides extra global properties, and provides hooks for the engine to interact with the outside world. Module resolution, reading data, printing messages, sending network requests, etc. are all runtime-level operations. Since its inception, JavaScript has been adopted in various environments, such as browsers (which provide APIs like DOM), Node.js (which provides APIs like file system access), etc. JavaScript has been successfully integrated in web (which was its primary purpose), mobile apps, desktop apps, server-side apps, serverless, embedded systems, and more. While you learn about JavaScript core features, it's also important to understand host-provided features in order to put the knowledge to use. For example, you can read about all web platform APIs, which are implemented by browsers, and sometimes non-browsers.

Further exploration

This page offers a very basic insight into how various JavaScript features compare with other languages. If you want to learn more about the language itself and the nuances of each feature, you can read the JavaScript guide and the JavaScript reference.

There are some essential parts of the language that we have omitted due to space and complexity, but you can explore on your own:

- Inheritance and the prototype chain
- Closures
- Regular expressions
- Iteration

JavaScript › Guide

JavaScript data types and data structures

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures · MDN page last modified 2025-07-08

Programming languages all have built-in data structures, but these often differ from one language to another. This article attempts to list the built-in data structures available in JavaScript and what properties they have. These can be used to build other data structures.

The language overview offers a similar summary of the common data types, but with more comparisons to other languages.

Dynamic and weak typing

JavaScript is a dynamic language with dynamic types. Variables in JavaScript are not directly associated with any particular value type, and any variable can be assigned (and re-assigned) values of all types:

```js
let foo = 42; // foo is now a number
foo = "bar"; // foo is now a string
foo = true; // foo is now a boolean
```

JavaScript is also a weakly typed language, which means it allows implicit type conversion when an operation involves mismatched types, instead of throwing type errors.

```js
const foo = 42; // foo is a number
const result = foo + "1"; // JavaScript coerces foo to a string, so it can be concatenated
with the other operand
console.log(result); // 421
```

Implicit coercions are very convenient, but can create subtle bugs when conversions happen where they are not expected, or where they are expected to happen in the other direction (for example, string to number instead of number to string). For symbols and BigInts, JavaScript has intentionally disallowed certain implicit type conversions.

Primitive values

All types except Object define immutable values represented directly at the lowest level of the language. We refer to values of these types as *primitive values*.

All primitive types, except `null`, can be tested by the `typeof` operator. `typeof null` returns `"object"`, so one has to use `=== null` to test for `null`.

All primitive types, except `null` and `undefined`, have their corresponding object wrapper types, which provide useful methods for working with the primitive values. For example, the `Number` object provides methods like `toExponential()`. When a property is accessed on a primitive value, JavaScript automatically wraps the value into the corresponding wrapper object and accesses the property on the object instead. However, accessing a property on `null` or `undefined` throws a `TypeError` exception, which necessitates the introduction of the optional chaining operator.

Type	typeof return value	Object wrapper
Null	`"object"`	N/A
Undefined	`"undefined"`	N/A

Type	typeof return value	Object wrapper
Boolean	`"boolean"`	`Boolean`
Number	`"number"`	`Number`
BigInt	`"bigint"`	`BigInt`
String	`"string"`	`String`
Symbol	`"symbol"`	`Symbol`

The object wrapper classes' reference pages contain more information about the methods and properties available for each type, as well as detailed descriptions for the semantics of the primitive types themselves.

Null type

The Null type is inhabited by exactly one value: `null`.

Undefined type

The Undefined type is inhabited by exactly one value: `undefined`.

Conceptually, `undefined` indicates the absence of a *value*, while `null` indicates the absence of an *object* (which could also make up an excuse for `typeof null === "object"`). The language usually defaults to `undefined` when something is devoid of a value:

- A `return` statement with no value (`return;`) implicitly returns `undefined`.
- Accessing a nonexistent object property (`obj.iDontExist`) returns `undefined`.
- A variable declaration without initialization (`let x;`) implicitly initializes the variable to `undefined`.
- Many methods, such as `Array.prototype.find()` and `Map.prototype.get()`, return `undefined` when no element is found.

`null` is used much less often in the core language. The most important place is the end of the prototype chain — subsequently, methods that interact with prototypes, such as `Object.getPrototypeOf()`, `Object.create()`, etc., accept or return `null` instead of `undefined`.

`null` is a keyword, but `undefined` is a normal identifier that happens to be a global property. In practice, the difference is minor, since `undefined` should not be redefined or shadowed.

Boolean type

The `Boolean` type represents a logical entity and is inhabited by two values: `true` and `false`.

Boolean values are usually used for conditional operations, including ternary operators, `if...else`, `while`, etc.

Number type

The `Number` type is a double-precision 64-bit binary format IEEE 754 value. It is capable of storing positive floating-point numbers between 2^{-1074} (`Number.MIN_VALUE`) and $2^{1023} \times (2 - 2^{-52})$ (`Number.MAX_VALUE`) as well as negative floating-point numbers of the same magnitude, but it can only safely store integers in the range $-(2^{53} - 1)$ (`Number.MIN_SAFE_INTEGER`) to $2^{53} - 1$ (`Number.MAX_SAFE_INTEGER`). Outside this range, JavaScript can no longer safely represent integers; they will instead be represented by a double-precision floating point approximation. You can check if a number is within the range of safe integers using `Number.isSafeInteger()`.

Values outside the representable range are automatically converted:

- Positive values greater than `Number.MAX_VALUE` are converted to `Infinity`.
- Positive values smaller than `Number.MIN_VALUE` are converted to `0`.
- Negative values smaller than `-Number.MAX_VALUE` are converted to `-Infinity`.
- Negative values greater than `-Number.MIN_VALUE` are converted to `-0`.

`Infinity` and `-Infinity` behave similarly to mathematical infinity, but with some slight differences; see `Number.POSITIVE_INFINITY` and `Number.NEGATIVE_INFINITY` for details.

The Number type has only one value with multiple representations: `0` is represented as both `-0` and `+0` (where `0` is an alias for `+0`). In practice, there is almost no difference between the different representations; for example, `+0 === -0` is `true`. However, you are able to notice this when you divide by zero:

```js
console.log(42 / +0); // Infinity
console.log(42 / -0); // -Infinity
```

`NaN` ("**N**ot **a** **N**umber") is a special kind of number value that's typically encountered when the result of an arithmetic operation cannot be expressed as a number. It is also the only value in JavaScript that is not equal to itself.

Although a number is conceptually a "mathematical value" and is always implicitly floating-point-encoded, JavaScript provides bitwise operators. When applying bitwise operators, the number is first converted to a 32-bit integer.

> **Note:** Although bitwise operators *can* be used to represent several Boolean values within a single number using bit masking, this is usually considered a bad practice. JavaScript offers other means to represent a set of Booleans (like an array of Booleans, or an object with Boolean values assigned to named properties). Bit masking also tends to make the code more difficult to read, understand, and maintain.

It may be necessary to use such techniques in very constrained environments, like when trying to cope with the limitations of local storage, or in extreme cases (such as when each bit over the network counts). This technique should only be considered when it is the last measure that can be taken to optimize size.

BigInt type

The `BigInt` type is a numeric primitive in JavaScript that can represent integers with arbitrary magnitude. With BigInts, you can safely store and operate on large integers even beyond the safe integer limit (`Number.MAX_SAFE_INTEGER`) for Numbers.

A BigInt is created by appending `n` to the end of an integer or by calling the `BigInt()` function.

This example demonstrates where incrementing the `Number.MAX_SAFE_INTEGER` returns the expected result:

```js
// BigInt
const x = BigInt(Number.MAX_SAFE_INTEGER); // 9007199254740991n
x + 1n === x + 2n; // false because 9007199254740992n and 9007199254740993n are unequal

// Number
Number.MAX_SAFE_INTEGER + 1 === Number.MAX_SAFE_INTEGER + 2; // true because both are
9007199254740992
```

You can use most operators to work with BigInts, including `+`, `*`, `-`, `**`, and `%` — the only forbidden one is `>>>`. A BigInt is not strictly equal to a Number with the same mathematical value, but it is loosely so.

BigInt values are neither always more precise nor always less precise than numbers, since BigInts cannot represent fractional numbers, but can represent big integers more accurately. Neither type entails the other, and they are not mutually substitutable. A `TypeError` is thrown if BigInt values are mixed with regular numbers in arithmetic expressions, or if they are implicitly converted to each other.

String type

The `String` type represents textual data and is encoded as a sequence of 16-bit unsigned integer values representing UTF-16 code units. Each element in the string occupies a position in the string. The first element is at index `0`, the next at index `1`, and so on. The length of a string is the number of UTF-16 code units in it, which may not correspond to the actual number of Unicode characters; see the `String` reference page for more details.

JavaScript strings are immutable. This means that once a string is created, it is not possible to modify it. String methods create new strings based on the content of the current string — for example:

- A substring of the original using `substring()`.
- A concatenation of two strings using the concatenation operator (`+`) or `concat()`.

Beware of "stringly-typing" your code!

It can be tempting to use strings to represent complex data. Doing this comes with short-term benefits:

- It is easy to build complex strings with concatenation.
- Strings are easy to debug (what you see printed is always what is in the string).
- Strings are the common denominator of a lot of APIs (input fields, local storage values, `fetch()` responses when using `Response.text()`, etc.) and it can be tempting to only work with strings.

With conventions, it is possible to represent any data structure in a string. This does not make it a good idea. For instance, with a separator, one could emulate a list (while a JavaScript array would be more suitable). Unfortunately, when the separator is used in one of the "list" elements, then, the list is broken. An escape character can be chosen, etc. All of this requires conventions and creates an unnecessary maintenance burden.

Use strings for textual data. When representing complex data, *parse* strings, and use the appropriate abstraction.

Symbol type

A `Symbol` is a **unique** and **immutable** primitive value and may be used as the key of an Object property (see below). In some programming languages, Symbols are called "atoms". The purpose of symbols is to create unique property keys that are guaranteed not to clash with keys from other code.

Objects

In computer science, an object is a value in memory which is possibly referenced by an identifier. In JavaScript, objects are the only mutable values. Functions are, in fact, also objects with the additional capability of being *callable*.

Properties

In JavaScript, objects can be seen as a collection of properties. With the object literal syntax, a limited set of properties are initialized; then properties can be added and removed. Object properties are equivalent to key-value pairs. Property keys are either strings or symbols. When other types (such as numbers) are used to index objects, the values are implicitly converted to strings. Property values can be values of any type, including other objects, which enables building complex data structures.

There are two types of object properties: The *data* property and the *accessor* property. Each property has corresponding *attributes*. Each attribute is accessed internally by the JavaScript engine, but you can set them through `Object.defineProperty()`, or read them through `Object.getOwnPropertyDescriptor()`. You can read more about the various nuances on the `Object.defineProperty()` page.

Data property

Data properties associate a key with a value. It can be described by the following attributes:

`value`
: The value retrieved by a get access of the property. Can be any JavaScript value.

`writable`
: A boolean value indicating if the property can be changed with an assignment.

`enumerable`
: A boolean value indicating if the property can be enumerated by a `for...in` loop. See also Enumerability and ownership of properties for how enumerability interacts with other functions and syntaxes.

`configurable`
: A boolean value indicating if the property can be deleted, can be changed to an accessor property, and can have its attributes changed.

Accessor property

Associates a key with one of two accessor functions (`get` and `set`) to retrieve or store a value.

> **Note:** It's important to recognize it's accessor *property* — not accessor *method*. We can give a JavaScript object class-like accessors by using a function as a value — but that doesn't make the object a class.

An accessor property has the following attributes:

`get`
: A function called with an empty argument list to retrieve the property value whenever a get access to the value is performed. See also getters. May be `undefined`.

`set`
: A function called with an argument that contains the assigned value. Executed whenever a specified property is attempted to be changed. See also setters. May be `undefined`.

`enumerable`

A boolean value indicating if the property can be enumerated by a `for...in` loop. See also Enumerability and ownership of properties for how enumerability interacts with other functions and syntaxes.

`configurable`

A boolean value indicating if the property can be deleted, can be changed to a data property, and can have its attributes changed.

The prototype of an object points to another object or to `null` — it's conceptually a hidden property of the object, commonly represented as `[[Prototype]]`. Properties of the object's `[[Prototype]]` can also be accessed on the object itself.

Objects are ad-hoc key-value pairs, so they are often used as maps. However, there can be ergonomics, security, and performance issues. Use a `Map` for storing arbitrary data instead. The `Map` reference contains a more detailed discussion of the pros & cons between plain objects and maps for storing key-value associations.

Dates

JavaScript provides two sets of APIs for representing dates: the legacy `Date` object and the modern `Temporal` object. `Date` has many undesirable design choices and should be avoided in new code if possible.

Indexed collections: Arrays and typed Arrays

Arrays are regular objects for which there is a particular relationship between integer-keyed properties and the `length` property.

Additionally, arrays inherit from `Array.prototype`, which provides a handful of convenient methods to manipulate arrays. For example, `indexOf()` searches a value in the array and `push()` appends an element to the array. This makes Arrays a perfect candidate to represent ordered lists.

Typed Arrays present an array-like view of an underlying binary data buffer, and offer many methods that have similar semantics to the array counterparts. "Typed array" is an umbrella term for a range of data structures, including `Int8Array`, `Float32Array`, etc. Check the typed array page for more information. Typed arrays are often used in conjunction with `ArrayBuffer` and `DataView`.

Keyed collections: Maps, Sets, WeakMaps, WeakSets

These data structures take object references as keys. `Set` and `WeakSet` represent a collection of unique values, while `Map` and `WeakMap` represent a collection of key-value associations.

You could implement `Maps` and `Sets` yourself. However, since objects cannot be compared (in the sense of < "less than", for instance), neither does the engine expose its hash function for objects, look-up performance would necessarily be linear. Native implementations of them (including `WeakMaps`) can have look-up performance that is approximately logarithmic to constant time.

Usually, to bind data to a DOM node, one could set properties directly on the object, or use `data-*` attributes. This has the downside that the data is available to any script running in the same context. `Maps` and `WeakMaps` make it easy to *privately* bind data to an object.

`WeakMap` and `WeakSet` only allow garbage-collectable values as keys, which are either objects or non-registered symbols, and the keys may be collected even when they remain in the collection. They are specifically used for memory usage optimization.

Structured data: JSON

JSON (**J**ava**S**cript **O**bject **N**otation) is a lightweight data-interchange format, derived from JavaScript, but used by many programming languages. JSON builds universal data structures that can be transferred between different environments and even across languages. See `JSON` for more details.

More objects in the standard library

JavaScript has a standard library of built-in objects. Read the reference to find out more about the built-in objects.

Type coercion

As mentioned above, JavaScript is a weakly typed language. This means that you can often use a value of one type where another type is expected, and the language will convert it to the right type for you. To do so, JavaScript defines a handful of coercion rules.

Primitive coercion

The primitive coercion process is used where a primitive value is expected, but there's no strong preference for what the actual type should be. This is usually when a string, a number, or a BigInt are equally acceptable. For example:

- The `Date()` constructor, when it receives one argument that's not a `Date` instance — strings represent date strings, while numbers represent timestamps.
- The `+` operator — if one operand is a string, string concatenation is performed; otherwise, numeric addition is performed.
- The `==` operator — if one operand is a primitive while the other is an object, the object is converted to a primitive value with no preferred type.

This operation does not do any conversion if the value is already a primitive. Objects are converted to primitives by calling its `[Symbol.toPrimitive]()` (with `"default"` as hint), `valueOf()`, and `toString()` methods, in that order. Note that primitive conversion calls `valueOf()` before `toString()`, which is similar to the behavior of number coercion but different from string coercion.

The `[Symbol.toPrimitive]()` method, if present, must return a primitive — returning an object results in a `TypeError`. For `valueOf()` and `toString()`, if one returns an object, the return value is ignored and the other's return value is used instead; if neither is present, or neither returns a primitive, a `TypeError` is thrown. For example, in the following code:

```js
console.log({} + []); // "[object Object]"
```

Neither `{}` nor `[]` have a `[Symbol.toPrimitive]()` method. Both `{}` and `[]` inherit `valueOf()` from `Object.prototype.valueOf`, which returns the object itself. Since the return value is an object, it is ignored. Therefore, `toString()` is called instead. `{}.toString()` returns `"[object Object]"`, while `[].toString()` returns `""`, so the result is their concatenation: `"[object Object]"`.

The `[Symbol.toPrimitive]()` method always takes precedence when doing conversion to any primitive type. Primitive conversion generally behaves like number conversion, because `valueOf()` is called in priority; however, objects with custom `[Symbol.toPrimitive]()` methods can choose to return any primitive. `Date` and `Symbol` objects are the only built-in objects that override the `[Symbol.toPrimitive]`

`()` method. `Date.prototype[Symbol.toPrimitive]()` treats the `"default"` hint as if it's `"string"`, while `Symbol.prototype[Symbol.toPrimitive]()` ignores the hint and always returns a symbol.

Numeric coercion

There are two numeric types: Number and BigInt. Sometimes the language specifically expects a number or a BigInt (such as `Array.prototype.slice()`, where the index must be a number); other times, it may tolerate either and perform different operations depending on the operand's type. For strict coercion processes that do not allow implicit conversion from the other type, see number coercion and BigInt coercion.

Numeric coercion is nearly the same as number coercion, except that BigInts are returned as-is instead of causing a `TypeError`. Numeric coercion is used by all arithmetic operators, since they are overloaded for both numbers and BigInts. The only exception is unary plus, which always does number coercion.

Other coercions

All data types, except Null, Undefined, and Symbol, have their respective coercion process. See string coercion, boolean coercion, and object coercion for more details.

As you may have noticed, there are three distinct paths through which objects may be converted to primitives:

- Primitive coercion: `[Symbol.toPrimitive]("default")` → `valueOf()` → `toString()`
- Numeric coercion, number coercion, BigInt coercion: `[Symbol.toPrimitive]("number")` → `valueOf()` → `toString()`
- String coercion: `[Symbol.toPrimitive]("string")` → `toString()` → `valueOf()`

In all cases, `[Symbol.toPrimitive]()`, if present, must be callable and return a primitive, while `valueOf` or `toString` will be ignored if they are not callable or return an object. At the end of the process, if successful, the result is guaranteed to be a primitive. The resulting primitive is then subject to further coercions depending on the context.

See also

- JavaScript Data Structures and Algorithms by Oleksii Trekhleb
- Computer Science in JavaScript by Nicholas C. Zakas

JavaScript › Guide

Equality comparisons and sameness

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Equality_comparisons_and_sameness · MDN page last modified 2026-09-27

JavaScript provides three different value-comparison operations:

- `===` — strict equality (triple equals)
- `==` — loose equality (double equals)
- `Object.is()`

Which operation you choose depends on what sort of comparison you are looking to perform. Briefly:

- Double equals (`==`) will perform a type conversion when comparing two things, and will handle `NaN`, `-0`, and `+0` specially to conform to IEEE 754 (so `NaN != NaN`, and `-0 == +0`);
- Triple equals (`===`) will do the same comparison as double equals (including the special handling for `NaN`, `-0`, and `+0`) but without type conversion; if the types differ, `false` is returned.
- `Object.is()` does no type conversion and no special handling for `NaN`, `-0`, and `+0` (giving it the same behavior as `===` except on those special numeric values).

They correspond to three of four equality algorithms in JavaScript:

- IsLooselyEqual: `==`
- IsStrictlyEqual: `===`
- SameValue: `Object.is()`
- SameValueZero: used by many built-in operations

Note that the distinctions between these all have to do with their handling of primitives; none of them compares whether the parameters are conceptually similar in structure. For any non-primitive objects `x` and `y` which have the same structure but are distinct objects themselves, all of the above forms will evaluate to `false`.

Comparing the contents of distinct objects or arrays recursively is called deep equality. JavaScript does not provide a general deep comparison operator; libraries and host APIs can provide comparison utilities with different rules.

Strict equality using ===

Strict equality compares two values for equality. Neither value is implicitly converted to some other value before being compared. If the values have different types, the values are considered unequal. If the values have the same type, are not numbers, and have the same value, they're considered equal. Finally, if both values are numbers, they're considered equal if they're both not `NaN` and are the same value, or if one is `+0` and one is `-0`.

```js
const num = 0;
const obj = new String("0");
const str = "0";

console.log(num === num); // true
console.log(obj === obj); // true
console.log(str === str); // true

console.log(num === obj); // false
console.log(num === str); // false
console.log(obj === str); // false
console.log(null === undefined); // false
console.log(obj === null); // false
console.log(obj === undefined); // false
```

Strict equality is almost always the correct comparison operation to use. For all values except numbers, it uses the obvious semantics: a value is only equal to itself. For numbers it uses slightly different semantics to gloss over two different edge cases. The first is that floating point zero is either positively or negatively signed. This is useful in representing certain mathematical solutions, but as most situations don't care about the difference between `+0` and `-0`, strict equality treats them as the same value. The second is that floating point includes the concept of a not-a-number value, `NaN`, to represent the solution to certain ill-defined mathematical problems: negative infinity added to positive infinity, for example. Strict equality treats `NaN` as unequal to every other value — including itself. (The only case in which `(x !== x)` is `true` is when `x` is `NaN`.)

Besides `===`, strict equality is also used by array index-finding methods including `Array.prototype.indexOf()`, `Array.prototype.lastIndexOf()`, `TypedArray.prototype.indexOf()`, `TypedArray.prototype.lastIndexOf()`, and `case`-matching. This means you cannot use `indexOf(NaN)` to find the index of a `NaN` value in an array, or use `NaN` as a `case` value in a `switch` statement and make it match anything.

```js
console.log([NaN].indexOf(NaN)); // -1
switch (NaN) {
  case NaN:
    console.log("Surprise"); // Nothing is logged
}
```

Loose equality using ==

Loose equality is *symmetric*: `A == B` always has identical semantics to `B == A` for any values of `A` and `B` (except for the order of applied conversions). The behavior for performing loose equality using `==` is as follows:

1. If the operands have the same type, they are compared as follows:
   - Object: return `true` only if both operands reference the same object.
   - String: return `true` only if both operands have the same characters in the same order.
   - Number: return `true` only if both operands have the same value. `+0` and `-0` are treated as the same value. If either operand is `NaN`, return `false`; so `NaN` is never equal to `NaN`.
   - Boolean: return `true` only if operands are both `true` or both `false`.
   - BigInt: return `true` only if both operands have the same value.
   - Symbol: return `true` only if both operands reference the same symbol.

2. If one of the operands is `null` or `undefined`, the other must also be `null` or `undefined` to return `true`. Otherwise return `false`.
3. If one of the operands is an object and the other is a primitive, convert the object to a primitive.
4. At this step, both operands are converted to primitives (one of String, Number, Boolean, Symbol, and BigInt). The rest of the conversion is done case-by-case.
   - If they are of the same type, compare them using step 1.
   - If one of the operands is a Symbol but the other is not, return `false`.
   - If one of the operands is a Boolean but the other is not, convert the boolean to a number: `true` is converted to 1, and `false` is converted to 0. Then compare the two operands loosely again.
   - Number to String: convert the string to a number. Conversion failure results in `NaN`, which will guarantee the equality to be `false`.
   - Number to BigInt: compare by their mathematical value. If the number is ±Infinity or `NaN`, return `false`.
   - String to BigInt: convert the string to a BigInt using the same algorithm as the `BigInt()` constructor. If conversion fails, return `false`.

Traditionally, and according to ECMAScript, all primitives and objects are loosely unequal to `undefined` and `null`. But most browsers permit a very narrow class of objects (specifically, the `document.all` object for any page), in some contexts, to act as if they *emulate* the value `undefined`. Loose equality is one such context: `null == A` and `undefined == A` evaluate to true if, and only if, A is an object that *emulates* `undefined`. In all other cases an object is never loosely equal to `undefined` or `null`.

In most cases, using loose equality is discouraged. The result of a comparison using strict equality is easier to predict, and may evaluate more quickly due to the lack of type coercion.

The following example demonstrates loose equality comparisons involving the number primitive `0`, the bigint primitive `0n`, the string primitive `'0'`, and an object whose `toString()` value is `'0'`.

js

```
const num = 0;
const big = 0n;
const str = "0";
const obj = new String("0");

console.log(num == str); // true
console.log(big == num); // true
console.log(str == big); // true

console.log(num == obj); // true
console.log(big == obj); // true
console.log(str == obj); // true
```

Loose equality is only used by the == operator.

Same-value equality using Object.is()

Same-value equality determines whether two values are *functionally identical* in all contexts. (This use case demonstrates an instance of the Liskov substitution principle.) One instance occurs when an attempt is made to mutate an immutable property:

```js
// Add an immutable NEGATIVE_ZERO property to the Number constructor.
Object.defineProperty(Number, "NEGATIVE_ZERO", {
  value: -0,
  writable: false,
  configurable: false,
  enumerable: false,
});

function attemptMutation(v) {
  Object.defineProperty(Number, "NEGATIVE_ZERO", { value: v });
}
```

`Object.defineProperty` will throw an exception when attempting to change an immutable property, but it does nothing if no actual change is requested. If `v` is `-0`, no change has been requested, and no error will be thrown. Internally, when an immutable property is redefined, the newly-specified value is compared against the current value using same-value equality.

Same-value equality is provided by the `Object.is` method. It's used almost everywhere in the language where a value of equivalent identity is expected.

Same-value-zero equality

Similar to same-value equality, but +0 and -0 are considered equal.

Same-value-zero equality is not exposed as a JavaScript API, but can be implemented with custom code:

```js
function sameValueZero(x, y) {
  if (typeof x === "number" && typeof y === "number") {
    // x and y are equal (may be -0 and 0) or they are both NaN
    return x === y || (x !== x && y !== y);
  }
  return x === y;
}
```

Same-value-zero only differs from strict equality by treating `NaN` as equivalent, and only differs from same-value equality by treating `-0` as equivalent to `0`. This makes it usually have the most sensible behavior during searching, especially when working with `NaN`. It's used by `Array.prototype.includes()`, `TypedArray.prototype.includes()`, as well as `Map` and `Set` methods for comparing key equality.

Comparing equality methods

People often compare double equals and triple equals by saying one is an "enhanced" version of the other. For example, double equals could be said as an extended version of triple equals, because the former does everything that the latter does, but with type conversion on its operands — for example, `6 == "6"`. Alternatively, it can be claimed that double equals is the baseline, and triple equals is an enhanced version, because it requires the two operands to be the same type, so it adds an extra constraint.

However, this way of thinking implies that the equality comparisons form a one-dimensional "spectrum" where "totally strict" lies on one end and "totally loose" lies on the other. This model falls short with `Object.is`, because it isn't "looser" than double equals or "stricter" than triple equals, nor does it fit somewhere in between (i.e., being both stricter than double equals, but looser than triple

equals). We can see from the sameness comparisons table below that this is due to the way that `Object.is` handles `NaN`. Notice that if `Object.is(NaN, NaN)` evaluated to `false`, we *could* say that it fits on the loose/strict spectrum as an even stricter form of triple equals, one that distinguishes between `-0` and `+0`. The `NaN` handling means this is untrue, however. Unfortunately, `Object.is` has to be thought of in terms of its specific characteristics, rather than its looseness or strictness with regard to the equality operators.

x	y	==	===	Object.is	SameValueZero
undefined	undefined	✓ true	✓ true	✓ true	✓ true
null	null	✓ true	✓ true	✓ true	✓ true
true	true	✓ true	✓ true	✓ true	✓ true
false	false	✓ true	✓ true	✓ true	✓ true
'foo'	'foo'	✓ true	✓ true	✓ true	✓ true
0	0	✓ true	✓ true	✓ true	✓ true
+0	-0	✓ true	✓ true	✗ false	✓ true
+0	0	✓ true	✓ true	✓ true	✓ true
-0	0	✓ true	✓ true	✗ false	✓ true
0n	-0n	✓ true	✓ true	✓ true	✓ true
0	false	✓ true	✗ false	✗ false	✗ false
""	false	✓ true	✗ false	✗ false	✗ false
""	0	✓ true	✗ false	✗ false	✗ false
'0'	0	✓ true	✗ false	✗ false	✗ false
'17'	17	✓ true	✗ false	✗ false	✗ false
[1, 2]	'1,2'	✓ true	✗ false	✗ false	✗ false
new String('foo')	'foo'	✓ true	✗ false	✗ false	✗ false
null	undefined	✓ true	✗ false	✗ false	✗ false
null	false	✗ false	✗ false	✗ false	✗ false
undefined	false	✗ false	✗ false	✗ false	✗ false
{ foo: 'bar' }	{ foo: 'bar' }	✗ false	✗ false	✗ false	✗ false
new String('foo')	new String('foo')	✗ false	✗ false	✗ false	✗ false
0	null	✗ false	✗ false	✗ false	✗ false
0	NaN	✗ false	✗ false	✗ false	✗ false
'foo'	NaN	✗ false	✗ false	✗ false	✗ false
NaN	NaN	✗ false	✗ false	✓ true	✓ true

When to use Object.is() versus triple equals

In general, the only time `Object.is`'s special behavior towards zeros is likely to be of interest is in the pursuit of certain meta-programming schemes, especially regarding property descriptors, when it is desirable for your work to mirror some of the characteristics of `Object.defineProperty`. If your use case does not require this, it is suggested to avoid `Object.is` and use `===` instead. Even if your requirements involve having comparisons between two `NaN` values evaluate to `true`, generally it is easier to special-case the `NaN` checks (using the `isNaN` method available from previous versions of ECMAScript) than it is to work out how surrounding computations might affect the sign of any zeros you encounter in your comparison.

Here's a non-exhaustive list of built-in methods and operators that might cause a distinction between `-0` and `+0` to manifest itself in your code:

`- (unary negation)`

Consider the following example:

```js
const stoppingForce = obj.mass * -obj.velocity;
```

If `obj.velocity` is `0` (or computes to `0`), a `-0` is introduced at that place and propagates out into `stoppingForce`.

`Math.atan2`, `Math.ceil`, `Math.pow`, `Math.round`

In some cases, it's possible for a `-0` to be introduced into an expression as a return value of these methods even when no `-0` exists as one of the parameters. For example, using `Math.pow` to raise `-Infinity` to the power of any negative, odd exponent evaluates to `-0`. Refer to the documentation for the individual methods.

`Math.floor`, `Math.max`, `Math.min`, `Math.sin`, `Math.sqrt`, `Math.tan`

It's possible to get a `-0` return value out of these methods in some cases where a `-0` exists as one of the parameters. E.g., `Math.min(-0, +0)` evaluates to `-0`. Refer to the documentation for the individual methods.

`~`, `<<`, `>>`

Each of these operators uses the ToInt32 algorithm internally. Since there is only one representation for 0 in the internal 32-bit integer type, `-0` will not survive a round trip after an inverse operation. E.g., both `Object.is(~~(-0), -0)` and `Object.is(-0 << 2 >> 2, -0)` evaluate to `false`.

Relying on `Object.is` when the sign of zeros is not taken into account can be hazardous. Of course, when the intent is to distinguish between `-0` and `+0`, it does exactly what's desired.

Caveat: Object.is() and NaN

`Object.is()` treats all instances of `NaN` as the same value. However, since typed arrays are available, we can have distinct floating point representations of `NaN` which don't behave identically in all contexts. For example:

```js
const f2b = (x) => new Uint8Array(new Float64Array([x]).buffer);
const b2f = (x) => new Float64Array(x.buffer)[0];
// Get a byte representation of NaN
const n = f2b(NaN);
// Change the first bit, which is the sign bit and doesn't matter for NaN
n[7] |= 0x80;
const nan2 = b2f(n);
console.log(nan2); // NaN
console.log(Object.is(nan2, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan2)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 255]
```

> **Note:** Implementations are allowed to canonicalize the bit representation of `NaN`, so `nan2`, when converted back to floating point, may have the same bit representation as the original `NaN`.

See also

- JS Comparison Table by dorey

JavaScript › Guide

Enumerability and ownership of properties

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Enumerability_and_ownership_of_properties · MDN page last modified 2025-07-08

Every property in JavaScript objects can be classified by three factors:

- Enumerable or non-enumerable;
- String or symbol;
- Own property or inherited property from the prototype chain.

Enumerable properties are those properties whose internal enumerable flag is set to true, which is the default for properties created via simple assignment or via a property initializer. Properties defined via `Object.defineProperty` and such are not enumerable by default. Most iteration means (such as `for...in` loops and `Object.keys`) only visit enumerable keys.

Ownership of properties is determined by whether the property belongs to the object directly and not to its prototype chain.

All properties, enumerable or not, string or symbol, own or inherited, can be accessed with dot notation or bracket notation. In this section, we will focus on the means provided by JavaScript to visit a group of object properties one-by-one.

Querying object properties

There are four built-in ways to query a property of an object. They all support both string and symbol keys. The following table summarizes when each method returns `true`.

	Enumerable, own	Enumerable, inherited	Non-enumerable, own	Non-enumerable, inherited
`propertyIsEnumerable()`	`true` ✓	`false` ✗	`false` ✗	`false` ✗
`hasOwnProperty()`	`true` ✓	`false` ✗	`true` ✓	`false` ✗
`Object.hasOwn()`	`true` ✓	`false` ✗	`true` ✓	`false` ✗
`in`	`true` ✓	`true` ✓	`true` ✓	`true` ✓

Traversing object properties

There are many methods in JavaScript that traverse a group of properties of an object. Sometimes, these properties are returned as an array; sometimes, they are iterated one-by-one in a loop; sometimes, they are used for constructing or mutating another object. The following table summarizes when a property may be visited.

Methods that only visit string properties or only symbol properties will have an extra note. ✓ means a property of this type will be visited; ✗ means it will not.

	Enumerable, own	Enumerable, inherited	Non-enumerable, own	Non-enumerable, inherited

`Object.keys`
`Object.values`
`Object.entries`
	✓ (strings)	✗	✗	✗

`Object.getOwnPropertyNames`	✓ (strings)	✗	✓ (strings)	✗

	Enumerable, own	Enumerable, inherited	Non-enumerable, own	Non-enumerable, inherited
`Object.getOwnPropertySymbols`	✓ (symbols)	✗	✓ (symbols)	✗
`Object.getOwnPropertyDescriptors`	✓	✗	✓	✗
`Reflect.ownKeys`	✓	✗	✓	✗
`for...in`	✓ (strings)	✓ (strings)	✗	✗
`Object.assign` (After the first parameter)	✓	✗	✗	✗
Object spread	✓	✗	✗	✗

Obtaining properties by enumerability/ownership

Note that this is not the most efficient algorithm for all cases, but useful for a quick demonstration.

- Detection can occur by `SimplePropertyRetriever.theGetMethodYouWant(obj).includes(prop)`
- Iteration can occur by `SimplePropertyRetriever.theGetMethodYouWant(obj).forEach((value, prop) => {});` (or use `filter()`, `map()`, etc.)

js

```
const SimplePropertyRetriever = {
  getOwnEnumProps(obj) {
    return this._getPropertyNames(obj, true, false, this._enumerable);
    // Or could use for...in filtered with Object.hasOwn or just this: return
Object.keys(obj);
  },
  getOwnNonEnumProps(obj) {
    return this._getPropertyNames(obj, true, false, this._notEnumerable);
  },
  getOwnProps(obj) {
    return this._getPropertyNames(
      obj,
      true,
      false,
      this._enumerableAndNotEnumerable,
    );
    // Or just use: return Object.getOwnPropertyNames(obj);
  },
  getPrototypeEnumProps(obj) {
    return this._getPropertyNames(obj, false, true, this._enumerable);
  },
  getPrototypeNonEnumProps(obj) {
    return this._getPropertyNames(obj, false, true, this._notEnumerable);
  },
  getPrototypeProps(obj) {
    return this._getPropertyNames(
      obj,
      false,
      true,
      this._enumerableAndNotEnumerable,
    );
  },
  getOwnAndPrototypeEnumProps(obj) {
    return this._getPropertyNames(obj, true, true, this._enumerable);
    // Or could use unfiltered for...in
```

(code continues on the next page)

(code continued from the previous page)

```
  },
  getOwnAndPrototypeNonEnumProps(obj) {
    return this._getPropertyNames(obj, true, true, this._notEnumerable);
  },
  getOwnAndPrototypeEnumAndNonEnumProps(obj) {
    return this._getPropertyNames(
      obj,
      true,
      true,
      this._enumerableAndNotEnumerable,
    );
  },
  // Private static property checker callbacks
  _enumerable(obj, prop) {
    return Object.prototype.propertyIsEnumerable.call(obj, prop);
  },
  _notEnumerable(obj, prop) {
    return !Object.prototype.propertyIsEnumerable.call(obj, prop);
  },
  _enumerableAndNotEnumerable(obj, prop) {
    return true;
  },
  // Inspired by http://stackoverflow.com/a/8024294/271577
  _getPropertyNames(obj, iterateSelf, iteratePrototype, shouldInclude) {
    const props = [];
    do {
      if (iterateSelf) {
        Object.getOwnPropertyNames(obj).forEach((prop) => {
          if (props.indexOf(prop) === -1 && shouldInclude(obj, prop)) {
            props.push(prop);
          }
        });
      }
      if (!iteratePrototype) {
        break;
      }
      iterateSelf = true;
      obj = Object.getPrototypeOf(obj);
    } while (obj);
    return props;
  },
};
```

See also

- in
- for...in
- Object.prototype.hasOwnProperty()
- Object.prototype.propertyIsEnumerable()
- Object.getOwnPropertyNames()
- Object.getOwnPropertySymbols()
- Object.keys()
- Object.getOwnPropertyDescriptors()
- Object.hasOwn()
- Reflect.ownKeys()

JavaScript › Guide

Closures

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures · MDN page last modified 2025-11-04

A **closure** is the combination of a function bundled together (enclosed) with references to its surrounding state (the **lexical environment**). In other words, a closure gives a function access to its outer scope. In JavaScript, closures are created every time a function is created, at function creation time.

Lexical scoping

Consider the following example code:

```js
function init() {
  var name = "Mozilla"; // name is a local variable created by init
  function displayName() {
    // displayName() is the inner function, that forms a closure
    console.log(name); // use variable declared in the parent function
  }
  displayName();
}
init();
```

`init()` creates a local variable called `name` and a function called `displayName()`. The `displayName()` function is an inner function that is defined inside `init()` and is available only within the body of the `init()` function. Note that the `displayName()` function has no local variables of its own. However, since inner functions have access to the variables of outer scopes, `displayName()` can access the variable `name` declared in the parent function, `init()`.

If you run this code in your console, you can see that the `console.log()` statement within the `displayName()` function successfully displays the value of the `name` variable, which is declared in its parent function. This is an example of *lexical scoping*, which describes how a parser resolves variable names when functions are nested. The word *lexical* refers to the fact that lexical scoping uses the location where a variable is declared within the source code to determine where that variable is available. Nested functions have access to variables declared in their outer scope.

Scoping with let and const

Traditionally (before ES6), JavaScript variables only had two kinds of scopes: *function scope* and *global scope*. Variables declared with `var` are either function-scoped or global-scoped, depending on whether they are declared within a function or outside a function. This can be tricky, because blocks with curly braces do not create scopes:

```js
if (Math.random() > 0.5) {
  var x = 1;
} else {
  var x = 2;
}
console.log(x);
```

For people from other languages (e.g., C, Java) where blocks create scopes, the above code should throw an error on the `console.log` line, because we are outside the scope of `x` in either block. However,

because blocks don't create scopes for `var`, the `var` statements here actually create a global variable. There is also a practical example introduced below that illustrates how this can cause actual bugs when combined with closures.

In ES6, JavaScript introduced the `let` and `const` declarations, which, among other things like temporal dead zones, allow you to create block-scoped variables.

```js
if (Math.random() > 0.5) {
  const x = 1;
} else {
  const x = 2;
}
console.log(x); // ReferenceError: x is not defined
```

In essence, blocks are finally treated as scopes in ES6, but only if you declare variables with `let` or `const`. In addition, ES6 introduced modules, which introduced another kind of scope. Closures are able to capture variables in all these scopes, which we will introduce later.

Closure

Consider the following code example:

```js
function makeFunc() {
  const name = "Mozilla";
  function displayName() {
    console.log(name);
  }
  return displayName;
}

const myFunc = makeFunc();
myFunc();
```

Running this code has exactly the same effect as the previous example of the `init()` function above. What's different (and interesting) is that the `displayName()` inner function is returned from the outer function *before being executed*.

At first glance, it might seem unintuitive that this code still works. In some programming languages, the local variables within a function exist for just the duration of that function's execution. Once `makeFunc()` finishes executing, you might expect that the `name` variable would no longer be accessible. However, because the code still works, this is obviously not the case in JavaScript.

The reason is that functions in JavaScript form closures. A *closure* is the combination of a function and the lexical environment within which that function was declared. This environment consists of any variables that were in-scope at the time the closure was created. In this case, `myFunc` is a reference to the instance of the function `displayName` that is created when `makeFunc` is run. The instance of `displayName` maintains a reference to its lexical environment, within which the variable `name` exists. For this reason, when `myFunc` is invoked, the variable `name` remains available for use, and "Mozilla" is passed to `console.log`.

Here's a slightly more interesting example—a `makeAdder` function:

```js
function makeAdder(x) {
  return function (y) {
    return x + y;
  };
}

const add5 = makeAdder(5);
const add10 = makeAdder(10);

console.log(add5(2)); // 7
console.log(add10(2)); // 12
```

In this example, we have defined a function `makeAdder(x)`, that takes a single argument `x`, and returns a new function. The function it returns takes a single argument `y`, and returns the sum of `x` and `y`.

In essence, `makeAdder` is a function factory. It creates functions that can add a specific value to their argument. In the above example, the function factory creates two new functions—one that adds five to its argument, and one that adds 10.

`add5` and `add10` both form closures. They share the same function body definition, but store different lexical environments. In `add5`'s lexical environment, `x` is 5, while in the lexical environment for `add10`, `x` is 10.

Practical closures

Closures are useful because they let you associate data (the lexical environment) with a function that operates on that data. This has obvious parallels to object-oriented programming, where objects allow you to associate data (the object's properties) with one or more methods.

Consequently, you can use a closure anywhere that you might normally use an object with only a single method.

Situations where you might want to do this are particularly common on the web. Much of the code written in front-end JavaScript is event-based. You define some behavior, and then attach it to an event that is triggered by the user (such as a click or a keypress). The code is attached as a callback (a single function that is executed in response to the event).

For instance, suppose we want to add buttons to a page to adjust the text size. One way of doing this is to specify the font-size of the `body` element (in pixels), and then set the size of the other elements on the page (such as headers) using the relative `em` unit:

```css
body {
  font-family: "Helvetica", "Arial", sans-serif;
  font-size: 12px;
}

h1 {
  font-size: 1.5em;
}

h2 {
  font-size: 1.2em;
}
```

Such interactive text size buttons can change the `font-size` property of the `body` element, and the adjustments are picked up by other elements on the page thanks to the relative units.

Here's the JavaScript:

```js
function makeSizer(size) {
  return () => {
    document.body.style.fontSize = `${size}px`;
  };
}

const size12 = makeSizer(12);
const size14 = makeSizer(14);
const size16 = makeSizer(16);
```

`size12`, `size14`, and `size16` are now functions that resize the body text to 12, 14, and 16 pixels, respectively. You can attach them to buttons as demonstrated in the following code example.

```js
document.getElementById("size-12").onclick = size12;
document.getElementById("size-14").onclick = size14;
document.getElementById("size-16").onclick = size16;
```

```html
<button id="size-12">12</button>
<button id="size-14">14</button>
<button id="size-16">16</button>
<p>This is some text that will change size when you click the buttons above.</p>
```

▷ *Live sample output ("practical closures sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures#practical_closures .*

Emulating private methods with closures

Languages such as Java allow you to declare methods as private, meaning that they can be called only by other methods in the same class.

JavaScript, prior to classes, didn't have a native way of declaring private methods, but it was possible to emulate private methods using closures. Private methods aren't just useful for restricting access to code. They also provide a powerful way of managing your global namespace.

The following code illustrates how to use closures to define public functions that can access private functions and variables. Note that these closures follow the Module Design Pattern.

```js
const counter = (function () {
  let privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
  }

  return {
    increment() {
      changeBy(1);
    },
```

(code continues on the next page)

(code continued from the previous page)

```
    decrement() {
      changeBy(-1);
    },

    value() {
      return privateCounter;
    },
  };
})();

console.log(counter.value()); // 0.

counter.increment();
counter.increment();
console.log(counter.value()); // 2.

counter.decrement();
console.log(counter.value()); // 1.
```

In previous examples, each closure had its own lexical environment. Here though, there is a single lexical environment that is shared by the three functions: `counter.increment`, `counter.decrement`, and `counter.value`.

The shared lexical environment is created in the body of an anonymous function, *which is executed as soon as it has been defined* (also known as an IIFE). The lexical environment contains two private items: a variable called `privateCounter`, and a function called `changeBy`. You can't access either of these private members from outside the anonymous function. Instead, you indirectly access them using the three public functions that are returned from the anonymous wrapper.

Those three public functions form closures that share the same lexical environment. Thanks to JavaScript's lexical scoping, they each have access to the `privateCounter` variable and the `changeBy` function.

js

```
function makeCounter() {
  let privateCounter = 0;
  function changeBy(val) {
    privateCounter += val;
  }
  return {
    increment() {
      changeBy(1);
    },

    decrement() {
      changeBy(-1);
    },

    value() {
      return privateCounter;
    },
  };
}
```

(code continues on the next page)

(code continued from the previous page)

```
const counter1 = makeCounter();
const counter2 = makeCounter();

console.log(counter1.value()); // 0.

counter1.increment();
counter1.increment();
console.log(counter1.value()); // 2.

counter1.decrement();
console.log(counter1.value()); // 1.
console.log(counter2.value()); // 0.
```

Notice how the two counters maintain their independence from one another. Each closure references a different version of the `privateCounter` variable through its own closure. Each time one of the counters is called, its lexical environment changes by changing the value of this variable. Changes to the variable value in one closure don't affect the value in the other closure.

> **Note:** Using closures in this way provides benefits that are normally associated with object-oriented programming. In particular, *data hiding* and *encapsulation*.

Closure scope chain

A nested function's access to the outer function's scope includes the enclosing scope of the outer function—effectively creating a chain of function scopes. To demonstrate, consider the following example code.

js

```
// global scope
const e = 10;
function sum(a) {
  return function (b) {
    return function (c) {
      // outer functions scope
      return function (d) {
        // local scope
        return a + b + c + d + e;
      };
    };
  };
}

console.log(sum(1)(2)(3)(4)); // 20
```

You can also write without anonymous functions:

js

```
// global scope
const e = 10;
function sum(a) {
  return function sum2(b) {
    return function sum3(c) {
      // outer functions scope
      return function sum4(d) {
        // local scope
        return a + b + c + d + e;
      };
```

(code continues on the next page)

(code continued from the previous page)

```
    };
  };
}

const sum2 = sum(1);
const sum3 = sum2(2);
const sum4 = sum3(3);
const result = sum4(4);
console.log(result); // 20
```

In the example above, there's a series of nested functions, all of which have access to the outer functions' scope. In this context, we can say that closures have access to *all* outer scopes.

Closures can capture variables in block scopes and module scopes as well. For example, the following creates a closure over the block-scoped variable `y`:

js

```
function outer() {
  let getY;
  {
    const y = 6;
    getY = () => y;
  }
  console.log(typeof y); // undefined
  console.log(getY()); // 6
}

outer();
```

Closures over modules can be more interesting.

js

```
// myModule.js
let x = 5;
export const getX = () => x;
export const setX = (val) => {
  x = val;
};
```

Here, the module exports a pair of getter-setter functions, which close over the module-scoped variable `x`. Even when `x` is not directly accessible from other modules, it can be read and written with the functions.

js

```
import { getX, setX } from "./myModule.js";

console.log(getX()); // 5
setX(6);
console.log(getX()); // 6
```

Closures can close over imported values as well, which are regarded as *live bindings*, because when the original value changes, the imported one changes accordingly.

js

```
// myModule.js
export let x = 1;
export const setX = (val) => {
  x = val;
};
```

js

```
// closureCreator.js
import { x } from "./myModule.js";

export const getX = () => x; // Close over an imported live binding
```

js

```
import { getX } from "./closureCreator.js";
import { setX } from "./myModule.js";

console.log(getX()); // 1
setX(2);
console.log(getX()); // 2
```

Creating closures in loops: A common mistake

Prior to the introduction of the `let` keyword, a common problem with closures occurred when you created them inside a loop. To demonstrate, consider the following example code.

html

```
<p id="help">Helpful notes will appear here</p>
<p>Email: <input type="text" id="email" name="email" /></p>
<p>Name: <input type="text" id="name" name="name" /></p>
<p>Age: <input type="text" id="age" name="age" /></p>
```

js · ✗ bad example

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    // Culprit is the use of `var` on this line
    var item = helpText[i];
    document.getElementById(item.id).onfocus = function () {
      showHelp(item.help);
    };
  }
}

setupHelp();
```

▷ *Live sample output ("closures bad sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures#closures_bad .*

The `helpText` array defines three helpful hints, each associated with the ID of an input field in the document. The loop cycles through these definitions, hooking up an `onfocus` event to each one that shows the associated help method.

If you try this code out, you'll see that it doesn't work as expected. No matter what field you focus on, the message about your age will be displayed.

The reason for this is that the functions assigned to `onfocus` form closures; they consist of the function definition and the captured environment from the `setupHelp` function's scope. Three closures have been created by the loop, but each one shares the same single lexical environment, which has a variable with changing values (`item`). This is because the variable `item` is declared with `var` and thus has function scope due to hoisting. The value of `item.help` is determined when the `onfocus` callbacks are executed. Because the loop has already run its course by that time, the `item` variable object (shared by all three closures) has been left pointing to the last entry in the `helpText` list.

One solution in this case is to use more closures: in particular, to use a function factory as described earlier:

html · hidden on the MDN page (supporting code for the live sample)

```
<p id="help">Helpful notes will appear here</p>
<p>Email: <input type="text" id="email" name="email" /></p>
<p>Name: <input type="text" id="name" name="name" /></p>
<p>Age: <input type="text" id="age" name="age" /></p>
```

js

```
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function makeHelpCallback(help) {
  return function () {
    showHelp(help);
  };
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    var item = helpText[i];
    document.getElementById(item.id).onfocus = makeHelpCallback(item.help);
  }
}

setupHelp();
```

▷ *Live sample output ("closures factory sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures#closures_factory .*

This works as expected. Rather than the callbacks all sharing a single lexical environment, the `makeHelpCallback` function creates *a new lexical environment* for each callback, in which `help` refers to

the corresponding string from the `helpText` array.

One other way to write the above using anonymous closures is:

```js
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  var helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (var i = 0; i < helpText.length; i++) {
    (function () {
      var item = helpText[i];
      document.getElementById(item.id).onfocus = function () {
        showHelp(item.help);
      };
    })(); // Immediate event listener attachment with the current value of item (preserved
until iteration).
  }
}

setupHelp();
```

If you don't want to use more closures, you can use the `let` or `const` keyword:

```js
function showHelp(help) {
  document.getElementById("help").textContent = help;
}

function setupHelp() {
  const helpText = [
    { id: "email", help: "Your email address" },
    { id: "name", help: "Your full name" },
    { id: "age", help: "Your age (you must be over 16)" },
  ];

  for (let i = 0; i < helpText.length; i++) {
    const item = helpText[i];
    document.getElementById(item.id).onfocus = () => {
      showHelp(item.help);
    };
  }
}

setupHelp();
```

This example uses `const` instead of `var`, so every closure binds the block-scoped variable, meaning that no additional closures are required.

If you are writing modern JavaScript anyway, you can consider more alternatives to the plain `for` loop, such as using `for...of` loop and declaring `item` as `let` or `const`, or using the `forEach()` method, which both avoid the closure problem.

```js
for (const item of helpText) {
  document.getElementById(item.id).onfocus = () => {
    document.getElementById("help").textContent = item.help;
  };
}

helpText.forEach((item) => {
  document.getElementById(item.id).onfocus = () => {
    showHelp(item.help);
  };
});
```

Performance considerations

As mentioned previously, each function instance manages its own scope and closure. Therefore, it is unwise to unnecessarily create functions within other functions if closures are not needed for a particular task, as it will negatively affect script performance both in terms of processing speed and memory consumption.

For instance, when creating a new object/class, methods should normally be associated to the object's prototype rather than defined into the object constructor. The reason is that whenever the constructor is called, the methods would get reassigned (that is, for every object creation).

Consider the following case:

```js
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
  this.getName = function () {
    return this.name;
  };

  this.getMessage = function () {
    return this.message;
  };
}
```

Because the previous code does not take advantage of the benefits of using closures in this particular instance, we could instead rewrite it to avoid using closures as follows:

```js
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
}
MyObject.prototype = {
  getName() {
    return this.name;
  },
  getMessage() {
    return this.message;
  },
};
```

However, redefining the prototype is not recommended. The following example instead appends to the existing prototype:

```js
function MyObject(name, message) {
  this.name = name.toString();
  this.message = message.toString();
}
MyObject.prototype.getName = function () {
  return this.name;
};
MyObject.prototype.getMessage = function () {
  return this.message;
};
```

In the two previous examples, the inherited prototype can be shared by all objects and the method definitions need not occur at every object creation. See Inheritance and the prototype chain for more.

JavaScript › Guide

Inheritance and the prototype chain

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Inheritance_and_the_prototype_chain · MDN page last modified 2025-11-22

In programming, *inheritance* refers to passing down characteristics from a parent to a child so that a new piece of code can reuse and build upon the features of an existing one. JavaScript implements inheritance by using objects. Each object has an internal link to another object called its *prototype*. That prototype object has a prototype of its own, and so on until an object is reached with `null` as its prototype. By definition, `null` has no prototype and acts as the final link in this **prototype chain**. It is possible to mutate any member of the prototype chain or even swap out the prototype at runtime, so concepts like static dispatching do not exist in JavaScript.

JavaScript is a bit confusing for developers experienced in class-based languages (like Java or C++), as it is dynamic and does not have static types. While this confusion is often considered to be one of JavaScript's weaknesses, the prototypal inheritance model itself is, in fact, more powerful than the classic model. It is, for example, fairly trivial to build a classic model on top of a prototypal model — which is how classes are implemented.

Although classes are now widely adopted and have become a new paradigm in JavaScript, classes do not bring a new inheritance pattern. While classes abstract most of the prototypal mechanism away, understanding how prototypes work under the hood is still useful.

Inheritance with the prototype chain

Inheriting properties

JavaScript objects are dynamic "bags" of properties (referred to as **own properties**). JavaScript objects have a link to a prototype object. When trying to access a property of an object, the property will not only be sought on the object, but also on the prototype of the object, the prototype of the prototype, and so on, until either a property with a matching name is found or the end of the prototype chain is reached.

> **Note:** Following the ECMAScript standard, the notation `someObject.[[Prototype]]` is used to designate the prototype of `someObject`. The `[[Prototype]]` internal slot can be accessed and modified with the `Object.getPrototypeOf()` and `Object.setPrototypeOf()` functions respectively. This is equivalent to the JavaScript accessor `__proto__` which is non-standard but de-facto implemented by many JavaScript engines. To prevent confusion while keeping it succinct, in our notation we will avoid using `obj.__proto__` but use `obj.[[Prototype]]` instead. This corresponds to `Object.getPrototypeOf(obj)`.
>
> It should not be confused with the `func.prototype` property of functions, which instead specifies the `[[Prototype]]` to be assigned to all *instances* of objects created by the given function when used as a constructor. We will discuss the `prototype` property of constructor functions in a later section.

There are several ways to specify the `[[Prototype]]` of an object, which are listed in a later section. For now, we will use the `__proto__` syntax for illustration. It's worth noting that the `{ __proto__: ... }` syntax is different from the `obj.__proto__` accessor: the former is standard and not deprecated.

In an object literal like `{ a: 1, b: 2, __proto__: c }`, the value `c` (which has to be either `null` or another object) will become the `[[Prototype]]` of the object represented by the literal, while the other keys like `a` and `b` will become the *own properties* of the object. This syntax reads very naturally, since `[[Prototype]]` is just an "internal property" of the object.

Here is what happens when trying to access a property:

```js
const o = {
  a: 1,
  b: 2,
  // __proto__ sets the [[Prototype]]. It's specified here
  // as another object literal.
  __proto__: {
    b: 3,
    c: 4,
  },
};

// o.[[Prototype]] has properties b and c.
// o.[[Prototype]].[[Prototype]] is Object.prototype (we will explain
// what that means later).
// Finally, o.[[Prototype]].[[Prototype]].[[Prototype]] is null.
// This is the end of the prototype chain, as null,
// by definition, has no [[Prototype]].
// Thus, the full prototype chain looks like:
// { a: 1, b: 2 } ---> { b: 3, c: 4 } ---> Object.prototype ---> null

console.log(o.a); // 1
// Is there an 'a' own property on o? Yes, and its value is 1.

console.log(o.b); // 2
// Is there a 'b' own property on o? Yes, and its value is 2.
// The prototype also has a 'b' property, but it's not visited.
// This is called Property Shadowing

console.log(o.c); // 4
// Is there a 'c' own property on o? No, check its prototype.
// Is there a 'c' own property on o.[[Prototype]]? Yes, its value is 4.

console.log(o.d); // undefined
// Is there a 'd' own property on o? No, check its prototype.
// Is there a 'd' own property on o.[[Prototype]]? No, check its prototype.
// o.[[Prototype]].[[Prototype]] is Object.prototype and
// there is no 'd' property by default, check its prototype.
// o.[[Prototype]].[[Prototype]].[[Prototype]] is null, stop searching,
// no property found, return undefined.
```

Setting a property to an object creates an own property. The only exception to the getting and setting behavior rules is when it's intercepted by a getter or setter.

Similarly, you can create longer prototype chains, and a property will be sought on all of them.

```js
const o = {
  a: 1,
  b: 2,
  // __proto__ sets the [[Prototype]]. It's specified here
  // as another object literal.
  __proto__: {
    b: 3,
    c: 4,
    __proto__: {
      d: 5,
    },
```

(code continues on the next page)

(code continued from the previous page)

```
  },
};

// { a: 1, b: 2 } ---> { b: 3, c: 4 } ---> { d: 5 } ---> Object.prototype ---> null

console.log(o.d); // 5
```

Inheriting "methods"

JavaScript does not have "methods" in the form that class-based languages define them. In JavaScript, any function can be added to an object in the form of a property. An inherited function acts just as any other property, including property shadowing as shown above (in this case, a form of *method overriding*).

When an inherited function is executed, the value of `this` points to the inheriting object, not to the prototype object where the function is an own property.

js

```
const parent = {
  value: 2,
  method() {
    return this.value + 1;
  },
};

console.log(parent.method()); // 3
// When calling parent.method in this case, 'this' refers to parent

// child is an object that inherits from parent
const child = {
  __proto__: parent,
};
console.log(child.method()); // 3
// When child.method is called, 'this' refers to child.
// So when child inherits the method of parent,
// The property 'value' is sought on child. However, since child
// doesn't have an own property called 'value', the property is
// found on the [[Prototype]], which is parent.value.

child.value = 4; // assign the value 4 to the property 'value' on child.
// This shadows the 'value' property on parent.
// The child object now looks like:
// { value: 4, __proto__: { value: 2, method: [Function] } }
console.log(child.method()); // 5
// Since child now has the 'value' property, 'this.value' means
// child.value instead
```

Constructors

The power of prototypes is that we can reuse a set of properties if they should be present on every instance — especially for methods. Suppose we are to create a series of boxes, where each box is an object that contains a value which can be accessed through a `getValue` function. A naive implementation would be:

```js
const boxes = [
  { value: 1, getValue() { return this.value; } },
  { value: 2, getValue() { return this.value; } },
  { value: 3, getValue() { return this.value; } },
];
```

This is subpar, because each instance has its own function property that does the same thing, which is redundant and unnecessary. Instead, we can move `getValue` to the `[[Prototype]]` of all boxes:

```js
const boxPrototype = {
  getValue() {
    return this.value;
  },
};

const boxes = [
  { value: 1, __proto__: boxPrototype },
  { value: 2, __proto__: boxPrototype },
  { value: 3, __proto__: boxPrototype },
];
```

This way, all boxes' `getValue` method will refer to the same function, lowering memory usage. However, manually binding the `__proto__` for every object creation is still very inconvenient. This is when we would use a *constructor* function, which automatically sets the `[[Prototype]]` for every object manufactured. Constructors are functions called with `new`.

```js
// A constructor function
function Box(value) {
  this.value = value;
}

// Properties all boxes created from the Box() constructor
// will have
Box.prototype.getValue = function () {
  return this.value;
};

const boxes = [new Box(1), new Box(2), new Box(3)];
```

We say that `new Box(1)` is an *instance* created from the `Box` constructor function. `Box.prototype` is not much different from the `boxPrototype` object we created previously — it's just a plain object. Every instance created from a constructor function will automatically have the constructor's `prototype` property as its `[[Prototype]]` — that is, `Object.getPrototypeOf(new Box()) === Box.prototype`. `Constructor.prototype` by default has one own property: `constructor`, which references the constructor function itself — that is, `Box.prototype.constructor === Box`. This allows one to access the original constructor from any instance.

> **Note:** If a non-primitive is returned from the constructor function, that value will become the result of the `new` expression. In this case the `[[Prototype]]` may not be correctly bound — but this should not happen much in practice.

The above constructor function can be rewritten in classes as:

```js
class Box {
  constructor(value) {
    this.value = value;
  }

  // Methods are created on Box.prototype
  getValue() {
    return this.value;
  }
}
```

Classes are syntax sugar over constructor functions, which means you can still manipulate `Box.prototype` to change the behavior of all instances. However, because classes are designed to be an abstraction over the underlying prototype mechanism, we will use the more-lightweight constructor function syntax for this tutorial to fully demonstrate how prototypes work.

Because `Box.prototype` references the same object as the `[[Prototype]]` of all instances, we can change the behavior of all instances by mutating `Box.prototype`.

```js
function Box(value) {
  this.value = value;
}
Box.prototype.getValue = function () {
  return this.value;
};
const box = new Box(1);

// Mutate Box.prototype after an instance has already been created
Box.prototype.getValue = function () {
  return this.value + 1;
};
box.getValue(); // 2
```

A corollary is, *re-assigning* `Constructor.prototype` (`Constructor.prototype = ...`) is a bad idea for two reasons:

- The `[[Prototype]]` of instances created before the reassignment is now referencing a different object from the `[[Prototype]]` of instances created after the reassignment — mutating one's `[[Prototype]]` no longer mutates the other.
- Unless you manually re-set the `constructor` property, the constructor function can no longer be traced from `instance.constructor`, which may break user expectation. Some built-in operations will read the `constructor` property as well, and if it is not set, they may not work as expected.

`Constructor.prototype` is only useful when constructing instances. It has nothing to do with `Constructor.[[Prototype]]`, which is the constructor function's *own* prototype, which is `Function.prototype` — that is, `Object.getPrototypeOf(Constructor) === Function.prototype`.

Implicit constructors of literals

Some literal syntaxes in JavaScript create instances that implicitly set the `[[Prototype]]`. For example:

```js
// Object literals (without the `__proto__` key) automatically
// have `Object.prototype` as their `[[Prototype]]`
const object = { a: 1 };
Object.getPrototypeOf(object) === Object.prototype; // true

// Array literals automatically have `Array.prototype` as their `[[Prototype]]`
const array = [1, 2, 3];
Object.getPrototypeOf(array) === Array.prototype; // true

// RegExp literals automatically have `RegExp.prototype` as their `[[Prototype]]`
const regexp = /abc/;
Object.getPrototypeOf(regexp) === RegExp.prototype; // true
```

We can "de-sugar" them into their constructor form.

```js
const array = new Array(1, 2, 3);
const regexp = new RegExp("abc");
```

For example, "array methods" like `map()` are simply methods defined on `Array.prototype`, which is why they are automatically available on all array instances.

> **Warning:** There is one misfeature that used to be prevalent — extending `Object.prototype` or one of the other built-in prototypes. An example of this misfeature is, defining `Array.prototype.myMethod = function () {...}` and then using `myMethod` on all array instances.
>
> This misfeature is called *monkey patching*. Doing monkey patching risks forward compatibility, because if the language adds this method in the future but with a different signature, your code will break. It has led to incidents like the SmooshGate, and can be a great nuisance for the language to advance since JavaScript tries to "not break the web".
>
> The **only** good reason for extending a built-in prototype is to backport the features of newer JavaScript engines, like `Array.prototype.forEach`.

It may be interesting to note that due to historical reasons, some built-in constructors' `prototype` property are instances themselves. For example, `Number.prototype` is a number 0, `Array.prototype` is an empty array, and `RegExp.prototype` is `/(?:)/`.

```js
Number.prototype + 1; // 1
Array.prototype.map((x) => x + 1); // []
String.prototype + "a"; // "a"
RegExp.prototype.source; // "(?:)"
Function.prototype(); // Function.prototype is a no-op function by itself
```

However, this is not the case for user-defined constructors, nor for modern constructors like `Map`.

```js
Map.prototype.get(1);
// Uncaught TypeError: get method called on incompatible Map.prototype
```

Building longer inheritance chains

The `Constructor.prototype` property will become the `[[Prototype]]` of the constructor's instances, as-is — including `Constructor.prototype`'s own `[[Prototype]]`. By default, `Constructor.prototype` is a *plain object* — that is, `Object.getPrototypeOf(Constructor.prototype) === Object.prototype`. The

only exception is `Object.prototype` itself, whose `[[Prototype]]` is `null` — that is, `Object.getPrototypeOf(Object.prototype) === null`. Therefore, a typical constructor will build the following prototype chain:

js

```
function Constructor() {}

const obj = new Constructor();
// obj ---> Constructor.prototype ---> Object.prototype ---> null
```

To build longer prototype chains, we can set the `[[Prototype]]` of `Constructor.prototype` via the `Object.setPrototypeOf()` function.

js

```
function Base() {}
function Derived() {}
// Set the `[[Prototype]]` of `Derived.prototype`
// to `Base.prototype`
Object.setPrototypeOf(Derived.prototype, Base.prototype);

const obj = new Derived();
// obj ---> Derived.prototype ---> Base.prototype ---> Object.prototype ---> null
```

In class terms, this is equivalent to using the `extends` syntax.

js

```
class Base {}
class Derived extends Base {}

const obj = new Derived();
// obj ---> Derived.prototype ---> Base.prototype ---> Object.prototype ---> null
```

You may also see some legacy code using `Object.create()` to build the inheritance chain. However, because this reassigns the `prototype` property and removes the `constructor` property, it can be more error-prone, while performance gains may not be apparent if the constructors haven't created any instances yet.

js · ✗ bad example

```
function Base() {}
function Derived() {}
// Re-assigns `Derived.prototype` to a new object
// with `Base.prototype` as its `[[Prototype]]`
// DON'T DO THIS — use Object.setPrototypeOf to mutate it instead
Derived.prototype = Object.create(Base.prototype);
```

Inspecting prototypes: a deeper dive

Let's look at what happens behind the scenes in a bit more detail.

In JavaScript, as mentioned above, functions are able to have properties. All functions have a special property named `prototype`. Please note that the code below is free-standing (it is safe to assume there is no other JavaScript on the webpage other than the below code). For the best learning experience, it is highly recommended that you open a console, navigate to the "console" tab, copy-and-paste in the below JavaScript code, and run it by pressing the Enter/Return key. (The console is included in most web browser's Developer Tools. More information is available for Firefox Developer Tools, Chrome DevTools, and Edge DevTools.)

```js
function doSomething() {}
console.log(doSomething.prototype);
// It does not matter how you declare the function; a
// function in JavaScript will always have a default
// prototype property — with one exception: an arrow
// function doesn't have a default prototype property:
const doSomethingFromArrowFunction = () => {};
console.log(doSomethingFromArrowFunction.prototype);
```

As seen above, `doSomething()` has a default `prototype` property, as demonstrated by the console. After running this code, the console should have displayed an object that looks similar to this.

```
{
  constructor: ƒ doSomething(),
  [[Prototype]]: {
    constructor: ƒ Object(),
    hasOwnProperty: ƒ hasOwnProperty(),
    isPrototypeOf: ƒ isPrototypeOf(),
    propertyIsEnumerable: ƒ propertyIsEnumerable(),
    toLocaleString: ƒ toLocaleString(),
    toString: ƒ toString(),
    valueOf: ƒ valueOf()
  }
}
```

> **Note:** The Chrome console uses `[[Prototype]]` to denote the object's prototype, following the spec's terms; Firefox uses `<prototype>`. For consistency we will use `[[Prototype]]`.

We can add properties to the prototype of `doSomething()`, as shown below.

```js
function doSomething() {}
doSomething.prototype.foo = "bar";
console.log(doSomething.prototype);
```

This results in:

```
{
  foo: "bar",
  constructor: ƒ doSomething(),
  [[Prototype]]: {
    constructor: ƒ Object(),
    hasOwnProperty: ƒ hasOwnProperty(),
    isPrototypeOf: ƒ isPrototypeOf(),
    propertyIsEnumerable: ƒ propertyIsEnumerable(),
    toLocaleString: ƒ toLocaleString(),
    toString: ƒ toString(),
    valueOf: ƒ valueOf()
  }
}
```

We can now use the `new` operator to create an instance of `doSomething()` based on this prototype. To use the new operator, call the function normally except prefix it with `new`. Calling a function with the `new` operator returns an object that is an instance of the function. Properties can then be added onto this object.

Try the following code:

```js
function doSomething() {}
doSomething.prototype.foo = "bar"; // add a property onto the prototype
const doSomeInstancing = new doSomething();
doSomeInstancing.prop = "some value"; // add a property onto the object
console.log(doSomeInstancing);
```

This results in an output similar to the following:

```
{
  prop: "some value",
  [[Prototype]]: {
    foo: "bar",
    constructor: ƒ doSomething(),
    [[Prototype]]: {
      constructor: ƒ Object(),
      hasOwnProperty: ƒ hasOwnProperty(),
      isPrototypeOf: ƒ isPrototypeOf(),
      propertyIsEnumerable: ƒ propertyIsEnumerable(),
      toLocaleString: ƒ toLocaleString(),
      toString: ƒ toString(),
      valueOf: ƒ valueOf()
    }
  }
}
```

As seen above, the `[[Prototype]]` of `doSomeInstancing` is `doSomething.prototype`. But, what does this do? When you access a property of `doSomeInstancing`, the runtime first looks to see if `doSomeInstancing` has that property.

If `doSomeInstancing` does not have the property, then the runtime looks for the property in `doSomeInstancing.[[Prototype]]` (a.k.a. `doSomething.prototype`). If `doSomeInstancing.[[Prototype]]` has the property being looked for, then that property on `doSomeInstancing.[[Prototype]]` is used.

Otherwise, if `doSomeInstancing.[[Prototype]]` does not have the property, then `doSomeInstancing.[[Prototype]].[[Prototype]]` is checked for the property. By default, the `[[Prototype]]` of any function's `prototype` property is `Object.prototype`. So, `doSomeInstancing.[[Prototype]].[[Prototype]]` (a.k.a. `doSomething.prototype.[[Prototype]]` (a.k.a. `Object.prototype`)) is then looked through for the property being searched for.

If the property is not found in `doSomeInstancing.[[Prototype]].[[Prototype]]`, then `doSomeInstancing.[[Prototype]].[[Prototype]].[[Prototype]]` is looked through. However, there is a problem: `doSomeInstancing.[[Prototype]].[[Prototype]].[[Prototype]]` does not exist, because `Object.prototype.[[Prototype]]` is `null`. Then, and only then, after the entire prototype chain of `[[Prototype]]`'s is looked through, the runtime asserts that the property does not exist and conclude that the value at the property is `undefined`.

Let's try entering some more code into the console:

```js
function doSomething() {}
doSomething.prototype.foo = "bar";
const doSomeInstancing = new doSomething();
doSomeInstancing.prop = "some value";
console.log("doSomeInstancing.prop:     ", doSomeInstancing.prop);
console.log("doSomeInstancing.foo:      ", doSomeInstancing.foo);
console.log("doSomething.prop:          ", doSomething.prop);
console.log("doSomething.foo:           ", doSomething.foo);
console.log("doSomething.prototype.prop:", doSomething.prototype.prop);
console.log("doSomething.prototype.foo: ", doSomething.prototype.foo);
```

This results in the following:

```
doSomeInstancing.prop:      some value
doSomeInstancing.foo:       bar
doSomething.prop:           undefined
doSomething.foo:            undefined
doSomething.prototype.prop: undefined
doSomething.prototype.foo:  bar
```

Different ways of creating and mutating prototype chains

We have encountered many ways to create objects and change their prototype chains. We will systematically summarize the different ways, comparing each approach's pros and cons.

Objects created with syntax constructs

```js
const o = { a: 1 };
// The newly created object o has Object.prototype as its [[Prototype]]
// Object.prototype has null as its [[Prototype]].
// o ---> Object.prototype ---> null

const b = ["yo", "sup", "?"];
// Arrays inherit from Array.prototype
// (which has methods indexOf, forEach, etc.)
// The prototype chain looks like:
// b ---> Array.prototype ---> Object.prototype ---> null

function f() {
  return 2;
}
// Functions inherit from Function.prototype
// (which has methods call, bind, etc.)
// f ---> Function.prototype ---> Object.prototype ---> null

const p = { b: 2, __proto__: o };
// It is possible to point the newly created object's [[Prototype]] to
// another object via the __proto__ literal property. (Not to be confused
// with Object.prototype.__proto__ accessors)
// p ---> o ---> Object.prototype ---> null
```

When using the `__proto__` key in object initializers, pointing the `__proto__` key to something that is not an object only fails silently without throwing an exception. Contrary to the `Object.prototype.__proto__` setter, `__proto__` in object literal initializers is standardized and

optimized, and can even be more performant than `Object.create`. Declaring extra own properties on the object at creation is more ergonomic than `Object.create`.

With constructor functions

```js
function Graph() {
  this.vertices = [];
  this.edges = [];
}

Graph.prototype.addVertex = function (v) {
  this.vertices.push(v);
};

const g = new Graph();
// g is an object with own properties 'vertices' and 'edges'.
// g.[[Prototype]] is the value of Graph.prototype when new Graph() is executed.
```

Constructor functions have been available since very early JavaScript. Therefore, it is very fast, very standard, and very JIT-optimizable. However, it's also hard to "do properly" because methods added this way are enumerable by default, which is inconsistent with the class syntax or how built-in methods behave. Doing longer inheritance chains is also error-prone, as previously demonstrated.

With Object.create()

Calling `Object.create()` creates a new object. The `[[Prototype]]` of this object is the first argument of the function:

```js
const a = { a: 1 };
// a ---> Object.prototype ---> null

const b = Object.create(a);
// b ---> a ---> Object.prototype ---> null
console.log(b.a); // 1 (inherited)

const c = Object.create(b);
// c ---> b ---> a ---> Object.prototype ---> null

const d = Object.create(null);
// d ---> null (d is an object that has null directly as its prototype)
console.log(d.hasOwnProperty);
// undefined, because d doesn't inherit from Object.prototype
```

Similar to the `__proto__` key in object initializers, `Object.create()` allows directly setting the prototype of an object at creation time, which permits the runtime to further optimize the object. It also allows the creation of objects with `null` prototype, using `Object.create(null)`. The second parameter of `Object.create()` allows you to precisely specify the attributes of each property in the new object, which can be a double-edged sword:

- It allows you to create non-enumerable properties, etc., during object creation, which is not possible with object literals.
- It is much more verbose and error-prone than object literals.
- It may be slower than object literals, especially when creating many properties.

With classes

```js
class Rectangle {
  constructor(height, width) {
    this.name = "Rectangle";
    this.height = height;
    this.width = width;
  }
}

class FilledRectangle extends Rectangle {
  constructor(height, width, color) {
    super(height, width);
    this.name = "Filled rectangle";
    this.color = color;
  }
}

const filledRectangle = new FilledRectangle(5, 10, "blue");
// filledRectangle ---> FilledRectangle.prototype ---> Rectangle.prototype --->
Object.prototype ---> null
```

Classes offer the highest readability and maintainability when defining complex inheritance structures. Private elements are a feature with no trivial replacement in prototypal inheritance. However, classes are less optimized than traditional constructor functions and are not supported in older environments.

With Object.setPrototypeOf()

While all methods above will set the prototype chain at object creation time, `Object.setPrototypeOf()` allows mutating the `[[Prototype]]` internal property of an existing object. It can even force a prototype on a prototype-less object created with `Object.create(null)` or remove the prototype of an object by setting it to `null`.

```js
const obj = { a: 1 };
const anotherObj = { b: 2 };
Object.setPrototypeOf(obj, anotherObj);
// obj ---> anotherObj ---> Object.prototype ---> null
```

However, you should set the prototype during creation if possible, because setting the prototype dynamically disrupts all optimizations that engines have made to the prototype chain. It might cause some engines to recompile your code for de-optimization, to make it work according to the specs.

With the __proto__ accessor

All objects inherit the `Object.prototype.__proto__` setter, which can be used to set the `[[Prototype]]` of an existing object (if the `__proto__` key is not overridden on the object).

> **Warning:** `Object.prototype.__proto__` accessors are **non-standard** and deprecated. You should almost always use `Object.setPrototypeOf` instead.

```js
const obj = {};
// DON'T USE THIS: for example only.
obj.__proto__ = { barProp: "bar val" };
obj.__proto__.__proto__ = { fooProp: "foo val" };
console.log(obj.fooProp);
console.log(obj.barProp);
```

Compared to `Object.setPrototypeOf`, setting `__proto__` to something that is not an object fails silently without throwing an exception. It also has slightly better browser support. However, it is non-standard and deprecated. You should almost always use `Object.setPrototypeOf` instead.

Performance

The lookup time for properties that are high up on the prototype chain can have a negative impact on the performance, and this may be significant in the code where performance is critical. Additionally, trying to access nonexistent properties will always traverse the full prototype chain.

Also, when iterating over the properties of an object, **every** enumerable property that is on the prototype chain will be enumerated. To check whether an object has a property defined on *itself* and not somewhere on its prototype chain, it is necessary to use the `hasOwnProperty` or `Object.hasOwn` methods. All objects, except those with `null` as `[[Prototype]]`, inherit `hasOwnProperty` from `Object.prototype` — unless it has been overridden further down the prototype chain. To give you a concrete example, let's take the above graph example code to illustrate it:

```js
function Graph() {
  this.vertices = [];
  this.edges = [];
}

Graph.prototype.addVertex = function (v) {
  this.vertices.push(v);
};

const g = new Graph();
// g ---> Graph.prototype ---> Object.prototype ---> null

g.hasOwnProperty("vertices"); // true
Object.hasOwn(g, "vertices"); // true

g.hasOwnProperty("nope"); // false
Object.hasOwn(g, "nope"); // false

g.hasOwnProperty("addVertex"); // false
Object.hasOwn(g, "addVertex"); // false

Object.getPrototypeOf(g).hasOwnProperty("addVertex"); // true
```

Note: It is **not** enough to check whether a property is `undefined`. The property might very well exist, but its value just happens to be set to `undefined`.

Conclusion

JavaScript may be a bit confusing for developers coming from Java or C++, as it's all dynamic, all runtime, and it has no static types at all. Everything is either an object (instance) or a function (constructor), and even functions themselves are instances of the `Function` constructor. Even the "classes" as syntax constructs are just constructor functions at runtime.

All constructor functions in JavaScript have a special property called `prototype`, which works with the `new` operator. The reference to the prototype object is copied to the internal `[[Prototype]]` property of the new instance. For example, when you do `const a1 = new A()`, JavaScript (after creating the object in memory and before running function `A()` with `this` defined to it) sets `a1.[[Prototype]] = A.prototype`. When you then access properties of the instance, JavaScript first checks whether they exist on that object directly, and if not, it looks in `[[Prototype]]`. `[[Prototype]]` is looked at *recursively*, i.e., `a1.doSomething`, `Object.getPrototypeOf(a1).doSomething`, `Object.getPrototypeOf(Object.getPrototypeOf(a1)).doSomething` etc., until it's found or `Object.getPrototypeOf` returns `null`. This means that all properties defined on `prototype` are effectively shared by all instances, and you can even later change parts of `prototype` and have the changes appear in all existing instances.

If, in the example above, you do `const a1 = new A(); const a2 = new A();`, then `a1.doSomething` would actually refer to `Object.getPrototypeOf(a1).doSomething` — which is the same as the `A.prototype.doSomething` you defined, i.e., `Object.getPrototypeOf(a1).doSomething === Object.getPrototypeOf(a2).doSomething === A.prototype.doSomething`.

It is essential to understand the prototypal inheritance model before writing complex code that makes use of it. Also, be aware of the length of the prototype chains in your code and break them up if necessary to avoid possible performance problems. Further, the native prototypes should **never** be extended unless it is for the sake of compatibility with newer JavaScript features.

JavaScript › Guide

Meta programming

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Meta_programming · MDN page last modified 2026-05-22

The `Proxy` and `Reflect` objects allow you to intercept and define custom behavior for fundamental language operations (e.g., property lookup, assignment, enumeration, function invocation, etc.). With the help of these two objects you are able to program at the meta level of JavaScript.

Proxies

`Proxy` objects allow you to intercept certain operations and to implement custom behaviors.

For example, getting a property on an object:

```js
const handler = {
  get(target, name) {
    return name in target ? target[name] : 42;
  },
};

const p = new Proxy({}, handler);
p.a = 1;
console.log(p.a, p.b); // 1, 42
```

The `Proxy` object defines a `target` (an empty object here) and a `handler` object, in which a `get` *trap* is implemented. Here, an object that is proxied will not return `undefined` when getting undefined properties, but will instead return the number `42`.

Additional examples are available on the `Proxy` reference page.

Terminology

The following terms are used when talking about the functionality of proxies.

handler
: Placeholder object which contains traps.

traps
: The methods that provide property access. (This is analogous to the concept of *traps* in operating systems.)

target
: Object which the proxy virtualizes. It is often used as storage backend for the proxy. Invariants (semantics that remain unchanged) regarding object non-extensibility or non-configurable properties are verified against the target.

invariants
: Semantics that remain unchanged when implementing custom operations are called *invariants*. If you violate the invariants of a handler, a `TypeError` will be thrown.

Handlers and traps

The following table summarizes the available traps available to `Proxy` objects. See the reference pages for detailed explanations and examples.

Handler / trap	Interceptions
handler.getPrototypeOf()	Object.getPrototypeOf() Reflect.getPrototypeOf() __proto__ Object.prototype.isPrototypeOf() instanceof
handler.setPrototypeOf()	Object.setPrototypeOf() Reflect.setPrototypeOf()
handler.isExtensible()	Object.isExtensible() Reflect.isExtensible()
handler.preventExtensions()	Object.preventExtensions() Reflect.preventExtensions()
handler.getOwnPropertyDescriptor()	Object.getOwnPropertyDescriptor() Reflect.getOwnPropertyDescriptor()
handler.defineProperty()	Object.defineProperty() Reflect.defineProperty()
handler.has()	**Property query** foo in proxy **Inherited property query** foo in Object.create(*proxy*) Reflect.has()
handler.get()	**Property access** *proxy*[foo] *proxy*.bar **Inherited property access** Object.create(*proxy*)[foo] Reflect.get()
handler.set()	**Property assignment** *proxy*[foo] = bar *proxy*.foo = bar **Inherited property assignment** Object.create(*proxy*)[foo] = bar Reflect.set()
handler.deleteProperty()	**Property deletion** delete *proxy*[foo] delete *proxy*.foo Reflect.deleteProperty()
handler.ownKeys()	Object.getOwnPropertyNames() Object.getOwnPropertySymbols() Object.keys() Reflect.ownKeys()
handler.apply()	proxy(..args) Function.prototype.apply() and Function.prototype.call() Reflect.apply()
handler.construct()	new proxy(...args) Reflect.construct()

Revocable `Proxy`

The `Proxy.revocable()` method is used to create a revocable `Proxy` object. This means that the proxy can be revoked via the function `revoke` and switches the proxy off.

Afterwards, any operation on the proxy leads to a `TypeError`.

js

```
const revocable = Proxy.revocable(
  {},
  {
    get(target, name) {
      return `[[${name}]]`;
    },
  },
);
const proxy = revocable.proxy;
console.log(proxy.foo); // "[[foo]]"

revocable.revoke();

console.log(proxy.foo); // TypeError: Cannot perform 'get' on a proxy that has been revoked
proxy.foo = 1; // TypeError: Cannot perform 'set' on a proxy that has been revoked
delete proxy.foo; // TypeError: Cannot perform 'deleteProperty' on a proxy that has been
revoked
console.log(typeof proxy); // "object", typeof doesn't trigger any trap
```

Reflection

`Reflect` is a built-in object that provides methods for interceptable JavaScript operations. The methods are the same as those of the proxy handler's.

`Reflect` is not a function object.

`Reflect` helps with forwarding default operations from the handler to the `target`.

With `Reflect.has()` for example, you get the `in` operator as a function:

js

```
Reflect.has(Object, "assign"); // true
```

A better apply() function

Before `Reflect`, you typically use the `Function.prototype.apply()` method to call a function with a given `this` value and `arguments` provided as an array (or an array-like object).

js

```
Function.prototype.apply.call(Math.floor, undefined, [1.75]);
```

With `Reflect.apply` this becomes less verbose and easier to understand:

```js
Reflect.apply(Math.floor, undefined, [1.75]);
// 1

Reflect.apply(String.fromCharCode, undefined, [104, 101, 108, 108, 111]);
// "hello"

Reflect.apply(RegExp.prototype.exec, /ab/, ["confabulation"]).index;
// 4

Reflect.apply("".charAt, "ponies", [3]);
// "i"
```

Checking if property definition has been successful

With `Object.defineProperty`, which returns an object if successful, or throws a `TypeError` otherwise, you would use a `try...catch` block to catch any error that occurred while defining a property. Because `Reflect.defineProperty()` returns a Boolean success status, you can just use an `if...else` block here:

```js
if (Reflect.defineProperty(target, property, attributes)) {
  // success
} else {
  // failure
}
```

JavaScript › Guide

Memory management

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Memory_management · MDN page last modified 2026-10-23

Low-level languages like C, have manual memory management primitives such as `malloc()` and `free()`. In contrast, JavaScript automatically allocates memory when objects are created and frees it when they are not used anymore (*garbage collection*). This automaticity is a potential source of confusion: it can give developers the false impression that they don't need to worry about memory management.

Memory life cycle

Regardless of the programming language, the memory life cycle is pretty much always the same:

1. Allocate the memory you need
2. Use the allocated memory (read, write)
3. Release the allocated memory when it is not needed anymore

The second part is explicit in all languages. The first and last parts are explicit in low-level languages but are mostly implicit in high-level languages like JavaScript.

Allocation in JavaScript

Value initialization

In order to not bother the programmer with allocations, JavaScript will automatically allocate memory when values are initially declared.

```js
const n = 123; // allocates memory for a number
const s = "string"; // allocates memory for a string

const o = {
  a: 1,
  b: null,
}; // allocates memory for an object and contained values

// (like object) allocates memory for the array and
// contained values
const a = [1, null, "str2"];

function f(a) {
  return a + 2;
} // allocates a function (which is a callable object)

// function expressions also allocate an object
someElement.addEventListener("click", () => {
  someElement.style.backgroundColor = "blue";
});
```

Allocation via function calls

Some function calls result in object allocation.

```js
const d = new Date(); // allocates a Date object

const e = document.createElement("div"); // allocates a DOM element
```

Some methods allocate new values or objects:

```js
const s = "string";
const s2 = s.substring(0, 3); // s2 is a new string
// Since strings are immutable values,
// JavaScript may decide to not allocate memory,
// but just store the [0, 3] range.

const a = ["yeah yeah", "no no"];
const a2 = ["generation", "no no"];
const a3 = a.concat(a2);
// new array with 4 elements being
// the concatenation of a and a2 elements.
```

Using values

Using values basically means reading and writing in allocated memory. This can be done by reading or writing the value of a variable or an object property or even passing an argument to a function.

Release when the memory is not needed anymore

The majority of memory management issues occur at this phase. The most difficult aspect of this stage is determining when the allocated memory is no longer needed.

Low-level languages require the developer to manually determine at which point in the program the allocated memory is no longer needed and to release it.

Some high-level languages, such as JavaScript, utilize a form of automatic memory management known as garbage collection (GC). The purpose of a garbage collector is to monitor memory allocation and determine when a block of allocated memory is no longer needed and reclaim it. This automatic process is an approximation since the general problem of determining whether or not a specific piece of memory is still needed is undecidable.

Garbage collection

As stated above, the general problem of automatically finding whether some memory "is not needed anymore" is undecidable. As a consequence, garbage collectors implement a restriction of a solution to the general problem. This section will explain the concepts that are necessary for understanding the main garbage collection algorithms and their respective limitations.

References

The main concept that garbage collection algorithms rely on is the concept of *reference*. Within the context of memory management, an object is said to reference another object if the former has access to the latter (either implicitly or explicitly). For instance, a JavaScript object has a reference to its prototype (implicit reference) and to its properties values (explicit reference).

In this context, the notion of an "object" is extended to something broader than regular JavaScript objects and also contain function scopes (or the global lexical scope).

Reference-counting garbage collection

> **Note:** No modern JavaScript engine uses reference-counting for garbage collection anymore.

This is the most naïve garbage collection algorithm. This algorithm reduces the problem from determining whether or not an object is still needed to determining if an object still has any other objects referencing it. An object is said to be "garbage", or collectible if there are zero references pointing to it.

For example:

```js
let x = {
  a: {
    b: 2,
  },
};
// 2 objects are created. One is referenced by the other as one of its properties.
// The other is referenced by virtue of being assigned to the 'x' variable.
// Obviously, none can be garbage-collected.

let y = x;
// The 'y' variable is the second thing that has a reference to the object.

x = 1;
// Now, the object that was originally in 'x' has a unique reference
// embodied by the 'y' variable.

let z = y.a;
// Reference to 'a' property of the object.
// This object now has 2 references: one as a property,
// the other as the 'z' variable.

y = "mozilla";
// The object that was originally in 'x' has now zero
// references to it. It can be garbage-collected.
// However its 'a' property is still referenced by
// the 'z' variable, so it cannot be freed.

z = null;
// The 'a' property of the object originally in x
// has zero references to it. It can be garbage collected.
```

There is a limitation when it comes to circular references. In the following example, two objects are created with properties that reference one another, thus creating a cycle. They will go out of scope after the function call has completed. At that point they become unneeded and their allocated memory should be reclaimed. However, the reference-counting algorithm will not consider them reclaimable since each of the two objects has at least one reference pointing to them, resulting in neither of them being marked for garbage collection. Circular references are a common cause of memory leaks.

```js
function f() {
  const x = {};
  const y = {};
  x.a = y; // x references y
  y.a = x; // y references x

  return "azerty";
}

f();
```

Mark-and-sweep algorithm

This algorithm reduces the definition of "an object is no longer needed" to "an object is unreachable".

This algorithm assumes the knowledge of a set of objects called *roots.* In JavaScript, the root is the global object. Periodically, the garbage collector will start from these roots, find all objects that are referenced from these roots, then all objects referenced from these, etc. Starting from the roots, the garbage collector will thus find all *reachable* objects and collect all non-reachable objects.

This algorithm is an improvement over the previous one since an object having zero references is effectively unreachable. The opposite does not hold true as we have seen with circular references.

Currently, all modern engines ship a mark-and-sweep garbage collector. All improvements made in the field of JavaScript garbage collection (generational/incremental/concurrent/parallel garbage collection) over the last few years are implementation improvements of this algorithm, but not improvements over the garbage collection algorithm itself nor its reduction of the definition of when "an object is no longer needed".

The immediate benefit of this approach is that cycles are no longer a problem. In the first example above, after the function call returns, the two objects are no longer referenced by any resource that is reachable from the global object. Consequently, they will be found unreachable by the garbage collector and have their allocated memory reclaimed.

However, the inability to manually control garbage collection remains. There are times when it would be convenient to manually decide when and what memory is released. In order to release the memory of an object, it needs to be made explicitly unreachable. It is also not possible to programmatically trigger garbage collection in JavaScript — and will likely never be within the core language, although engines may expose APIs behind opt-in flags.

Configuring an engine's memory model

JavaScript engines typically offer flags that expose the memory model. For example, Node.js offers additional options and tools that expose the underlying V8 mechanisms for configuring and debugging memory issues. This configuration may not be available in browsers, and even less so for web pages (via HTTP headers, etc.).

The max amount of available heap memory can be increased with a flag:

bash

```
node --max-old-space-size=6000 index.js
```

We can also expose the garbage collector for debugging memory issues using a flag and the Chrome Debugger:

bash

```
node --expose-gc --inspect index.js
```

Data structures aiding memory management

Although JavaScript does not directly expose the garbage collector API, the language offers several data structures that indirectly observe garbage collection and can be used to manage memory usage.

WeakMaps and WeakSets

`WeakMap` and `WeakSet` are data structures whose APIs closely mirror their non-weak counterparts: `Map` and `Set`. `WeakMap` allows you to maintain a collection of key-value pairs, while `WeakSet` allows you to maintain a collection of unique values, both with performant addition, deletion, and querying.

`WeakMap` and `WeakSet` got the name from the concept of *weakly held* values. If `x` is weakly held by `y`, it means that although you can access the value of `x` via `y`, the mark-and-sweep algorithm won't consider `x` as reachable if nothing else *strongly holds* to it. Most data structures, except the ones discussed here, strongly hold to the objects passed in so that you can retrieve them at any time. The keys of `WeakMap` and `WeakSet` can be garbage-collected (for `WeakMap` objects, the values would then be eligible for garbage collection as well) as long as nothing else in the program is referencing the key. This is ensured by two characteristics:

- `WeakMap` and `WeakSet` can only store objects or symbols. This is because only objects are garbage collected — primitive values can always be forged (that is, `1 === 1` but `{} !== {}`), making them stay in the collection forever. Registered symbols (like `Symbol.for("key")`) can also be forged and thus are not garbage collectable, but symbols created with `Symbol("key")` are garbage collectable. Well-known symbols like `Symbol.iterator` come in a fixed set and are unique throughout the lifetime of the program, similar to intrinsic objects such as `Array.prototype`, so they are also allowed as keys.
- `WeakMap` and `WeakSet` are not iterable. This prevents you from using `Array.from(map.keys()).length` to observe the liveliness of objects, or get hold of an arbitrary key which should otherwise be eligible for garbage collection. (Garbage collection should be as invisible as possible.)

In typical explanations of `WeakMap` and `WeakSet` (such as the one above), it's often implied that the key is garbage-collected first, freeing the value for garbage collection as well. However, consider the case of the value referencing the key:

```js
const wm = new WeakMap();
const key = {};
wm.set(key, { key });
// Now `key` cannot be garbage collected,
// because the value holds a reference to the key,
// and the value is strongly held in the map!
```

If `key` is stored as an actual reference, it would create a cyclic reference and make both the key and value ineligible for garbage collection, even when nothing else references `key` — because if `key` is garbage collected, it means that at some particular instant, `value.key` would point to a non-existent address, which is not legal. To fix this, the entries of `WeakMap` and `WeakSet` aren't actual references, but ephemerons, an enhancement to the mark-and-sweep mechanism. Barros et al. offers a good summary of the algorithm (page 4). To quote a paragraph:

> Ephemerons are a refinement of weak pairs where neither the key nor the value can be classified as weak or strong. The connectivity of the key determines the connectivity of the value, but the connectivity of the value does not affect the connectivity of the key. [...] when the garbage collection offers support to ephemerons, it occurs in three phases instead of two (mark and sweep).

As a rough mental model, think of a `WeakMap` as the following implementation:

> **Warning:** This is not a polyfill, nor is it anywhere close to how it's implemented in the engine (which hooks into the garbage collection mechanism).

```js
class MyWeakMap {
  #marker = Symbol("MyWeakMapData");
  get(key) {
    return key[this.#marker];
  }
  set(key, value) {
    key[this.#marker] = value;
  }
  has(key) {
    return this.#marker in key;
  }
  delete(key) {
    delete key[this.#marker];
  }
}
```

As you can see, the `MyWeakMap` never actually holds a collection of keys. It simply adds metadata to each object being passed in. The object is then garbage-collectable via mark-and-sweep. Therefore, it's not possible to iterate over the keys in a `WeakMap`, nor clear the `WeakMap` (as that also relies on the knowledge of the entire key collection).

For more information on their APIs, see the keyed collections guide.

WeakRefs and FinalizationRegistry

> **Note:** `WeakRef` and `FinalizationRegistry` offer direct introspection into the garbage collection machinery. Avoid using them where possible because the runtime semantics are almost completely unguaranteed.

All variables with an object as value are references to that object. However, such references are *strong* — their existence would prevent the garbage collector from marking the object as eligible for collection. A `WeakRef` is a *weak reference* to an object that allows the object to be garbage collected, while still retaining the ability to read the object's content during its lifetime.

One use case for `WeakRef` is a cache system which maps string URLs to large objects. We cannot use a `WeakMap` for this purpose, because `WeakMap` objects have their *keys* weakly held, but not their *values* — if you access a key, you would always deterministically get the value (since having access to the key means it's still alive). Here, we are okay to get `undefined` for a key (if the corresponding value is no longer alive) since we can just re-compute it, but we don't want unreachable objects to stay in the cache. In this case, we can use a normal `Map`, but with each value being a `WeakRef` of the object instead of the actual object value.

```js
function cached(getter) {
  // A Map from string URLs to WeakRefs of results
  const cache = new Map();
  return async (key) => {
    if (cache.has(key)) {
      const dereferencedValue = cache.get(key).deref();
      if (dereferencedValue !== undefined) {
        return dereferencedValue;
```

(code continues on the next page)

(code continued from the previous page)

```
      }
    }
    const value = await getter(key);
    cache.set(key, new WeakRef(value));
    return value;
  };
}

const getImage = cached((url) => fetch(url).then((res) => res.blob()));
```

`FinalizationRegistry` provides an even stronger mechanism to observe garbage collection. It allows you to register objects and be notified when they are garbage collected. For example, for the cache system exemplified above, even when the blobs themselves are free for collection, the `WeakRef` objects that hold them are not — and over time, the `Map` may accumulate a lot of useless entries. Using a `FinalizationRegistry` allows one to perform cleanup in this case.

js

```
function cached(getter) {
  // A Map from string URLs to WeakRefs of results
  const cache = new Map();
  // After each value is garbage collected, the callback might
  // eventually be called with the key in the cache as argument,
  // allowing us to remove the cache entry
  const registry = new FinalizationRegistry((key) => {
    // Note: it's important to test that the WeakRef is indeed empty.
    // Otherwise, the callback may be called after a new object has been
    // added with this key, and that new, alive object gets deleted
    if (!cache.get(key)?.deref()) {
      cache.delete(key);
    }
  });
  return async (key) => {
    if (cache.has(key)) {
      // Note: there are no guarantees when (or even if) the registry's
      // callback will fire, so it is still possible to see a WeakRef in
      // the cache even if it is empty
      const dereferencedValue = cache.get(key).deref();
      if (dereferencedValue !== undefined) {
        return dereferencedValue;
      }
    }
    const value = await getter(key);
    cache.set(key, new WeakRef(value));
    registry.register(value, key);
    return value;
  };
}

const getImage = cached((url) => fetch(url).then((res) => res.blob()));
```

Due to performance and security concerns, there is no guarantee of when the callback will be called, or if it will be called at all. It should only be used for cleanup — and non-critical cleanup. There are other ways for more deterministic resource management, such as `try...finally`, which will always execute the `finally` block. `WeakRef` and `FinalizationRegistry` exist solely for optimization of memory usage in long-running programs.

For more information on the API of `WeakRef` and `FinalizationRegistry`, see their reference pages.

JavaScript › Guide

Agent communication and coordination

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Agent_communication_and_coordination · MDN page last modified 2026-10-28

Agents is the JavaScript formal terminology for describing autonomous JavaScript executors. Most JavaScript you write gets executed in a single thread, but you may create more threads with specific platform APIs such as web workers.

This guide explains general computer science concepts related to thread coordination and how JavaScript implements them. It also introduces the APIs for enabling cross-thread communication: `postMessage()`, `SharedArrayBuffer`, and `Atomics`.

It is recommended to understand Typed arrays before reading this. Afterwards, you can continue with the Execution model reference to dive into the technical details.

> **Note:** In this article, we will use "thread" and "agent" interchangeably. An agent is a JavaScript engine abstraction, while a thread is an implementation detail; an agent may or may not run in a separate operating system thread. Technically, we should have used "agents" everywhere because you can only interact with agents in JavaScript, but when talking about general computer science concepts like locks, it's much more natural to use "threads".
>
> Also, this guide has nothing to do with AI agents.

Communication between two agents

The JavaScript language spec does not provide APIs for creating agents, getting a reference to another agent, or exchanging data between them. Although the concept of "workers" exists in both the web and in Node.js, their APIs are largely distinct. This guide uses web workers.

Creating agents

A parent agent creates and holds a reference to a child agent using the `Worker()` constructor:

```js
// -- main.js --
const worker = new Worker("./worker.js", { type: "module" });
```

The parent agent can be a window or another worker. For example, to run this example in a browser, you can reference `main.js` in an HTML file:

```html
<script src="main.js" type="module"></script>
```

Communication via messages

Two agents communicate using *messages*. Messages are sent over *message channels*. The parent agent (`main.js` here) and its child agent (`worker.js` here) have a message channel by default, which routes to their default ports:

- `main.js` sends messages using `worker.postMessage()`, and receives messages using the `message` event on the `worker` object.
- `worker.js` sends messages using `self.postMessage()`, and receives messages using the `message` event on `self`.

However, this is limited to communication between the creator and the agent it creates. When you want any two agents to communicate, you need to use a dedicated `MessageChannel`. You pass one of its `MessagePort` objects to the worker over the default channel, and then exclusively use that port:

```js
// -- main.js --
const worker = new Worker("./worker.js", { type: "module" });
const { port1, port2 } = new MessageChannel();

// We'll explain the [port2] parameter soon
worker.postMessage({ hereIsYourPort: port2 }, [port2]);

// Mark the port as ready for receiving messages
port1.start();
```

```js
// -- worker.js --
// Use top-level await (only available inside modules) so code following it
// only evaluates when the port is received
const port = await new Promise((resolve, reject) => {
  self.addEventListener(
    "message",
    (e) => {
      const port = e.data.hereIsYourPort;
      if (!(port instanceof MessagePort)) {
        reject(new Error("I'm expecting a dedicated port"));
        return;
      }
      resolve(port);
    },
    { once: true },
  );
});

// Mark the port as ready for receiving messages
port.start();
```

> **Note:** In all subsequent examples, we'll assume that these lines exist at the top of `main.js` and `worker.js`. We won't repeat them.

Now, each agent can call `postMessage()` and listen for `message` events on the `MessagePort` object.

Using message channels, you can also connect two agents that aren't in a parent-child relationship, such as two workers created by the same thread:

```js
const { port1, port2 } = new MessageChannel();
const worker1 = new Worker("./worker1.js", { type: "module" });
const worker2 = new Worker("./worker2.js", { type: "module" });

worker1.postMessage({ hereIsYourPort: port1 }, [port1]);
worker2.postMessage({ hereIsYourPort: port2 }, [port2]);
```

As a practical example, we implement a number-guessing game. `main.js` defines a secret number, and `worker.js` tries to guess it. Each time, `main.js` tells `worker.js` whether the guess is correct, too low, or too high, and the worker adjusts its guess. Here's how `main.js` can receive guesses from the worker using `MessageEvent.data`, and provide feedback using `port1.postMessage()`:

```js
// -- main.js --
const secretNumber = 63;

port1.addEventListener("message", (e) => {
  const workerGuess = e.data;
  console.log("Worker guessed:", workerGuess);
  if (workerGuess < secretNumber) {
    port1.postMessage("Too low");
  } else if (workerGuess > secretNumber) {
    port1.postMessage("Too high");
  } else {
    port1.postMessage("Correct");
    console.log("Worker successfully guessed the secret number!");
  }
});
```

And here's how the worker can submit guesses to `main.js` and adjust based on the feedback from it:

```js
// -- worker.js --
let low = 0;
let high = 100;
let lastGuess = Math.floor((low + high) / 2);

// First guess submitted immediately after initialization
port.postMessage(lastGuess);
port.addEventListener("message", (e) => {
  const feedback = e.data;
  if (feedback === "Correct") {
    console.log("Yay");
    return;
  }
  if (feedback === "Too low") {
    low = lastGuess + 1;
  } else if (feedback === "Too high") {
    high = lastGuess - 1;
  }
  if (low > high) {
    throw new Error(
      "Sorry, the feedback from main.js is inconsistent and I suspect main is cheating",
    );
  }
  lastGuess = Math.floor((low + high) / 2);
  port.postMessage(lastGuess);
});

port.start();
```

If you run this in a browser, both the worker and the main thread will output to the same DevTools console, so you will see the output as:

```
Worker guessed: 50                              main.js
Worker guessed: 75                              main.js
Worker guessed: 62                              main.js
Worker guessed: 68                              main.js
Worker guessed: 65                              main.js
```

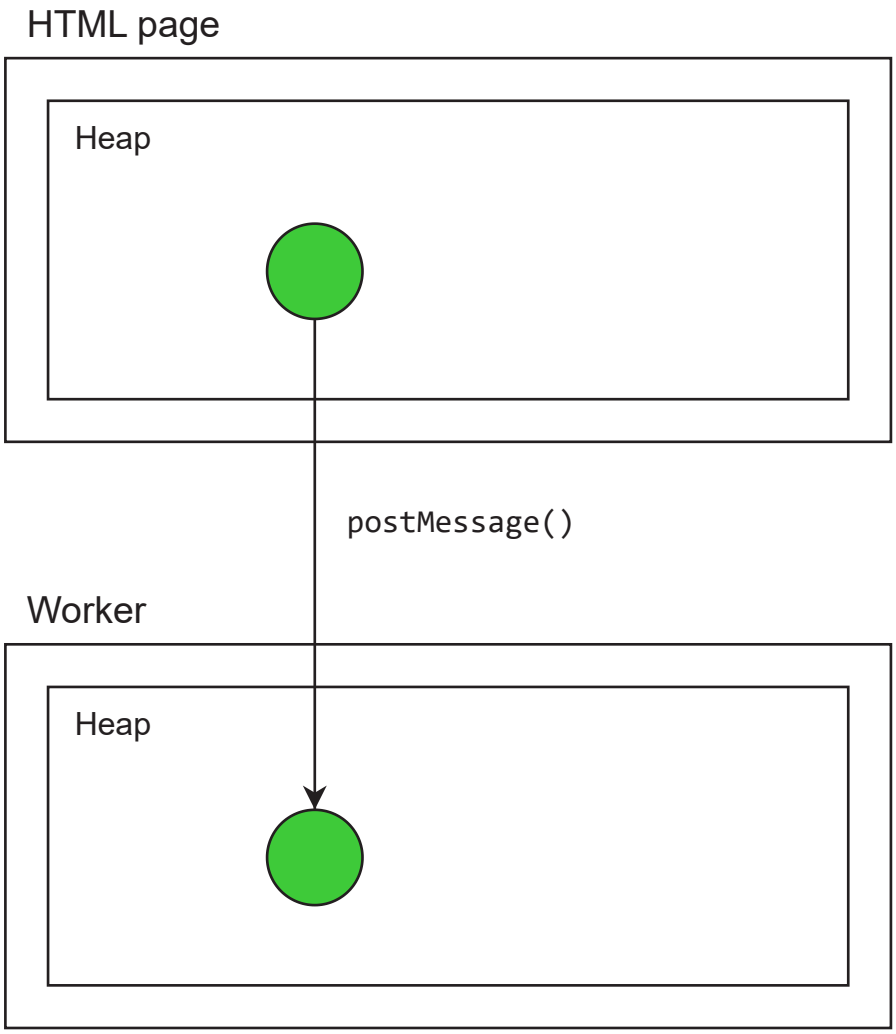
(code continues on the next page)

(code continued from the previous page)

```
Worker guessed: 63                                main.js
Worker successfully guessed the secret number!  main.js
Yay                                               worker.js
```

Sending messages by copying or transferring

You can pass many kinds of JavaScript values to `postMessage`. In the number-guessing example, we pass numbers for guesses, strings for feedback, and objects for the `MessagePort`. By default, each agent has its own memory space, so when an object is sent from one heap to another, a copy of it is created in the target heap. The copy uses the structured clone algorithm, which only supports a fixed set of data types. When you attempt to copy an unsupported object, a `DOMException` is thrown.

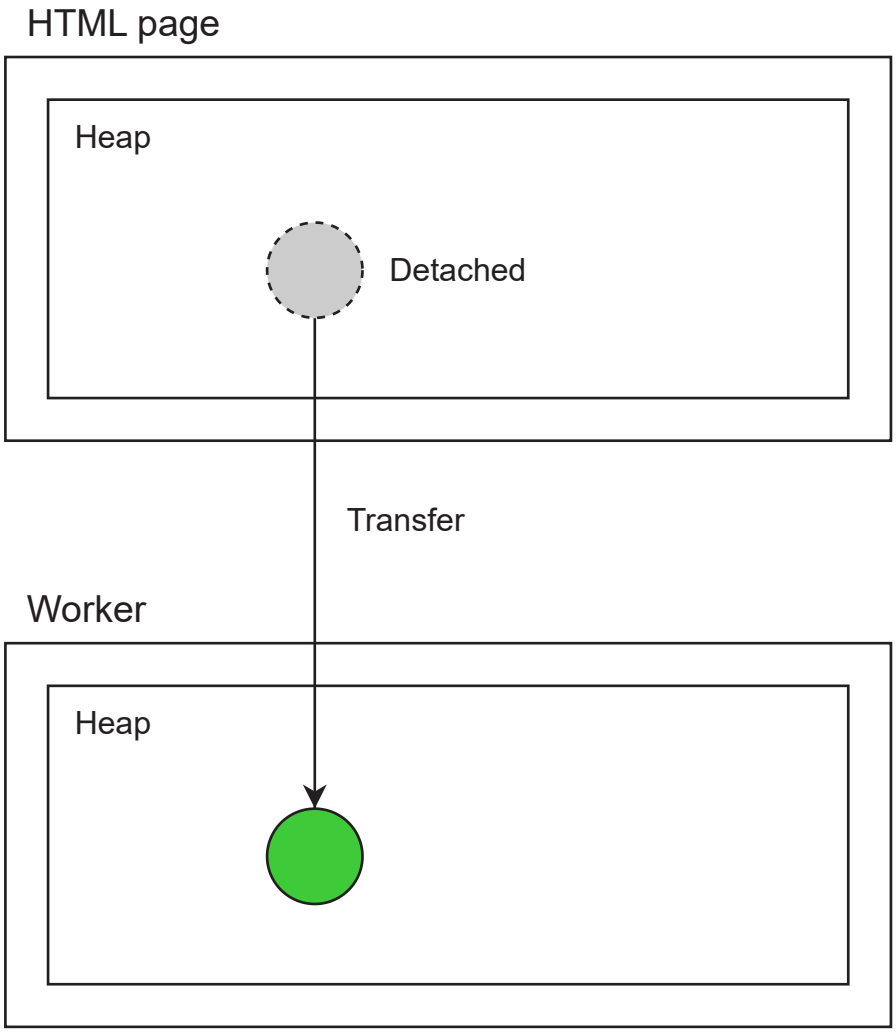


Copying an object allows the sender and the recipient to independently manipulate the object without affecting the other. The downside is that the object is re-constructed in the recipient's scope, which means:

- It's not tethered to the original creator, so `MessagePort` can't work (indeed, if you read the list of serializable objects, you can see that `MessagePort` is not listed here).
- There are two copies of the same data, which increases memory usage.

To solve these problems, instead of copying objects, you can pass them by *transferring* instead. Transferring an object uses a slightly different algorithm for constructing it in the target realm, and

marks it as *detached* in the source realm. The source object still exists, but its transferred resource is now available through the new object in the target realm:



To transfer an object, you still pass it (or other objects containing it) to `postMessage()`. The second parameter of `postMessage()` takes a list of objects inside the message that should be transferred instead of copied. This explains the following line in the number guessing game:

```js
worker.postMessage({ hereIsYourPort: port2 }, [port2]);
```

The whole object `{ hereIsYourPort: port2 }` is copied as usual and can be used in both agents, though it isn't assigned to a variable in the sender, so it will be immediately dropped over there. The `port2` resource is taken over by the new object in the worker agent, while the source instance is marked as detached. The effects of detachment depend on the object type, but generally operations either throw an error or do nothing. For a `MessagePort`, the original object loses its connection to the other port.

Other than `MessagePort`, another widely-used transferable object is `ArrayBuffer`, which happens to be both copyable and transferable. When an `ArrayBuffer` is copied, the source and recipient each hold an independent copy, and mutations on one side do not affect the other. When an `ArrayBuffer` is transferred, it is again marked as *detached*. A detached array buffer still exists as an object, but no longer provides access to the underlying memory. This means that:

- `byteLength` becomes `0` (in both the buffer and the associated typed array views).
- Methods, such as `resize()` and `slice()`, throw a `TypeError` when invoked. The associated typed array views' methods also throw a `TypeError`.

You can check whether an `ArrayBuffer` is detached via its `detached` property.

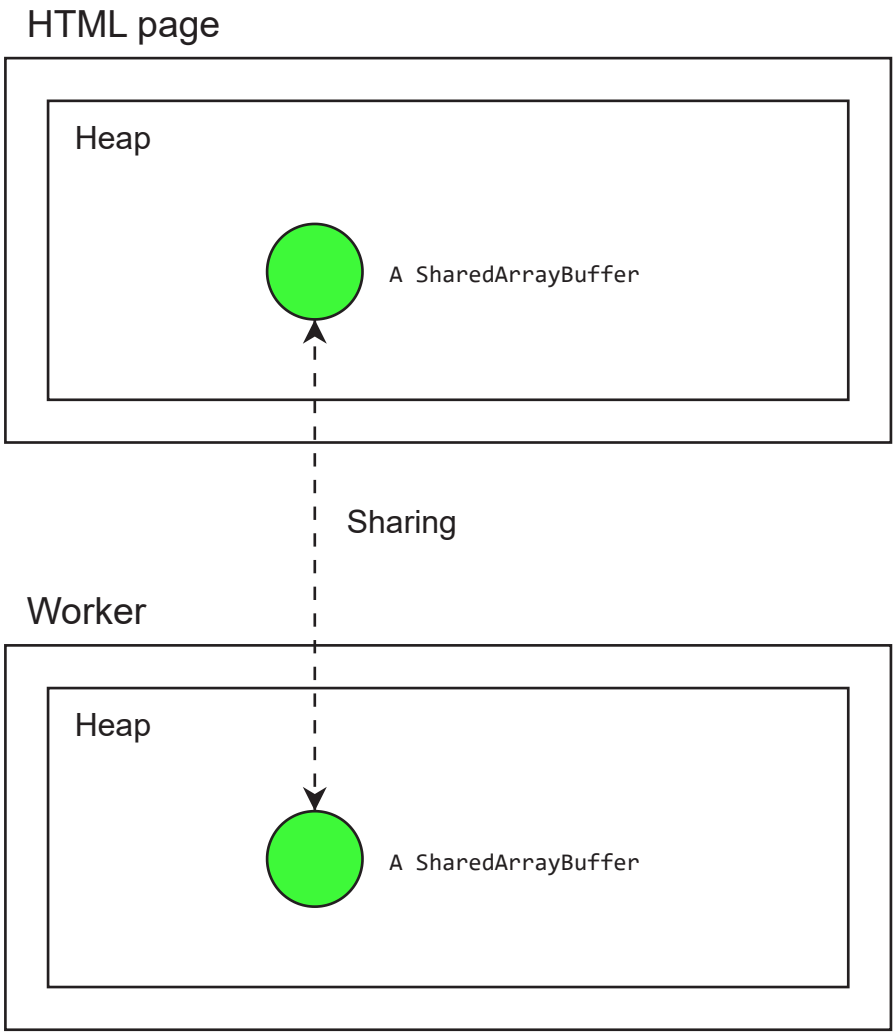
In pure JavaScript, you can also transfer the ownership of memory from one `ArrayBuffer` to another using its `transfer()` or `transferToFixedLength()` method.

```js
const buffer1 = new ArrayBuffer(1024);
console.log(buffer1.detached); // false
console.log(buffer1.byteLength); // 1024

const buffer2 = buffer1.transfer();
console.log(buffer1.detached); // true
console.log(buffer1.byteLength); // 0
console.log(buffer2.byteLength); // 1024
```

Sharing memory with SharedArrayBuffer

Other than copying and transferring, there's a third model of data passing: *sharing*. In sharing, both sides hold an object that shares the underlying memory. Memory sharing is available only to the `SharedArrayBuffer` object. The sender and recipient still hold distinct instances of the `SharedArrayBuffer` object, but they represent pointers to the same memory span and mutations on one side are visible to the other side.



Enabling shared memory

Memory sharing isn't inherently risky; however, it allows implementing a high-precision clock (we won't go into the details here), which was revealed to enable Spectre attacks. In response to this, browsers restrict shared memory access to cross-origin isolated websites only.

> **Note:** If you are using Node.js for this tutorial (which means you had been adapting the previous examples to Node-specific APIs), you can ignore this section, because shared memory is available by default.

If you are using a local HTTP server to experiment with these examples, you must make your website cross-origin isolated, for example by sending the following headers:

http

```
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Opener-Policy: same-origin
```

After you do this, you can perform the following checks to confirm that shared memory is enabled:

js

```
console.log(window.crossOriginIsolated); // true
console.log("SharedArrayBuffer" in window); // true
```

Now you can share memory, but only within an agent cluster—you can't share memory with agents outside of its cluster, even when they have a message channel. A window and its dedicated worker are in the same cluster, so our examples will work fine.

Using a shared buffer

The `SharedArrayBuffer` (SAB) interface is mostly identical to `ArrayBuffer`. You can construct a `SharedArrayBuffer`, which allocates the specified number of bytes.

```js
const sab = new SharedArrayBuffer(1024);
```

You can read and write the memory contents by constructing a view, such as a `Int8Array`.

```js
const int8 = new Int8Array(sab);
int8[0] = 63;
```

You can create a slice of the SAB, which allocates a new shared memory span and copies its contents.

```js
const sab2 = sab.slice();
// Entire sab is copied
const int82 = new Int8Array(sab2);
console.log(int82[0]); // 63
```

There are several differences between `SharedArrayBuffer` and `ArrayBuffer` (other than memory-sharing behavior, of course):

- Resizable SABs can only be *grown*, while normal resizable buffers can both be grown and shrunk.
- SABs cannot be transferred/detached.

To create a resizable/growable SAB, pass the `maxByteLength` option specifying the maximum length it's allowed to grow to. Then, you can use `grow()`.

```js
const sabResizable = new SharedArrayBuffer(1024, { maxByteLength: 2048 });
console.log(sabResizable.byteLength); // 1024

sabResizable.grow(2000);
console.log(sabResizable.byteLength); // 2000
```

Posting SharedArrayBuffers

`SharedArrayBuffer` objects are always shared when passed to `postMessage()` (as long as the security requirements are met); no extra options are necessary. Both sides can mutate this memory span and observe the other side's mutations. You can also pass a view of a `SharedArrayBuffer` directly—the view and the `buffer` object are both copied, but the underlying memory span is still shared.

In the following examples, we'll share a buffer between `main.js` and `worker.js` together with the message port. Replace the line in `main.js`:

```js
worker.postMessage({ hereIsYourPort: port2 }, [port2]);
```

With:

```js
const sab = new SharedArrayBuffer(1024);
worker.postMessage({ hereIsYourPort: port2, andHereIsOurSAB: sab }, [port2]);
```

And inside `worker.js`, replace the initial reception code with:

```js
const { port, sab } = await new Promise((resolve, reject) => {
  self.addEventListener(
    "message",
    (e) => {
      const port = e.data.hereIsYourPort;
      if (!(port instanceof MessagePort)) {
        reject(new Error("I'm expecting a dedicated port"));
        return;
      }
      const sab = e.data.andHereIsOurSAB;
      if (!(sab instanceof SharedArrayBuffer)) {
        reject(new Error("I'm expecting a shared array buffer"));
        return;
      }
      resolve({ port, sab });
    },
    { once: true },
  );
});
```

To verify that this setup works, try coordinating a short write-read exchange using a message. The worker immediately updates the `SharedArrayBuffer` and posts a message to the main thread. The main thread receives that message and reads the new value. We'll talk about more efficient communications later.

```js
// -- main.js --
const int8 = new Int8Array(sab);

port1.addEventListener("message", (e) => {
  console.log(int8[0]); // 63
});
```

```js
// -- worker.js --
const int8 = new Int8Array(sab);
int8[0] = 63;
port.postMessage("I've modified the memory. Please check");
```

Accessing shared memory with Atomics

In the example above, we use a message to force the following order:

1. Worker writes to memory
2. Worker posts message
3. Main agent receives message and runs event listener
4. Main agent reads memory

The problem with event-based coordination is that events are asynchronous: the time between steps 2 and 3 is dependent on the browser's event loop. In this case, you might as well pass values back and forth without relying on shared memory. The advantage of shared memory is exactly that you can (mostly) communicate synchronously, without yielding the job queue to other async tasks.

In a single agent, the run-to-completion semantics ensure well-behaved effect ordering with respect to multiple synchronous jobs:

```js
const promise = Promise.resolve();
let i = 0;
promise.then(() => {
  i += 1;
  console.log(i);
});
promise.then(() => {
  i += 1;
  console.log(i);
});
```

Here, the overall order is always `i += 1; console.log(i); i += 1; console.log(i);` and never `i += 1; i += 1; console.log(i); console.log(i);`.

There aren't such guarantees when the synchronous jobs belong to different agents. In the example below, the worker sends the main agent a `started` message and immediately starts incrementing the shared integer in a loop. Upon receipt of the message, the main agent runs the same loop at the same time. When the worker finishes, it sends a `finished` message, and the main agent logs the final value.

```js
// -- main.js --
const i = new Int32Array(sab, 0, 1);

port1.addEventListener("message", ({ data }) => {
  if (data === "started") {
    for (let n = 0; n < 10_000_000; n++) {
      i[0] += 1;
    }
  } else if (data === "finished") {
    console.log(i[0]); // Usually less than 20000000
  }
});
```

```js
// -- worker.js --
const i = new Int32Array(sab, 0, 1);

port.postMessage("started");
for (let n = 0; n < 10_000_000; n++) {
  i[0] += 1;
}
port.postMessage("finished");
```

Because messages to a port are delivered in order, the `finished` handler always runs after the main agent's own loop has completed, so both agents have finished counting by the time the value is logged. Still, the logged value is probably less than `20000000`, and it differs between runs. This is because `i[0] += 1` is not a single step: it reads `i[0]`, adds 1, and writes the result back. There are at least two orders possible; the first one, which doesn't lose an increment, is:

1. `main.js` reads `i[0]` as `currentI`
2. `main.js` writes `i[0] = currentI + 1`
3. `worker.js` reads `i[0]` as `currentI + 1`
4. `worker.js` writes `i[0] = currentI + 2`

The second one, which loses an increment, is:

1. `main.js` reads `i[0]` as `currentI`
2. `worker.js` reads `i[0]` as `currentI`
3. `main.js` writes `i[0] = currentI + 1`
4. `worker.js` writes `i[0] = currentI + 1`

To solve this problem, we must use a mechanism to ensure execution order across agents, so a statement in `worker.js` cannot execute until a statement in `main.js` finishes and its effect becomes observable, and vice versa. Methods from the `Atomics` namespace provide this capability.

Reading and writing

The most basic `Atomics` primitives are `Atomics.load()` and `Atomics.store()`, which read and write at a particular index, respectively. They can be used with any signed or unsigned integer arrays (except `Uint8ClampedArray`).

The problem with the index read/write syntax is that the effect of a write is not required to be observable immediately to other agents. Consider the following indexed read/write:

```js
// -- main.js --
const a = new Int32Array(sab);
a[0] = 63;
console.log("Main:", a[1]);
```

```js
// -- worker.js --
const a = new Int32Array(sab);
a[1] = 15;
console.log("Worker:", a[0]);
```

Theoretically, no matter how these operations are ordered, as long as each agent executes its code in order, at least one log entry must be non-`0` (try listing all possible orderings of the four statements, with the constraint that `a[0] = 63` is before `console.log(a[1])`, and `a[1] = 15` is before `console.log(a[0])`!). However, in reality, both logs may print 0. This may arise from the following order:

1. `main.js` reads `a[1]` as `0`.
2. `worker.js` reads `a[0]` as `0`.
3. `main.js` writes `a[0] = 63`.
4. `worker.js` writes `a[1] = 15`.
5. `main.js` and `worker.js` execute their `console.log` using already-retrieved `a[1]` and `a[0]` values, respectively.

Note how the expressions that read `a[1]` and `a[0]`, which come towards the end of the source code, are reordered to be executed before the `a[0] = 63` and `a[1] = 15` statements. This kind of instruction reordering can happen in both the JavaScript compiler and the CPU in order to optimize consecutive

memory operations. Within each single agent, this ordering perfectly preserves the sequential-execution guarantees you would expect. For example, for `main.js`, the `a[0] = 63` write is never read back, so it's perfectly fine to reorder it after the `a[1]` read. However, this causes problems with multiple agents.

With `Atomics.load()` and `Atomics.store()`, you explicitly tell the compiler and the CPU that such reordering is not permitted, and the order of memory read/write must be consistent with the order of JavaScript code execution.

```js
// -- main.js --
const a = new Int32Array(sab);
Atomics.store(a, 0, 63);
console.log("Main:", Atomics.load(a, 1));
```

```js
// -- worker.js --
const a = new Int32Array(sab);
Atomics.store(a, 1, 15);
console.log("Worker:", Atomics.load(a, 0));
```

Here, although you may still see `0` in one of the logs (for example, because the main `load` happens before the worker `store`), they are guaranteed to never both be `0`.

> **Note:** If you want to go further and guarantee that both new values are observed, you need to make both agents wait before the `Atomics.load()` call, such as using a `while (Atomics.load(a, 1) === 0);` loop.

Furthermore, atomic load/store operations establish a partial ordering for other *non-atomic* read/write operations. Specifically, if agent A runs `Atomics.store()`, and then agent B reads the same value with `Atomics.load()`, then all non-atomic writes executed by A before the atomic store must also become observable to all non-atomic reads executed by B after the atomic load. This essentially create a "synchronization point" between two threads, much like a `message` event. For example:

```js
// -- main.js --
a[0] = 63;
// Set a "Ready" flag in a[1]; assume initially it's not 1.
Atomics.store(a, 1, 1);
```

```js
// -- worker.js --
// Spin until a[1] receives the "Ready" flag
while (Atomics.load(a, 1) !== 1);
console.log(a[0]); // 63
```

> **Note:** Within a single agent, the `worker.js` code would definitely be a bug because it's not possible for `a[1]` to have changed in the loop if the loop is never updating the value. Many synchronous infinite loops only make sense with shared memory!
>
> In Waiting and notifying, we'll see how to make this code slightly more performant and less like a bug.

The only possible order is the following:

1. `main.js` writes `a[0] = 63`. `worker.js` keeps spinning in the `while` loop because `a[1]` isn't the right value.

2. `main.js` stores `a[1]` as `1`.
3. `worker.js` loads the new `a[1]` value and exits the `while` loop.
4. `worker.js` reads `a[0]` as `63`.

We have successfully established an ordering from 1 to 4 with the help of the store/load pair, so `worker.js` can observe the new value of `a[0]` written by `main.js`, despite not using atomic operations.

Atomic updates

The next set of operations directly replace several compound assignment operators. As we already demonstrated, the `+=` operator is not atomic: other instructions may execute between the read and the write, causing the final write to not write the value you expect. To fix this, you can replace it with `Atomics.add()`:

```js
// -- main.js --
const i = new Int32Array(sab, 0, 1);

port1.addEventListener("message", ({ data }) => {
  if (data === "started") {
    for (let n = 0; n < 10_000_000; n++) {
      Atomics.add(i, 0, 1);
    }
  } else if (data === "finished") {
    console.log(i[0]); // Guaranteed 20000000
  }
});
```

```js
// -- worker.js --
const i = new Int32Array(sab, 0, 1);

port.postMessage("started");
for (let n = 0; n < 10_000_000; n++) {
  Atomics.add(i, 0, 1);
}
port.postMessage("finished");
```

The only difference from the previous data-race example is that we've replaced both `i[0] += 1` statements with `Atomics.add(i, 0, 1)`. But now, not only does it have the same "sequentially consistent" guarantee of `load/store`, but each block of read-modify-write sequence is also guaranteed to be atomic, so there are two atomic blocks that can be freely ordered but never interleaved:

1. Atomic block:
    1. `main.js` reads `i[0]` as `currentI`
    2. `main.js` writes `i[0] = currentI + 1`
2. Atomic block:
    1. `worker.js` reads `i[0]` as `currentI + 1`
    2. `worker.js` writes `i[0] = currentI + 2`

A few other compound assignment operators have their respective `Atomics` methods:

Operator	Method
`+=`	`Atomics.add()`

Operator	Method
-=	Atomics.sub()
&=	Atomics.and()
|=	Atomics.or()
^=	Atomics.xor()

If your operation does not correspond to one of the methods above, you can still use the general `Atomics.compareExchange()` method instead. This method is a replacement for the following sequence:

```js
const oldValue = array[index];
if (oldValue === expectedValue) {
  array[index] = newValue;
}
return oldValue;
```

For example, if you have an operation `array[index] *= multiplier`, you can make it atomic using the following:

```js
function atomicMultiply(array, index, multiplier) {
  let oldValue = Atomics.load(array, index);

  while (true) {
    const observed = Atomics.compareExchange(
      array,
      index,
      oldValue,
      oldValue * multiplier,
    );

    if (observed === oldValue) {
      return oldValue;
    }

    oldValue = observed;
  }
}
```

This function spins in a loop. Each time, it tries to write `oldValue * multiplier`, on the condition that at the time of its atomic block, the value at that index is still `oldValue`. If the value has changed, it re-attempts with the new `oldValue`.

There's also an unconditional exchange method, `Atomics.exchange()`. This method is a replacement for the following sequence:

```js
const oldValue = array[index];
array[index] = newValue;
return oldValue;
```

Use this method when you want to reset some memory while obtaining its value at the time of reset. For example, it can be used for a clock:

```js
// -- worker.js --
while (true) {
  Atomics.add(a, 0, 1);
}
```

```js
// -- main.js --
const ticksSinceLastCheck = Atomics.exchange(a, 0, 0);
// ticksSinceLastCheck records how many times the worker has spun.
```

If you split this into a `load` and a `store` operation, then the worker may have performed another `add` in the meantime, which is silently dropped.

Waiting and notifying

In the examples above, wherever an agent is waiting on another agent to complete its operation, we use an infinite loop, like the `while (Atomics.load(a, 1) !== 1);` example. This is called busy waiting. Essentially, it blocks the entire thread from executing other jobs, until some other thread writes the value it expects. You can make this slightly more efficient by calling `Atomics.pause()` inside the `while` loop. This method may tell the CPU to allocate fewer resources because it's just waiting and not actually doing computation, without asking the CPU to be immediately yielded to other threads.

```js
// -- worker.js --
while (Atomics.load(a, 1) !== 1) {
  Atomics.pause();
}
```

Alternatively, you can change it to an explicit notification-based system while still keeping it sync. The `Atomics.wait()` and `Atomics.notify()` methods allow you to do so—the waiting thread doesn't have to keep checking the value; the writing thread notifies it whenever the writing is done.

> **Note:** Unlike the previously-introduced methods, `wait()`, `waitAsync()`, and `notify()` can only operate on `Int32Array` and `BigInt64Array` views.

Here, the `worker.js` thread is halted if `a[1] === 0`.

```js
// -- worker.js --
Atomics.wait(a, 1, 0);
console.log(a[0]);
```

It stays halted until another thread—in our case, `main.js`—notifies it:

```js
// -- main.js --
a[0] = 63;
Atomics.store(a, 1, 1);
Atomics.notify(a, 1);
```

Some important notes to call out:

- `Atomics.wait()` also blocks the thread from executing other jobs. On the web, the main thread cannot be blocked, so calling `Atomics.wait()` throws an error there.

- `Atomics.wait()` returns whenever it has been notified, regardless of whether `a[1]` has actually changed.
- Other than checking the initial value, the memory index that `store` and `wait` operate on is entirely arbitrary: they do not care what that value actually represents. The index serves as the ID of the list of waiting agents. The initial flag check ensures that if the worker `wait` happens after `notify` is already called, the `wait` continues immediately instead of sleeping forever, so you should set and only set `a[1]` to `1` (representing "Ready" in our case) immediately before calling `notify`.

You can also specify a `timeout`, after which the thread resumes despite not receiving a notification:

```js
// -- worker.js --
const result = Atomics.wait(a, 1, 0, 100); // Wait about 100ms
if (result === "timed-out") {
  // Decide what to do here: wait again? Throw an error?
  // Change to an asynchronous wait instead?
} else {
  // result can be either "not-equal" - i.e., a[1] is already "Ready"
  // without waiting or "ok" - i.e., some other thread notified it
  console.log(a[0]);
}
```

The `Atomics.waitAsync()` method provides the same functionality as `wait()`, but in the form of a promise, thus allowing other asynchronous jobs to proceed in the meantime.

```js
// -- main.js --
const result = Atomics.waitAsync(a, 1, 0, 100); // Wait about 100ms
if (!result.async) {
  // Already ready
  // (Note: because timeout is not 0, result.value is always "not-equal")
  console.log(a[0]);
} else {
  const resultValue = await result.value;
  if (resultValue === "ok") {
    console.log(a[0]);
  } else {
    // resultValue is "timed-out" here; decide what to do
  }
}
```

You must carefully weigh the impacts of the three waiting mechanisms:

- Busy waiting blocks the thread without yielding the CPU. When it finishes, the CPU can immediately resume without context switching, but while it's spinning, this core is occupied and cannot take other jobs (although the operating system may decide to schedule in some other thread if the current thread has used the CPU for too long). This is the fastest mechanism when you expect the wait to be very short, even shorter than the time taken by two context switches.
- Synchronous `Atomics.wait()` blocks the thread but yields the CPU. The CPU core can take other threads and may need to context switch when this thread continues. This is the most efficient mechanism when you expect the wait to be longer than two context switches and it might be better to let other threads use the CPU in the meantime, but you prefer to keep the code synchronous.
- Asynchronous `Atomics.waitAsync()` uses normal promise semantics: the code that waits for its completion comes back as another job in the agent, but other code in the same thread keeps executing. This is useful when the current thread has other jobs that are equally important and must

proceed in the meantime, such as the main thread, which needs to render the page and respond to user interactions.

Useful shared memory patterns

Let's look at a few common techniques and concepts for ensuring the performance and correctness of multi-threading.

A shared memory protocol needs to answer these questions:

- Who may access each part of the memory?
- Which operations must happen together without interleaving operations from other threads?
- How does an agent know that data is ready to read or safe to overwrite?
- How much work does the protocol require when an agent is working alone, and what happens when many agents compete for access?

Locks and mutexes

A *lock*, also called a *mutex* (for "mutual exclusion"), is a mechanism that ensures that while a thread has access to a particular resource, no other thread can have access to the same resource. Each thread acquires the lock before accessing protected data and releases it afterwards. The code between acquisition and release is called a *critical section*.

> **Note:** A *futex* is short for a "fast user-space mutex", but somewhat confusingly, it is not a mutex—it is a waiting mechanism that can be used to implement a mutex. In JavaScript, futexes are the only built-in waiting mechanism, provided by `Atomics.wait()`/`Atomics.waitAsync()` and `Atomics.notify()`; our mutex examples build upon this mechanism.

A mutex is merely a protocol and does not physically prevent another thread from accessing the memory. As long as the protocol is upheld, different threads will never have overlapping critical sections, thus protecting a whole sequence of operations without making each individual read and write atomic.

For example, create a buffer with space for a lock and a value. In our protocol, each participating agent will create an `Int32Array` view where the first element represents the lock (`1` means unlocked, `0` means locked) and the second holds the protected data.

```js
const sab = new SharedArrayBuffer(8);
const shared = new Int32Array(sab);
shared[0] = 1;
```

Here's how one participating agent may implement that protocol. When an agent calls `lock()` to try to acquire the lock:

- `Atomics.compareExchange()` is called, attempting to set the first element to `0` (locked), but only if it's currently `1` (available).
- If that fails—i.e., when it's already `0` because another agent is holding the lock—`Atomics.wait()` puts the caller to sleep, provided the lock still contains `0` (the lock may have been released between `compareExchange()` and `wait()`).
- The agent will only wake up when the agent currently holding the lock calls `unlock()`, which both resets the lock element to `1` and wakes the earliest waiting agent in the queue with

`Atomics.notify()`.

- After waking, the agent tries to acquire the lock again, but it may still fail if there's a newcomer (for example, if the releasing agent immediately calls `lock()` again) who got the lock first, in which case this agent goes back to sleep again due to the `while()` loop.
- The loop only exits when lock acquisition succeeds.

```js
function lock() {
  while (Atomics.compareExchange(shared, 0, 1, 0) !== 1) {
    Atomics.wait(shared, 0, 0);
  }
}

function unlock() {
  Atomics.store(shared, 0, 1);
  Atomics.notify(shared, 0, 1);
}
```

Here's how the lock-unlock pair can be called. The critical section must be wrapped in a `try...finally` block, with `unlock()` called in `finally`, so the lock is released even if the critical section throws an exception:

```js
lock();
try {
  shared[1] *= 2;
  shared[1] += 1;
} finally {
  unlock();
}
```

Although the critical section uses ordinary indexed read/write, other agents will never observe stale values, because the atomic operations in `lock()`/`unlock()` establish a sequentially-consistent ordering.

One mutex around all shared data is a straightforward starting point, but it serializes all protected work. If agents can operate on independent regions, giving those regions separate locks—or assigning each region to a single writer—can allow more concurrency.

This simple mutex has multiple opportunities for improvement:

What if an agent accidentally calls `unlock()` when the lock wasn't actually held by it?

We can solve this by replacing the boolean flag with a unique ID for each agent, so `unlock()` only proceeds if the ID in the lock matches the agent ID.

What if an agent accidentally calls `lock()` when it's already holding the lock (this is called *re-entry*)? This would cause it to wait on itself indefinitely, and all other agents waiting for the lock will also never get it.

We can solve this by making `lock()` only wait if the owner ID in the lock doesn't match the current ID.

> **Note:** Re-entry is commonly seen in nested function calls; for example if a critical section calls another function that also tries to establish a critical section using the same lock. If the lock is non-reentrant (it doesn't implement this mechanism), then each agent must ensure that the critical section is never re-entered.

What if the agent that re-enters the lock calls `unlock()`? This would cause the lock to be released early, despite there being another `lock()` call that expects the lock to still be held.

We need to add a counter that increments on each `lock()` and decrements on each `unlock()`, and only actually release the lock when the counter reaches zero.

What if the waiting agent always loses to a newcomer and never gets the lock?

We need some scheduling mechanism to ensure that the newcomer doesn't jump the queue.

Spinlocks

In the example above, an agent waiting for the lock goes to sleep and requires the other agent to notify it with `unlock()`. This is efficient when contention is high, because the lock may not be acquired quickly and it's best to yield the CPU to other processes not needing the lock. But when contention is low, scheduling the thread in and out of the CPU creates extra overhead compared to just occupying the CPU while waiting.

A *spinlock* uses busy waiting instead of putting contenders to sleep. To turn the previous mutex into a spinlock, replace its functions with:

```js
function lock() {
  while (Atomics.compareExchange(shared, 0, 1, 0) !== 1) {
    Atomics.pause();
  }
}

function unlock() {
  Atomics.store(shared, 0, 1);
}
```

The `Atomics.pause()` here is just a performance hint to the CPU. You can remove it and still keep the same functionality.

> **Warning:** Using spinlocking on the main thread is not recommended, as it will freeze the entire page. In general, unless designed very carefully, spinlocks may not actually be more performant than a regular wait.

Sometimes, whether contention is high or low is not for you to decide while writing the code. The same code may run when there are only two contenders and busy-waiting is more efficient, or when there are a thousand contenders and it must use a regular wait. In this case, you can use a hybrid approach, which is to first spin for a little while (say, 10 iterations), and if the lock is still not acquired, then call `Atomics.wait()`. There's an example on the `Atomics.pause()` page.

Semaphores

A mutex only allows one agent to access the resource at a time; a *semaphore*, on the other hand, generalizes that to allow up to a specified number of simultaneous agents.

To implement this, the shared memory maintains a count of available *permits*: acquiring a permit decrements the count, releasing one increments it, and an agent waits if no permits are available.

```js
const sab = new SharedArrayBuffer(4);
const permits = new Int32Array(sab);
permits[0] = 3;
```

Each worker creates its own view of that buffer and uses the following functions:

```js
const permits = new Int32Array(sab);

function acquire() {
  while (true) {
    const available = Atomics.load(permits, 0);
    if (available === 0) {
      Atomics.wait(permits, 0, 0);
    } else if (
      Atomics.compareExchange(permits, 0, available, available - 1) ===
      available
    ) {
      return;
    }
  }
}

function release() {
  Atomics.add(permits, 0, 1);
  Atomics.notify(permits, 0, 1);
}

acquire();
try {
  // Perform a task that requires one permit.
} finally {
  release();
}
```

You might notice that if we set the maximum number of permits to 1, we arrive at essentially the same code as the mutex example—indeed, our simple mutex is just a *binary semaphore* (at most 1 permit).

A semaphore with multiple permits (called a *counting semaphore*) still runs into problems when the concurrent agents try to access the same resource. The goal of a semaphore is to moderate access to a limited resource pool (for example, if there are only 4 available buffers). You still need a separate mechanism to assign these resources to the agents to avoid simultaneous access.

Avoiding deadlocks

A *deadlock* occurs when agents wait on each other in a cycle, so none can make progress. For example:

1. Agent A acquires lock X.
2. Agent B acquires lock Y.
3. Agent A waits to acquire Y, while still holding X.
4. Agent B waits to acquire X, while still holding Y.

```js
// -- a.js --
lockX.lock();
try {
  lockY.lock(); // Waiting for lockY.unlock() in b.js...
  try {
    // Work requiring both locks
  } finally {
    lockY.unlock();
  }
} finally {
  lockX.unlock();
}
```

```js
// -- b.js --
lockY.lock();
try {
  lockX.lock(); // Waiting for lockX.unlock() in a.js...
  try {
    // Work requiring both locks
  } finally {
    lockX.unlock();
  }
} finally {
  lockY.unlock();
}
```

Neither agent can reach the code that releases its lock.

Waiting on a lock is just one way to enter a deadlock: two agents may also wait on a message from the other agent, etc.

Preventing deadlocks requires very high discipline within each agent. For example, when an operation needs multiple locks, the protocol must specify the order in which these locks are acquired. Each agent must make sure that locks are eventually released, especially in case of errors (by using `try...finally`). How such protocols can be enforced at the compiler or static checker level is an active area of research.

Lock-free algorithms

Locks are powerful and convenient (when implemented correctly), but they are also a bit heavyweight, especially when there's no contention. For every critical section—which may be as short as a single read/write operation—the mutex above incurs the overhead of an `Atomics.compareExchange()`, an `Atomics.store()`, and—except when using spinlocks—an `Atomics.notify()`.

In fact, even something as simple as `Atomics.load(uint16, 0)` might need an internal lock! If you think about it, an element in a `Uint16Array` is comprised of two bytes. If an extremely simple CPU only allows reading memory byte-by-byte, it would look like the following (assuming little-endian byte order):

```js
const firstByte = uint8[0]; // uint8 is the same memory span viewed as Uint8Array
const secondByte = uint8[1];

const value = firstByte + (secondByte << 8);
```

If another agent writes a new two-byte value between the first and second statements, the final read may combine bytes from the old and new values, rather than being a coherent `uint16` value. This is called a *torn read*. Atomic reads and writes of the same element never tear, because the engine either puts a lock on the bytes of interest or uses native CPU instructions that provide the required atomicity. Locking involves two extra steps (acquisition and release) and can be much slower than a lock-free implementation; whether the former or the latter is used depends on the operating system, CPU architecture, JavaScript engine, etc., as well as the byte size of the operation. To know if atomic operations on a specific byte size in your current environment are implemented without locks, use `Atomics.isLockFree()`.

```js
Atomics.isLockFree(4); // true: required for 4-byte atomic operations
Atomics.isLockFree(7); // false: not a supported atomic element size
```

Not all operations are *tear-free*. Read/write on `Int16`, `Uint16`, `Int32`, `Uint32` types never tear, whether using `Atomics` or indexed access. Other data types may tear if you use indexed access, so you need to use `Atomics` to avoid tears, or—for unsupported types like floating point—implement a lock yourself. `Atomics.isLockFree` tells you whether such tear-free guarantee comes from a lock or a native instruction.

You can also avoid locks in your own code. Recall the atomic multiplication code from above:

```js
function atomicMultiply(array, index, multiplier) {
  let oldValue = Atomics.load(array, index);

  while (true) {
    const observed = Atomics.compareExchange(
      array,
      index,
      oldValue,
      oldValue * multiplier,
    );

    if (observed === oldValue) {
      return oldValue;
    }

    oldValue = observed;
  }
}
```

There's no explicit lock here. The agent doesn't actively enforce a critical section; it just repeatedly retries the read-write sequence until there turns out to be no contender in between. This makes the no-contender case as fast as possible, but may retry many times when contention is high.

Producer–consumer handoff

A common case where two threads need to access the same memory is when one is a *producer* and the other is a *consumer*. With a normal lock, each time, both agents compete for the lock, and if the data isn't ready yet (for example, the producer finds that the consumer hasn't read it, or the consumer finds that the producer hasn't written it; you also need a separate readiness flag to track this), that agent goes back to sleep and lets the other agent take the lock; this is inefficient. The *handoff* pattern lets two agents take turns, coalescing the lock and the readiness flag into the same protocol.

Again, we start with a 2-element shared memory. The first element is a state flag: `0` means empty and `1` means full. The second element holds one value.

```js
const sab = new SharedArrayBuffer(8);
const shared = new Int32Array(sab);
```

The producer waits while the value slot is "full" (i.e., the consumer has not read it). When the slot is "empty", it writes the new value and notifies the consumer.

```js
// -- producer.js --
function produce(value) {
  while (Atomics.load(shared, 0) !== 0) {
    Atomics.wait(shared, 0, 1);
  }

  shared[1] = value;
  Atomics.store(shared, 0, 1);
  Atomics.notify(shared, 0, 1);
}

produce(63);
produce(64);
```

Conversely, the consumer waits while the value slot is "empty", and reads the new value when it becomes "full".

```js
// -- consumer.js --
function consume() {
  while (Atomics.load(shared, 0) !== 1) {
    Atomics.wait(shared, 0, 0);
  }

  const value = shared[1];
  Atomics.store(shared, 0, 0);
  Atomics.notify(shared, 0, 1);
  return value;
}

console.log(consume()); // 63
console.log(consume()); // 64
```

This protocol depends on having exactly one producer and one consumer. With multiple producers, two agents could both see an empty slot and overwrite each other's data, so you would still need a mutex among the producers. The same goes for consumers. This is still an improvement over a shared lock among all producers and consumers though: it lets control alternate between the *groups* of producers and consumers without competition.

Number-guessing game using shared memory

To close off this guide, let's re-implement our number-guessing game with this handoff pattern. Both agents alternate between producing and consuming: the worker produces a guess, and the main thread consumes it and produces feedback. They share two integers:

- `shared[0]` indicates whose turn it is: `0` for the worker and `1` for the main thread.

- `shared[1]` holds either the guess or the feedback, depending on whose turn it is. Feedback is `-1` for "Too low", `1` for "Too high", and `0` for "Correct".

js

```
// -- main.js --
const worker = new Worker("./worker.js", { type: "module" });
const sab = new SharedArrayBuffer(8);
const shared = new Int32Array(sab);
const secretNumber = 63;

// The turn flag is initially 0, so the worker takes the first turn
worker.postMessage(sab);

// Wait until the worker has started. The browser may not start the
// worker until the main thread returns to the event loop.
await new Promise((resolve) => {
  worker.addEventListener("message", resolve, { once: true });
});

while (true) {
  // Wait until the worker finishes its turn
  while (Atomics.load(shared, 0) !== 1) {
    // We use busy waiting here because we expect the worker to come
    // back very quickly and the main thread can't use Atomics.wait().
    // In more complex scenarios, you probably want to use Atomics.waitAsync()
    // to avoid blocking the main thread.
    Atomics.pause();
  }

  const workerGuess = shared[1];
  console.log("Worker guessed:", workerGuess);
  const feedback =
    workerGuess < secretNumber ? -1 : workerGuess > secretNumber ? 1 : 0;

  shared[1] = feedback;
  Atomics.store(shared, 0, 0);
  Atomics.notify(shared, 0, 1);

  if (feedback === 0) {
    console.log("Worker successfully guessed the secret number!");
    break;
  }
}
```

js

```
// -- worker.js --
const sab = await new Promise((resolve) => {
  self.addEventListener("message", (e) => resolve(e.data), { once: true });
});
const shared = new Int32Array(sab);
let low = 0;
let high = 100;

// Tell the main thread that the worker has started
self.postMessage("ready");
```

(code continues on the next page)

(code continued from the previous page)

```
while (true) {
  const lastGuess = Math.floor((low + high) / 2);
  // Worker initiates the handoff here by making a guess
  shared[1] = lastGuess;
  Atomics.store(shared, 0, 1);
  Atomics.notify(shared, 0, 1);

  // Wait until the main agent writes the feedback
  while (Atomics.load(shared, 0) !== 0) {
    Atomics.wait(shared, 0, 1);
  }

  const feedback = shared[1];
  if (feedback === 0) {
    console.log("Yay");
    break;
  }
  if (feedback === -1) {
    low = lastGuess + 1;
  } else if (feedback === 1) {
    high = lastGuess - 1;
  }
  if (low > high) {
    throw new Error(
      "Sorry, the feedback from main.js is inconsistent and I suspect main is cheating",
    );
  }
}
```

This implements the same functionality, and note how both loops are synchronous—this should be much more efficient than passing messages back and forth!

See also

- JavaScript execution model
- Web workers API
- `SharedArrayBuffer`
- `Atomics`
- `Worker.postMessage()`